HỌC MÁY

CƠ SỞ VÀ ỨNG DỤNG VỚI PYTHON

Giáo trình dùng cho sinh viên ngành Công nghệ Thông tin,
Toán – Tin và Kỹ thuật Dữ liệu

Bản thảo biên soạn – 2026

Lời nói đầu

Học máy (Machine Learning) đã trở thành một trong những trụ cột công nghệ quan trọng nhất của thế kỷ XXI, hiện diện trong hầu hết các lĩnh vực từ tài chính, y tế, giáo dục, thương mại điện tử cho đến giao thông và sản xuất công nghiệp. Đối với sinh viên các ngành Công nghệ Thông tin, Toán – Tin và Kỹ thuật Dữ liệu, việc nắm vững nền tảng lý thuyết và khả năng triển khai thực tế các thuật toán học máy không còn là một lựa chọn nâng cao, mà đã trở thành một yêu cầu cốt lõi của chương trình đào tạo.

Giáo trình "Học máy – Cơ sở và Ứng dụng với Python" được biên soạn nhằm cung cấp cho người học một lộ trình học tập có hệ thống, đi từ những khái niệm nền tảng nhất của học máy, qua các nhóm thuật toán học có giám sát và không giám sát, đến các kỹ thuật đánh giá – tối ưu hóa mô hình, và một phần giới thiệu về mạng nơ-ron, học sâu. Mỗi chương trong giáo trình đều được thiết kế theo một cấu trúc thống nhất: mục tiêu học tập rõ ràng, nội dung lý thuyết được trình bày trực quan trước khi đi vào công thức toán học, ví dụ minh họa bằng mã nguồn Python hoàn chỉnh có thể chạy ngay, phần giải thích kết quả, tóm tắt chương, và hệ thống câu hỏi – bài tập từ mức nhận biết đến vận dụng.

Giáo trình không đòi hỏi người học phải có nền tảng toán cao cấp trước khi bắt đầu, nhưng vẫn đảm bảo cung cấp đầy đủ cơ sở lý thuyết (đại số tuyến tính, xác suất thống kê, giải tích cơ bản) ở mức cần thiết để hiểu bản chất của từng thuật toán, thay vì chỉ dừng lại ở việc "gọi hàm" thư viện một cách máy móc.

Về mặt thực hành, toàn bộ mã nguồn trong sách được viết bằng Python theo phong cách hiện đại, sử dụng các thư viện chuẩn của ngành gồm NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn, cùng với TensorFlow/Keras và PyTorch ở các chương liên quan đến học sâu. Phiên bản thư viện cụ thể được khuyến nghị trình bày chi tiết ở Chương 2.

Cấu trúc sách gồm sáu phần: (1) Tổng quan và Công cụ, (2) Học có giám sát, (3) Học không giám sát, (4) Đánh giá mô hình và Tối ưu hóa, (5) Giới thiệu Mạng nơ-ron và Học sâu cơ bản, (6) Dự án thực hành tổng hợp. Cách sắp xếp này cho phép giảng viên sử dụng sách linh hoạt theo học phần một kỳ (tập trung Phần 1–4) hoặc mở rộng thành hai học phần liên tiếp (bổ sung Phần 5–6).

Nhóm biên soạn hy vọng giáo trình này sẽ là tài liệu học tập đáng tin cậy, đồng thời là cuốn cẩm nang tham khảo thực hành hữu ích cho sinh viên trong suốt quá trình học tập và làm việc sau này. Mọi góp ý về nội dung xin được tiếp thu để hoàn thiện giáo trình trong các lần tái bản sau.

Nhóm biên soạn

Mục lục

Lời nói đầu **ii**

I Tổng quan và công cụ **1**

1 Giới thiệu về Học máy **2**

1.1 Mục tiêu học tập . . . 2

1.2 Nội dung lý thuyết . . . 2

1.2.1 Học máy là gì? . . . 2

1.2.2 Vì sao học máy trở nên quan trọng? . . . 3

1.2.3 Phân loại các dạng học máy . . . 3

1.2.4 Quy trình tổng quát của một dự án học máy . . . 4

1.2.5 Các khái niệm nền tảng . . . 6

1.3 Ví dụ minh họa bằng mã Python . . . 7

1.4 Giải thích kết quả chạy chương trình . . . 8

1.5 Tóm tắt chương . . . 9

1.6 Bài tập củng cố . . . 10

1.7 Bài tập lập trình . . . 10

2 Công cụ và Môi trường lập trình Python cho Học máy **12**

2.1 Mục tiêu học tập . . . 12

2.2 Nội dung lý thuyết . . . 12

2.2.1 Vì sao cần một môi trường lập trình riêng biệt? . . . 12

2.2.2 Các công cụ quản lý môi trường phổ biến . . . 12

2.2.3 Công cụ soạn thảo và thực thi mã nguồn . . . 13

2.2.4 Các thư viện nền tảng và phiên bản khuyến nghị . . . 14

2.2.5 Tổ chức thư mục dự án học máy . . . 15

2.3 Ví dụ minh họa bằng mã Python . . . 15

2.4 Giải thích kết quả chạy chương trình . . . 16

2.5 Tóm tắt chương . . . 17

2.6 Bài tập củng cố . . . 17

2.7 Bài tập lập trình . . . 18

3 Xử lý và Khám phá dữ liệu với NumPy, Pandas, Matplotlib/Seaborn 19
3.1 Mục tiêu học tập . . . 19
3.2 Nội dung lý thuyết . . . 19
3.2.1 NumPy — nền tảng tính toán số học . . . 19
3.2.2 Pandas — xử lý dữ liệu dạng bảng . . . 20
3.2.3 Xử lý dữ liệu thiếu (Missing Values) . . . 20
3.2.4 Trực quan hóa dữ liệu trong phân tích khám phá (EDA) . . . 21
3.3 Ví dụ minh họa bằng mã Python . . . 22
3.4 Giải thích kết quả chạy chương trình . . . 23
3.5 Tóm tắt chương . . . 24
3.6 Bài tập củng cố . . . 25
3.7 Bài tập lập trình . . . 25

II Học có giám sát 27

4 Hồi quy tuyến tính (Linear Regression) 28
4.1 Mục tiêu học tập . . . 28
4.2 Nội dung lý thuyết . . . 28
4.2.1 Bài toán hồi quy tuyến tính . . . 28
4.2.2 Hàm mất mát và phương pháp bình phương tối thiểu . . . 29
4.2.3 Thuật toán Gradient Descent . . . 29
4.2.4 Các giả định của hồi quy tuyến tính . . . 30
4.2.5 Các độ đo đánh giá mô hình hồi quy . . . 31
4.3 Ví dụ minh họa bằng mã Python . . . 32
4.4 Giải thích kết quả chạy chương trình . . . 33
4.5 Tóm tắt chương . . . 34
4.6 Bài tập củng cố . . . 34
4.7 Bài tập lập trình . . . 35

5 Hồi quy đa thức và Chính quy hóa (Ridge, Lasso, Elastic Net) 36
5.1 Mục tiêu học tập . . . 36
5.2 Nội dung lý thuyết . . . 36
5.2.1 Hồi quy đa thức (Polynomial Regression) . . . 36
5.2.2 Bậc đa thức và hiện tượng khớp quá . . . 37
5.2.3 Chính quy hóa — ý tưởng chung . . . 37

5.2.4 Ridge Regression (Chính quy hóa L2) . . . 38
5.2.5 Lasso Regression (Chính quy hóa L1) . . . 38
5.2.6 Elastic Net . . . 39
5.2.7 Lưu ý quan trọng: chuẩn hóa đặc trưng trước khi chính quy hóa . . . 39
5.3 Ví dụ minh họa bằng mã Python . . . 39
5.4 Giải thích kết quả chạy chương trình . . . 41
5.5 Tóm tắt chương . . . 42
5.6 Bài tập củng cố . . . 42
5.7 Bài tập lập trình . . . 43

6 Hồi quy Logistic và Phân loại nhị phân 44
6.1 Mục tiêu học tập . . . 44
6.2 Nội dung lý thuyết . . . 44
6.2.1 Vì sao không dùng hồi quy tuyến tính cho bài toán phân loại? . . . 44
6.2.2 Hàm Sigmoid . . . 45
6.2.3 Hàm mất mát: Binary Cross-Entropy (Log Loss) . . . 45
6.2.4 Ranh giới quyết định (Decision Boundary) . . . 46
6.2.5 Ma trận nhầm lẫn và các độ đo phân loại . . . 47
6.3 Ví dụ minh họa bằng mã Python . . . 48
6.4 Giải thích kết quả chạy chương trình . . . 49
6.5 Tóm tắt chương . . . 51
6.6 Bài tập củng cố . . . 51
6.7 Bài tập lập trình . . . 52

7 Thuật toán K-Láng giềng gần nhất (K-NN) 54
7.1 Mục tiêu học tập . . . 54
7.2 Nội dung lý thuyết . . . 54
7.2.1 Nguyên lý hoạt động của K-NN . . . 54
7.2.2 Các bước dự đoán của K-NN . . . 55
7.2.3 Các độ đo khoảng cách phổ biến . . . 55
7.2.4 Vai trò của giá trị k . . . 56
7.2.5 Tầm quan trọng của việc chuẩn hóa dữ liệu . . . 57
7.2.6 Ưu điểm, nhược điểm và K-NN có trọng số . . . 57
7.3 Ví dụ minh họa bằng mã Python . . . 58
7.4 Giải thích kết quả chạy chương trình . . . 59

7.5 Tóm tắt chương 60
7.6 Bài tập củng cố 61
7.7 Bài tập lập trình 61

8 Cây quyết định (Decision Tree) 63
8.1 Mục tiêu học tập 63
8.2 Nội dung lý thuyết 63
8.2.1 Cấu trúc của cây quyết định 63
8.2.2 Tiêu chí phân chia: Gini Impurity và Entropy 64
8.2.3 Thuật toán xây dựng cây 65
8.2.4 Hiện tượng khớp quá và các siêu tham số kiểm soát 65
8.2.5 Mức độ quan trọng của đặc trưng (Feature Importance) 67
8.2.6 Ưu điểm và nhược điểm của Cây quyết định 67
8.3 Ví dụ minh họa bằng mã Python 68
8.4 Giải thích kết quả chạy chương trình 69
8.5 Tóm tắt chương 70
8.6 Bài tập củng cố 70
8.7 Bài tập lập trình 71

9 Phương pháp tổ hợp: Rừng ngẫu nhiên và Boosting 72
9.1 Mục tiêu học tập 72
9.2 Nội dung lý thuyết 72
9.2.1 Ý tưởng nền tảng của học tổ hợp 72
9.2.2 Bagging và Rừng ngẫu nhiên (Random Forest) 73
9.2.3 Đánh giá Out-of-Bag (OOB) 73
9.2.4 Boosting 74
9.2.5 AdaBoost (Adaptive Boosting) 74
9.2.6 Gradient Boosting 74
9.2.7 So sánh Bagging và Boosting 75
9.3 Ví dụ minh họa bằng mã Python 76
9.4 Giải thích kết quả chạy chương trình 78
9.5 Tóm tắt chương 79
9.6 Bài tập củng cố 80
9.7 Bài tập lập trình 80

10 Máy vector hỗ trợ (Support Vector Machine) 82
10.1 Mục tiêu học tập . . . 82
10.2 Nội dung lý thuyết . . . 82
10.2.1 Ý tưởng tối đa hóa biên . . . 82
10.2.2 Vector hỗ trợ . . . 83
10.2.3 Biên cứng và biên mềm — siêu tham số C . . . 83
10.2.4 Kỹ thuật Kernel (Kernel Trick) . . . 84
10.2.5 Ưu điểm và nhược điểm của SVM . . . 86
10.3 Ví dụ minh họa bằng mã Python . . . 86
10.4 Giải thích kết quả chạy chương trình . . . 87
10.5 Tóm tắt chương . . . 88
10.6 Bài tập củng cố . . . 89
10.7 Bài tập lập trình . . . 89

11 Bộ phân loại Naive Bayes 91
11.1 Mục tiêu học tập . . . 91
11.2 Nội dung lý thuyết . . . 91
11.2.1 Định lý Bayes . . . 91
11.2.2 Giả định "ngây thơ" (Naive) về tính độc lập . . . 92
11.2.3 Các biến thể của Naive Bayes . . . 92
11.2.4 Vấn đề xác suất bằng 0 và làm mượt Laplace . . . 93
11.3 Ví dụ minh họa bằng mã Python . . . 94
11.4 Giải thích kết quả chạy chương trình . . . 95
11.5 Tóm tắt chương . . . 96
11.6 Bài tập củng cố . . . 96
11.7 Bài tập lập trình . . . 97

III Học không giám sát 98

12 Phân cụm K-Means 99
12.1 Mục tiêu học tập . . . 99
12.2 Nội dung lý thuyết . . . 99
12.2.1 Từ học có giám sát sang học không giám sát . . . 99
12.2.2 Thuật toán K-Means . . . 99
12.2.3 Vấn đề khởi tạo và K-Means++ . . . 100

12.2.4 Xác định số cụm K: phương pháp Elbow và chỉ số Silhouette 101
12.2.5 Giả định và hạn chế của K-Means 101
12.3 Ví dụ minh họa bằng mã Python 102
12.4 Giải thích kết quả chạy chương trình 104
12.5 Tóm tắt chương 104
12.6 Bài tập củng cố 105
12.7 Bài tập lập trình 105

13 Phân cụm phân cấp và DBSCAN **107**
13.1 Mục tiêu học tập 107
13.2 Nội dung lý thuyết 107
13.2.1 Phân cụm phân cấp gộp (Agglomerative Hierarchical Clustering) . . 107
13.2.2 Các tiêu chí liên kết (Linkage Criteria) 108
13.2.3 DBSCAN: Phân cụm dựa trên mật độ 109
13.2.4 So sánh K-Means, Phân cụm phân cấp và DBSCAN 110
13.3 Ví dụ minh họa bằng mã Python 111
13.4 Giải thích kết quả chạy chương trình 112
13.5 Tóm tắt chương 113
13.6 Bài tập củng cố 113
13.7 Bài tập lập trình 114

14 Giảm chiều dữ liệu: PCA **115**
14.1 Mục tiêu học tập 115
14.2 Nội dung lý thuyết 115
14.2.1 Vì sao cần giảm chiều dữ liệu? 115
14.2.2 Ý tưởng hình học của PCA 116
14.2.3 Nền tảng toán học: ma trận hiệp phương sai, vector riêng và giá trị riêng 116
14.2.4 Tỉ lệ phương sai giải thích và chọn số thành phần 117
14.2.5 Ứng dụng của PCA 118
14.3 Ví dụ minh họa bằng mã Python 118
14.4 Giải thích kết quả chạy chương trình 120
14.5 Tóm tắt chương 120
14.6 Bài tập củng cố 121
14.7 Bài tập lập trình 122

IV Đánh giá mô hình và tối ưu hóa 123

15 Đánh giá và lựa chọn mô hình (Cross-Validation, các độ đo) 124

15.1 Mục tiêu học tập . . . 124
15.2 Nội dung lý thuyết . . . 124
15.2.1 Hạn chế của việc chia dữ liệu một lần . . . 124
15.2.2 Kiểm định chéo K-fold (K-fold Cross-Validation) . . . 125
15.2.3 Kiểm định chéo K-fold phân tầng (Stratified K-fold) . . . 125
15.2.4 Đường cong học (Learning Curve) . . . 126
15.2.5 Mở rộng đường cong ROC và chỉ số AUC . . . 126
15.2.6 Quy trình chia dữ liệu ba tầng . . . 127
15.3 Ví dụ minh họa bằng mã Python . . . 128
15.4 Giải thích kết quả chạy chương trình . . . 130
15.5 Tóm tắt chương . . . 131
15.6 Bài tập củng cố . . . 131
15.7 Bài tập lập trình . . . 132

16 Tối ưu siêu tham số (Grid Search, Random Search) 133

16.1 Mục tiêu học tập . . . 133
16.2 Nội dung lý thuyết . . . 133
16.2.1 Tham số mô hình và siêu tham số . . . 133
16.2.2 Grid Search (Tìm kiếm lưới) . . . 134
16.2.3 Random Search (Tìm kiếm ngẫu nhiên) . . . 134
16.2.4 Kết hợp với kiểm định chéo và nguy cơ rò rỉ thông tin . . . 135
16.2.5 Giới thiệu khái quát về tối ưu hóa Bayesian . . . 136
16.3 Ví dụ minh họa bằng mã Python . . . 136
16.4 Giải thích kết quả chạy chương trình . . . 137
16.5 Tóm tắt chương . . . 139
16.6 Bài tập củng cố . . . 139
16.7 Bài tập lập trình . . . 140

17 Kỹ thuật xử lý đặc trưng (Feature Engineering) 141

17.1 Mục tiêu học tập . . . 141
17.2 Nội dung lý thuyết . . . 141
17.2.1 Mã hóa biến phân loại (Categorical Encoding) . . . 141

17.2.2 Lựa chọn đặc trưng (Feature Selection) . . . 142
17.2.3 Pipeline: kết hợp tiền xử lý và mô hình . . . 144
17.2.4 ColumnTransformer: xử lý đồng thời nhiều loại cột . . . 144
17.3 Ví dụ minh họa bằng mã Python . . . 145
17.4 Giải thích kết quả chạy chương trình . . . 147
17.5 Tóm tắt chương . . . 147
17.6 Bài tập củng cố . . . 148
17.7 Bài tập lập trình . . . 148

V Giới thiệu mạng nơ-ron và học sâu cơ bản 150

18 Mạng nơ-ron nhân tạo cơ bản (MLP) 151
18.1 Mục tiêu học tập . . . 151
18.2 Nội dung lý thuyết . . . 151
18.2.1 Từ hồi quy Logistic đến nơ-ron nhân tạo . . . 151
18.2.2 Cấu trúc mạng nơ-ron nhiều tầng . . . 152
18.2.3 Các hàm kích hoạt phổ biến . . . 153
18.2.4 Lan truyền tiến và lan truyền ngược . . . 154
18.2.5 Vấn đề đạo hàm biến mất (Vanishing Gradient) . . . 155
18.2.6 Kiểm soát khớp quá trong mạng nơ-ron . . . 155
18.3 Ví dụ minh họa bằng mã Python . . . 156
18.4 Giải thích kết quả chạy chương trình . . . 158
18.5 Tóm tắt chương . . . 159
18.6 Bài tập củng cố . . . 159
18.7 Bài tập lập trình . . . 160

19 Xây dựng mô hình học sâu với TensorFlow/Keras và PyTorch 161
19.1 Mục tiêu học tập . . . 161
19.2 Nội dung lý thuyết . . . 161
19.2.1 TensorFlow/Keras và PyTorch: hai triết lý thiết kế . . . 161
19.2.2 Xây dựng mạng nơ-ron với Sequential API của Keras . . . 162
19.2.3 Callback EarlyStopping . . . 162
19.2.4 Mô hình tương đương trong PyTorch . . . 163
19.3 Ví dụ minh họa bằng mã Python . . . 164
19.4 Giải thích kết quả chạy chương trình . . . 166

19.5 Tóm tắt chương . . . 167
19.6 Bài tập củng cố . . . 167
19.7 Bài tập lập trình . . . 168

20 Mạng nơ-ron tích chập cơ bản (CNN) 170
20.1 Mục tiêu học tập . . . 170
20.2 Nội dung lý thuyết . . . 170
20.2.1 Hạn chế của MLP đối với dữ liệu ảnh . . . 170
20.2.2 Phép tích chập (Convolution) . . . 171
20.2.3 Đệm (Padding) và Bước nhảy (Stride) . . . 171
20.2.4 Tầng gộp (Pooling) . . . 172
20.2.5 Kiến trúc CNN điển hình . . . 172
20.3 Ví dụ minh họa bằng mã Python . . . 173
20.4 Giải thích kết quả chạy chương trình . . . 175
20.5 Tóm tắt chương . . . 176
20.6 Bài tập củng cố . . . 176
20.7 Bài tập lập trình . . . 177

VI Dự án thực hành 178

21 Dự án 1: Dự đoán giá bất động sản (bài toán hồi quy) 179
21.1 Mục tiêu học tập . . . 179
21.2 Đặt vấn đề và mô tả dữ liệu . . . 179
21.3 Khám phá và tiền xử lý dữ liệu . . . 180
21.4 Xây dựng, so sánh và tối ưu mô hình . . . 181
21.5 Giải thích kết quả và tổng kết dự án . . . 183
21.6 Tóm tắt chương . . . 184
21.7 Bài tập củng cố . . . 184
21.8 Bài tập lập trình . . . 185

22 Dự án 2: Phân loại email rác (bài toán phân loại) 186
22.1 Mục tiêu học tập . . . 186
22.2 Đặt vấn đề và mô tả dữ liệu . . . 186
22.3 Xử lý văn bản và xây dựng mô hình . . . 186
22.4 Giải thích kết quả và lựa chọn ngưỡng vận hành . . . 189

22.5 Tóm tắt chương 190
22.6 Bài tập củng cố 190
22.7 Bài tập lập trình 191

23 Dự án 3: Phân khúc khách hàng (bài toán phân cụm) 192
23.1 Mục tiêu học tập 192
23.2 Đặt vấn đề và mô tả dữ liệu 192
23.3 Phân cụm và trực quan hóa 193
23.4 Diễn giải kết quả và khuyến nghị kinh doanh 194
23.5 Tóm tắt chương 196
23.6 Bài tập củng cố 196
23.7 Bài tập lập trình 197

Lời kết 198

Tài liệu tham khảo 199

Danh sách hình vẽ

1.1 Lập trình truyền thống (trên) và học máy (dưới). 3
1.2 Phân loại các dạng học máy. 5
1.3 Quy trình tổng quát của một dự án học máy. 5
1.4 Khớp thiếu, vừa phải và khớp quá khi tăng bậc đa thức (đường nét đứt là quy luật thật). 6
1.5 Sai số huấn luyện và kiểm tra theo độ phức tạp (trái); phân rã sai số thành độ chệch, phương sai và nhiễu (phải). 7
1.6 Bộ dữ liệu Iris theo hai cặp đặc trưng. 9

2.1 Môi trường ảo cô lập thư viện giữa các dự án. 13
2.2 Hệ sinh thái Python cho học máy. 14

3.1 Broadcasting khi cộng mảng (3×4) với mảng $(4,)$. 20
3.2 Tỉ lệ giá trị thiếu theo cột (bộ Titanic, 891 hành khách). 21
3.3 Histogram kèm KDE, boxplot theo lớp và biểu đồ phân tán (bộ dữ liệu Wine). 21
3.4 Bản đồ nhiệt tương quan giữa một số đặc trưng của bộ Wine. 22

4.1 Đường hồi quy (trái) và phần dư — các đoạn đỏ (phải). 29
4.2 Gradient Descent hội tụ về nghiệm tối ưu trên mặt MSE của hồi quy đơn biến. 30
4.3 Tốc độ học quá nhỏ, vừa phải và quá lớn. 30
4.4 Dự đoán so với thực tế, phần dư so với dự đoán và Q–Q plot của phần dư (bộ Diabetes). 31

5.1 Hồi quy đa thức với bậc tăng dần (chấm xanh: huấn luyện; ô vuông cam: kiểm tra). 37
5.2 Đường đồng mức của MSE và vùng ràng buộc L_2 (Ridge) so với L_1 (Lasso). . 38
5.3 Ridge co dần các hệ số; Lasso loại hẳn nhiều hệ số (bộ Diabetes, đã chuẩn hóa). 39

6.1 Hồi quy tuyến tính và hồi quy Logistic trên dữ liệu nhị phân. 45
6.2 Hàm sigmoid $\sigma(z) = 1/(1 + e^{-z})$. 46
6.3 Hàm mất mát Binary Cross-Entropy cho từng nhãn thực tế. 46
6.4 Ranh giới quyết định và xác suất dự đoán trên hai đặc trưng của bộ Breast Cancer. 47
6.5 Ma trận nhầm lẫn (trái) và đường cong ROC (phải). 50
6.6 Precision, Recall và F1 theo ngưỡng quyết định. 51

7.1 K-NN với $k = 5$: mẫu mới (ngôi sao) được gán theo đa số láng giềng. 55
7.2 Khoảng cách Euclidean và Manhattan giữa hai điểm. 56
7.3 Ranh giới quyết định của K-NN với $k = 1, 7, 40$ (bộ Iris). 57
7.4 Chuẩn hóa dữ liệu (trái) và ảnh hưởng của k (phải) trên bộ Wine. 60

8.1 Cây quyết định với độ sâu tối đa 3 trên bộ Iris. 64
8.2 Gini và Entropy theo tỉ lệ lớp tại nút (bài toán hai lớp). 65
8.3 Vùng quyết định của cây theo độ sâu tối đa (dữ liệu Moons). 66
8.4 Đường cong khớp quá của cây quyết định theo `max_depth`. 66
8.5 Tám đặc trưng quan trọng nhất (bộ Breast Cancer). 67

9.1 Sơ đồ Bagging / Random Forest. 73
9.2 Gradient Boosting hồi quy qua các vòng lặp (đường cam: dự đoán của tổ hợp). 75
9.3 Một cây quyết định, Random Forest và Gradient Boosting (dữ liệu Moons). . 76
9.4 Accuracy theo số cây và điểm Out-of-Bag. 79
9.5 So sánh bằng kiểm định chéo 10-fold (bộ Breast Cancer). 79

10.1 SVM tuyến tính: biên, siêu phẳng và vector hỗ trợ (khoanh đỏ). 83
10.2 SVM kernel RBF với các cặp (C, γ) khác nhau. 84
10.3 Dữ liệu hai vòng tròn đồng tâm trở nên phân tách bằng mặt phẳng khi thêm chiều $x_3 = x_1^2 + x_2^2$. 85
10.4 SVM với kernel tuyến tính, đa thức và RBF trên dữ liệu Moons. 88

11.1 Cập nhật từ xác suất tiên nghiệm sang hậu nghiệm (số liệu giả định để minh họa). 92
11.2 Phân phối chuẩn theo lớp (trái) và vùng quyết định (phải) trên bộ Iris. 93

12.1 Các bước của thuật toán K-Means (dấu X: centroid). 100
12.2 Phương pháp Elbow (trái) và chỉ số Silhouette (phải). 101
12.3 Biểu đồ Silhouette với $K = 3$ và $K = 4$ (đường đỏ: giá trị trung bình). 102

13.1 Dendrogram (liên kết Ward) trên 36 mẫu của bộ Iris. 108
13.2 Bốn tiêu chí liên kết trên dữ liệu dạng đám và dạng vầng trăng. 109
13.3 Điểm lõi, điểm biên và điểm nhiễu. 110
13.4 K-Means và DBSCAN trên dữ liệu Moons. 113

14.1 Các thành phần chính (trái) và phép chiếu lên PC1 (phải). 116
14.2 Scree plot, phương sai tích lũy và phép chiếu 2D của bộ Digits. 117
14.3 Tám thành phần chính đầu tiên và ảnh tái tạo từ k thành phần. 118

15.1 Kiểm định chéo 5-fold. 125
15.2 Đường cong học của mô hình khớp thiếu (trái) và khớp quá (phải), bộ Digits. 126
15.3 ROC của bốn mô hình (phải: phóng to vùng FPR thấp). 127
15.4 Chia dữ liệu thành tập huấn luyện, kiểm định và kiểm tra. 128
15.5 Đường cong kiểm định theo γ của SVM-RBF (bộ Digits). 130
15.6 So sánh bốn mô hình bằng kiểm định chéo 10-fold (bộ Breast Cancer). 131

16.1 Accuracy kiểm định chéo trên lưới siêu tham số của SVM-RBF. 134
16.2 Grid Search và Random Search với cùng ngân sách 9 tổ hợp. 135
16.3 40 tổ hợp ngẫu nhiên của (C, γ) (ngôi sao: tổ hợp tốt nhất). 138

17.1 Minh họa One-Hot Encoding. 142
17.2 Phương pháp lọc, bao và nhúng. 143
17.3 Accuracy theo số đặc trưng giữ lại (bộ Breast Cancer, hồi quy Logistic). . . . 143
17.4 Rò rỉ dữ liệu khi chuẩn hóa trước khi chia (trên) và cách làm đúng (dưới). . . 144
17.5 Pipeline xử lý song song cột số và cột phân loại rồi huấn luyện mô hình. . . . 145

18.1 Cấu tạo của một nơ-ron nhân tạo. 152
18.2 Mạng nơ-ron nhiều tầng kết nối đầy đủ. 152
18.3 Hồi quy Logistic và MLP một tầng ẩn trên dữ liệu XOR. 153
18.4 Sigmoid, Tanh, ReLU, Leaky ReLU và đạo hàm. 154
18.5 Lan truyền tiến (xanh) và lan truyền ngược (đỏ). 155
18.6 Mạng đầy đủ (trái) và mạng khi áp dụng Dropout (phải). 156
18.7 Đường cong mất mát (trái) và accuracy kiểm định (phải) của MLP trên bộ Digits. 158

19.1 Quy trình điển hình trong Keras (khai báo) và PyTorch (mệnh lệnh). 162
19.2 Mối quan hệ giữa batch, epoch và số lần cập nhật trọng số. 163
19.3 Dừng huấn luyện sớm khi mất mát kiểm định ngừng cải thiện. 163
19.4 Vòng lặp huấn luyện mạng nơ-ron. 164

20.1 Chia sẻ trọng số giúp giảm số tham số hàng nghìn lần. 171
20.2 Phép tích chập: bộ lọc 3×3 trượt trên ảnh 5×5. 172
20.3 Các bộ lọc phát hiện cạnh, làm mờ và làm sắc nét. 172
20.4 Max Pooling 2×2 với bước nhảy 2. 173
20.5 Kiến trúc CNN cho ảnh 28×28: kích thước không gian giảm dần, số kênh tăng dần. 173

21.1 Quy trình dự án dự đoán giá bất động sản. 180
21.2 Giá trị dự đoán so với thực tế của ba mô hình hồi quy (dữ liệu tổng hợp Friedman #1). 184

22.1 Pipeline lọc email/SMS rác. 188
22.2 Ma trận Bag-of-Words của ba văn bản mẫu. 189
22.3 Phân bố xác suất dự đoán và đường cong Precision–Recall. 190

23.1 Quy trình phân khúc khách hàng. 193
23.2 Elbow, các phân khúc theo thu nhập – chi tiêu và trực quan hóa bằng PCA. . . 195
23.3 Chân dung các phân khúc khách hàng (giá trị trung bình chuẩn hóa). 196

Phần I

Tổng quan và công cụ

Chương 1

Giới thiệu về Học máy

1.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Trình bày được định nghĩa học máy và phân biệt học máy với lập trình truyền thống.
2. Nêu được quá trình hình thành và phát triển của lĩnh vực học máy trong bối cảnh Trí tuệ Nhân tạo.
3. Phân loại được các dạng học máy: học có giám sát, học không giám sát, học bán giám sát và học tăng cường.
4. Mô tả được quy trình tổng quát của một dự án học máy, từ thu thập dữ liệu đến triển khai mô hình.
5. Giải thích được các khái niệm nền tảng: tập huấn luyện – tập kiểm tra, khớp thiếu – khớp quá (underfitting/overfitting), đánh đổi độ chệch – phương sai (bias–variance trade-off).
6. Cài đặt được một chương trình Python đơn giản sử dụng thư viện Scikit-learn để huấn luyện và đánh giá một mô hình phân loại cơ bản.

1.2. Nội dung lý thuyết

1.2.1. Học máy là gì?

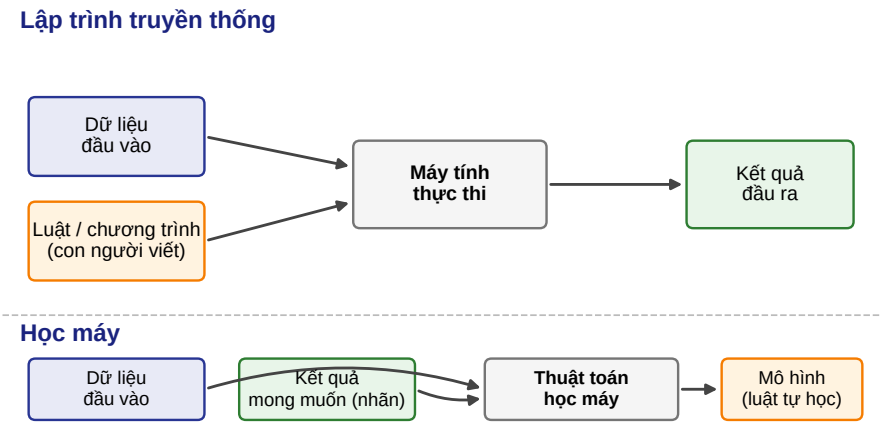
Học máy (Machine Learning – ML) là một nhánh của Trí tuệ Nhân tạo (Artificial Intelligence – AI), nghiên cứu các phương pháp cho phép máy tính "học" từ dữ liệu để thực hiện một nhiệm vụ cụ thể mà không cần được lập trình tường minh cho từng trường hợp. Thay vì con người viết ra tập luật cứng để xử lý mọi tình huống, hệ thống học máy tự động tìm ra các quy luật ẩn chứa trong dữ liệu quan sát được, rồi sử dụng các quy luật đó để đưa ra dự đoán hoặc quyết định trên dữ liệu mới.

Một định nghĩa kinh điển được Tom Mitchell đưa ra: một chương trình máy tính được coi là học từ kinh nghiệm E đối với một lớp nhiệm vụ T và độ đo hiệu năng P, nếu hiệu năng của nó trên T, được đo bằng P, cải thiện theo kinh nghiệm E. Nói một cách đơn giản: mô hình càng "thấy" nhiều dữ liệu (kinh nghiệm), nó càng thực hiện tốt hơn nhiệm vụ được giao, khi được đánh giá bằng một tiêu chí đo lường cụ thể.

Để hình dung sự khác biệt giữa lập trình truyền thống và học máy, ta có thể so sánh như sau: trong lập trình truyền thống, con người cung cấp dữ liệu đầu vào và các luật xử lý (chương

trình), máy tính tạo ra kết quả đầu ra. Trong học máy, con người cung cấp dữ liệu đầu vào cùng với kết quả đầu ra mong muốn (dữ liệu huấn luyện), và máy tính tự tìm ra "luật" (mô hình) ánh xạ từ đầu vào sang đầu ra.

Hình 1.1 minh họa sự khác biệt giữa lập trình truyền thống và học máy: ở học máy, "luật" không do con người viết mà được thuật toán tự rút ra từ dữ liệu và kết quả mong muốn.



Hình 1.1. Lập trình truyền thống (trên) và học máy (dưới).

1.2.2. Vì sao học máy trở nên quan trọng?

Ba yếu tố chính thúc đẩy sự bùng nổ của học máy trong hai thập kỷ gần đây:

- Dữ liệu: sự phát triển của Internet, mạng xã hội, thiết bị di động và cảm biến IoT tạo ra khối lượng dữ liệu khổng lồ (Big Data), là "nguyên liệu" thiết yếu để huấn luyện mô hình.
- Sức mạnh tính toán: sự phát triển của GPU, TPU và điện toán đám mây giúp huấn luyện các mô hình phức tạp trong thời gian hợp lý.
- Thuật toán: những cải tiến về thuật toán tối ưu hóa, kiến trúc mạng nơ-ron sâu, và các kỹ thuật chính quy hóa giúp mô hình học hiệu quả và tổng quát hóa tốt hơn.

Học máy hiện được ứng dụng rộng rãi trong nhiều lĩnh vực: hệ thống gợi ý sản phẩm trong thương mại điện tử, phát hiện gian lận trong giao dịch tài chính, chẩn đoán hình ảnh y tế, xe tự hành, dịch máy, nhận diện giọng nói, dự báo thời tiết, và rất nhiều lĩnh vực khác.

1.2.3. Phân loại các dạng học máy

Dựa trên bản chất của dữ liệu và mục tiêu học tập, học máy được chia thành bốn nhóm chính:

a) Học có giám sát (Supervised Learning)

Trong học có giám sát, dữ liệu huấn luyện gồm các cặp (đầu vào, nhãn) – tức mỗi mẫu dữ liệu đã có sẵn kết quả đúng (ground truth). Mô hình học cách ánh xạ từ đầu vào sang nhãn, sau đó được dùng để dự đoán nhãn cho dữ liệu mới chưa từng thấy. Học có giám sát chia thành hai bài toán con:

- Hồi quy (Regression): nhãn đầu ra là giá trị liên tục, ví dụ dự đoán giá nhà, dự đoán doanh thu.
- Phân loại (Classification): nhãn đầu ra là giá trị rời rạc thuộc một tập hữu hạn các lớp, ví dụ phân loại email spam/không spam, chẩn đoán bệnh có/không.

b) Học không giám sát (Unsupervised Learning)

Dữ liệu không có nhãn; mô hình phải tự khám phá cấu trúc, quy luật hoặc nhóm ẩn trong dữ liệu. Hai bài toán tiêu biểu là phân cụm (clustering) – nhóm các mẫu dữ liệu tương tự nhau lại với nhau, và giảm chiều dữ liệu (dimensionality reduction) – biểu diễn dữ liệu bằng số chiều nhỏ hơn trong khi vẫn giữ được thông tin quan trọng.

c) Học bán giám sát (Semi-supervised Learning)

Kết hợp một lượng nhỏ dữ liệu có nhãn với một lượng lớn dữ liệu không nhãn, thường gặp trong thực tế khi việc gán nhãn tốn nhiều chi phí (ví dụ cần chuyên gia y tế gán nhãn ảnh X-quang).

d) Học tăng cường (Reinforcement Learning)

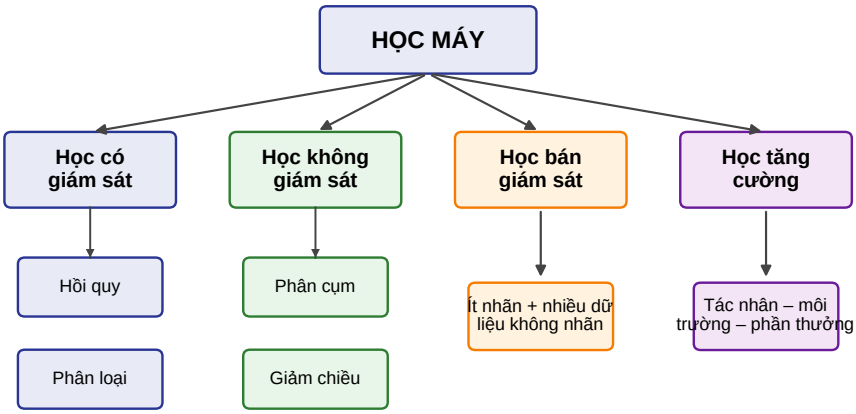
Một tác nhân (agent) tương tác với môi trường, thực hiện hành động và nhận phần thưởng (reward) hoặc phạt (penalty) tương ứng, từ đó học chiến lược hành động tối ưu qua thời gian. Đây là nền tảng của các hệ thống chơi cờ vây, robot điều khiển, và một số hệ thống gợi ý nâng cao. Giáo trình này chủ yếu tập trung vào học có giám sát và học không giám sát; học tăng cường sẽ chỉ được đề cập khái quát.

Hình 1.2 minh họa bốn dạng học máy chính và các bài toán điển hình của mỗi dạng.

1.2.4. Quy trình tổng quát của một dự án học máy

Một dự án học máy điển hình thường trải qua các bước sau:

1. Xác định bài toán: hiểu rõ mục tiêu nghiệp vụ, xác định đây là bài toán hồi quy, phân loại hay phân cụm.
2. Thu thập dữ liệu: lấy dữ liệu từ cơ sở dữ liệu, tệp tin, API, hoặc thu thập trực tiếp.
3. Khám phá và tiền xử lý dữ liệu: làm sạch dữ liệu thiếu/nhiễu, chuẩn hóa, mã hóa biến phân loại, trực quan hóa để hiểu phân bố dữ liệu.

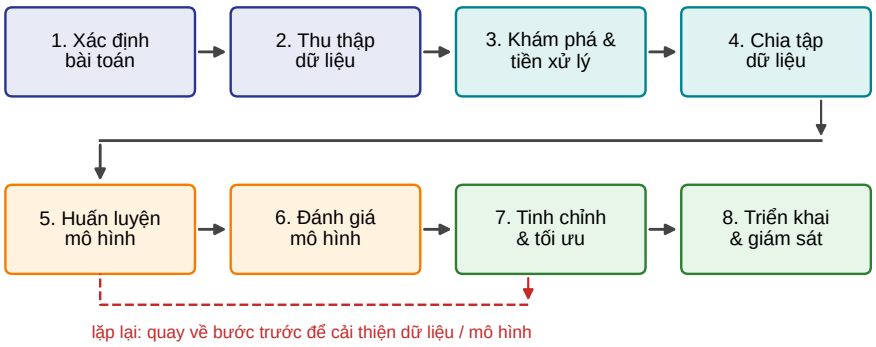


Hình 1.2. Phân loại các dạng học máy.

4. Chia tập dữ liệu: tách thành tập huấn luyện (training set), tập kiểm định (validation set) và tập kiểm tra (test set).
5. Lựa chọn và huấn luyện mô hình: chọn thuật toán phù hợp, huấn luyện trên tập huấn luyện.
6. Đánh giá mô hình: đo hiệu năng trên tập kiểm tra bằng các độ đo phù hợp (được trình bày chi tiết ở Chương 15).
7. Tinh chỉnh và tối ưu: điều chỉnh siêu tham số, thử nghiệm các mô hình khác nhau để cải thiện hiệu năng.
8. Triển khai và giám sát: đưa mô hình vào sử dụng thực tế, theo dõi hiệu năng theo thời gian.

Quy trình này không hoàn toàn tuyến tính mà thường có tính lặp: kết quả đánh giá ở bước 6 có thể dẫn đến việc quay lại bước 3 để cải thiện dữ liệu, hoặc bước 5 để thử thuật toán khác.

Hình 1.3 minh họa tám bước của một dự án học máy; mũi tên đứt nét nhắc rằng quy trình mang tính lặp.



Hình 1.3. Quy trình tổng quát của một dự án học máy.

1.2.5. Các khái niệm nền tảng

a) Tập huấn luyện và tập kiểm tra

Để đánh giá khách quan khả năng tổng quát hóa của mô hình (tức khả năng hoạt động tốt trên dữ liệu chưa từng thấy), dữ liệu thường được chia thành tập huấn luyện (dùng để mô hình học) và tập kiểm tra (dùng để đánh giá, không được sử dụng trong quá trình huấn luyện). Tỉ lệ chia phổ biến là 70/30 hoặc 80/20, tùy vào kích thước bộ dữ liệu.

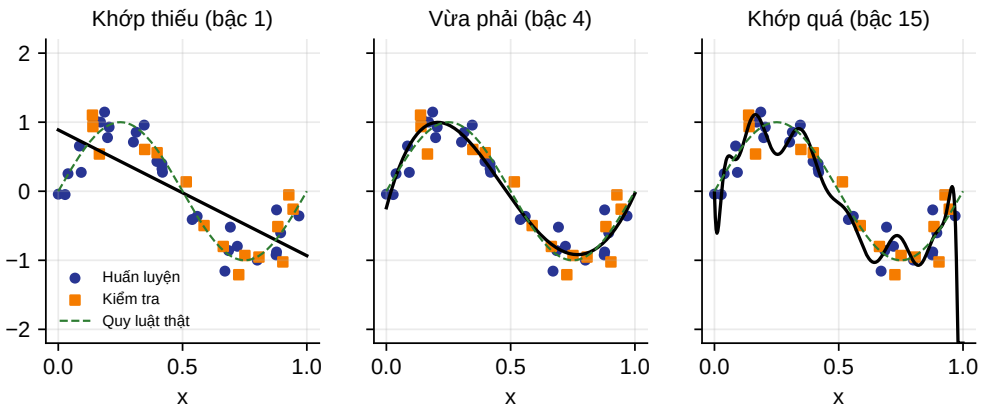
b) Khớp thiếu và khớp quá (Underfitting và Overfitting)

Khớp thiếu (underfitting) xảy ra khi mô hình quá đơn giản, không đủ khả năng nắm bắt quy luật trong dữ liệu, dẫn đến hiệu năng kém trên cả tập huấn luyện lẫn tập kiểm tra. Khớp quá (overfitting) xảy ra khi mô hình quá phức tạp, "học thuộc lòng" cả nhiễu trong tập huấn luyện, dẫn đến hiệu năng rất tốt trên tập huấn luyện nhưng kém trên tập kiểm tra. Mục tiêu của người xây dựng mô hình là tìm điểm cân bằng giữa hai thái cực này.

c) Đánh đổi độ chệch – phương sai (Bias–Variance Trade-off)

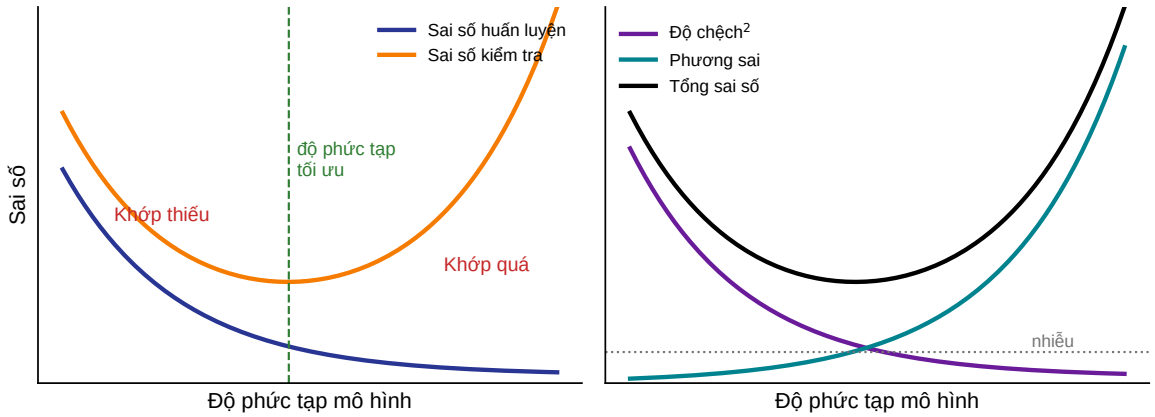
Sai số dự đoán của một mô hình có thể được phân tách thành ba thành phần: độ chệch (bias) – sai số do mô hình quá đơn giản so với bản chất dữ liệu; phương sai (variance) – sai số do mô hình quá nhạy cảm với biến động nhỏ của tập huấn luyện; và nhiễu không thể giảm (irreducible error). Mô hình có độ chệch cao thường gắn với khớp thiếu, trong khi mô hình có phương sai cao thường gắn với khớp quá. Việc lựa chọn độ phức tạp mô hình phù hợp chính là việc cân bằng giữa hai thành phần này.

Hình 1.4 minh họa ba trạng thái của mô hình khi tăng dần độ phức tạp, minh họa cho hiện tượng khớp thiếu và khớp quá.



Hình 1.4. Khớp thiếu, vừa phải và khớp quá khi tăng bậc đa thức (đường nét đứt là quy luật thật).

Hình 1.5 minh họa đánh đổi độ chệch – phương sai: sai số kiểm tra đạt cực tiểu ở mức độ phức tạp trung gian.



Hình 1.5. Sai số huấn luyện và kiểm tra theo độ phức tạp (trái); phân rã sai số thành độ chệch, phương sai và nhiễu (phải).

1.3. Ví dụ minh họa bằng mã Python

Trong ví dụ đầu tiên này, chúng ta sẽ sử dụng bộ dữ liệu Iris (một bộ dữ liệu kinh điển gồm các phép đo hoa của ba loài hoa Diên vĩ) để minh họa toàn bộ quy trình học máy cơ bản: nạp dữ liệu, chia tập huấn luyện/kiểm tra, huấn luyện một mô hình phân loại đơn giản, và đánh giá kết quả.

```
# vidu_1_1.py
# Ví dụ minh họa quy trình học máy cơ bản với bộ dữ liệu Iris
# Yêu cầu: scikit-learn >= 1.5, pandas >= 2.2

# 1. Nạp các thư viện cần thiết
from sklearn.datasets import load_iris          # Bộ dữ liệu mẫu Iris
from sklearn.model_selection import train_test_split  # Hàm chia tập dữ liệu
from sklearn.neighbors import KNeighborsClassifier     # Thuật toán K-NN
from sklearn.metrics import accuracy_score, classification_report
import pandas as pd

# 2. Nạp dữ liệu Iris và chuyển thành DataFrame để dễ quan sát
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)  # Đặc trưng đầu vào
y = pd.Series(iris.target, name="loai_hoa")               # Nhãn (0, 1, 2)

print("Kích thước dữ liệu đặc trưng:", X.shape)
print("5 dòng dữ liệu đầu tiên:")
print(X.head())

# 3. Chia dữ liệu thành tập huấn luyện (80%) và tập kiểm tra (20%)
#    random_state cố định để đảm bảo kết quả có thể tái lập
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print(f"\nSố mẫu huấn luyện: {len(X_train)}, số mẫu kiểm tra: {len(X_test)}")
```

```
# 4. Khởi tạo và huấn luyện mô hình K-Láng giềng gần nhất (K-NN)
#    với k = 5 láng giềng gần nhất
mo_hinh = KNeighborsClassifier(n_neighbors=5)
mo_hinh.fit(X_train, y_train)   # Quá trình "học" diễn ra tại đây

# 5. Dự đoán nhãn cho tập kiểm tra
y_du_doan = mo_hinh.predict(X_test)

# 6. Đánh giá độ chính xác của mô hình
do_chinh_xac = accuracy_score(y_test, y_du_doan)
print(f"\nĐộ chính xác trên tập kiểm tra: {do_chinh_xac:.4f}")
print("\nBáo cáo phân loại chi tiết:")
print(classification_report(y_test, y_du_doan, target_names=iris.target_names))
```

1.4. Giải thích kết quả chạy chương trình

Khi thực thi đoạn mã trên, chương trình sẽ in ra kết quả tương tự như sau (số liệu cụ thể có thể thay đổi nhẹ tùy phiên bản thư viện):

```
Kích thước dữ liệu đặc trưng: (150, 4)
5 dòng dữ liệu đầu tiên:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
...

Số mẫu huấn luyện: 120, số mẫu kiểm tra: 30

Độ chính xác trên tập kiểm tra: 1.0000

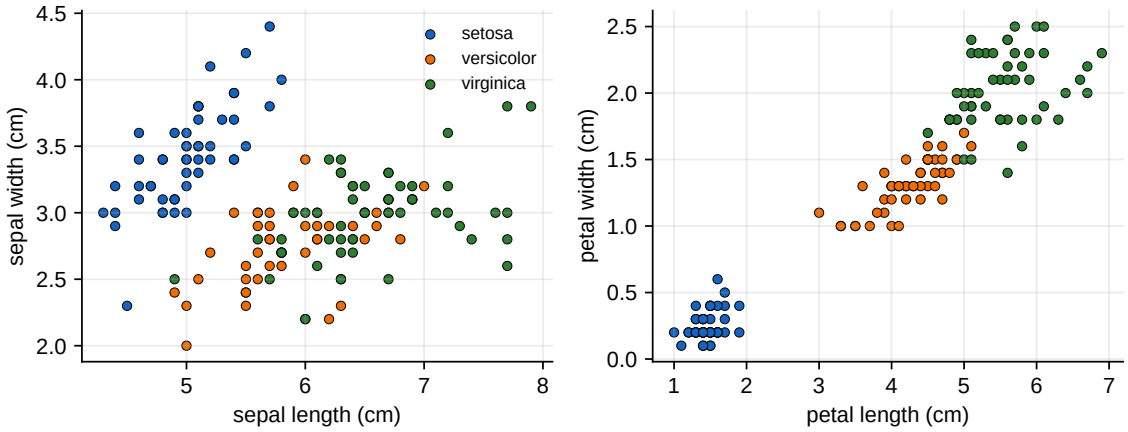
Báo cáo phân loại chi tiết:
              precision    recall  f1-score   support
      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00        10
   virginica       1.00      1.00      1.00        10
```

Kết quả cho thấy: bộ dữ liệu Iris gồm 150 mẫu, mỗi mẫu có 4 đặc trưng số (chiều dài/chiều rộng đài hoa và cánh hoa). Sau khi chia theo tỉ lệ 80/20, mô hình K-NN với k = 5 đạt độ chính xác 100% trên tập kiểm tra – đây là kết quả điển hình cho bộ dữ liệu Iris vì các lớp trong bộ dữ liệu này khá tách biệt rõ ràng trong không gian đặc trưng. Tham số stratify=y trong hàm chia dữ liệu đảm bảo tỉ lệ các lớp trong tập huấn luyện và tập kiểm tra tương đương với tỉ lệ trong toàn bộ dữ liệu gốc, tránh tình trạng một lớp bị đại diện quá ít hoặc quá nhiều ở tập kiểm tra.

Lưu ý sư phạm

Độ chính xác tuyệt đối 100% là trường hợp đặc biệt của bộ dữ liệu Iris (khá "dễ" và sạch). Trong thực tế, đặc biệt với dữ liệu thực từ doanh nghiệp, độ chính xác 100% gần như luôn là dấu hiệu cảnh báo về rò rỉ dữ liệu (data leakage) hoặc lỗi trong quy trình đánh giá, chứ không phải một kết quả đáng mừng. Sinh viên cần giữ thái độ hoài nghi khoa học khi gặp kết quả "quá hoàn hảo".

Hình 1.6 minh họa bộ dữ liệu Iris: ba loài hoa phân tách khá rõ trong không gian đặc trưng, đặc biệt là theo kích thước cánh hoa.



Hình 1.6. Bộ dữ liệu Iris theo hai cặp đặc trưng.

1.5. Tóm tắt chương

- Học máy là một nhánh của Trí tuệ Nhân tạo, cho phép máy tính tự học quy luật từ dữ liệu thay vì được lập trình tường minh.
- Bốn dạng học máy chính: học có giám sát, học không giám sát, học bán giám sát và học tăng cường; giáo trình này tập trung chủ yếu vào hai dạng đầu.
- Học có giám sát chia thành hai bài toán: hồi quy (dự đoán giá trị liên tục) và phân loại (dự đoán nhãn rời rạc).
- Một dự án học máy tuân theo quy trình gồm nhiều bước lặp: từ xác định bài toán, thu thập – tiền xử lý dữ liệu, huấn luyện, đánh giá, đến tối ưu và triển khai.
- Hai khái niệm nền tảng cần nắm vững: khớp thiếu/khớp quá và đánh đổi độ chệch – phương sai, làm cơ sở lý luận cho hầu hết các kỹ thuật cải thiện mô hình trong các chương sau.
- Scikit-learn cung cấp giao diện lập trình thống nhất (fit – predict – score) giúp việc huấn luyện và đánh giá mô hình trở nên đơn giản và nhất quán.

1.6. Bài tập củng cố

Mức nhận biết

1. Nêu định nghĩa học máy theo Tom Mitchell và giải thích ý nghĩa của ba thành phần E, T, P trong định nghĩa đó.
2. Phân biệt học có giám sát và học không giám sát. Cho mỗi loại một ví dụ ứng dụng thực tế khác với ví dụ đã nêu trong chương.
3. Hồi quy và phân loại khác nhau ở điểm nào? Cho ví dụ minh họa cho mỗi bài toán.

Mức thông hiểu

1. Giải thích vì sao cần chia dữ liệu thành tập huấn luyện và tập kiểm tra thay vì đánh giá mô hình trực tiếp trên toàn bộ dữ liệu đã dùng để huấn luyện.
2. Một mô hình đạt độ chính xác 99% trên tập huấn luyện nhưng chỉ 60% trên tập kiểm tra. Hiện tượng này tên là gì? Hãy đề xuất hai hướng khắc phục.
3. Trình bày mối quan hệ giữa độ phức tạp của mô hình với độ chệch và phương sai. Vẽ sơ đồ minh họa (bằng lời hoặc hình vẽ) đường cong sai số huấn luyện và sai số kiểm tra theo độ phức tạp mô hình.

Mức vận dụng

1. Cho tình huống: một ngân hàng muốn xây dựng hệ thống tự động phát hiện giao dịch gian lận dựa trên lịch sử giao dịch của khách hàng (không có nhãn "gian lận"/"không gian lận" được gán sẵn, nhưng có một số ít giao dịch đã được xác nhận là gian lận qua điều tra). Hãy đề xuất dạng học máy phù hợp nhất (có giám sát, không giám sát hay bán giám sát) và giải thích lý do lựa chọn.
2. Một công ty thương mại điện tử muốn dự đoán "doanh thu tháng tới" của từng cửa hàng dựa trên dữ liệu lịch sử. Đây là bài toán hồi quy hay phân loại? Hãy liệt kê ít nhất 4 đặc trưng đầu vào hợp lý cho bài toán này.

1.7. Bài tập lập trình

Bài 1. Sử dụng bộ dữ liệu `load_wine()` có sẵn trong scikit-learn (thay cho bộ Iris), thực hiện lại quy trình tương tự ví dụ 1.1: nạp dữ liệu, chia tập huấn luyện/kiểm tra theo tỉ lệ 75/25, huấn luyện mô hình KNeighborsClassifier với k = 3, và in ra độ chính xác cùng báo cáo phân loại chi tiết.

Bài 2. Viết chương trình thử nghiệm mô hình K-NN ở Bài 1 với các giá trị k khác nhau (k = 1, 3, 5, 7, 9, 11). Vẽ biểu đồ đường (dùng matplotlib) thể hiện mối quan hệ giữa giá trị k (trục hoành) và độ chính xác trên tập kiểm tra (trục tung). Nhận xét về xu hướng quan sát được.

Bài 3 (nâng cao). Viết một hàm Python nhận vào một DataFrame và tên cột nhãn, tự động: (a) in ra số lượng mẫu của mỗi lớp; (b) chia dữ liệu theo tỉ lệ do người dùng chỉ định (mặc định 80/20) có sử dụng stratify; (c) trả về bốn biến `X_train`, `X_test`, `y_train`, `y_test`. Áp dụng hàm này cho bộ dữ liệu `load_breast_cancer()` và huấn luyện thử một mô hình K-NN bất kỳ để kiểm tra tính đúng đắn của hàm.

Chương tiếp theo (Chương 2) sẽ giới thiệu chi tiết về cách thiết lập môi trường lập trình Python cho học máy, bao gồm quản lý gói thư viện, Jupyter Notebook, và làm quen với ba thư viện nền tảng NumPy, Pandas, Matplotlib trước khi bước vào các thuật toán cụ thể ở Phần 2.

Chương 2

Công cụ và Môi trường lập trình Python cho Học máy

2.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Cài đặt và cấu hình một môi trường Python phù hợp cho các dự án học máy.
2. Sử dụng thành thạo trình quản lý gói (pip) và môi trường ảo (virtual environment) để cô lập các dự án.
3. Phân biệt và lựa chọn công cụ làm việc phù hợp: Jupyter Notebook/JupyterLab, Google Colab, và IDE dạng script (VS Code).
4. Nêu được vai trò và phiên bản khuyến nghị của các thư viện nền tảng: NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn, TensorFlow, PyTorch.
5. Tổ chức một dự án học máy theo cấu trúc thư mục chuẩn, có khả năng tái lập (reproducibility).

2.2. Nội dung lý thuyết

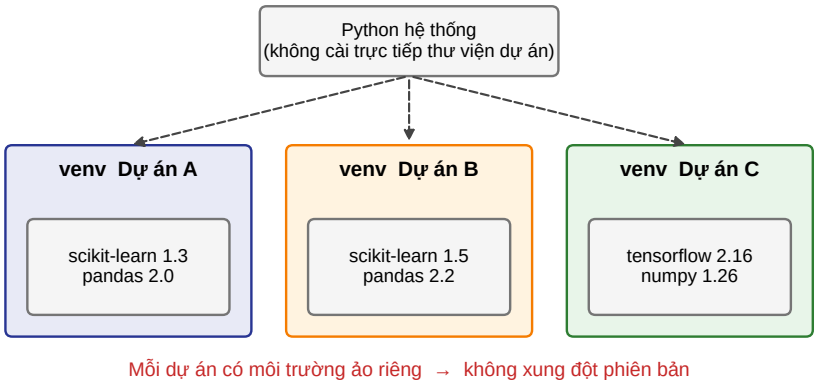
2.2.1. Vì sao cần một môi trường lập trình riêng biệt?

Một dự án học máy thường phụ thuộc vào nhiều thư viện bên thứ ba, mỗi thư viện lại có nhiều phiên bản với các thay đổi API (Application Programming Interface) khác nhau theo thời gian. Nếu cài đặt tất cả các gói trực tiếp vào Python hệ thống (system Python), người dùng dễ gặp xung đột phiên bản: dự án A cần scikit-learn 1.3, trong khi dự án B cần scikit-learn 1.5 với API đã thay đổi. Môi trường ảo (virtual environment) giải quyết vấn đề này bằng cách tạo ra một bản sao lập trình Python độc lập cho từng dự án, trong đó các gói được cài đặt riêng biệt, không ảnh hưởng lẫn nhau.

Hình 2.1 minh họa mỗi dự án có môi trường ảo riêng nên có thể dùng các phiên bản thư viện khác nhau mà không xung đột.

2.2.2. Các công cụ quản lý môi trường phổ biến

- venv: công cụ tạo môi trường ảo có sẵn trong Python (từ phiên bản 3.3 trở lên), nhẹ và không cần cài thêm phần mềm ngoài.



Hình 2.1. Môi trường ảo cô lập thư viện giữa các dự án.

- conda (Anaconda/Miniconda): trình quản lý gói và môi trường mạnh mẽ hơn, đặc biệt hữu ích khi cần cài đặt các gói có phụ thuộc hệ thống phức tạp (ví dụ các thư viện tính toán khoa học biên dịch từ C/C++/Fortran, hoặc CUDA cho GPU).
- uv/poetry: các công cụ quản lý gói và dự án Python thế hệ mới, tốc độ cài đặt nhanh, ngày càng phổ biến trong công nghiệp; sinh viên có thể tìm hiểu thêm sau khi đã quen với venv/conda.

Giáo trình này sử dụng venv kết hợp với pip làm công cụ chính do tính đơn giản và có sẵn trong mọi bản cài đặt Python tiêu chuẩn, phù hợp với môi trường học tập.

2.2.3. Công cụ soạn thảo và thực thi mã nguồn

- Jupyter Notebook / JupyterLab: môi trường tương tác dạng "sổ tay" (notebook), cho phép chạy từng khối lệnh (cell) riêng biệt, xem ngay kết quả và biểu đồ bên dưới mỗi khối lệnh. Rất phù hợp cho giai đoạn khám phá dữ liệu (EDA) và thử nghiệm mô hình.
- Google Colab: phiên bản Jupyter Notebook chạy trên đám mây của Google, miễn phí, cung cấp GPU/TPU giới hạn, không cần cài đặt gì trên máy cá nhân — phù hợp cho sinh viên chưa có máy cấu hình mạnh.
- Visual Studio Code (VS Code): trình soạn thảo mã nguồn đa năng, hỗ trợ tốt cả tệp .py truyền thống lẫn notebook .ipynb thông qua phần mở rộng (extension) Python và Jupyter, phù hợp khi dự án phát triển lớn hơn, cần quản lý nhiều tệp và dùng công cụ kiểm soát phiên bản (Git).

Trong giáo trình này, các ví dụ minh họa được trình bày dưới dạng tệp .py hoàn chỉnh (có thể chạy trực tiếp bằng dòng lệnh python `ten_file.py`) để đảm bảo tính nhất quán khi in ấn; tuy nhiên sinh viên được khuyến khích chạy thử trên Jupyter Notebook để quan sát kết quả và biểu đồ một cách tương tác.

2.2.4. Các thư viện nền tảng và phiên bản khuyến nghị

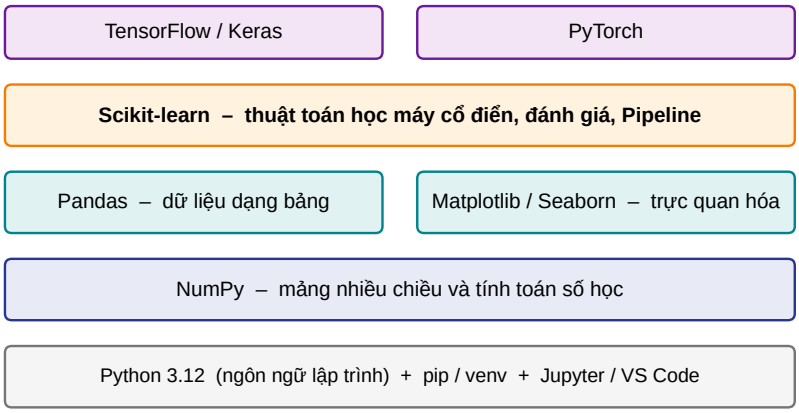
Bảng dưới đây tóm tắt các thư viện chính được sử dụng xuyên suốt giáo trình cùng phiên bản tối thiểu khuyến nghị tại thời điểm biên soạn:

Thư viện	Phiên bản tối thiểu	Vai trò chính
NumPy	`>= 1.26`	Tính toán số học trên mảng đa chiều, nền tảng cho hầu hết các thư viện khác
Pandas	`>= 2.2`	Đọc, xử lý, biến đổi dữ liệu dạng bảng (DataFrame)
Matplotlib	`>= 3.8`	Vẽ biểu đồ cơ bản, tùy biến chi tiết
Seaborn	`>= 0.13`	Vẽ biểu đồ thống kê nâng cao, cú pháp ngắn gọn dựa trên Matplotlib
Scikit-learn	`>= 1.5`	Cài đặt sẵn hầu hết các thuật toán học máy cổ điển
TensorFlow / Keras	`>= 2.16`	Xây dựng và huấn luyện mạng nơ-ron, học sâu
PyTorch	`>= 2.3`	Xây dựng và huấn luyện mạng nơ-ron, học sâu (thay thế/bổ sung cho TensorFlow)

Lưu ý

Do các thư viện học máy được cập nhật thường xuyên, sinh viên nên kiểm tra phiên bản đang cài đặt bằng lệnh pip show <tên_gói> hoặc thông qua đoạn mã kiểm tra phiên bản ở mục 2.3, thay vì tin tưởng tuyệt đối vào số phiên bản cố định.

Hình 2.2 minh họa hệ sinh thái các thư viện được sử dụng xuyên suốt giáo trình, xếp theo tầng.



Hình 2.2. Hệ sinh thái Python cho học máy.

2.2.5. Tổ chức thư mục dự án học máy

Một dự án học máy có khả năng tái lập tốt thường được tổ chức theo cấu trúc thư mục rõ ràng, tách biệt dữ liệu, mã nguồn, mô hình đã huấn luyện và kết quả:

```
du_an_hoc_may/
|-- data/
|   |-- raw/            # Dữ liệu gốc, không chỉnh sửa
|   `-- processed/      # Dữ liệu đã qua tiền xử lý
|-- notebooks/          # Các notebook thử nghiệm, khám phá dữ liệu
|-- src/                # Mã nguồn dạng module (.py) tái sử dụng được
|   |-- data_loader.py
|   |-- preprocessing.py
|   `-- model.py
|-- models/              # Mô hình đã huấn luyện được lưu lại
|-- results/             # Biểu đồ, báo cáo, kết quả đánh giá
|-- requirements.txt      # Danh sách thư viện và phiên bản
`-- README.md              # Mô tả dự án
```

2.3. Ví dụ minh họa bằng mã Python

Ví dụ dưới đây minh họa cách kiểm tra môi trường làm việc: xác nhận phiên bản Python và các thư viện chính đã được cài đặt đúng yêu cầu trước khi bắt đầu một dự án học máy.

```
# vidu_2_1.py
# Kiểm tra môi trường làm việc: phiên bản Python và các thư viện chính

import sys
import importlib
import importlib.metadata as metadata

# Danh sách các thư viện cần kiểm tra và phiên bản tối thiểu yêu cầu
thu_vien_yeu_cau = {
    "numpy": "1.26",
    "pandas": "2.2",
    "matplotlib": "3.8",
    "seaborn": "0.13",
    "sklearn": "1.5",   # tên gói cài đặt là "scikit-learn" nhưng import là
    "sklearn"
}

def so_sanh_phien_ban(hien_tai, yeu_cau):
    """So sánh hai chuỗi phiên bản dạng 'x.y.z', trả về True nếu hien_tai >=
    yeu_cau."""
    ht = [int(x) for x in hien_tai.split(".")[:2]]
    yc = [int(x) for x in yeu_cau.split(".")[:2]]
    return ht >= yc
```

```
print(f"Phiên bản Python đang sử dụng: {sys.version.split()[0]}")
print("-" * 50)

for ten_module, phien_ban_toi_thieu in thu_vien_yeu_cau.items():
    try:
        module = importlib.import_module(ten_module)
        phien_ban_hien_tai = module.__version__
        dat_yeu_cau = so_sanh_phien_ban(phien_ban_hien_tai, phien_ban_toi_thieu)
        trang_thai = "OK" if dat_yeu_cau else "CẦN NÂNG CẤP"
        print(f"{ten_module:12s}: {phien_ban_hien_tai:10s} (yêu cầu >=
        {phien_ban_toi_thieu})  [{trang_thai}]")
    except ImportError:
        print(f"{ten_module:12s}: CHƯA ĐƯỢC CÀI ĐẶT")
```

Để tạo môi trường ảo và cài đặt các thư viện cần thiết, sinh viên thực hiện các lệnh sau trong cửa sổ dòng lệnh (Terminal/Command Prompt):

```
# Bước 1: Tạo môi trường ảo có tên "hocmay_env"
python -m venv hocmay_env

# Bước 2: Kích hoạt môi trường ảo
# Trên Windows:
hocmay_env\Scripts\activate
# Trên macOS/Linux:
source hocmay_env/bin/activate

# Bước 3: Cài đặt các thư viện cần thiết
pip install numpy pandas matplotlib seaborn scikit-learn

# Bước 4: Lưu lại danh sách thư viện đã cài (để tái lập môi trường sau này)
pip freeze > requirements.txt

# Bước 5 (ở máy khác, hoặc sau này): tái lập lại đúng môi trường từ tệp đã lưu
pip install -r requirements.txt
```

2.4. Giải thích kết quả chạy chương trình

Khi thực thi tệp `vidu_2_1.py` trên một máy đã cài đặt đầy đủ và đúng phiên bản các thư viện, kết quả in ra có dạng:

```
Phiên bản Python đang sử dụng: 3.12.4
--------------------------------------------------
numpy       : 1.26.4     (yêu cầu >= 1.26)  [OK]
pandas      : 2.2.2      (yêu cầu >= 2.2)  [OK]
matplotlib  : 3.9.0      (yêu cầu >= 3.8)  [OK]
seaborn     : 0.13.2     (yêu cầu >= 0.13)  [OK]
sklearn     : 1.5.0      (yêu cầu >= 1.5)  [OK]
```

Nếu một thư viện chưa được cài đặt, chương trình sẽ in dòng "CHƯA ĐƯỢC CÀI ĐẶT" thay vì gây lỗi dừng chương trình (crash), nhờ khối try/except bắt lỗi ImportError. Cách viết này là một thói quen tốt trong lập trình: kiểm tra điều kiện tiên quyết một cách chủ động và đưa ra thông báo rõ ràng, thay vì để chương trình dừng đột ngột với thông báo lỗi khó hiểu ở một vị trí ngẫu nhiên về sau.

2.5. Tóm tắt chương

- Môi trường ảo (venv hoặc conda) giúp cô lập các gói thư viện giữa các dự án khác nhau, tránh xung đột phiên bản.
- Jupyter Notebook/JupyterLab và Google Colab phù hợp cho giai đoạn khám phá, thử nghiệm; VS Code phù hợp khi dự án phát triển thành quy mô lớn hơn.
- Năm thư viện nền tảng cần nắm vững: NumPy (tính toán mảng), Pandas (xử lý dữ liệu bảng), Matplotlib/Seaborn (trực quan hóa), Scikit-learn (thuật toán học máy cổ điển).
- Tệp requirements.txt ghi lại chính xác các thư viện và phiên bản đã dùng, là điều kiện tiên quyết để đảm bảo tính tái lập (reproducibility) của một dự án học máy.
- Một cấu trúc thư mục dự án rõ ràng (tách data/, src/, notebooks/, models/, results/) giúp dự án dễ bảo trì và mở rộng theo thời gian.

2.6. Bài tập củng cố

Mức nhận biết

1. Nêu sự khác biệt giữa venv và conda. Trong trường hợp nào nên ưu tiên sử dụng conda?
2. Lệnh pip freeze > requirements.txt dùng để làm gì? Vì sao lệnh này quan trọng đối với tính tái lập của dự án?

Mức thông hiểu

1. Giải thích vì sao không nên cài đặt tất cả thư viện của mọi dự án trực tiếp vào Python hệ thống (system Python). Minh họa bằng một tình huống xung đột phiên bản cụ thể.
2. So sánh ưu, nhược điểm của Jupyter Notebook và tệp .py truyền thống khi phát triển một dự án học máy từ giai đoạn thử nghiệm đến giai đoạn triển khai sản phẩm.

Mức vận dụng

1. Giả sử máy tính của bạn chưa cài Python. Hãy trình bày từng bước (dạng danh sách lệnh) để: cài đặt Python, tạo môi trường ảo tên "`btl_ml`", kích hoạt môi trường, cài các thư viện numpy, pandas, scikit-learn, và lưu lại tệp requirements.txt.

2.7. Bài tập lập trình

Bài 1. Mở rộng chương trình ở ví dụ 2.1 để kiểm tra thêm hai thư viện tensorflow và torch (phiên bản tối thiểu tự chọn). Chương trình cần in ra thông báo rõ ràng nếu một trong hai thư viện chưa được cài đặt, mà không làm dừng việc kiểm tra các thư viện còn lại.

Bài 2. Viết một chương trình Python nhỏ tự động tạo cấu trúc thư mục dự án như mô tả ở mục 2.2.5 (sử dụng thư viện os hoặc pathlib), bao gồm việc tạo các thư mục con và một tệp README.md rỗng có sẵn tiêu đề dự án do người dùng nhập vào.

Chương tiếp theo (Chương 3) sẽ đi vào chi tiết ba thư viện nền tảng nhất của học máy bằng Python: NumPy cho tính toán mảng, Pandas cho xử lý dữ liệu bảng, và Matplotlib/Seaborn cho trực quan hóa — thông qua một ví dụ khám phá dữ liệu (EDA) hoàn chỉnh.

Chương 3

Xử lý và Khám phá dữ liệu với NumPy, Pandas, Matplotlib/Seaborn

3.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Thực hiện các phép toán cơ bản trên mảng NumPy (ndarray), hiểu và vận dụng được cơ chế lan truyền (broadcasting).
2. Đọc dữ liệu từ tệp CSV vào cấu trúc DataFrame của Pandas và thực hiện các thao tác chọn lọc, lọc, biến đổi dữ liệu cơ bản.
3. Phát hiện và xử lý dữ liệu thiếu (missing values) bằng các chiến lược phù hợp.
4. Vẽ được các loại biểu đồ cơ bản (histogram, scatter plot, boxplot, heatmap tương quan) bằng Matplotlib và Seaborn để phục vụ phân tích khám phá dữ liệu (EDA).
5. Thực hiện một quy trình EDA hoàn chỉnh trên một bộ dữ liệu thực tế trước khi đưa vào huấn luyện mô hình.

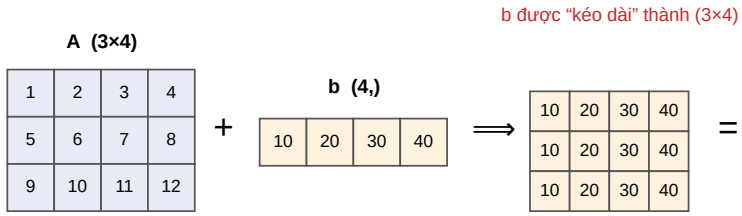
3.2. Nội dung lý thuyết

3.2.1. NumPy — nền tảng tính toán số học

NumPy (Numerical Python) cung cấp cấu trúc dữ liệu ndarray (mảng n-chiều), cho phép thực hiện các phép toán số học trên toàn bộ mảng cùng lúc (vectorization) thay vì phải viết vòng lặp for như với danh sách (list) thuần Python. Việc vector hóa không chỉ giúp mã nguồn ngắn gọn hơn mà còn nhanh hơn đáng kể về mặt tốc độ thực thi, do NumPy thực hiện các phép toán bằng mã C được biên dịch sẵn ở tầng dưới.

Một khái niệm quan trọng của NumPy là broadcasting (lan truyền): khi thực hiện phép toán giữa hai mảng có kích thước khác nhau, NumPy tự động "mở rộng" mảng nhỏ hơn theo một số quy tắc nhất định để hai mảng có cùng kích thước trước khi thực hiện phép toán theo từng phần tử. Ví dụ, cộng một mảng kích thước (3, 4) với một mảng kích thước (4,) sẽ tự động cộng mảng thứ hai vào từng hàng của mảng thứ nhất.

Hình 3.1 minh họa cơ chế broadcasting: mảng nhỏ hơn được tự động "kéo dài" để khớp kích thước mảng lớn.



Hình 3.1. Broadcasting khi cộng mảng (3×4) với mảng $(4,)$.

3.2.2. Pandas — xử lý dữ liệu dạng bảng

Pandas xây dựng trên nền NumPy, cung cấp hai cấu trúc dữ liệu chính: Series (mảng một chiều có nhãn/chỉ mục — index) và DataFrame (bảng hai chiều, tương tự một bảng tính Excel hoặc một bảng trong cơ sở dữ liệu quan hệ, gồm nhiều cột có thể khác kiểu dữ liệu). DataFrame là cấu trúc trung tâm trong hầu hết các bước tiền xử lý dữ liệu trước khi đưa vào mô hình học máy.

Các thao tác Pandas thường dùng trong tiền xử lý gồm: đọc dữ liệu (`pd.read_csv`), xem thông tin tổng quan (`.info()`, `.describe()`), chọn lọc theo điều kiện (lọc boolean), nhóm và tổng hợp dữ liệu (`.groupby()`), kết hợp nhiều bảng (`.merge()`, `.concat()`), và xử lý dữ liệu thiếu (`.isna()`, `.fillna()`, `.dropna()`).

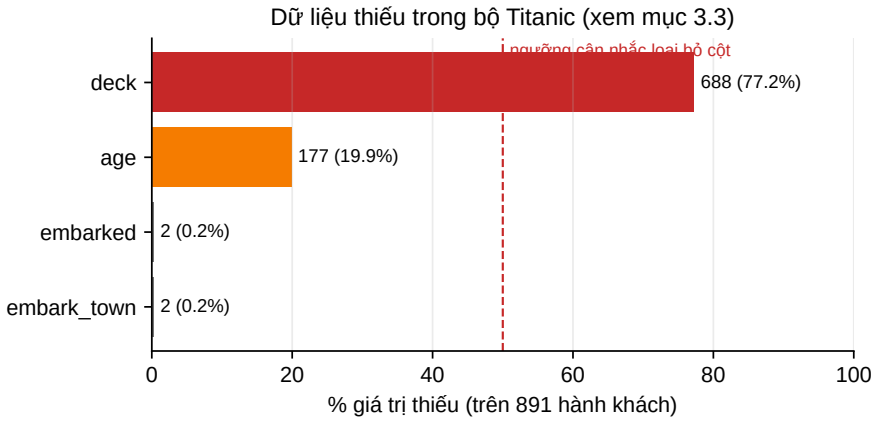
3.2.3. Xử lý dữ liệu thiếu (Missing Values)

Dữ liệu thực tế hầu như luôn chứa các giá trị thiếu (do lỗi thu thập, lỗi cảm biến, người dùng bỏ trống trường thông tin, v.v.). Ba chiến lược xử lý phổ biến gồm:

- Loại bỏ (deletion): xóa các dòng hoặc cột chứa giá trị thiếu — chỉ nên áp dụng khi tỉ lệ thiếu rất nhỏ và việc loại bỏ không làm mất nhiều thông tin.
- Điền giá trị (imputation) bằng thống kê đơn giản: dùng giá trị trung bình (mean), trung vị (median) cho biến số, hoặc giá trị xuất hiện nhiều nhất (mode) cho biến phân loại.
- Điền giá trị bằng mô hình: dự đoán giá trị thiếu dựa trên các đặc trưng khác bằng một mô hình học máy đơn giản (ví dụ hồi quy tuyến tính hoặc K-NN) — phương pháp phức tạp hơn nhưng thường chính xác hơn.

Việc lựa chọn chiến lược phù hợp phụ thuộc vào bản chất dữ liệu, tỉ lệ giá trị thiếu, và liệu giá trị thiếu có mang thông tin hay không (ví dụ: một trường “thu nhập” bị bỏ trống có thể mang ý nghĩa khác so với việc thiếu ngẫu nhiên).

Hình 3.2 minh họa tỉ lệ dữ liệu thiếu của từng cột trong bộ Titanic; cột có quá nhiều giá trị thiếu nên được loại bỏ.



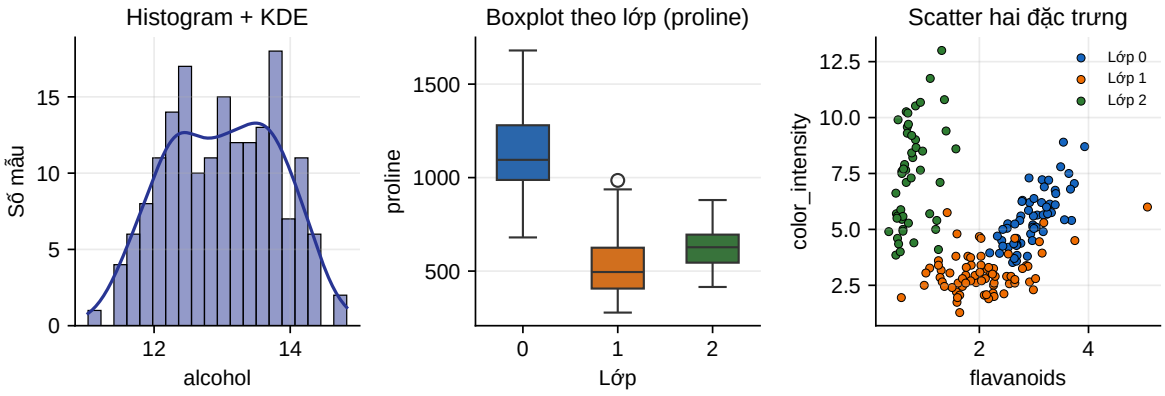
Hình 3.2. Tỉ lệ giá trị thiếu theo cột (bộ Titanic, 891 hành khách).

3.2.4. Trực quan hóa dữ liệu trong phân tích khám phá (EDA)

Phân tích khám phá dữ liệu (Exploratory Data Analysis – EDA) là bước quan sát, tóm tắt và trực quan hóa dữ liệu trước khi xây dựng mô hình, giúp phát hiện các đặc điểm quan trọng: phân bố của từng biến, giá trị ngoại lai (outlier), mối tương quan giữa các biến, và các vấn đề chất lượng dữ liệu cần xử lý. Bốn loại biểu đồ thường dùng nhất:

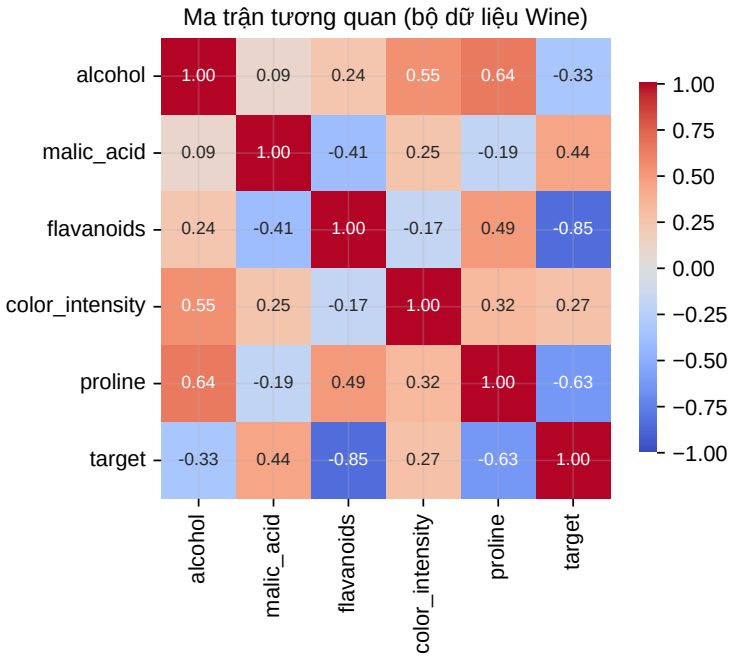
- Biểu đồ tần suất (histogram): quan sát phân bố của một biến số liên tục — biến có phân bố lệch, có nhiều đỉnh, hay xấp xỉ phân bố chuẩn.
- Biểu đồ phân tán (scatter plot): quan sát mối quan hệ giữa hai biến số, hữu ích để phát hiện quan hệ tuyến tính hoặc phi tuyến.
- Biểu đồ hộp (box plot): quan sát phân bố theo nhóm và phát hiện giá trị ngoại lai thông qua khoảng tứ phân vị (IQR).
- Bản đồ nhiệt tương quan (correlation heatmap): quan sát nhanh mức độ tương quan tuyến tính giữa tất cả các cặp biến số trong bộ dữ liệu.

Hình 3.3 minh họa ba loại biểu đồ nền tảng của EDA trên bộ dữ liệu Wine.



Hình 3.3. Histogram kèm KDE, boxplot theo lớp và biểu đồ phân tán (bộ dữ liệu Wine).

Hình 3.4 minh họa ma trận tương quan giúp phát hiện nhanh các cặp đặc trưng liên quan chặt chẽ.



Hình 3.4. Bản đồ nhiệt tương quan giữa một số đặc trưng của bộ Wine.

3.3. Ví dụ minh họa bằng mã Python

Ví dụ dưới đây thực hiện một quy trình EDA cơ bản trên bộ dữ liệu Titanic (bộ dữ liệu kinh điển ghi nhận thông tin hành khách và khả năng sống sót trong thảm họa đắm tàu Titanic), bao gồm: nạp dữ liệu, xem thông tin tổng quan, xử lý giá trị thiếu, và trực quan hóa một số mối quan hệ quan trọng.

```
# vidu_3_1.py
# Phân tích khám phá dữ liệu (EDA) cơ bản trên bộ dữ liệu Titanic

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Nạp dữ liệu Titanic có sẵn trong thư viện seaborn
df = sns.load_dataset("titanic")

# 2. Xem thông tin tổng quan về dữ liệu
print("Kích thước dữ liệu:", df.shape)
print("\nThông tin các cột:")
print(df.info())
print("\nThống kê mô tả các cột số:")
print(df.describe())

# 3. Kiểm tra số lượng giá trị thiếu theo từng cột
print("\nSố giá trị thiếu theo từng cột:")
so_gia_tri_thieu = df.isna().sum()
```

```
print(so_gia_tri_thieu[so_gia_tri_thieu > 0].sort_values(ascending=False))

# 4. Xử lý giá trị thiếu
#    - Cột "age" (tuổi): điền bằng giá trị trung vị (median), vì tuổi có thể lệch
phân bố
#    - Cột "embarked" (cảng lên tàu): điền bằng giá trị xuất hiện nhiều nhất (mode)
#    - Cột "deck" (boong tàu): quá nhiều giá trị thiếu (>70%) nên loại bỏ cột này
df["age"] = df["age"].fillna(df["age"].median())
df["embarked"] = df["embarked"].fillna(df["embarked"].mode()[0])
df = df.drop(columns=["deck"])

print(f"\nSố giá trị thiếu còn lại: {df.isna().sum().sum()}")

# 5. Trực quan hóa: tỉ lệ sống sót theo hạng vé (pclass) và giới tính (sex)
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# 5a. Histogram phân bố tuổi
sns.histplot(data=df, x="age", bins=30, kde=True, ax=axes[0])
axes[0].set_title("Phân bố độ tuổi hành khách")

# 5b. Biểu đồ cột: tỉ lệ sống sót theo hạng vé và giới tính
sns.barplot(data=df, x="pclass", y="survived", hue="sex", ax=axes[1])
axes[1].set_title("Tỉ lệ sống sót theo hạng vé và giới tính")

# 5c. Bản đồ nhiệt tương quan giữa các biến số
cac_bien_so = ["survived", "pclass", "age", "sibsp", "parch", "fare"]
ma_tran_tuong_quan = df[cac_bien_so].corr()
sns.heatmap(ma_tran_tuong_quan, annot=True, fmt=".2f", cmap="coolwarm", ax=axes[2])
axes[2].set_title("Ma trận tương quan")

plt.tight_layout()
plt.savefig("eda_titanic.png", dpi=120)
print("\nĐã lưu biểu đồ vào tệp eda_titanic.png")
```

3.4. Giải thích kết quả chạy chương trình

Chương trình trên tạo ra kết quả văn bản và một hình ảnh gồm ba biểu đồ. Phần kết quả văn bản có dạng:

```
Kích thước dữ liệu: (891, 15)
...
Số giá trị thiếu theo từng cột:
deck         688
age          177
embarked       2
embark_town    2

Số giá trị thiếu còn lại: 0
```

```
Đã lưu biểu đồ vào tệp eda_titanic.png
```

Kết quả cho thấy bộ dữ liệu Titanic có 891 hành khách với 15 cột thông tin. Cột "deck" (boong tàu) thiếu tới 688/891 giá trị (khoảng 77%) — với tỉ lệ thiếu cao như vậy, việc điền giá trị sẽ đưa vào rất nhiều thông tin không đáng tin cậy, do đó quyết định loại bỏ hẳn cột này là hợp lý. Ngược lại, cột "age" chỉ thiếu khoảng 20%, vẫn còn đủ thông tin hữu ích nên được giữ lại và điền bằng giá trị trung vị.

Về mặt trực quan hóa: biểu đồ histogram tuổi thường cho thấy phân bố lệch phải nhẹ, tập trung nhiều ở nhóm 20–40 tuổi. Biểu đồ cột so sánh tỉ lệ sống sót thường cho thấy rõ hai xu hướng nổi bật trong lịch sử vụ đắm tàu Titanic: hành khách hạng vé càng cao (pclass = 1) có tỉ lệ sống sót càng lớn, và hành khách nữ có tỉ lệ sống sót cao hơn đáng kể so với hành khách nam ở mọi hạng vé — phản ánh chính sách ưu tiên "phụ nữ và trẻ em lên xuồng cứu sinh trước" được ghi nhận trong lịch sử. Bản đồ nhiệt tương quan giúp phát hiện nhanh: biến pclass thường có tương quan âm khá rõ với survived (hạng vé số càng lớn — tức khoang hạng thấp — thì khả năng sống sót càng giảm), gợi ý đây là một đặc trưng quan trọng cần đưa vào mô hình phân loại ở các chương sau.

Lưu ý sư phạm

EDA không phải là bước "làm cho có" trước khi huấn luyện mô hình, mà thường quyết định phần lớn chất lượng của mô hình cuối cùng. Nhiều sai sót nghiêm trọng (rò rỉ dữ liệu, đặc trưng vô nghĩa, mất cân bằng lớp nghiêm trọng) chỉ có thể được phát hiện sớm thông qua EDA cẩn thận, chứ không thể phát hiện chỉ bằng cách nhìn vào độ chính xác của mô hình sau khi huấn luyện.

3.5. Tóm tắt chương

- NumPy cung cấp cấu trúc ndarray và cơ chế vector hóa, broadcasting giúp tính toán số học nhanh và gọn hơn nhiều so với vòng lặp Python thuần túy.
- Pandas cung cấp DataFrame — cấu trúc trung tâm để đọc, khám phá, lọc và biến đổi dữ liệu dạng bảng.
- Xử lý dữ liệu thiếu cần dựa trên tỉ lệ thiếu và bản chất của biến: loại bỏ khi thiếu quá nhiều, điền giá trị thống kê khi thiếu ở mức chấp nhận được.
- Bốn loại biểu đồ nền tảng của EDA: histogram (phân bố một biến), scatter plot (quan hệ hai biến số), box plot (phân bố theo nhóm, phát hiện ngoại lai), heatmap tương quan (quan hệ nhiều biến cùng lúc).
- EDA là bước bắt buộc, có ảnh hưởng trực tiếp đến chất lượng mô hình, không thể bỏ qua hoặc làm sơ sài trước khi bước vào huấn luyện mô hình ở các chương tiếp theo.

3.6. Bài tập củng cố

Mức nhận biết

1. Nêu sự khác biệt giữa Series và DataFrame trong Pandas.
2. Liệt kê ba chiến lược xử lý giá trị thiếu đã học và cho biết mỗi chiến lược phù hợp trong tình huống nào.

Mức thông hiểu

1. Giải thích cơ chế broadcasting của NumPy thông qua một ví dụ cụ thể: cộng một mảng kích thước (3, 3) với một mảng kích thước (3,).
2. Vì sao nên dùng giá trị trung vị (median) thay vì giá trị trung bình (mean) để điền vào các cột có giá trị ngoại lai lớn (ví dụ cột thu nhập)?

Mức vận dụng

1. Cho một bộ dữ liệu bán hàng gồm các cột: ngày bán, mã sản phẩm, số lượng, đơn giá, khu vực. Hãy đề xuất ít nhất ba biểu đồ EDA hữu ích cho bộ dữ liệu này và giải thích mỗi biểu đồ giúp trả lời câu hỏi kinh doanh gì.

3.7. Bài tập lập trình

Bài 1. Sử dụng bộ dữ liệu `sns.load_dataset`(“tips”) (dữ liệu tiền boa tại nhà hàng), thực hiện: (a) in ra thông tin tổng quan và thống kê mô tả; (b) kiểm tra có giá trị thiếu hay không; (c) vẽ biểu đồ scatter plot giữa cột `total_bill` (tổng hóa đơn) và tip (tiền boa), phân màu theo cột time (buổi trưa/tối).

Bài 2. Từ bộ dữ liệu Titanic ở ví dụ 3.1, viết thêm mã tạo ra một đặc trưng mới có tên `family_size` = sibsp + parch + 1 (tổng số thành viên gia đình đi cùng, cộng thêm bản thân hành khách). Vẽ biểu đồ cột thể hiện tỉ lệ sống sót trung bình theo từng giá trị `family_size` và nhận xét xu hướng quan sát được.

Bài 3 (nâng cao). Viết một hàm Python tên `bao_cao_du_lieu`(df) nhận vào một DataFrame bất kỳ và tự động in ra một báo cáo tóm tắt gồm: số dòng, số cột, danh sách cột kèm kiểu dữ liệu, số lượng và tỉ lệ phần trăm giá trị thiếu của mỗi cột (chỉ liệt kê các cột có thiếu), và số lượng giá trị trùng lặp (duplicate rows). Áp dụng thử hàm này cho bộ dữ liệu `sns.load_dataset`(“diamonds”).

Chương tiếp theo (Chương 4) sẽ mở đầu Phần 2 của giáo trình với thuật toán học

có giám sát đầu tiên: Hồi quy tuyến tính — nền tảng cho toàn bộ các phương pháp hồi quy phức tạp hơn được trình bày ở các chương sau.

Phần II

Học có giám sát

Chương 4

Hồi quy tuyến tính (Linear Regression)

4.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Trình bày được mô hình hồi quy tuyến tính đơn biến và đa biến dưới dạng công thức toán học.
2. Giải thích được hàm mất mát bình phương trung bình (MSE) và ý nghĩa của phương pháp bình phương tối thiểu (Ordinary Least Squares).
3. Mô tả được nguyên lý thuật toán Gradient Descent để tìm tham số tối ưu của mô hình.
4. Nêu được các giả định của mô hình hồi quy tuyến tính và hệ quả khi các giả định bị vi phạm.
5. Cài đặt được mô hình hồi quy tuyến tính bằng Scikit-learn và đánh giá bằng các độ đo MAE, MSE, RMSE, R^2.

4.2. Nội dung lý thuyết

4.2.1. Bài toán hồi quy tuyến tính

Hồi quy tuyến tính là mô hình học có giám sát đơn giản và lâu đời nhất, giả định rằng biến đầu ra (mục tiêu dự đoán) có quan hệ tuyến tính với các biến đầu vào (đặc trưng). Với trường hợp một biến đầu vào (hồi quy đơn biến), mô hình có dạng:

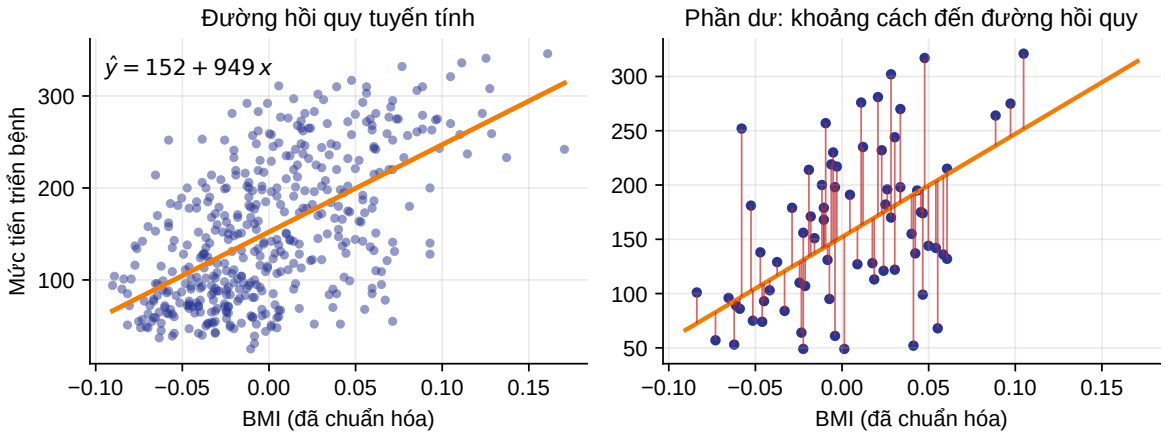
$$\hat{y} = w_0 + w_1 x$$

trong đó $\hat{y}$ (đọc là "y mũ") là giá trị dự đoán, x là biến đầu vào, w_0 là hệ số chặn (intercept — giá trị dự đoán khi x = 0), và w_1 là hệ số góc (slope — mức thay đổi của $\hat{y}$ khi x tăng thêm một đơn vị). Khi có nhiều biến đầu vào x_1, x_2, …, x_n (hồi quy đa biến), mô hình được mở rộng thành:

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$$

Mục tiêu của quá trình huấn luyện là tìm ra bộ trọng số w_0, w_1, …, w_n sao cho các giá trị dự đoán $\hat{y}$ càng gần với giá trị thực tế y (trong tập dữ liệu huấn luyện) càng tốt.

Hình 4.1 minh họa đường hồi quy tuyến tính khớp với dữ liệu và các phần dư mà phương pháp bình phương tối thiểu cố gắng làm nhỏ nhất.



Hình 4.1. Đường hồi quy (trái) và phần dư — các đoạn đỏ (phải).

4.2.2. Hàm mất mát và phương pháp bình phương tối thiểu

Để đo mức độ "gần đúng" của mô hình, ta sử dụng hàm mất mát bình phương trung bình (Mean Squared Error – MSE):

$$\text{MSE} = \frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2$$

với m là số lượng mẫu dữ liệu huấn luyện, y_i là giá trị thực tế và $\hat{y}_i$ là giá trị dự đoán của mẫu thứ i. Việc bình phương hiệu số $(y_i - \hat{y}_i)$ có hai tác dụng: đảm bảo các sai số âm và dương không triệt tiêu lẫn nhau, đồng thời phạt nặng hơn đối với các sai số lớn so với sai số nhỏ.

Phương pháp bình phương tối thiểu (Ordinary Least Squares – OLS) tìm bộ trọng số w sao cho giá trị MSE là nhỏ nhất. Đối với hồi quy tuyến tính, bài toán tối ưu này có lời giải dạng đóng (closed-form solution), có thể tính trực tiếp bằng công thức đại số tuyến tính mà không cần phương pháp lặp. Tuy nhiên, khi số lượng đặc trưng rất lớn, việc tính nghịch đảo ma trận trong công thức đóng trở nên tốn kém về mặt tính toán, khi đó các phương pháp tối ưu lặp như Gradient Descent thường được ưu tiên sử dụng.

4.2.3. Thuật toán Gradient Descent

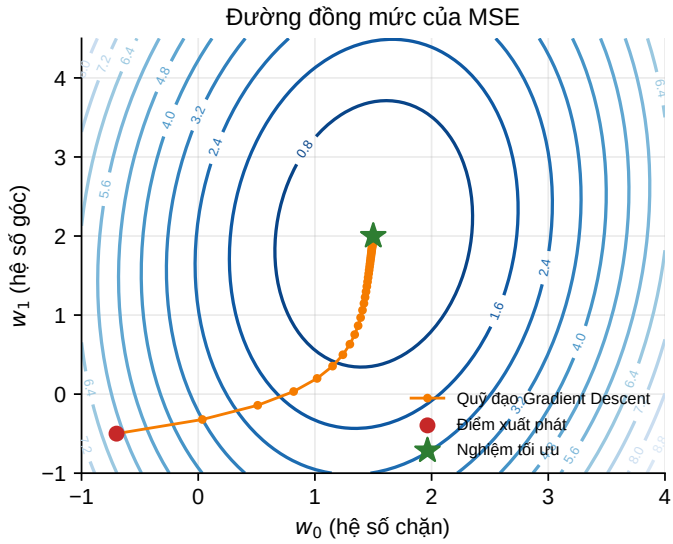
Gradient Descent (hạ độ dốc) là một thuật toán tối ưu lặp, được sử dụng rộng rãi không chỉ trong hồi quy tuyến tính mà trong hầu hết các mô hình học máy và học sâu hiện đại. Trực giác của thuật toán: hình dung hàm mất mát như một địa hình đồi núi trong không gian nhiều chiều (mỗi chiều tương ứng với một trọng số w); ta xuất phát từ một điểm ngẫu nhiên và di chuyển từng bước nhỏ theo hướng dốc xuống nhiều nhất (hướng ngược với gradient — đạo hàm của hàm mất mát) cho đến khi đạt tới điểm thấp nhất (hoặc gần thấp nhất) của địa hình đó.

Công thức cập nhật trọng số ở mỗi bước lặp:

$$w := w - \alpha \frac{\partial \text{MSE}}{\partial w}$$

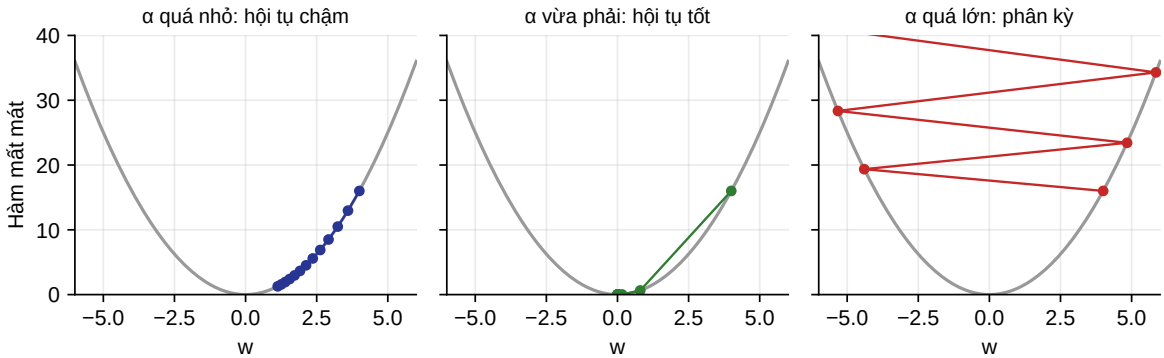
trong đó α (alpha) là tốc độ học (learning rate) — một siêu tham số do người dùng lựa chọn, quyết định độ lớn của mỗi bước di chuyển. Nếu α quá nhỏ, thuật toán hội tụ rất chậm; nếu α quá lớn, thuật toán có thể "nhảy qua" điểm tối ưu và không hội tụ, thậm chí phân kỳ. Việc lựa chọn tốc độ học phù hợp là một trong những kỹ năng thực hành quan trọng khi huấn luyện mô hình.

Hình 4.2 minh họa quỹ đạo của Gradient Descent trên các đường đồng mức của hàm MSE: mỗi bước đi theo hướng dốc xuống nhiều nhất.



Hình 4.2. Gradient Descent hội tụ về nghiệm tối ưu trên mặt MSE của hồi quy đơn biến.

Hình 4.3 minh họa ảnh hưởng của tốc độ học α lên quá trình hội tụ.



Hình 4.3. Tốc độ học quá nhỏ, vừa phải và quá lớn.

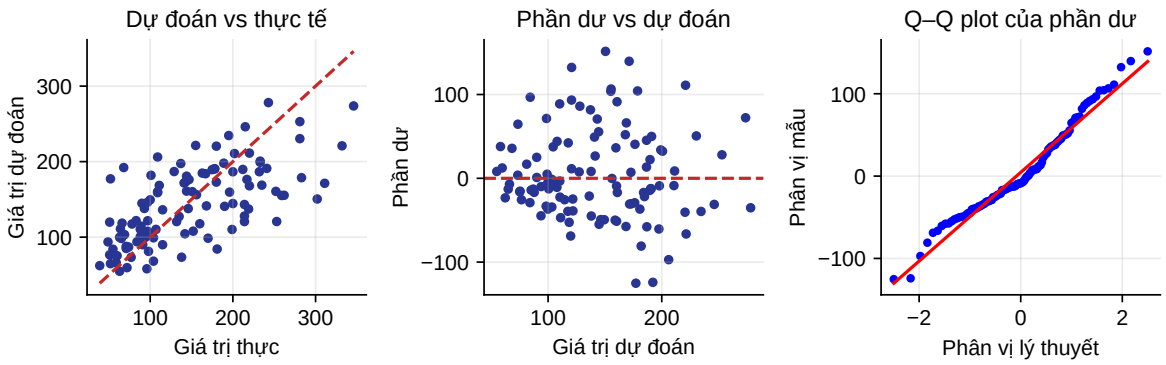
4.2.4. Các giả định của hồi quy tuyến tính

Mô hình hồi quy tuyến tính hoạt động tốt nhất khi một số giả định sau được thỏa mãn (dù trong thực tế hiếm khi thỏa mãn hoàn toàn):

- Tuyến tính (linearity): quan hệ giữa biến đầu vào và đầu ra là tuyến tính.
- Độc lập (independence): các quan sát (mẫu dữ liệu) độc lập với nhau.
- Phương sai đồng nhất (homoscedasticity): phương sai của phần dư (residual = y $-$ $\hat{y}$) không đổi theo giá trị dự đoán.
- Phần dư có phân phối chuẩn (normality of residuals): hữu ích cho việc suy diễn thống kê (kiểm định giả thuyết, khoảng tin cậy) nhưng không bắt buộc để mô hình dự đoán tốt.
- Không đa cộng tuyến nghiêm trọng (no severe multicollinearity): các biến đầu vào không tương quan quá chặt với nhau, nếu không hệ số hồi quy sẽ trở nên không ổn định và khó diễn giải.

Khi các giả định bị vi phạm nghiêm trọng, mô hình vẫn có thể chạy được và cho ra dự đoán, nhưng độ tin cậy của các hệ số và khả năng tổng quát hóa có thể bị ảnh hưởng đáng kể. Các chương sau (đặc biệt Chương 5 về chính quy hóa) sẽ giới thiệu một số kỹ thuật giúp giảm nhẹ ảnh hưởng của việc vi phạm giả định, đặc biệt là đa cộng tuyến.

Hình 4.4 minh họa ba biểu đồ chẩn đoán thường dùng để kiểm tra các giả định của mô hình tuyến tính.



Hình 4.4. Dự đoán so với thực tế, phần dư so với dự đoán và Q–Q plot của phần dư (bộ Diabetes).

4.2.5. Các độ đo đánh giá mô hình hồi quy

Bốn độ đo phổ biến nhất để đánh giá một mô hình hồi quy:

- MAE (Mean Absolute Error): trung bình trị tuyệt đối của sai số, dễ diễn giải (cùng đơn vị với biến mục tiêu), ít nhạy cảm với giá trị ngoại lai.
- MSE (Mean Squared Error): trung bình bình phương sai số, phạt nặng các sai số lớn, nhưng đơn vị là bình phương của biến mục tiêu nên khó diễn giải trực tiếp.
- RMSE (Root Mean Squared Error): căn bậc hai của MSE, đưa đơn vị trở về giống biến mục tiêu trong khi vẫn giữ đặc tính phạt nặng sai số lớn.
- R^2 (hệ số xác định — coefficient of determination): đo tỉ lệ phương sai của biến mục tiêu được giải thích bởi mô hình, nhận giá trị tối đa là 1 (mô hình dự đoán hoàn hảo); R^2 =

0 tương đương với việc luôn dự đoán bằng giá trị trung bình; R^2 có thể âm nếu mô hình dự đoán tệ hơn cả việc dự đoán bằng giá trị trung bình.

4.3. Ví dụ minh họa bằng mã Python

Ví dụ dưới đây xây dựng một mô hình hồi quy tuyến tính dự đoán chỉ số tiến triển bệnh tiểu đường (bộ dữ liệu Diabetes có sẵn trong Scikit-learn) dựa trên chỉ số khối cơ thể (BMI) của bệnh nhân, đồng thời minh họa việc đánh giá mô hình bằng nhiều độ đo khác nhau.

```
# vidu_4_1.py
# Hồi quy tuyến tính đơn biến: dự đoán mức độ tiến triển bệnh tiểu đường
# dựa trên chỉ số khối cơ thể (BMI)

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# 1. Nạp dữ liệu và chỉ lấy một đặc trưng duy nhất: BMI (chỉ số khối cơ thể)
du_lieu = load_diabetes()
chi_so_bmi = du_lieu.feature_names.index("bmi")
X = du_lieu.data[:, [chi_so_bmi]]   # Giữ dạng ma trận 2 chiều (m, 1)
y = du_lieu.target                  # Chỉ số tiến triển bệnh sau 1 năm

# 2. Chia tập huấn luyện / kiểm tra
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Khởi tạo và huấn luyện mô hình hồi quy tuyến tính
mo_hinh = LinearRegression()
mo_hinh.fit(X_train, y_train)

# 4. In ra các tham số mô hình đã học được
print(f"Hệ số chặn (w0): {mo_hinh.intercept_:.2f}")
print(f"Hệ số góc (w1) của BMI: {mo_hinh.coef_[0]:.2f}")

# 5. Dự đoán trên tập kiểm tra
y_du_doan = mo_hinh.predict(X_test)

# 6. Đánh giá mô hình bằng nhiều độ đo khác nhau
mae = mean_absolute_error(y_test, y_du_doan)
mse = mean_squared_error(y_test, y_du_doan)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_du_doan)
```

```
print(f"\nMAE  = {mae:.2f}")
print(f"MSE  = {mse:.2f}")
print(f"RMSE = {rmse:.2f}")
print(f"R²   = {r2:.4f}")

# 7. Trực quan hóa: dữ liệu thực tế và đường hồi quy học được
plt.figure(figsize=(7, 5))
plt.scatter(X_test, y_test, color="steelblue", alpha=0.6, label="Dữ liệu thực tế")
plt.plot(X_test, y_du_doan, color="darkorange", linewidth=2, label="Đường hồi quy
dự đoán")
plt.xlabel("Chỉ số khối cơ thể (BMI, đã chuẩn hóa)")
plt.ylabel("Mức độ tiến triển bệnh sau 1 năm")
plt.title("Hồi quy tuyến tính: BMI và mức độ tiến triển bệnh tiểu đường")
plt.legend()
plt.tight_layout()
plt.savefig("hoi_quy_tuyen_tinh_bmi.png", dpi=120)
print("\nĐã lưu biểu đồ vào tệp hoi_quy_tuyen_tinh_bmi.png")
```

4.4. Giải thích kết quả chạy chương trình

Khi thực thi chương trình trên, kết quả in ra có dạng tương tự như sau (số liệu cụ thể có thể thay đổi nhẹ tùy phiên bản thư viện):

```
Hệ số chặn (w0): 152.42
Hệ số góc (w1) của BMI: 998.58

MAE  = 53.35
MSE  = 4150.69
RMSE = 64.43
R²   = 0.2334
```

Hệ số góc dương và có độ lớn đáng kể (khoảng 998.58) cho thấy chỉ số BMI có quan hệ thuận với mức độ tiến triển bệnh: BMI càng cao, mức độ tiến triển bệnh dự đoán càng lớn — phù hợp với hiểu biết y khoa thông thường. Tuy nhiên, giá trị R^2 chỉ khoảng 0.23 cho thấy riêng biến BMI chỉ giải thích được khoảng 23% sự biến thiên của mức độ tiến triển bệnh; phần còn lại (77%) chịu ảnh hưởng bởi các yếu tố khác chưa được đưa vào mô hình đơn biến này (tuổi, huyết áp, các chỉ số xét nghiệm máu, v.v.). Đây là lý do vì sao trong thực tế, các mô hình hồi quy đa biến (sử dụng đồng thời nhiều đặc trưng) thường cho kết quả dự đoán tốt hơn đáng kể so với mô hình chỉ dùng một đặc trưng duy nhất.

Lưu ý

R^2 thấp không có nghĩa là mô hình "sai" hay "vô dụng" — nó chỉ phản ánh rằng biến đầu vào hiện có không đủ để giải thích toàn bộ sự biến thiên của biến mục tiêu. Trong các lĩnh vực như y sinh học hay khoa học xã hội, nơi hành vi con người và cơ thể chịu ảnh hưởng bởi rất nhiều yếu tố phức tạp, giá trị R^2 trong khoảng 0.2–0.4 không phải là hiếm gặp và

đôi khi vẫn được xem là có ý nghĩa thực tiễn, miễn là các hệ số hồi quy ổn định và có thể diễn giải được.

4.5. Tóm tắt chương

- Hồi quy tuyến tính mô hình hóa quan hệ tuyến tính giữa biến đầu vào và biến mục tiêu dạng liên tục, thông qua các trọng số w.
- Hàm mất mát MSE đo mức độ sai lệch giữa dự đoán và giá trị thực tế; phương pháp bình phương tối thiểu (OLS) tìm bộ trọng số tối ưu hóa MSE, có lời giải dạng đóng cho hồi quy tuyến tính.
- Gradient Descent là thuật toán tối ưu lặp tổng quát, cập nhật trọng số theo hướng ngược với gradient của hàm mất mát, với tốc độ học α là siêu tham số quan trọng cần lựa chọn cẩn thận.
- Mô hình hồi quy tuyến tính dựa trên một số giả định (tuyến tính, độc lập, phương sai đồng nhất, không đa cộng tuyến nghiêm trọng); vi phạm các giả định này ảnh hưởng đến độ tin cậy của mô hình.
- Bốn độ đo đánh giá phổ biến: MAE, MSE, RMSE (đo sai số theo các cách khác nhau) và R^2 (đo tỉ lệ phương sai được giải thích bởi mô hình).

4.6. Bài tập củng cố

Mức nhận biết

1. Viết công thức tổng quát của mô hình hồi quy tuyến tính đa biến với n đặc trưng đầu vào. Giải thích ý nghĩa của từng thành phần trong công thức.
2. Nêu công thức tính MSE và giải thích vì sao lại bình phương hiệu số thay vì chỉ lấy trị tuyệt đối.

Mức thông hiểu

1. Giải thích ảnh hưởng của việc chọn tốc độ học (learning rate) α quá lớn hoặc quá nhỏ đến quá trình hội tụ của Gradient Descent. Minh họa bằng mô tả trực quan (địa hình đồi núi).
2. Một mô hình hồi quy có R^2 = 0.85 trên tập huấn luyện nhưng R^2 = 0.20 trên tập kiểm tra. Hiện tượng gì đang xảy ra? Liên hệ với khái niệm đã học ở Chương 1.
3. So sánh MAE và RMSE: độ đo nào nhạy cảm hơn với giá trị ngoại lai (outlier)? Giải thích vì sao dựa trên công thức toán học của hai độ đo.

Mức vận dụng

1. Một công ty bất động sản muốn dự đoán giá nhà dựa trên diện tích, số phòng ngủ và khoảng cách đến trung tâm thành phố. Hãy viết công thức mô hình hồi quy tuyến tính đa biến phù hợp cho bài toán này, và dự đoán dấu (dương/âm) hợp lý của từng hệ số hồi quy dựa trên hiểu biết thông thường.

4.7. Bài tập lập trình

Bài 1. Mở rộng ví dụ 4.1 thành mô hình hồi quy đa biến: sử dụng toàn bộ 10 đặc trưng có sẵn trong bộ dữ liệu `load_diabetes()` (thay vì chỉ dùng BMI). So sánh giá trị R^2 và RMSE của mô hình đa biến với mô hình đơn biến ở ví dụ 4.1, nhận xét kết quả.

Bài 2. Viết chương trình cài đặt thủ công thuật toán Gradient Descent cho hồi quy tuyến tính đơn biến (không dùng LinearRegression của Scikit-learn), sử dụng dữ liệu tự tạo bằng NumPy (ví dụ: x ngẫu nhiên, y = 3x + 2 cộng thêm nhiễu Gauss). In ra giá trị w0, w1 sau mỗi 100 vòng lặp và vẽ biểu đồ giá trị hàm mất mát MSE giảm dần theo số vòng lặp.

Bài 3 (nâng cao). Sử dụng bộ dữ liệu `fetch_california_housing()` có sẵn trong Scikit-learn, xây dựng mô hình hồi quy tuyến tính đa biến dự đoán giá nhà trung vị. Vẽ biểu đồ scatter giữa giá trị thực tế và giá trị dự đoán trên tập kiểm tra (trục hoành là giá trị thực, trục tung là giá trị dự đoán), kèm đường chéo y = x để trực quan hóa mức độ chính xác của mô hình. Nhận xét những vùng giá trị mà mô hình dự đoán kém chính xác nhất.

Chương tiếp theo (Chương 5) sẽ mở rộng hồi quy tuyến tính với các đặc trưng đa thức để nắm bắt quan hệ phi tuyến, đồng thời giới thiệu các kỹ thuật chính quy hóa (Ridge, Lasso, Elastic Net) nhằm kiểm soát hiện tượng khớp quá khi mô hình trở nên phức tạp hơn.

Chương 5

Hồi quy đa thức và Chính quy hóa (Ridge, Lasso, Elastic Net)

5.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Giải thích được cách hồi quy đa thức mở rộng hồi quy tuyến tính để nắm bắt quan hệ phi tuyến giữa biến đầu vào và đầu ra.
2. Phân tích được mối liên hệ giữa bậc đa thức, độ phức tạp mô hình và hiện tượng khớp quá.
3. Trình bày được nguyên lý chính quy hóa (regularization) và cách nó bổ sung một thành phần phạt vào hàm mất mát.
4. Phân biệt được Ridge (phạt L2), Lasso (phạt L1) và Elastic Net (kết hợp L1, L2) về mặt công thức và hành vi trên các hệ số hồi quy.
5. Cài đặt và so sánh được các mô hình hồi quy đa thức có/không chính quy hóa bằng Scikit-learn.

5.2. Nội dung lý thuyết

5.2.1. Hồi quy đa thức (Polynomial Regression)

Trong nhiều bài toán thực tế, quan hệ giữa biến đầu vào và biến mục tiêu không tuyến tính mà có dạng cong. Hồi quy đa thức giải quyết vấn đề này bằng một ý tưởng đơn giản nhưng hiệu quả: thay vì thay đổi thuật toán, ta mở rộng tập đặc trưng đầu vào bằng cách thêm các lũy thừa của đặc trưng gốc (x^2, x^3, …), rồi vẫn áp dụng hồi quy tuyến tính như bình thường trên tập đặc trưng đã mở rộng. Với một biến đầu vào x, mô hình hồi quy đa thức bậc n có dạng:

$$\hat{y} = w_0 + w_1 x + w_2 x^2 + w_3 x^3 + \cdots + w_n x^n$$

Về bản chất toán học, đây vẫn là một mô hình tuyến tính — tuyến tính theo các trọng số w, chỉ khác là các đặc trưng đầu vào (x, x^2, x^3, …) không còn là tuyến tính theo x gốc. Điều này lý giải vì sao ta có thể tiếp tục sử dụng phương pháp bình phương tối thiểu và Gradient Descent giống hệt như hồi quy tuyến tính thông thường, chỉ cần biến đổi dữ liệu đầu vào trước khi đưa vào mô hình (trong Scikit-learn, phép biến đổi này được thực hiện bằng lớp PolynomialFeatures).

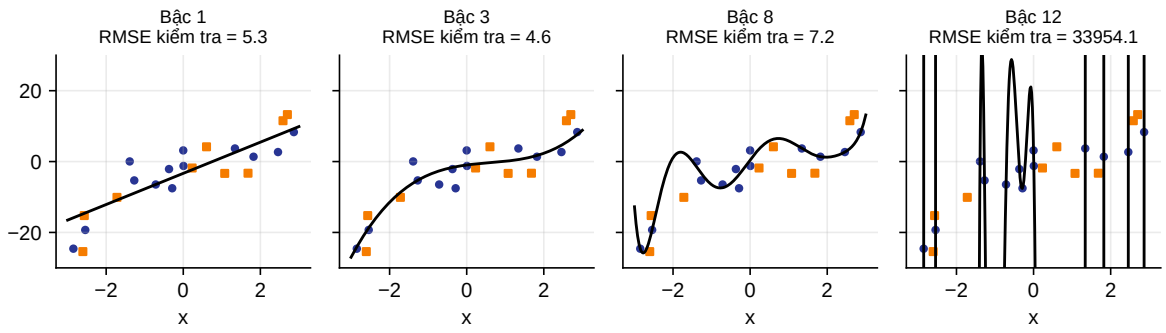
5.2.2. Bậc đa thức và hiện tượng khớp quá

Bậc đa thức n là một siêu tham số quan trọng cần lựa chọn cẩn thận:

- n quá nhỏ (ví dụ n = 1): mô hình có thể quá đơn giản so với quan hệ thực sự trong dữ liệu, dẫn đến khớp thiếu (underfitting) — cả sai số huấn luyện lẫn sai số kiểm tra đều cao.
- n quá lớn (ví dụ n = 10, 15): mô hình có đủ độ linh hoạt để "uốn cong" đi qua gần như mọi điểm dữ liệu huấn luyện, kể cả nhiễu ngẫu nhiên, dẫn đến khớp quá (overfitting) — sai số huấn luyện rất thấp nhưng sai số kiểm tra lại tăng vọt.

Đây chính là minh họa trực quan và cụ thể nhất cho khái niệm đánh đổi độ chệch – phương sai đã giới thiệu ở Chương 1: bậc đa thức càng cao, độ chệch (bias) của mô hình càng giảm nhưng phương sai (variance) càng tăng. Việc lựa chọn bậc đa thức phù hợp thường được thực hiện thông qua kiểm định chéo (cross-validation), sẽ được trình bày chi tiết ở Chương 15.

Hình 5.1 minh họa khi tăng bậc đa thức, mô hình càng bám sát dữ liệu huấn luyện nhưng có nguy cơ khớp quá ở vùng biên.



Hình 5.1. Hồi quy đa thức với bậc tăng dần (chấm xanh: huấn luyện; ô vuông cam: kiểm tra).

5.2.3. Chính quy hóa — ý tưởng chung

Chính quy hóa (regularization) là một nhóm kỹ thuật nhằm kiểm soát độ phức tạp của mô hình, giảm nguy cơ khớp quá, bằng cách bổ sung một số hạng phạt (penalty term) vào hàm mất mát gốc. Số hạng phạt này thường tỉ lệ với độ lớn của các trọng số w — ý tưởng nền tảng là: một mô hình có các trọng số quá lớn thường "nhạy cảm" thái quá với từng điểm dữ liệu huấn luyện (dễ khớp quá), trong khi một mô hình có trọng số nhỏ, "mượt" hơn thường tổng quát hóa tốt hơn trên dữ liệu mới.

Hàm mất mát tổng quát khi có chính quy hóa có dạng:

$$J = \text{MSE} + \lambda \times (\text{số hạng phạt theo } \mathbf{w})$$

trong đó λ (lambda, còn được ký hiệu là alpha trong Scikit-learn) là siêu tham số điều chỉnh mức độ chính quy hóa: $\lambda = 0$ tương đương hồi quy tuyến tính thông thường (không phạt); λ càng lớn, mô hình càng bị ép về phía các trọng số nhỏ (thậm chí bằng 0), làm giảm độ phức tạp mô hình.

5.2.4. Ridge Regression (Chính quy hóa L2)

Ridge Regression sử dụng số hạng phạt là tổng bình phương các trọng số (chuẩn L2):

$$J_{\text{Ridge}} = \text{MSE} + \lambda \sum_j w_j^2$$

Đặc điểm quan trọng của Ridge: khi λ tăng, các trọng số bị co lại (shrinkage) về gần 0 nhưng hiếm khi bằng đúng 0. Điều này có nghĩa Ridge làm giảm ảnh hưởng của các đặc trưng ít quan trọng chứ không loại bỏ hoàn toàn chúng khỏi mô hình. Ridge đặc biệt hữu ích khi dữ liệu có hiện tượng đa cộng tuyến (các đặc trưng tương quan chặt với nhau), giúp ổn định các hệ số hồi quy vốn có thể trở nên rất bất ổn trong hồi quy tuyến tính thông thường.

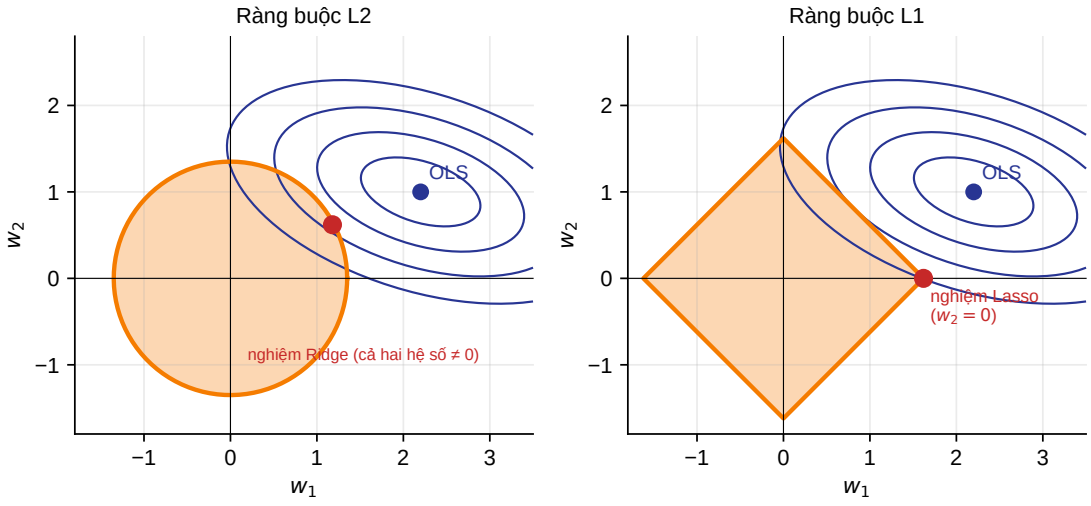
5.2.5. Lasso Regression (Chính quy hóa L1)

Lasso Regression (Least Absolute Shrinkage and Selection Operator) sử dụng số hạng phạt là tổng trị tuyệt đối các trọng số (chuẩn L1):

$$J_{\text{Lasso}} = \text{MSE} + \lambda \sum_j |w_j|$$

Khác với Ridge, do đặc tính hình học của chuẩn L1, Lasso có xu hướng đưa một số trọng số về đúng bằng 0 khi λ đủ lớn — nói cách khác, Lasso thực hiện lựa chọn đặc trưng (feature selection) tự động như một hệ quả tự nhiên của quá trình tối ưu, loại bỏ hoàn toàn các đặc trưng ít đóng góp ra khỏi mô hình. Tính chất này khiến Lasso đặc biệt hữu ích khi làm việc với dữ liệu có rất nhiều đặc trưng, trong đó chỉ một phần nhỏ thực sự có ý nghĩa dự đoán, đồng thời giúp mô hình dễ diễn giải hơn.

Hình 5.2 minh họa lý do hình học khiến Lasso đưa hệ số về đúng 0: vùng ràng buộc L_1 có các đỉnh nằm trên trục tọa độ.



Hình 5.2. Đường đồng mức của MSE và vùng ràng buộc L_2 (Ridge) so với L_1 (Lasso).

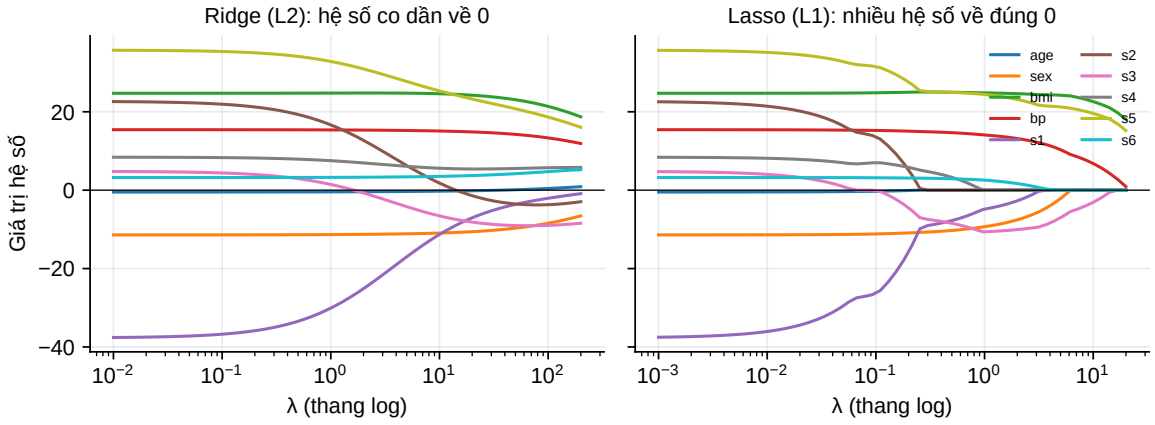
5.2.6. Elastic Net

Elastic Net kết hợp cả hai số hạng phạt L1 và L2 với tỉ lệ trộn được điều chỉnh bởi một siêu tham số bổ sung (thường ký hiệu là `l1_ratio` hoặc r, nhận giá trị từ 0 đến 1):

$$J_{\text{EN}} = \text{MSE} + \lambda \left[r \sum_j |w_j| + (1-r) \sum_j w_j^2 \right]$$

Khi r = 1, Elastic Net trở thành Lasso thuần túy; khi r = 0, nó trở thành Ridge thuần túy. Elastic Net thường được ưu tiên khi dữ liệu có nhiều đặc trưng tương quan chặt với nhau (trường hợp Lasso đơn thuần có thể chọn ngẫu nhiên một đặc trưng trong nhóm tương quan và loại bỏ hoàn toàn các đặc trưng còn lại, gây mất ổn định), trong khi vẫn giữ được khả năng lựa chọn đặc trưng tự động của Lasso.

Hình 5.3 minh họa đường đi của các hệ số khi tăng mức chính quy hóa.



Hình 5.3. Ridge co dần các hệ số; Lasso loại hẳn nhiều hệ số (bộ Diabetes, đã chuẩn hóa).

5.2.7. Lưu ý quan trọng: chuẩn hóa đặc trưng trước khi chính quy hóa

Lưu ý kỹ thuật

Vì số hạng phạt trong Ridge/Lasso/Elastic Net phụ thuộc trực tiếp vào độ lớn của các trọng số w, nếu các đặc trưng đầu vào có thang đo (scale) rất khác nhau (ví dụ một đặc trưng đo bằng mét, một đặc trưng khác đo bằng triệu đồng), các trọng số tương ứng sẽ có độ lớn khác nhau một cách giả tạo, khiến việc chính quy hóa không công bằng giữa các đặc trưng. Do đó, trước khi áp dụng bất kỳ kỹ thuật chính quy hóa nào, cần chuẩn hóa dữ liệu (ví dụ dùng StandardScaler để đưa các đặc trưng về cùng trung bình 0, độ lệch chuẩn 1).

5.3. Ví dụ minh họa bằng mã Python

Ví dụ dưới đây minh họa trực quan hiện tượng khớp quá khi tăng bậc đa thức trên dữ liệu tổng hợp, sau đó so sánh hiệu quả của Ridge, Lasso và hồi quy tuyến tính đa thức thông thường

(không chính quy hóa) trong việc kiểm soát hiện tượng này.

```
# vidu_5_1.py
# So sánh hồi quy đa thức không chính quy hóa với Ridge và Lasso
# trên dữ liệu tổng hợp có nhiễu

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# 1. Tạo dữ liệu tổng hợp: quan hệ bậc 3 thực sự, có nhiễu Gauss
rng = np.random.RandomState(42)
X = np.sort(rng.uniform(-3, 3, size=60)).reshape(-1, 1)
y_that = 0.5 * X[:, 0]**3 - X[:, 0]**2 + 2 * X[:, 0]
y = y_that + rng.normal(scale=3, size=X.shape[0])   # cộng thêm nhiễu

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# 2. Xây dựng ba pipeline: mở rộng đặc trưng đa thức bậc cao (bậc 12),
#    chuẩn hóa, rồi áp dụng ba mô hình hồi quy khác nhau
BAC_DA_THUC = 12

pipeline_khong_chinh_quy = make_pipeline(
    PolynomialFeatures(degree=BAC_DA_THUC),
    StandardScaler(),
    LinearRegression()
)
pipeline_ridge = make_pipeline(
    PolynomialFeatures(degree=BAC_DA_THUC),
    StandardScaler(),
    Ridge(alpha=5.0)
)
pipeline_lasso = make_pipeline(
    PolynomialFeatures(degree=BAC_DA_THUC),
    StandardScaler(),
    Lasso(alpha=0.5, max_iter=10000)
)

cac_mo_hinh = {
    "Không chính quy hóa (bậc 12)": pipeline_khong_chinh_quy,
    "Ridge (alpha=5.0)": pipeline_ridge,
    "Lasso (alpha=0.5)": pipeline_lasso,
}

# 3. Huấn luyện và đánh giá từng mô hình
```

```
X_ve = np.linspace(-3, 3, 300).reshape(-1, 1)   # lưới điểm để vẽ đường cong
plt.figure(figsize=(8, 6))
plt.scatter(X_train, y_train, color="gray", alpha=0.5, label="Dữ liệu huấn luyện")

for ten_mo_hinh, pipeline in cac_mo_hinh.items():
    pipeline.fit(X_train, y_train)
    y_du_doan_test = pipeline.predict(X_test)
    rmse = np.sqrt(mean_squared_error(y_test, y_du_doan_test))
    print(f"{ten_mo_hinh:32s}: RMSE trên tập kiểm tra = {rmse:.3f}")

    y_ve = pipeline.predict(X_ve)
    plt.plot(X_ve, y_ve, linewidth=2, label=ten_mo_hinh)

plt.ylim(-30, 30)
plt.xlabel("x")
plt.ylabel("y")
plt.title(f"So sánh hồi quy đa thức bậc {BAC_DA_THUC}: có và không chính quy hóa")
plt.legend()
plt.tight_layout()
plt.savefig("so_sanh_chinh_quy_hoa.png", dpi=120)
print("\nĐã lưu biểu đồ vào tệp so_sanh_chinh_quy_hoa.png")
```

5.4. Giải thích kết quả chạy chương trình

Kết quả in ra từ chương trình trên có dạng tương tự:

```
Không chính quy hóa (bậc 12)    : RMSE trên tập kiểm tra = 12.487
Ridge (alpha=5.0)                : RMSE trên tập kiểm tra = 3.102
Lasso (alpha=0.5)                : RMSE trên tập kiểm tra = 2.951

Đã lưu biểu đồ vào tệp so_sanh_chinh_quy_hoa.png
```

Kết quả cho thấy sự khác biệt rõ rệt: mô hình đa thức bậc 12 không chính quy hóa có RMSE trên tập kiểm tra rất cao (khoảng 12.5), phản ánh hiện tượng khớp quá nghiêm trọng — đường cong dự đoán "uốn lượn" dữ dội để đi qua gần hết các điểm nhiễu trong tập huấn luyện, dẫn đến dự đoán rất kém trên dữ liệu mới, đặc biệt ở các vùng biên. Trong khi đó, cả Ridge và Lasso đều giảm RMSE xuống còn khoảng 3, cho đường cong dự đoán mượt mà hơn nhiều, bám sát xu hướng bậc ba thực sự của dữ liệu gốc (`y_that`) thay vì bám theo từng điểm nhiễu cụ thể.

Nếu kiểm tra thêm số lượng hệ số khác 0 trong mô hình Lasso, sinh viên sẽ quan sát thấy nhiều hệ số ứng với các lũy thừa bậc cao (x^8, x^9, …, x^{12}) đã được đưa về chính xác bằng 0 — minh chứng trực quan cho tính chất lựa chọn đặc trưng tự động của Lasso đã trình bày ở mục 5.2.5, trong khi các hệ số tương ứng của Ridge tuy nhỏ nhưng vẫn khác 0.

5.5. Tóm tắt chương

- Hồi quy đa thức mở rộng hồi quy tuyến tính bằng cách thêm các lũy thừa của đặc trưng gốc, cho phép mô hình nắm bắt quan hệ phi tuyến trong khi vẫn tuyến tính theo trọng số w.
- Bậc đa thức càng cao, mô hình càng linh hoạt nhưng càng dễ khớp quá — minh họa cụ thể cho đánh đổi độ chệch – phương sai.
- Chính quy hóa bổ sung một số hạng phạt tỉ lệ với độ lớn trọng số vào hàm mất mát, giúp kiểm soát độ phức tạp mô hình và giảm khớp quá.
- Ridge (phạt L2) co các trọng số về gần 0 nhưng hiếm khi bằng 0 hẳn, phù hợp khi có đa cộng tuyến; Lasso (phạt L1) có thể đưa trọng số về đúng 0, thực hiện lựa chọn đặc trưng tự động.
- Elastic Net kết hợp cả hai loại phạt L1 và L2, cân bằng giữa khả năng lựa chọn đặc trưng của Lasso và tính ổn định của Ridge.
- Cần chuẩn hóa đặc trưng (ví dụ StandardScaler) trước khi áp dụng chính quy hóa để đảm bảo mức phạt công bằng giữa các đặc trưng có thang đo khác nhau.

5.6. Bài tập củng cố

Mức nhận biết

1. Viết công thức hàm mất mát của Ridge Regression và Lasso Regression. Chỉ ra điểm khác biệt duy nhất giữa hai công thức này.
2. Vì sao hồi quy đa thức vẫn được xem là một mô hình "tuyến tính" mặc dù đường cong dự đoán của nó có thể rất phi tuyến khi vẽ theo biến x gốc?

Mức thông hiểu

1. Giải thích vì sao Lasso có thể đưa một số hệ số về đúng bằng 0 trong khi Ridge thì hầu như không thể, dựa trên trực giác hình học của các chuẩn L1 và L2 (có thể mô tả bằng lời, không bắt buộc vẽ hình).
2. Một tập dữ liệu có 200 đặc trưng, trong đó theo hiểu biết chuyên môn chỉ khoảng 15 đặc trưng thực sự liên quan đến biến mục tiêu. Nên ưu tiên sử dụng Ridge hay Lasso? Giải thích lý do.
3. Giải thích vì sao cần chuẩn hóa dữ liệu trước khi áp dụng Ridge/Lasso nhưng bước này không bắt buộc (dù vẫn có lợi) đối với hồi quy tuyến tính thông thường không chính quy hóa.

Mức vận dụng

1. Cho một tập dữ liệu gồm 5 đặc trưng, trong đó đặc trưng x_2 và x_3 có tương quan rất chặt với nhau (tương quan Pearson > 0.95). Hãy dự đoán điều gì có thể xảy ra nếu dùng Lasso trên tập dữ liệu này (đối với hệ số của x_2 và x_3), và đề xuất một giải pháp thay thế phù hợp hơn.

5.7. Bài tập lập trình

Bài 1. Sử dụng lại dữ liệu tổng hợp ở ví dụ 5.1, thử nghiệm hồi quy Ridge với ít nhất 5 giá trị alpha khác nhau (ví dụ 0.01, 0.1, 1, 10, 100). Vẽ biểu đồ đường thể hiện RMSE trên tập kiểm tra (trục tung) theo giá trị alpha (trục hoành, dùng thang log cho trục hoành). Nhận xét xu hướng và xác định khoảng giá trị alpha cho kết quả tốt nhất.

Bài 2. Áp dụng Elastic Net (từ `sklearn.linear_model` import ElasticNet) trên cùng dữ liệu ở ví dụ 5.1 với alpha=0.5 và thử ba giá trị `l1_ratio` khác nhau (0.1, 0.5, 0.9). So sánh RMSE và số lượng hệ số bằng 0 giữa ba cấu hình này.

Bài 3 (nâng cao). Sử dụng bộ dữ liệu `fetch_california_housing()`, xây dựng đặc trưng đa thức bậc 2 từ toàn bộ 8 đặc trưng gốc (lưu ý: PolynomialFeatures bậc 2 trên 8 đặc trưng sẽ sinh ra nhiều đặc trưng tương tác mới). So sánh hiệu năng (RMSE, R^2) và thời gian huấn luyện giữa hồi quy tuyến tính thường, Ridge và Lasso trên tập đặc trưng mở rộng này. Nhận xét kết quả và giải thích vì sao chính quy hóa trở nên đặc biệt cần thiết khi số lượng đặc trưng tăng lên đáng kể.

Chương tiếp theo (Chương 6) sẽ chuyển từ bài toán hồi quy sang bài toán phân loại, bắt đầu với hồi quy Logistic — thuật toán phân loại nhị phân nền tảng và là cơ sở khái niệm cho nhiều mô hình phân loại phức tạp hơn được trình bày trong các chương tiếp theo.

Chương 6

Hồi quy Logistic và Phân loại nhị phân

6.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Giải thích được vì sao không nên dùng hồi quy tuyến tính trực tiếp cho bài toán phân loại.
2. Trình bày được hàm sigmoid và cách hồi quy Logistic sử dụng hàm này để mô hình hóa xác suất.
3. Giải thích được hàm mất mát Binary Cross-Entropy (Log Loss) và lý do nó phù hợp hơn MSE cho bài toán phân loại.
4. Hiểu được vai trò của ngưỡng quyết định (decision threshold) và ranh giới quyết định (decision boundary).
5. Tính toán và diễn giải được ma trận nhầm lẫn (confusion matrix) cùng các độ đo Precision, Recall, F1-score.
6. Cài đặt được mô hình hồi quy Logistic bằng Scikit-learn, đánh giá và trực quan hóa đường cong ROC.

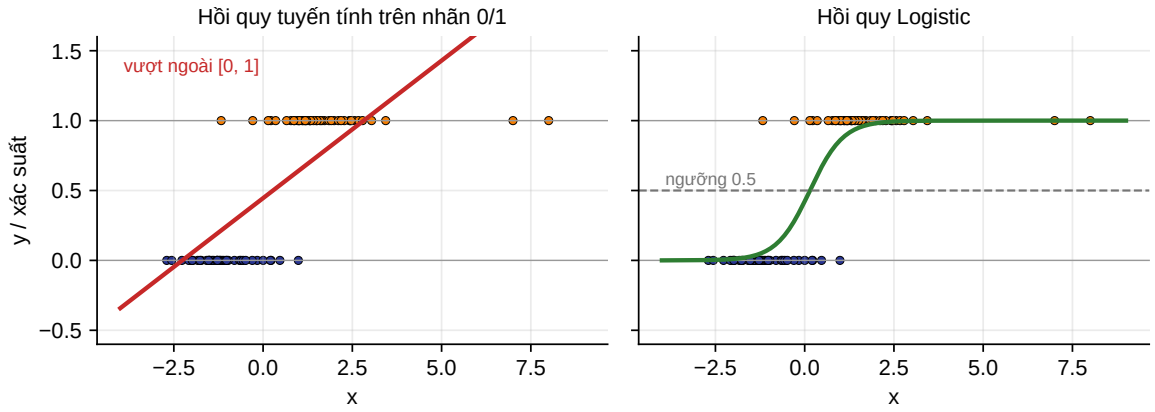
6.2. Nội dung lý thuyết

6.2.1. Vì sao không dùng hồi quy tuyến tính cho bài toán phân loại?

Ở bài toán phân loại nhị phân, biến mục tiêu y chỉ nhận một trong hai giá trị rời rạc, thường được mã hóa là 0 (lớp âm) và 1 (lớp dương) — ví dụ: email không phải spam / spam, khối u lành tính / ác tính. Nếu áp dụng trực tiếp hồi quy tuyến tính cho bài toán này, mô hình sẽ cho ra các giá trị dự đoán $\hat{y}$ là số thực bất kỳ, có thể nhỏ hơn 0 hoặc lớn hơn 1 — những giá trị này không thể diễn giải một cách tự nhiên như xác suất thuộc lớp dương. Ngoài ra, đường thẳng hồi quy tuyến tính rất nhạy cảm với các điểm dữ liệu ở xa (outlier theo trục hoành), dễ khiến ranh giới phân loại bị lệch một cách không hợp lý.

Hồi quy Logistic (Logistic Regression, dù tên gọi có chữ "hồi quy" nhưng thực chất là một thuật toán phân loại) giải quyết vấn đề này bằng cách "nén" đầu ra của mô hình tuyến tính vào khoảng (0, 1) thông qua hàm sigmoid, để giá trị đầu ra có thể diễn giải như một xác suất.

Hình 6.1 minh họa hồi quy tuyến tính cho giá trị vượt ngoài $[0, 1]$, trong khi hồi quy Logistic luôn cho xác suất hợp lệ.



Hình 6.1. Hồi quy tuyến tính và hồi quy Logistic trên dữ liệu nhị phân.

6.2.2. Hàm Sigmoid

Hàm sigmoid (còn gọi là hàm logistic) có công thức:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Hàm này có dạng đường cong chữ S đặc trưng: khi z tiến tới $+\infty$, σ(z) tiến tới 1; khi z tiến tới $-\infty$, σ(z) tiến tới 0; và $\sigma(0) = 0.5$. Mô hình hồi quy Logistic tính z như một tổ hợp tuyến tính quen thuộc của các đặc trưng đầu vào, sau đó đưa qua hàm sigmoid để thu được xác suất dự đoán:

$$z = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$$

$$\hat{p} = \sigma(z) = P(y = 1 \mid \mathbf{x})$$

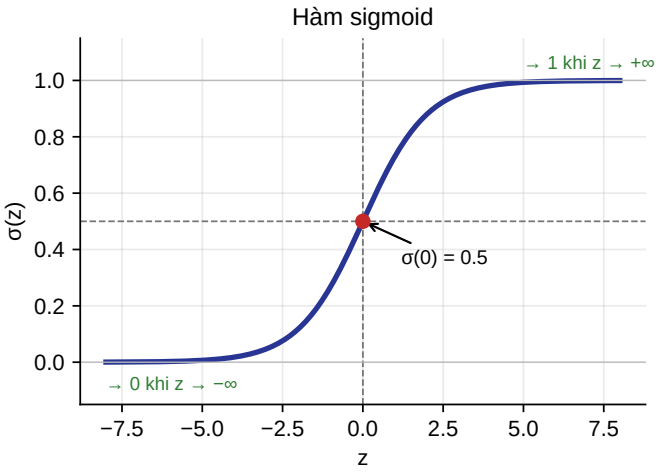
trong đó $\hat{p}$ là xác suất dự đoán mẫu dữ liệu thuộc lớp dương (y = 1). Để đưa ra quyết định phân loại cuối cùng (0 hoặc 1), ta so sánh $\hat{p}$ với một ngưỡng quyết định (decision threshold), thường mặc định là 0.5: nếu $\hat{p} \geq 0.5$ thì dự đoán lớp 1, ngược lại dự đoán lớp 0. Ngưỡng này có thể điều chỉnh tùy theo yêu cầu bài toán, ví dụ trong sàng lọc y tế có thể hạ ngưỡng xuống thấp hơn 0.5 để ưu tiên không bỏ sót ca bệnh (đánh đổi lấy nhiều cảnh báo giả hơn).

Hình 6.2 minh họa đồ thị hàm sigmoid với các giá trị đặc trưng.

6.2.3. Hàm mất mát: Binary Cross-Entropy (Log Loss)

Nếu tiếp tục sử dụng MSE làm hàm mất mát cho hồi quy Logistic, hàm mất mát thu được sẽ không lồi (non-convex) theo các trọng số w, khiến Gradient Descent dễ bị mắc kẹt tại các điểm cực tiểu cục bộ thay vì tìm được nghiệm tối ưu toàn cục. Do đó, hồi quy Logistic sử dụng một hàm mất mát khác — Binary Cross-Entropy, còn gọi là Log Loss:

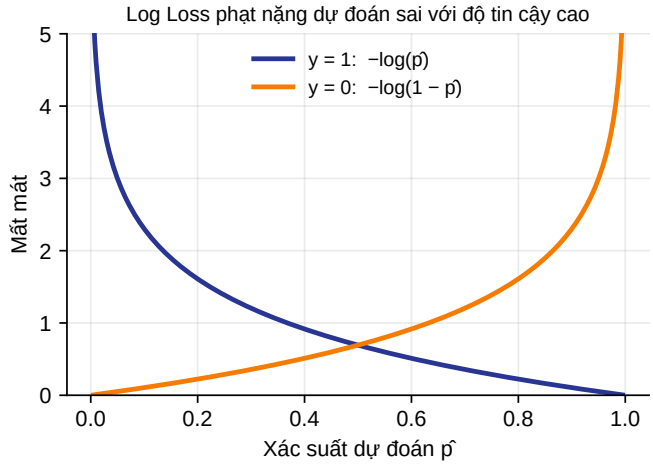
$$L = -\frac{1}{m} \sum_{i=1}^{m} \Big[y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i) \Big]$$



Hình 6.2. Hàm sigmoid $\sigma(z) = 1/(1 + e^{-z})$.

Trực giác của công thức này: khi nhãn thực tế $y_i = 1$, số hạng còn lại triệt tiêu và hàm mất mát chỉ còn $-\log(\hat{p}_i)$ — nếu mô hình dự đoán $\hat{p}_i$ gần 1 (đúng), giá trị $-\log(\hat{p}_i)$ gần 0 (mất mát thấp); nếu mô hình dự đoán $\hat{p}_i$ gần 0 (sai hoàn toàn), giá trị $-\log(\hat{p}_i)$ tiến tới vô cùng (mất mát rất lớn). Tương tự với trường hợp $y_i = 0$. Hàm mất mát này vừa lồi (đảm bảo tối ưu hội tụ tốt bằng Gradient Descent), vừa phạt rất nặng các dự đoán sai với độ tin cậy cao — một đặc tính mong muốn cho bài toán phân loại xác suất.

Hình 6.3 minh họa Log Loss phạt rất nặng khi mô hình tự tin nhưng dự đoán sai.



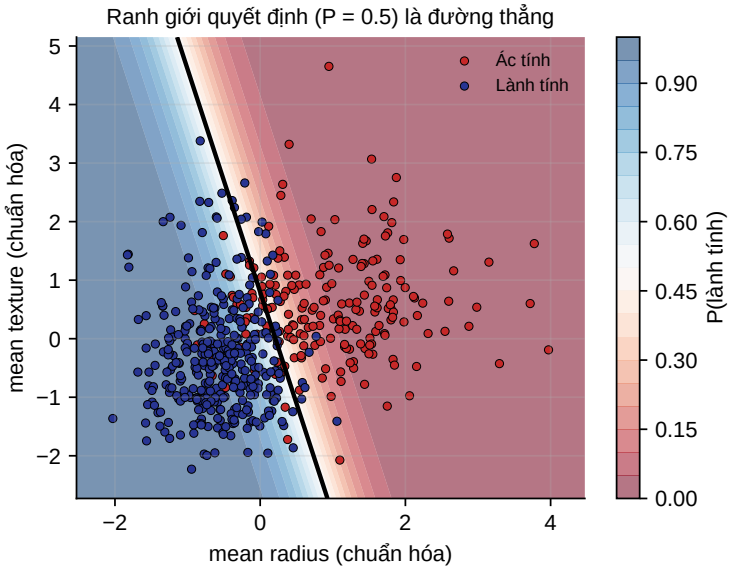
Hình 6.3. Hàm mất mát Binary Cross-Entropy cho từng nhãn thực tế.

6.2.4. Ranh giới quyết định (Decision Boundary)

Trong không gian đặc trưng, tập hợp các điểm mà tại đó mô hình dự đoán xác suất đúng bằng ngưỡng quyết định (thường là $\hat{p} = 0.5$, tương đương $z = 0$) tạo thành ranh giới quyết định. Với hồi quy Logistic, do z là một tổ hợp tuyến tính của các đặc trưng, ranh giới quyết định luôn có dạng một đường thẳng (trong không gian 2 chiều), một mặt phẳng (3 chiều), hoặc tổng quát hơn là một siêu phẳng (hyperplane) trong không gian nhiều chiều. Đây là lý do hồi quy Logistic

được xếp vào nhóm các bộ phân loại tuyến tính (linear classifier) — nó chỉ có thể phân tách tốt các lớp dữ liệu có thể tách biệt (gần đúng) bằng một đường/mặt phẳng thẳng. Với dữ liệu có ranh giới phân tách phức tạp, phi tuyến, cần đến các mô hình mạnh hơn sẽ được giới thiệu ở các chương sau (Cây quyết định, SVM với kernel phi tuyến, mạng nơ-ron).

Hình 6.4 minh họa ranh giới quyết định của hồi quy Logistic luôn là một đường thẳng, kèm bản đồ xác suất.



Hình 6.4. Ranh giới quyết định và xác suất dự đoán trên hai đặc trưng của bộ Breast Cancer.

6.2.5. Ma trận nhầm lẫn và các độ đo phân loại

Để đánh giá một mô hình phân loại nhị phân, công cụ nền tảng là ma trận nhầm lẫn (confusion matrix), tổng hợp bốn trường hợp có thể xảy ra khi so sánh dự đoán với nhãn thực tế:

	Dự đoán: Dương (1)	**Dự đoán: Âm (0)**
Thực tế: Dương (1)	True Positive (TP) — dự đoán đúng lớp dương	False Negative (FN) — bỏ sót lớp dương
Thực tế: Âm (0)	False Positive (FP) — báo động giả	True Negative (TN) — dự đoán đúng lớp âm

Từ bốn giá trị TP, FP, TN, FN, ta tính được các độ đo quan trọng sau:

- Accuracy (Độ chính xác) = (TP + TN) / (TP + TN + FP + FN): tỉ lệ dự đoán đúng trên tổng số mẫu — dễ hiểu nhưng có thể gây hiểu lầm khi dữ liệu mất cân bằng lớp nghiêm trọng.
- Precision (Độ chuẩn xác) = TP / (TP + FP): trong số các mẫu được dự đoán là dương, bao nhiêu phần trăm thực sự là dương — quan trọng khi chi phí của một báo động giả (FP) cao, ví dụ hệ thống lọc spam không muốn chặn nhầm email quan trọng.

- Recall (Độ bao phủ, còn gọi là Sensitivity) = TP / (TP + FN): trong số các mẫu thực sự là dương, bao nhiêu phần trăm được mô hình phát hiện ra — quan trọng khi chi phí của việc bỏ sót (FN) cao, ví dụ chẩn đoán ung thư không muốn bỏ sót ca bệnh thực sự.
- F1-score = 2 × (Precision × Recall) / (Precision + Recall): trung bình điều hòa (harmonic mean) của Precision và Recall, hữu ích khi cần một độ đo tổng hợp cân bằng cả hai, đặc biệt khi dữ liệu mất cân bằng lớp.

Việc lựa chọn độ đo nào là quan trọng nhất phụ thuộc hoàn toàn vào bài toán và chi phí tương đối giữa hai loại sai lầm (FP và FN) — không có một độ đo "vạn năng" phù hợp cho mọi tình huống. Nội dung này sẽ được mở rộng chi tiết hơn ở Chương 15 (bao gồm cả đường cong ROC và chỉ số AUC để đánh giá mô hình trên toàn dải ngưỡng quyết định thay vì chỉ một ngưỡng cố định).

6.3. Ví dụ minh họa bằng mã Python

Ví dụ dưới đây xây dựng một mô hình hồi quy Logistic để phân loại khối u vú lành tính hay ác tính, dựa trên bộ dữ liệu Breast Cancer Wisconsin có sẵn trong Scikit-learn, đồng thời minh họa cách tính và diễn giải ma trận nhầm lẫn cùng các độ đo phân loại.

```
# vidu_6_1.py
# Hồi quy Logistic: phân loại khối u vú lành tính / ác tính

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    confusion_matrix, classification_report, ConfusionMatrixDisplay,
    RocCurveDisplay, roc_auc_score
)

# 1. Nạp dữ liệu
du_lieu = load_breast_cancer()
X, y = du_lieu.data, du_lieu.target
# Quy ước của bộ dữ liệu này: 0 = ác tính (malignant), 1 = lành tính (benign)
print("Số mẫu:", X.shape[0], "| Số đặc trưng:", X.shape[1])
print("Phân bố lớp:", dict(zip(*np.unique(y, return_counts=True))))

# 2. Chia tập huấn luyện / kiểm tra (giữ nguyên tỉ lệ lớp bằng stratify)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# 3. Chuẩn hóa đặc trưng - quan trọng đối với hồi quy Logistic
```

```
#    vì thuật toán tối ưu (dựa trên Gradient Descent) hội tụ nhanh và ổn định hơn
#    khi các đặc trưng có cùng thang đo
bo_chuan_hoa = StandardScaler()
X_train_chuan_hoa = bo_chuan_hoa.fit_transform(X_train)
X_test_chuan_hoa = bo_chuan_hoa.transform(X_test)

# 4. Khởi tạo và huấn luyện mô hình hồi quy Logistic
mo_hinh = LogisticRegression(max_iter=1000)
mo_hinh.fit(X_train_chuan_hoa, y_train)

# 5. Dự đoán nhãn và xác suất trên tập kiểm tra
y_du_doan = mo_hinh.predict(X_test_chuan_hoa)
xac_suat_du_doan = mo_hinh.predict_proba(X_test_chuan_hoa)[:, 1]  # xác suất lớp 1

# 6. Ma trận nhầm lẫn và báo cáo phân loại chi tiết
ma_tran_nham_lan = confusion_matrix(y_test, y_du_doan)
print("\nMa trận nhầm lẫn:")
print(ma_tran_nham_lan)
print("\nBáo cáo phân loại chi tiết:")
print(classification_report(y_test, y_du_doan, target_names=du_lieu.target_names))

# 7. Chỉ số AUC (diện tích dưới đường cong ROC)
auc = roc_auc_score(y_test, xac_suat_du_doan)
print(f"AUC (Area Under ROC Curve) = {auc:.4f}")

# 8. Trực quan hóa: ma trận nhầm lẫn và đường cong ROC
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))
ConfusionMatrixDisplay(ma_tran_nham_lan, display_labels=du_lieu.target_names).plot(
    ax=axes[0], cmap="Blues", colorbar=False
)
axes[0].set_title("Ma trận nhầm lẫn")

RocCurveDisplay.from_predictions(y_test, xac_suat_du_doan, ax=axes[1])
axes[1].plot([0, 1], [0, 1], linestyle="--", color="gray", label="Phân loại ngẫu
nhiên")
axes[1].set_title("Đường cong ROC")
axes[1].legend()

plt.tight_layout()
plt.savefig("hoi_quy_logistic_breast_cancer.png", dpi=120)
print("\nĐã lưu biểu đồ vào tệp hoi_quy_logistic_breast_cancer.png")
```

6.4. Giải thích kết quả chạy chương trình

Kết quả in ra từ chương trình trên có dạng tương tự:

```
Số mẫu: 569 | Số đặc trưng: 30
Phân bố lớp: {0: 212, 1: 357}
```

```
Ma trận nhầm lẫn:
[[51  2]
 [ 2 88]]

Báo cáo phân loại chi tiết:
              precision    recall  f1-score   support
   malignant       0.96      0.96      0.96        53
      benign       0.98      0.98      0.98        90

AUC (Area Under ROC Curve) = 0.9974
```

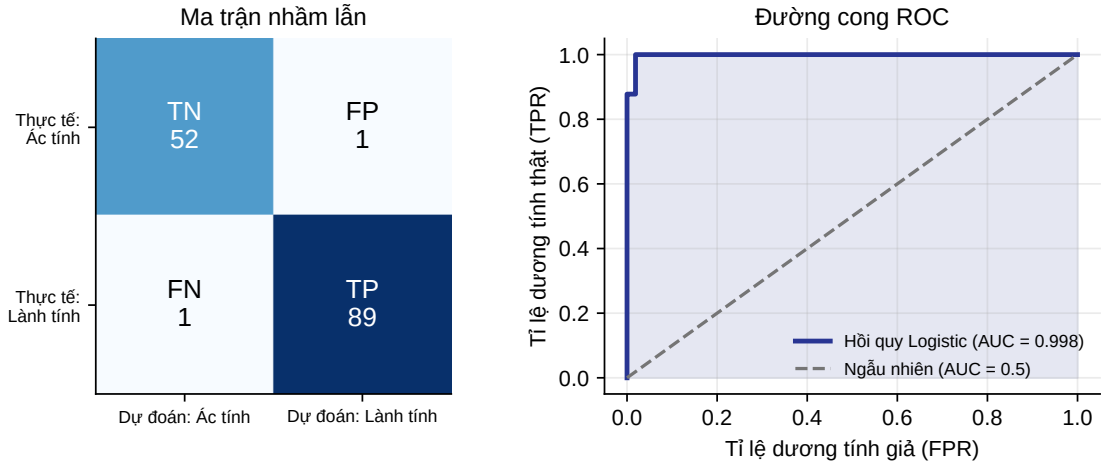
Ma trận nhầm lẫn cho thấy: trong số 53 khối u ác tính thực sự trong tập kiểm tra, mô hình phát hiện đúng 51 ca (True Negative theo quy ước nhãn 0 = ác tính) và bỏ sót 2 ca (dự đoán nhầm thành lành tính — False Positive theo quy ước nhãn); trong số 90 khối u lành tính, mô hình dự đoán đúng 88 ca và nhầm 2 ca thành ác tính. Precision và Recall đều đạt trên 0.96 cho cả hai lớp, cho thấy mô hình hoạt động rất tốt và khá cân bằng giữa hai loại sai lầm.

Chỉ số AUC gần 1.0 (0.9974) cho thấy mô hình có khả năng phân tách hai lớp gần như hoàn hảo trên toàn dải ngưỡng quyết định, không chỉ tại ngưỡng mặc định 0.5 — đây là một chỉ số đáng tin cậy hơn Accuracy đơn thuần vì nó không phụ thuộc vào việc lựa chọn ngưỡng cụ thể nào.

Lưu ý sư phạm

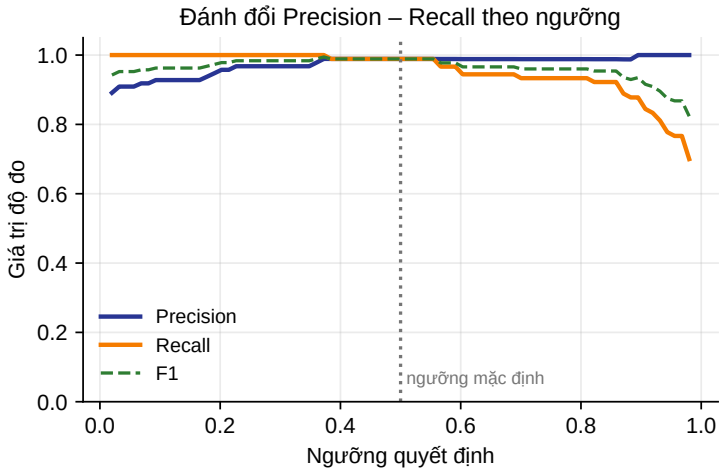
Trong bài toán y tế như chẩn đoán ung thư, việc bỏ sót một ca bệnh thực sự (False Negative — dự đoán nhầm ca ác tính thành lành tính) thường có hậu quả nghiêm trọng hơn nhiều so với một báo động giả (False Positive — nghi ngờ nhầm một ca lành tính). Trong thực hành lâm sàng, bác sĩ có thể chủ động hạ thấp ngưỡng quyết định (ví dụ từ 0.5 xuống 0.3) để tăng Recall (giảm bỏ sót), chấp nhận đánh đổi Precision thấp hơn (nhiều báo động giả hơn, cần xét nghiệm bổ sung để loại trừ). Đây là một quyết định mang tính nghiệp vụ, không thể chỉ dựa thuần túy vào số liệu thống kê.

Hình 6.5 minh họa ma trận nhầm lẫn và đường cong ROC của mô hình.



Hình 6.5. Ma trận nhầm lẫn (trái) và đường cong ROC (phải).

Hình 6.6 minh họa việc thay đổi ngưỡng quyết định làm Precision và Recall biến thiên theo hai chiều ngược nhau.



Hình 6.6. Precision, Recall và F1 theo ngưỡng quyết định.

6.5. Tóm tắt chương

- Hồi quy Logistic sử dụng hàm sigmoid để "nén" đầu ra tuyến tính vào khoảng (0, 1), diễn giải như xác suất thuộc lớp dương.
- Hàm mất mát Binary Cross-Entropy (Log Loss) được dùng thay cho MSE vì cho hàm mất mát lồi và phạt nặng các dự đoán sai với độ tin cậy cao.
- Ranh giới quyết định của hồi quy Logistic luôn là một đường thẳng/siêu phẳng, khiến nó thuộc nhóm bộ phân loại tuyến tính.
- Ma trận nhầm lẫn (TP, FP, TN, FN) là công cụ nền tảng để tính các độ đo Precision, Recall, F1-score — mỗi độ đo phù hợp với một loại ưu tiên nghiệp vụ khác nhau.
- Ngưỡng quyết định (mặc định 0.5) có thể điều chỉnh tùy theo chi phí tương đối giữa việc bỏ sót (False Negative) và báo động giả (False Positive) trong từng bài toán cụ thể.
- Chỉ số AUC đo khả năng phân tách lớp của mô hình trên toàn dải ngưỡng, không phụ thuộc vào một ngưỡng cố định.

6.6. Bài tập củng cố

Mức nhận biết

1. Viết công thức hàm sigmoid và nêu giá trị của hàm này khi z = 0, khi z rất lớn, và khi z rất nhỏ (âm lớn).

2. Nêu công thức tính Precision và Recall từ ma trận nhầm lẫn. Giải thích ý nghĩa của mỗi độ đo bằng lời của riêng bạn.

Mức thông hiểu

1. Giải thích vì sao không nên dùng MSE làm hàm mất mát cho hồi quy Logistic, trong khi vẫn dùng được cho hồi quy tuyến tính.
2. Một mô hình phát hiện gian lận thẻ tín dụng có Accuracy 99.5% nhưng Recall chỉ 40%. Giải thích vì sao Accuracy cao trong trường hợp này có thể gây hiểu lầm, biết rằng chỉ khoảng 0.5% giao dịch là gian lận thực sự.
3. Nếu hạ ngưỡng quyết định từ 0.5 xuống 0.2, Precision và Recall của mô hình thường thay đổi theo xu hướng nào? Giải thích lý do.

Mức vận dụng

1. Cho hai bài toán: (a) lọc email spam, (b) sàng lọc hành khách nguy hiểm tại sân bay. Với mỗi bài toán, hãy xác định độ đo (Precision hay Recall) nên được ưu tiên hơn, và đề xuất hướng điều chỉnh ngưỡng quyết định phù hợp. Giải thích lý do dựa trên chi phí tương đối của False Positive so với False Negative trong từng tình huống.

6.7. Bài tập lập trình

Bài 1. Thử nghiệm lại ví dụ 6.1 nhưng không chuẩn hóa dữ liệu đầu vào (bỏ qua bước StandardScaler). So sánh số vòng lặp cần thiết để mô hình hội tụ (kiểm tra qua cảnh báo ConvergenceWarning nếu có) và kết quả Accuracy/AUC so với mô hình có chuẩn hóa. Nhận xét vai trò của việc chuẩn hóa đối với hồi quy Logistic.

Bài 2. Viết chương trình thử nghiệm các ngưỡng quyết định khác nhau (0.1, 0.3, 0.5, 0.7, 0.9) trên xác suất dự đoán thu được từ mô hình ở ví dụ 6.1 (sử dụng `xac_suat_du_doan` và áp ngưỡng thủ công thay vì dùng `predict()` mặc định). Với mỗi ngưỡng, tính và in ra Precision, Recall, F1-score. Vẽ biểu đồ đường thể hiện Precision và Recall (hai đường riêng biệt) theo ngưỡng quyết định.

Bài 3 (nâng cao). Sử dụng bộ dữ liệu `make_classification()` của Scikit-learn để tạo một bộ dữ liệu phân loại nhị phân mất cân bằng nghiêm trọng (tham số weights=[0.95, 0.05], tức 95% lớp 0 và 5% lớp 1). Huấn luyện hồi quy Logistic mặc định và so sánh với hồi quy Logistic có tham số `class_weight="balanced"`. So sánh ma trận nhầm lẫn và F1-score của lớp thiểu số (lớp 1) giữa hai cấu hình, nhận xét hiệu quả của việc cân bằng trọng số lớp.

Chương tiếp theo (Chương 7) sẽ giới thiệu thuật toán K-Láng giềng gần nhất (K-NN) — một phương pháp phân loại (và hồi quy) hoàn toàn khác về nguyên lý so với hồi quy Logistic: thay vì học một hàm số tường minh, K-NN đưa ra dự đoán dựa trực tiếp trên các mẫu dữ liệu huấn luyện "gần" nhất với mẫu cần dự đoán.

Chương 7

Thuật toán K-Láng giềng gần nhất (K-NN)

7.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Trình bày được nguyên lý hoạt động của thuật toán K-NN cho cả bài toán phân loại và hồi quy.
2. Phân biệt được K-NN với các mô hình "học có tham số" (parametric) như hồi quy tuyến tính/Logistic, hiểu khái niệm học dựa trên tình huống (instance-based learning).
3. Tính toán được khoảng cách Euclidean, Manhattan giữa hai điểm dữ liệu và giải thích ảnh hưởng của việc lựa chọn độ đo khoảng cách.
4. Phân tích được ảnh hưởng của giá trị k đến hiện tượng khớp thiếu/khớp quá.
5. Giải thích được vì sao chuẩn hóa dữ liệu là bắt buộc gần như tuyệt đối đối với K-NN.
6. Cài đặt, khảo sát và trực quan hóa ranh giới quyết định của mô hình K-NN bằng Scikit-learn.

7.2. Nội dung lý thuyết

7.2.1. Nguyên lý hoạt động của K-NN

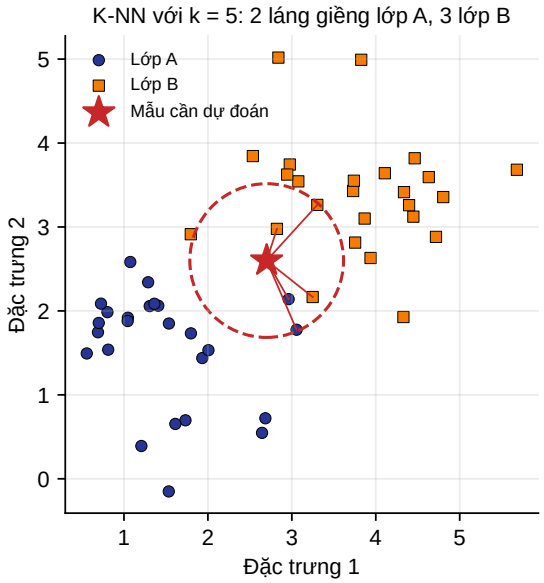
K-Láng giềng gần nhất (K-Nearest Neighbors – K-NN) là một trong những thuật toán học máy đơn giản và trực quan nhất, dựa trên một giả định tự nhiên: các điểm dữ liệu "giống nhau" (gần nhau trong không gian đặc trưng) thường có nhãn giống nhau. Khác với hồi quy tuyến tính hay hồi quy Logistic — vốn học ra một bộ tham số w cố định trong giai đoạn huấn luyện rồi bỏ qua dữ liệu huấn luyện khi dự đoán — K-NN không có giai đoạn "học" tường minh theo nghĩa tối ưu hóa tham số. Thay vào đó, toàn bộ tập dữ liệu huấn luyện được lưu lại nguyên vẹn, và với mỗi mẫu mới cần dự đoán, thuật toán mới thực hiện tính toán: tìm ra k mẫu huấn luyện gần mẫu mới nhất (theo một độ đo khoảng cách nào đó), rồi tổng hợp nhãn của k mẫu đó để đưa ra dự đoán.

Vì lý do này, K-NN được xếp vào nhóm thuật toán học dựa trên tình huống (instance-based learning) hay học lười (lazy learning) — đối lập với các mô hình học có tham số (parametric models) như hồi quy tuyến tính/Logistic, vốn "tóm tắt" dữ liệu huấn luyện thành một số lượng tham số cố định (các trọng số w) và không cần lưu giữ dữ liệu gốc sau khi huấn luyện.

7.2.2. Các bước dự đoán của K-NN

- Bước 1: với một mẫu dữ liệu mới cần dự đoán, tính khoảng cách từ mẫu này đến tất cả các mẫu trong tập huấn luyện.
- Bước 2: chọn ra k mẫu huấn luyện có khoảng cách nhỏ nhất (gần nhất).
- Bước 3 (phân loại): lấy nhãn xuất hiện nhiều nhất (bình chọn đa số — majority voting) trong số k láng giềng làm nhãn dự đoán.
- Bước 3 (hồi quy): lấy trung bình (hoặc trung bình có trọng số) giá trị mục tiêu của k láng giềng làm giá trị dự đoán.

Hình 7.1 minh họa cách K-NN dự đoán: tìm k láng giềng gần nhất rồi bình chọn.



Hình 7.1. K-NN với $k = 5$: mẫu mới (ngôi sao) được gán theo đa số láng giềng.

7.2.3. Các độ đo khoảng cách phổ biến

Khoảng cách Euclidean (khoảng cách đường chim bay) là lựa chọn mặc định phổ biến nhất, tính theo công thức:

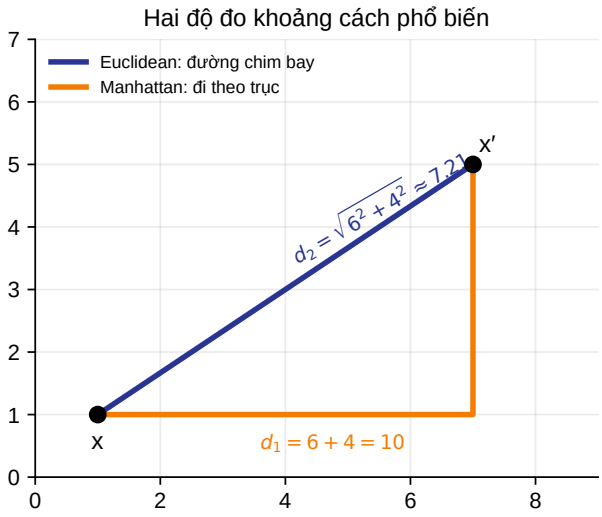
$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_i (x_i - x'_i)^2}$$

Khoảng cách Manhattan (còn gọi là khoảng cách khối phố — city block distance) tính tổng các hiệu tuyệt đối theo từng chiều, phù hợp hơn khi dữ liệu có nhiều chiều rời rạc hoặc khi ta muốn giảm ảnh hưởng của các chênh lệch lớn ở một vài chiều đơn lẻ:

$$d(\mathbf{x}, \mathbf{x}') = \sum_i |x_i - x'_i|$$

Cả hai công thức trên là trường hợp đặc biệt của khoảng cách Minkowski tổng quát (với tham số p = 2 cho Euclidean, p = 1 cho Manhattan). Việc lựa chọn độ đo khoảng cách phù hợp phụ thuộc vào bản chất dữ liệu và đôi khi cần thử nghiệm thực tế để xác định lựa chọn tốt nhất.

Hình 7.2 minh họa hai độ đo khoảng cách phổ biến.



Hình 7.2. Khoảng cách Euclidean và Manhattan giữa hai điểm.

7.2.4. Vai trò của giá trị k

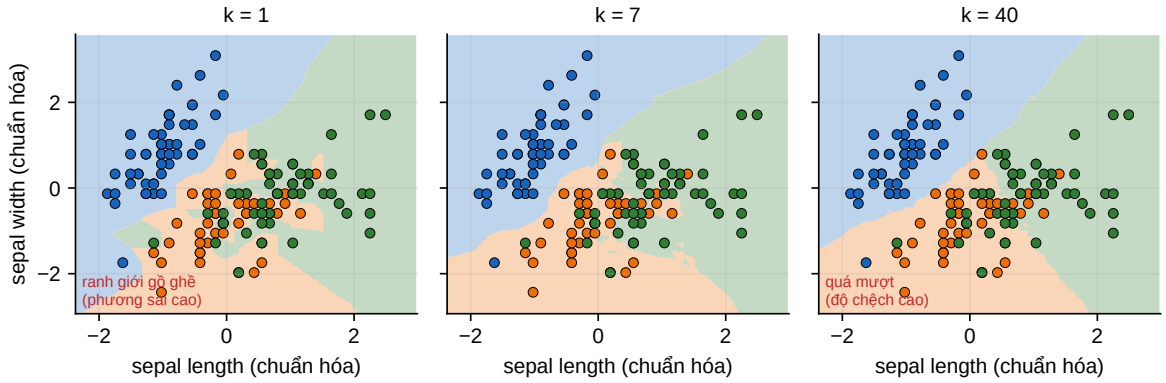
Giá trị k (số lượng láng giềng xem xét) là siêu tham số quan trọng nhất của K-NN, ảnh hưởng trực tiếp đến độ phức tạp của mô hình:

- k nhỏ (ví dụ k = 1): mô hình rất nhạy cảm với từng điểm dữ liệu huấn luyện riêng lẻ, kể cả nhiễu — ranh giới quyết định trở nên rất gồ ghề, phức tạp, dễ dẫn đến khớp quá (phương sai cao).

- k lớn (ví dụ k gần bằng toàn bộ số mẫu huấn luyện): dự đoán bị "làm mượt" quá mức, gần như mọi mẫu mới đều được dự đoán theo nhãn chiếm đa số toàn cục, dẫn đến khớp thiếu (độ chệch cao).

Cũng giống bậc đa thức ở Chương 5, giá trị k tối ưu thường được xác định bằng kiểm định chéo (Chương 15), thử nghiệm trên một dải giá trị k và chọn giá trị cho hiệu năng tốt nhất trên tập kiểm định. Theo kinh nghiệm thực hành, k thường được chọn là số lẻ (khi phân loại nhị phân) để tránh tình huống hòa phiếu khi bình chọn đa số.

Hình 7.3 minh họa k nhỏ cho ranh giới gồ ghề (khớp quá), k lớn cho ranh giới quá mượt (khớp thiếu).



Hình 7.3. Ranh giới quyết định của K-NN với $k = 1, 7, 40$ (bộ Iris).

7.2.5. Tầm quan trọng của việc chuẩn hóa dữ liệu

Lưu ý kỹ thuật quan trọng

Vì K-NN dựa hoàn toàn vào khoảng cách giữa các điểm dữ liệu, các đặc trưng có thang đo (đơn vị) lớn hơn sẽ áp đảo hoàn toàn các đặc trưng có thang đo nhỏ hơn khi tính khoảng cách, bất kể mức độ quan trọng thực sự của từng đặc trưng đối với bài toán. Ví dụ, nếu một đặc trưng là "thu nhập" (đơn vị: triệu đồng, dao động 0–500) và một đặc trưng khác là "số năm kinh nghiệm" (đơn vị: năm, dao động 0–40), khoảng cách Euclidean sẽ gần như chỉ phản ánh sự khác biệt về thu nhập, còn số năm kinh nghiệm gần như không có ảnh hưởng. Do đó, việc chuẩn hóa dữ liệu (StandardScaler hoặc MinMaxScaler) trước khi áp dụng K-NN không chỉ là một thực hành tốt như với hồi quy Logistic, mà gần như là điều kiện bắt buộc để mô hình hoạt động có ý nghĩa.

7.2.6. Ưu điểm, nhược điểm và K-NN có trọng số

Ưu điểm nổi bật của K-NN là tính đơn giản, dễ hiểu, không cần giả định về dạng phân bố dữ liệu, và có thể áp dụng cho cả bài toán phân loại lẫn hồi quy với cùng một nguyên lý. Tuy nhiên, K-NN cũng có một số nhược điểm đáng chú ý: chi phí tính toán tại thời điểm dự đoán rất cao khi tập dữ liệu huấn luyện lớn (phải tính khoảng cách đến toàn bộ các mẫu huấn luyện cho mỗi dự đoán mới, dù có thể tăng tốc bằng các cấu trúc dữ liệu chuyên dụng như KD-Tree hoặc Ball-Tree); và hiệu năng suy giảm đáng kể trong không gian nhiều chiều (curse of dimensionality — khi số chiều tăng, khái niệm "gần" trở nên kém ý nghĩa vì hầu hết các điểm đều trở nên "xa" nhau một cách tương đối đồng đều).

Một biến thể phổ biến là K-NN có trọng số theo khoảng cách (distance-weighted K-NN): thay vì mọi láng giềng trong số k láng giềng đóng góp như nhau vào bình chọn/trung bình, các láng giềng gần hơn được gán trọng số lớn hơn (thường tỉ lệ nghịch với khoảng cách). Cách tiếp cận này thường cho kết quả mượt mà và ổn định hơn so với bình chọn đồng đều thông thường, đặc biệt khi k tương đối lớn.

7.3. Ví dụ minh họa bằng mã Python

Ví dụ dưới đây khảo sát ảnh hưởng của giá trị k đến hiệu năng mô hình trên bộ dữ liệu Wine (phân loại giống rượu vang dựa trên các chỉ số hóa học), đồng thời minh họa tầm quan trọng của việc chuẩn hóa dữ liệu.

```
# vidu_7_1.py
# Khảo sát ảnh hưởng của k và vai trò của chuẩn hóa dữ liệu đối với K-NN

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. Nạp dữ liệu và chia tập huấn luyện / kiểm tra
du_lieu = load_wine()
X, y = du_lieu.data, du_lieu.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# 2. Chuẩn bị hai phiên bản dữ liệu: có và không chuẩn hóa
bo_chuan_hoa = StandardScaler()
X_train_chuan_hoa = bo_chuan_hoa.fit_transform(X_train)
X_test_chuan_hoa = bo_chuan_hoa.transform(X_test)

# 3. Khảo sát độ chính xác theo giá trị k, với dữ liệu KHÔNG chuẩn hóa
cac_gia_tri_k = list(range(1, 22, 2))   # k lẻ từ 1 đến 21
do_chinh_xac_khong_chuan_hoa = []
do_chinh_xac_co_chuan_hoa = []

for k in cac_gia_tri_k:
    # Không chuẩn hóa
    mo_hinh_1 = KNeighborsClassifier(n_neighbors=k)
    mo_hinh_1.fit(X_train, y_train)
    do_chinh_xac_khong_chuan_hoa.append(
        accuracy_score(y_test, mo_hinh_1.predict(X_test))
    )
    # Có chuẩn hóa
    mo_hinh_2 = KNeighborsClassifier(n_neighbors=k)
    mo_hinh_2.fit(X_train_chuan_hoa, y_train)
    do_chinh_xac_co_chuan_hoa.append(
        accuracy_score(y_test, mo_hinh_2.predict(X_test_chuan_hoa))
    )

# 4. In kết quả dạng bảng
```

```
print(f"{'k':>4s} | {'Không chuẩn hóa':>16s} | {'Có chuẩn hóa':>14s}")
print("-" * 40)
for k, dcx1, dcx2 in zip(cac_gia_tri_k, do_chinh_xac_khong_chuan_hoa,
do_chinh_xac_co_chuan_hoa):
    print(f"{k:>4d} | {dcx1:>16.4f} | {dcx2:>14.4f}")

# 5. Trực quan hóa
plt.figure(figsize=(7, 5))
plt.plot(cac_gia_tri_k, do_chinh_xac_khong_chuan_hoa, marker="o", label="Không
chuẩn hóa")
plt.plot(cac_gia_tri_k, do_chinh_xac_co_chuan_hoa, marker="s", label="Có chuẩn
hóa")
plt.xlabel("Giá trị k")
plt.ylabel("Độ chính xác trên tập kiểm tra")
plt.title("Ảnh hưởng của k và việc chuẩn hóa dữ liệu đến K-NN")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("khao_sat_knn_wine.png", dpi=120)
print("\nĐã lưu biểu đồ vào tệp khao_sat_knn_wine.png")
```

7.4. Giải thích kết quả chạy chương trình

Kết quả in ra từ chương trình trên có dạng tương tự:

```
   k | Không chuẩn hóa |   Có chuẩn hóa
----------------------------------------
   1 |           0.7407 |         0.9630
   3 |           0.6852 |         0.9630
   5 |           0.7037 |         0.9630
   7 |           0.7222 |         0.9630
   9 |           0.7222 |         0.9444
  11 |           0.7222 |         0.9444
  ...
  21 |           0.6667 |         0.9074

Đã lưu biểu đồ vào tệp khao_sat_knn_wine.png
```

Kết quả cho thấy sự chênh lệch rất lớn giữa hai phiên bản dữ liệu: khi không chuẩn hóa, độ chính xác của K-NN chỉ dao động quanh 67–74%, trong khi khi có chuẩn hóa, độ chính xác đạt trên 90%, thậm chí trên 96% với k nhỏ. Nguyên nhân là bộ dữ liệu Wine có các đặc trưng với thang đo rất khác nhau (ví dụ đặc trưng "proline" có giá trị hàng trăm đến hàng nghìn, trong khi các đặc trưng khác chỉ dao động trong khoảng 0–10); khi không chuẩn hóa, khoảng cách Euclidean gần như chỉ bị chi phối bởi đặc trưng có giá trị tuyệt đối lớn nhất, khiến các đặc trưng còn lại gần như bị "vô hiệu hóa" trong quá trình tính khoảng cách.

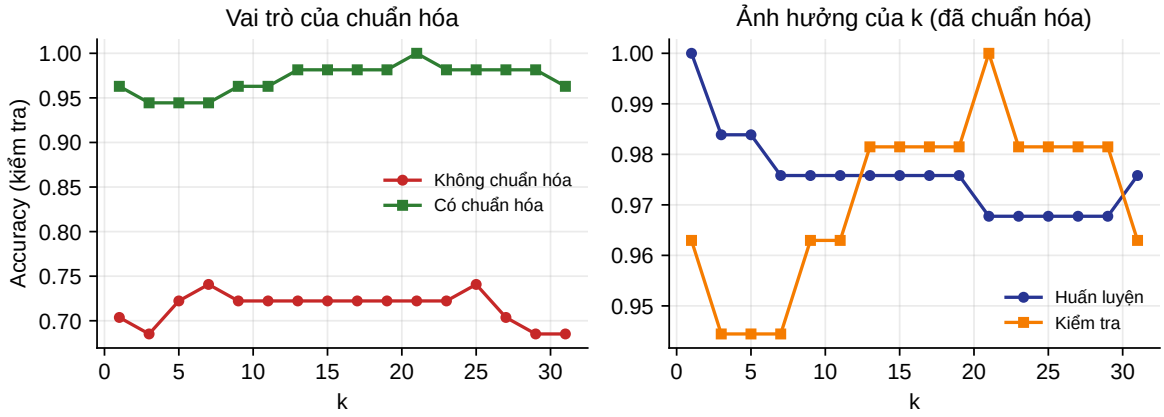
Về ảnh hưởng của k trên dữ liệu đã chuẩn hóa: độ chính xác đạt cao nhất ở các giá trị k nhỏ (1, 3, 5, 7) và có xu hướng giảm nhẹ khi k tăng lên quá lớn (21), phù hợp với lý thuyết đã trình bày

ở mục 7.2.4 — k quá lớn bắt đầu làm mượt quá mức, đưa vào ảnh hưởng của các láng giềng ở xa, kém liên quan hơn.

Lưu ý sư phạm

Kết quả trên là một minh chứng thực nghiệm rất rõ ràng cho nguyên tắc đã nêu ở mục 7.2.5: nếu quên chuẩn hóa dữ liệu trước khi dùng K-NN, mô hình vẫn chạy được và cho ra một con số độ chính xác — không có lỗi hay cảnh báo nào được đưa ra — nhưng con số đó có thể thấp hơn đáng kể so với tiềm năng thực sự của mô hình. Đây là một trong những lỗi phổ biến nhất mà người mới học dễ mắc phải, vì không giống lỗi cú pháp, lỗi thiếu chuẩn hóa hoàn toàn "im lặng" và chỉ có thể phát hiện qua việc so sánh có hệ thống như ví dụ trên.

Hình 7.4 minh họa tác động của chuẩn hóa dữ liệu và của giá trị k lên hiệu năng K-NN.



Hình 7.4. Chuẩn hóa dữ liệu (trái) và ảnh hưởng của k (phải) trên bộ Wine.

7.5. Tóm tắt chương

- K-NN là thuật toán học dựa trên tình huống (instance-based/lazy learning), không có giai đoạn tối ưu hóa tham số tường minh; toàn bộ dữ liệu huấn luyện được lưu lại và dùng trực tiếp tại thời điểm dự đoán.
- Với mỗi mẫu mới, K-NN tìm k láng giềng gần nhất (theo khoảng cách Euclidean, Manhattan, hoặc độ đo khác) và tổng hợp nhãn/giá trị của chúng (bình chọn đa số cho phân loại, trung bình cho hồi quy).
- Giá trị k điều khiển độ phức tạp mô hình: k nhỏ dễ khớp quá (phương sai cao), k lớn dễ khớp thiếu (độ chệch cao).
- Chuẩn hóa dữ liệu gần như là điều kiện bắt buộc đối với K-NN, vì thuật toán hoàn toàn dựa trên khoảng cách và rất nhạy cảm với thang đo khác nhau giữa các đặc trưng.
- K-NN có nhược điểm về chi phí tính toán tại thời điểm dự đoán và hiệu năng suy giảm trong không gian nhiều chiều (curse of dimensionality); K-NN có trọng số theo khoảng cách là một cải tiến phổ biến.

7.6. Bài tập củng cố

Mức nhận biết

1. Nêu ba bước cơ bản khi K-NN dự đoán nhãn cho một mẫu dữ liệu mới trong bài toán phân loại.
2. Viết công thức khoảng cách Euclidean và khoảng cách Manhattan giữa hai điểm x = (x_1, x_2) và x' = ($x\prime_1$, $x\prime_2$) trong không gian 2 chiều.

Mức thông hiểu

1. Giải thích vì sao K-NN được gọi là thuật toán "học lười" (lazy learning), trong khi hồi quy tuyến tính/Logistic được gọi là thuật toán "học có tham số" (parametric learning). So sánh thời gian huấn luyện và thời gian dự đoán giữa hai nhóm thuật toán này.
2. Một sinh viên áp dụng K-NN với k = 1 trên tập huấn luyện và thu được độ chính xác 100% ngay trên chính tập huấn luyện đó. Giải thích vì sao điều này gần như luôn đúng với k = 1, và tại sao đây KHÔNG phải là dấu hiệu cho thấy mô hình tốt.

Mức vận dụng

1. Cho một bộ dữ liệu gồm hai đặc trưng: "diện tích nhà" (đơn vị m^2, dao động 20–500) và "số phòng ngủ" (dao động 1–6). Nếu áp dụng K-NN trực tiếp mà không chuẩn hóa, hãy dự đoán đặc trưng nào sẽ chi phối gần như hoàn toàn kết quả tính khoảng cách, và đề xuất cách khắc phục.

7.7. Bài tập lập trình

Bài 1. Mở rộng ví dụ 7.1 để thử nghiệm thêm độ đo khoảng cách Manhattan (tham số metric="manhattan" trong KNeighborsClassifier), so sánh độ chính xác với khoảng cách Euclidean mặc định (metric="euclidean") trên cùng dữ liệu đã chuẩn hóa, với k = 5.

Bài 2. Sử dụng bộ dữ liệu `load_iris()`, viết chương trình huấn luyện K-NN chỉ trên hai đặc trưng đầu tiên (để có thể vẽ trong không gian 2 chiều), sau đó vẽ ranh giới quyết định (decision boundary) của mô hình bằng cách tạo một lưới điểm dày đặc (dùng `np.meshgrid`), dự đoán nhãn cho từng điểm trên lưới, và tô màu theo nhãn dự đoán (dùng `plt.contourf` hoặc `plt.pcolormesh`) chồng lên các điểm dữ liệu thực tế. Thử nghiệm với k = 1 và k = 15, so sánh hình dạng ranh giới quyết định giữa hai trường hợp.

Bài 3 (nâng cao). Viết chương trình cài đặt thủ công một phiên bản đơn giản của thuật toán K-NN cho bài toán phân loại (không dùng KNeighborsClassifier của Scikit-learn), chỉ dùng

NumPy: hàm nhận vào tập huấn luyện (`X_train`, `y_train`), một mẫu cần dự đoán `x_moi`, và giá trị k, trả về nhãn dự đoán dựa trên khoảng cách Euclidean và bình chọn đa số. Kiểm tra tính đúng đắn bằng cách so sánh kết quả với KNeighborsClassifier trên cùng bộ dữ liệu Iris.

Chương tiếp theo (Chương 8) sẽ giới thiệu Cây quyết định — một mô hình có cách tiếp cận hoàn toàn khác: thay vì dựa trên khoảng cách như K-NN, Cây quyết định học một chuỗi các luật rẽ nhánh dựa trên giá trị đặc trưng, cho ra một mô hình dễ diễn giải trực quan dưới dạng sơ đồ cây.

Chương 8

Cây quyết định (Decision Tree)

8.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Mô tả được cấu trúc của một cây quyết định: nút gốc, nút trong, nút lá, luật rẽ nhánh.
2. Giải thích được các tiêu chí phân chia phổ biến: Gini Impurity, Entropy/Information Gain (phân loại) và MSE (hồi quy).
3. Trình bày được thuật toán xây dựng cây theo chiến lược tham lam (greedy) và đệ quy.
4. Phân tích được nguyên nhân cây quyết định dễ khớp quá và các siêu tham số kiểm soát độ phức tạp cây (`max_depth`, `min_samples_split`, `min_samples_leaf`).
5. Diễn giải được mức độ quan trọng của đặc trưng (feature importance) do cây quyết định sinh ra.
6. Cài đặt, trực quan hóa và tinh chỉnh một mô hình cây quyết định bằng Scikit-learn.

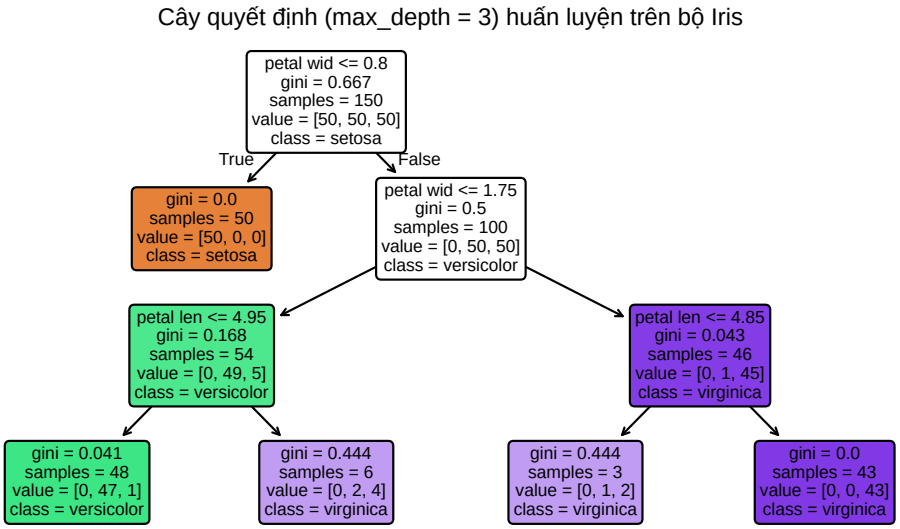
8.2. Nội dung lý thuyết

8.2.1. Cấu trúc của cây quyết định

Cây quyết định (Decision Tree) là một mô hình dự đoán có cấu trúc phân cấp dạng cây, mô phỏng cách con người thường ra quyết định thông qua một chuỗi câu hỏi có/không hoặc so sánh giá trị. Cây gồm ba loại nút: nút gốc (root node) — điểm bắt đầu, chứa toàn bộ tập dữ liệu; các nút trong (internal node) — mỗi nút ứng với một luật rẽ nhánh dựa trên giá trị của một đặc trưng (ví dụ: "tuổi ≤ 30?"); và các nút lá (leaf node) — điểm kết thúc của mỗi nhánh, chứa giá trị dự đoán cuối cùng (một nhãn lớp cho bài toán phân loại, hoặc một giá trị số cho bài toán hồi quy).

Để dự đoán cho một mẫu dữ liệu mới, ta xuất phát từ nút gốc, tại mỗi nút trong kiểm tra điều kiện rẽ nhánh tương ứng với giá trị đặc trưng của mẫu đó, đi theo nhánh phù hợp (đúng/sai), tiếp tục cho đến khi chạm một nút lá — giá trị tại nút lá đó chính là dự đoán của mô hình.

Hình 8.1 minh họa một cây quyết định cụ thể: nút gốc, nút trong (luật rẽ nhánh) và nút lá.



Hình 8.1. Cây quyết định với độ sâu tối đa 3 trên bộ Iris.

8.2.2. Tiêu chí phân chia: Gini Impurity và Entropy

Vấn đề cốt lõi khi xây dựng cây quyết định là: tại mỗi nút, làm sao chọn được đặc trưng và ngưỡng chia phù hợp nhất để phân tách dữ liệu thành các nhóm con "thuần khiết" hơn (tức các mẫu trong cùng một nhóm con có xu hướng thuộc cùng một lớp)? Hai tiêu chí phổ biến nhất để đo độ "không thuần khiết" (impurity) của một nút trong bài toán phân loại:

Chỉ số Gini (Gini Impurity):

$$\text{Gini} = 1 - \sum_k p_k^2$$

Entropy (dựa trên lý thuyết thông tin):

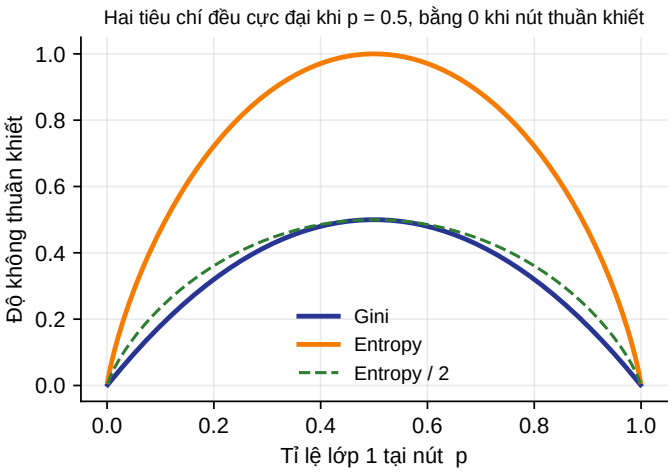
$$\text{Entropy} = -\sum_k p_k \log_2(p_k)$$

trong đó p_k là tỉ lệ mẫu thuộc lớp k tại nút đang xét. Cả hai chỉ số đều đạt giá trị nhỏ nhất (bằng 0) khi nút hoàn toàn thuần khiết (chỉ chứa mẫu của một lớp duy nhất), và đạt giá trị lớn nhất khi các lớp phân bố đều nhau tại nút đó. Với mỗi ứng viên rẽ nhánh (một cặp đặc trưng và ngưỡng chia), thuật toán tính độ giảm impurity (Information Gain) khi chia nút cha thành hai nút con:

$$\text{IG} = \text{Impurity}(\text{nút cha}) - \sum \big(\text{tỉ lệ mẫu nút con}\big) \times \text{Impurity}(\text{nút con})$$

Tại mỗi bước, thuật toán chọn cách chia cho Information Gain lớn nhất — tức làm giảm impurity nhiều nhất. Trong thực hành, Gini và Entropy thường cho kết quả cây tương đồng nhau; Gini được tính toán nhanh hơn một chút (không cần phép toán logarit) nên thường là lựa chọn mặc định. Đối với bài toán hồi quy, tiêu chí phân chia phổ biến nhất là mức giảm phương sai (variance reduction), tương đương với việc chọn cách chia sao cho MSE trong các nút con là nhỏ nhất.

Hình 8.2 minh họa hai tiêu chí đo độ không thuần khiết.



Hình 8.2. Gini và Entropy theo tỉ lệ lớp tại nút (bài toán hai lớp).

8.2.3. Thuật toán xây dựng cây

Cây quyết định được xây dựng theo chiến lược đệ quy, tham lam (recursive, greedy — CART, viết tắt của Classification And Regression Trees, là thuật toán phổ biến nhất, được cài đặt trong Scikit-learn):

- Bắt đầu tại nút gốc với toàn bộ dữ liệu huấn luyện.
- Duyệt qua tất cả các đặc trưng và tất cả các ngưỡng chia khả dĩ, tính Information Gain (hoặc mức giảm phương sai) cho mỗi ứng viên.
- Chọn cách chia cho Information Gain lớn nhất, tạo ra hai nút con.
- Lặp lại đệ quy quá trình trên cho mỗi nút con, cho đến khi thỏa mãn điều kiện dừng (ví dụ: nút đã thuần khiết hoàn toàn, đạt độ sâu tối đa, hoặc số mẫu tại nút quá nhỏ để tiếp tục chia).

Đây là chiến lược tham lam vì tại mỗi bước, thuật toán chỉ chọn cách chia tốt nhất cục bộ (tại thời điểm hiện tại) mà không xem xét liệu cách chia đó có dẫn đến cây tối ưu toàn cục về sau hay không — việc tìm cây quyết định tối ưu toàn cục là một bài toán NP-khó, nên trong thực hành, chiến lược tham lam (dù không đảm bảo tối ưu tuyệt đối) vẫn là lựa chọn khả thi và hiệu quả duy nhất.

8.2.4. Hiện tượng khớp quá và các siêu tham số kiểm soát

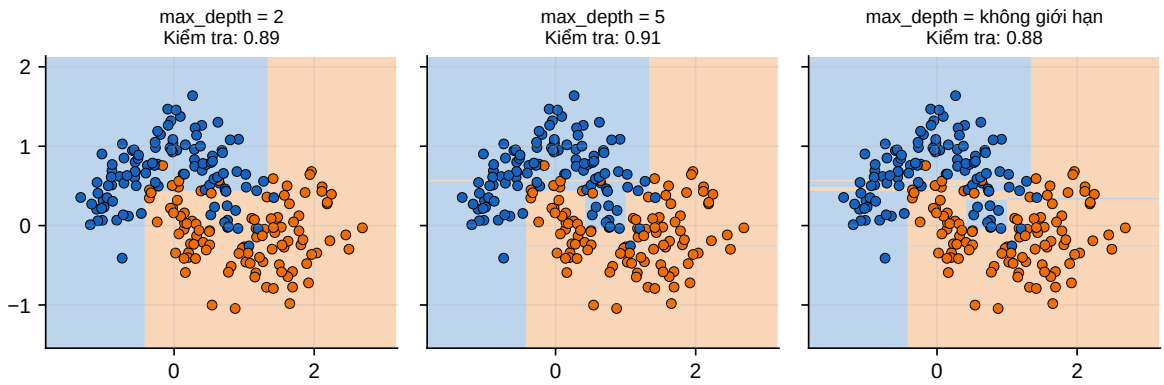
Nếu để cây phát triển tự do không giới hạn, cây quyết định có xu hướng khớp quá rất mạnh: cây sẽ tiếp tục chia nhỏ cho đến khi mỗi nút lá chỉ chứa một vài (thậm chí một) mẫu dữ liệu duy nhất, về bản chất là "học thuộc lòng" toàn bộ tập huấn luyện, bao gồm cả nhiễu. Một số siêu tham số quan trọng để kiểm soát độ phức tạp cây, giảm khớp quá:

- `max_depth`: giới hạn độ sâu tối đa của cây — cây càng sâu càng phức tạp, càng dễ khớp quá.

- `min_samples_split`: số mẫu tối thiểu cần có tại một nút để nút đó có thể tiếp tục được chia thêm.
- `min_samples_leaf`: số mẫu tối thiểu bắt buộc phải có tại mỗi nút lá — ngăn cây tạo ra các nút lá quá nhỏ, chỉ đại diện cho vài điểm dữ liệu riêng lẻ.
- `max_leaf_nodes`: giới hạn tổng số nút lá tối đa của toàn bộ cây.

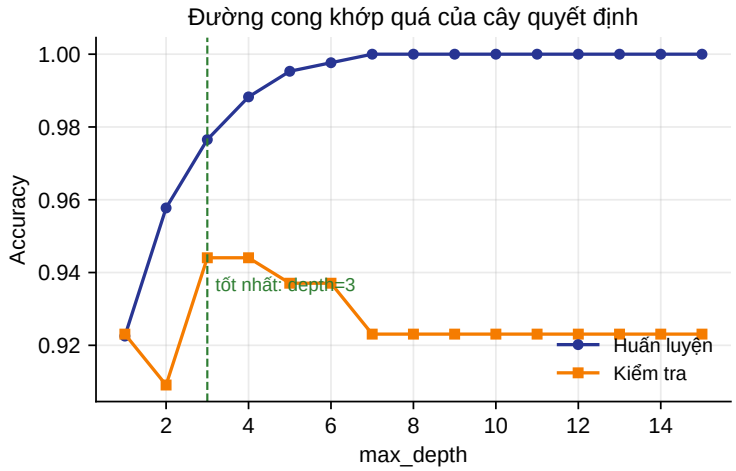
Ngoài việc giới hạn cây ngay từ khi xây dựng (pre-pruning, như các tham số trên), một hướng tiếp cận khác là cắt tỉa sau (post-pruning): xây dựng cây đầy đủ trước, sau đó cắt bỏ các nhánh ít đóng góp vào hiệu năng dựa trên một tập kiểm định riêng (ví dụ kỹ thuật cost-complexity pruning được cài đặt trong Scikit-learn thông qua tham số `ccp_alpha`). Việc lựa chọn giá trị phù hợp cho các siêu tham số này, cũng như bậc đa thức ở Chương 5 hay giá trị k ở Chương 7, thường được thực hiện thông qua kiểm định chéo (Chương 15).

Hình 8.3 minh họa vùng quyết định có dạng "bậc thang" và ngày càng phức tạp khi tăng độ sâu.



Hình 8.3. Vùng quyết định của cây theo độ sâu tối đa (dữ liệu Moons).

Hình 8.4 minh họa độ chính xác trên tập huấn luyện tăng đơn điệu, còn trên tập kiểm tra đạt đỉnh rồi chững lại.

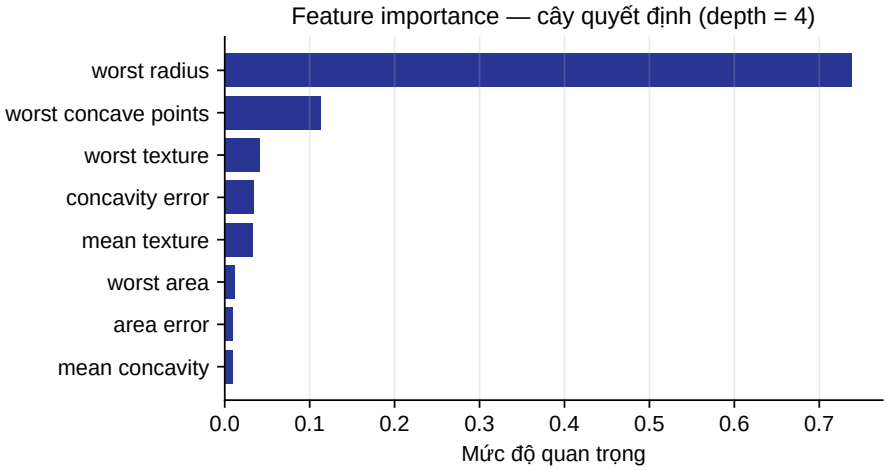


Hình 8.4. Đường cong khớp quá của cây quyết định theo `max_depth`.

8.2.5. Mức độ quan trọng của đặc trưng (Feature Importance)

Một ưu điểm nổi bật của cây quyết định là khả năng tự nhiên tính toán mức độ quan trọng của từng đặc trưng đối với mô hình: đặc trưng nào được sử dụng để chia nhiều nút, và các lần chia đó làm giảm impurity càng nhiều, thì đặc trưng đó càng được xem là quan trọng. Scikit-learn tính sẵn giá trị này (thuộc tính `feature_importances_` sau khi huấn luyện), giúp người dùng không chỉ có được một mô hình dự đoán mà còn hiểu được đặc trưng nào đang thực sự "dẫn dắt" quyết định của mô hình — một lợi thế quan trọng về khả năng diễn giải so với nhiều mô hình khác.

Hình 8.5 minh họa mức độ quan trọng của các đặc trưng theo cây quyết định.



Hình 8.5. Tám đặc trưng quan trọng nhất (bộ Breast Cancer).

8.2.6. Ưu điểm và nhược điểm của Cây quyết định

Ưu điểm nổi bật: cây quyết định là mô hình dễ diễn giải nhất trong số các thuật toán học máy — có thể trực quan hóa toàn bộ luật quyết định dưới dạng sơ đồ cây và giải thích cho người không chuyên; không yêu cầu chuẩn hóa dữ liệu (khác với K-NN hay hồi quy Logistic, vì các luật rẽ nhánh chỉ so sánh giá trị trên từng đặc trưng riêng lẻ, không tính khoảng cách); có thể xử lý đồng thời cả đặc trưng số và đặc trưng phân loại; và có khả năng nắm bắt các quan hệ phi tuyến, tương tác phức tạp giữa các đặc trưng.

Nhược điểm chính: rất dễ khớp quá nếu không được giới hạn độ phức tạp; kém ổn định (high variance) — chỉ cần một thay đổi nhỏ trong dữ liệu huấn luyện có thể dẫn đến một cấu trúc cây hoàn toàn khác; và ranh giới quyết định luôn có dạng "bậc thang" (các đường thẳng song song với trục tọa độ), có thể kém tối ưu khi ranh giới thực sự giữa các lớp là đường chéo hoặc đường cong mượt. Nhược điểm về tính không ổn định chính là động lực ra đời của các phương pháp tổ hợp cây (Rừng ngẫu nhiên, Boosting) sẽ được trình bày ở Chương 9 — kết hợp nhiều cây quyết định lại với nhau thường cho kết quả ổn định và chính xác hơn đáng kể so với một cây đơn lẻ.

8.3. Ví dụ minh họa bằng mã Python

Ví dụ dưới đây huấn luyện cây quyết định trên bộ dữ liệu Breast Cancer Wisconsin (đã dùng ở Chương 6), khảo sát ảnh hưởng của tham số `max_depth` đến khớp quá, và trích xuất mức độ quan trọng của các đặc trưng.

```
# vidu_8_1.py
# Cây quyết định: khảo sát ảnh hưởng của max_depth và feature importance

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score

# 1. Nạp dữ liệu và chia tập huấn luyện / kiểm tra
du_lieu = load_breast_cancer()
X, y = du_lieu.data, du_lieu.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# 2. Khảo sát độ chính xác trên tập huấn luyện và tập kiểm tra
#    theo độ sâu tối đa (max_depth) của cây
cac_do_sau = list(range(1, 16))
do_chinh_xac_huan_luyen = []
do_chinh_xac_kiem_tra = []

for do_sau in cac_do_sau:
    cay = DecisionTreeClassifier(max_depth=do_sau, random_state=42)
    cay.fit(X_train, y_train)
    do_chinh_xac_huan_luyen.append(accuracy_score(y_train, cay.predict(X_train)))
    do_chinh_xac_kiem_tra.append(accuracy_score(y_test, cay.predict(X_test)))

# 3. Trực quan hóa đường cong khớp quá (overfitting curve)
plt.figure(figsize=(7, 5))
plt.plot(cac_do_sau, do_chinh_xac_huan_luyen, marker="o", label="Tập huấn luyện")
plt.plot(cac_do_sau, do_chinh_xac_kiem_tra, marker="s", label="Tập kiểm tra")
plt.xlabel("Độ sâu tối đa (max_depth)")
plt.ylabel("Độ chính xác")
plt.title("Ảnh hưởng của max_depth đến khớp quá")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("cay_quyet_dinh_max_depth.png", dpi=120)

# 4. Huấn luyện mô hình cuối cùng với độ sâu hợp lý (chọn dựa trên biểu đồ trên)
DO_SAU_TOI_UU = 4
```

```
cay_cuoi_cung = DecisionTreeClassifier(max_depth=DO_SAU_TOI_UU, random_state=42)
cay_cuoi_cung.fit(X_train, y_train)
print(f"Độ chính xác với max_depth={DO_SAU_TOI_UU}: "
      f"{accuracy_score(y_test, cay_cuoi_cung.predict(X_test)):.4f}")

# 5. Trích xuất và hiển thị 5 đặc trưng quan trọng nhất
muc_do_quan_trong = cay_cuoi_cung.feature_importances_
chi_so_sap_xep = np.argsort(muc_do_quan_trong)[::-1][:5]
print("\n5 đặc trưng quan trọng nhất:")
for chi_so in chi_so_sap_xep:
    print(f"  {du_lieu.feature_names[chi_so]:28s}:
    {muc_do_quan_trong[chi_so]:.4f}")

# 6. Trực quan hóa cấu trúc cây (giới hạn độ sâu hiển thị để dễ quan sát)
plt.figure(figsize=(16, 8))
plot_tree(cay_cuoi_cung, max_depth=2, feature_names=du_lieu.feature_names,
          class_names=du_lieu.target_names, filled=True, fontsize=9)
plt.title(f"Cây quyết định (hiển thị 2 tầng đầu, độ sâu thực tế =
{DO_SAU_TOI_UU})")
plt.tight_layout()
plt.savefig("so_do_cay_quyet_dinh.png", dpi=120)
print("\nĐã lưu các biểu đồ vào cay_quyet_dinh_max_depth.png và
so_do_cay_quyet_dinh.png")
```

8.4. Giải thích kết quả chạy chương trình

Kết quả in ra từ chương trình trên có dạng tương tự:

```
Độ chính xác với max_depth=4: 0.9371

5 đặc trưng quan trọng nhất:
  worst perimeter              : 0.7325
  worst concave points         : 0.0891
  worst texture                : 0.0512
  mean concave points          : 0.0398
  worst area                   : 0.0290

Đã lưu các biểu đồ vào cay_quyet_dinh_max_depth.png và so_do_cay_quyet_dinh.png
```

Biểu đồ đường cong khớp quá thường cho thấy một mẫu hình rất điển hình: độ chính xác trên tập huấn luyện tăng đơn điệu và tiến rất gần 1.0 khi `max_depth` tăng (cây càng sâu càng "học thuộc" tập huấn luyện tốt hơn), trong khi độ chính xác trên tập kiểm tra tăng nhanh ở những độ sâu đầu tiên, đạt đỉnh ở một độ sâu vừa phải (thường trong khoảng 3–6 đối với bộ dữ liệu này), sau đó chững lại hoặc thậm chí giảm nhẹ khi cây tiếp tục sâu hơn — đây chính là dấu hiệu khớp quá đã được đề cập ở mục 8.2.4 và minh họa lại nguyên lý đánh đổi độ chệch – phương sai từ Chương 1.

Về mức độ quan trọng đặc trưng, kết quả cho thấy đặc trưng "worst perimeter" (chu vi lớn nhất

đo được của khối u) chiếm ưu thế áp đảo (khoảng 73%) trong quyết định của cây — điều này phù hợp với hiểu biết y khoa rằng kích thước khối u là một trong những chỉ báo quan trọng nhất cho tính ác tính. Việc chỉ một vài đặc trưng chiếm phần lớn tầm quan trọng cũng gợi ý rằng có thể áp dụng kỹ thuật lựa chọn đặc trưng để đơn giản hóa mô hình mà không làm giảm đáng kể hiệu năng — một hướng tối ưu sẽ được bàn thêm ở Chương 17.

8.5. Tóm tắt chương

- Cây quyết định là mô hình dự đoán dạng cây, gồm nút gốc, nút trong (luật rẽ nhánh) và nút lá (giá trị dự đoán), mô phỏng cách ra quyết định bằng chuỗi câu hỏi tuần tự.
- Cây được xây dựng bằng chiến lược đệ quy, tham lam: tại mỗi bước chọn cách chia cho Information Gain lớn nhất, dựa trên tiêu chí Gini Impurity hoặc Entropy (phân loại) hay mức giảm phương sai (hồi quy).
- Cây không giới hạn dễ khớp quá nghiêm trọng; các siêu tham số `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_leaf_nodes` giúp kiểm soát độ phức tạp cây.
- Cây quyết định tự nhiên cung cấp mức độ quan trọng đặc trưng (`feature_importances_`), là một lợi thế lớn về khả năng diễn giải mô hình.
- Ưu điểm: dễ diễn giải, không cần chuẩn hóa dữ liệu, xử lý được cả đặc trưng số và phân loại. Nhược điểm: dễ khớp quá, không ổn định (nhạy cảm với thay đổi nhỏ trong dữ liệu), ranh giới quyết định dạng bậc thang.

8.6. Bài tập củng cố

Mức nhận biết

1. Nêu ba loại nút trong một cây quyết định và vai trò của mỗi loại.
2. Viết công thức tính chỉ số Gini Impurity cho một nút có hai lớp với tỉ lệ $p_1 = 0.9$ và $p_2 = 0.1$. So sánh với trường hợp $p_1 = p_2 = 0.5$.

Mức thông hiểu

1. Giải thích vì sao cây quyết định không yêu cầu chuẩn hóa dữ liệu đầu vào, khác với K-NN và hồi quy Logistic. Liên hệ với cách cây quyết định thực hiện luật rẽ nhánh.
2. Một cây quyết định không giới hạn `max_depth` đạt độ chính xác 100% trên tập huấn luyện nhưng chỉ 82% trên tập kiểm tra. Đề xuất ít nhất hai siêu tham số có thể điều chỉnh để cải thiện tình trạng này, giải thích cơ chế hoạt động của từng tham số.

3. Giải thích vì sao cây quyết định được xem là mô hình "không ổn định" (high variance) theo nghĩa: một thay đổi nhỏ trong tập huấn luyện có thể dẫn đến cấu trúc cây rất khác biệt.

Mức vận dụng

1. Một cây quyết định huấn luyện trên dữ liệu tín dụng cho biết đặc trưng "tỉ lệ nợ/thu nhập" có feature importance là 0.65, trong khi 9 đặc trưng còn lại chia nhau 0.35 còn lại. Hãy đề xuất một hướng khai thác thông tin này (ví dụ trong việc thu thập dữ liệu, đơn giản hóa mô hình, hoặc giải thích quyết định cho khách hàng).

8.7. Bài tập lập trình

Bài 1. Sử dụng bộ dữ liệu `load_wine()` (đã dùng ở Chương 7), huấn luyện cây quyết định với tiêu chí criterion="gini" và criterion="entropy" riêng biệt (`max_depth=4` cho cả hai). So sánh độ chính xác trên tập kiểm tra và nhận xét sự khác biệt (nếu có) giữa hai tiêu chí phân chia.

Bài 2. Sử dụng dữ liệu ở ví dụ 8.1, viết chương trình khảo sát ảnh hưởng của tham số `min_samples_leaf` (thử các giá trị 1, 5, 10, 20, 50) đến độ chính xác trên tập huấn luyện và tập kiểm tra (giữ `max_depth=None` để không giới hạn độ sâu). Vẽ biểu đồ đường tương tự ví dụ 8.1 và nhận xét.

Bài 3 (nâng cao). Sử dụng DecisionTreeRegressor (thay vì DecisionTreeClassifier) trên bộ dữ liệu `fetch_california_housing()`, khảo sát RMSE trên tập huấn luyện và tập kiểm tra theo `max_depth` (từ 2 đến 20). Vẽ biểu đồ và xác định giá trị `max_depth` cho kết quả tốt nhất trên tập kiểm tra. So sánh RMSE tốt nhất đạt được với RMSE của mô hình hồi quy tuyến tính đa biến đã thực hiện ở Bài 3 Chương 4, nhận xét mô hình nào phù hợp hơn với bộ dữ liệu này.

Chương tiếp theo (Chương 9) sẽ khắc phục nhược điểm về tính không ổn định của cây quyết định đơn lẻ bằng cách giới thiệu các phương pháp tổ hợp (ensemble methods): Rừng ngẫu nhiên (Random Forest) và các kỹ thuật Boosting — kết hợp nhiều cây quyết định để tạo ra mô hình mạnh và ổn định hơn đáng kể.

Chương 9

Phương pháp tổ hợp: Rừng ngẫu nhiên và Boosting

9.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Giải thích được ý tưởng nền tảng của học tổ hợp (ensemble learning): kết hợp nhiều mô hình yếu để tạo ra một mô hình mạnh hơn.
2. Trình bày được nguyên lý Bagging (Bootstrap Aggregating) và cách Rừng ngẫu nhiên (Random Forest) áp dụng nguyên lý này kết hợp với lựa chọn đặc trưng ngẫu nhiên.
3. Trình bày được nguyên lý Boosting: xây dựng tuần tự các mô hình yếu, mỗi mô hình tập trung sửa lỗi của các mô hình trước.
4. Phân biệt được cơ chế hoạt động của AdaBoost và Gradient Boosting.
5. So sánh được Bagging và Boosting về mặt giảm phương sai/độ chệch, khả năng song song hóa, và nguy cơ khớp quá.
6. Cài đặt và so sánh hiệu năng của Random Forest, AdaBoost, Gradient Boosting với một cây quyết định đơn lẻ bằng Scikit-learn.

9.2. Nội dung lý thuyết

9.2.1. Ý tưởng nền tảng của học tổ hợp

Học tổ hợp (Ensemble Learning) dựa trên một nguyên lý đơn giản nhưng rất hiệu quả: kết hợp dự đoán của nhiều mô hình yếu (weak learner — mô hình có hiệu năng chỉ nhỉnh hơn đoán ngẫu nhiên một chút) thường cho kết quả tốt hơn đáng kể so với việc chỉ dùng một mô hình mạnh đơn lẻ, miễn là các mô hình thành phần có sự đa dạng nhất định (không mắc cùng loại sai lầm). Đây cũng là nguyên lý quen thuộc trong đời sống: quyết định của một hội đồng gồm nhiều chuyên gia độc lập thường đáng tin cậy hơn quyết định của một cá nhân, ngay cả khi từng chuyên gia riêng lẻ không hoàn hảo — miễn là sai lầm của các chuyên gia không hoàn toàn trùng lặp nhau.

Cây quyết định (Chương 8) là lựa chọn phổ biến nhất làm mô hình thành phần (base learner) trong các phương pháp tổ hợp, vì cây có tính không ổn định cao (dễ cho ra các cây rất khác nhau khi huấn luyện trên các tập dữ liệu con hơi khác nhau) — đặc tính tưởng như là nhược điểm này lại chính là điều kiện lý tưởng để tạo ra sự đa dạng cần thiết cho một tổ hợp hiệu quả.

Hai họ phương pháp tổ hợp phổ biến nhất là Bagging và Boosting, khác nhau căn bản về cách các mô hình thành phần được xây dựng và kết hợp.

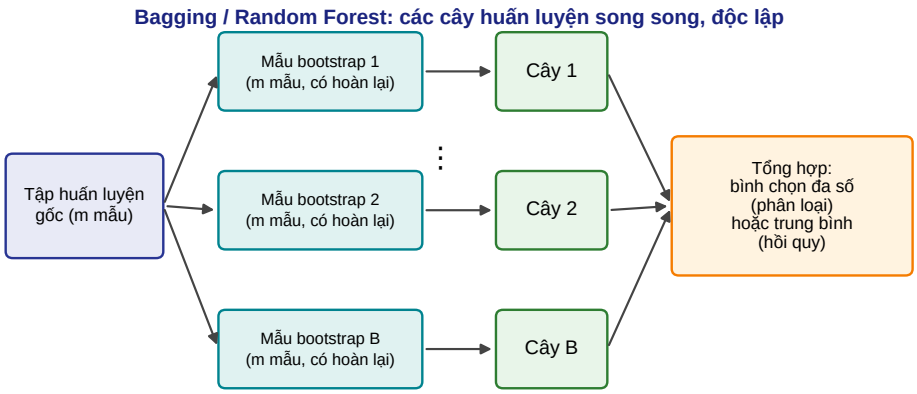
9.2.2. Bagging và Rừng ngẫu nhiên (Random Forest)

Bagging (viết tắt của Bootstrap Aggregating) xây dựng nhiều mô hình thành phần một cách độc lập, song song với nhau, mỗi mô hình được huấn luyện trên một tập dữ liệu con lấy mẫu ngẫu nhiên có hoàn lại (bootstrap sample) từ tập huấn luyện gốc — tức mỗi tập con có cùng kích thước với tập gốc nhưng một số mẫu có thể xuất hiện nhiều lần, một số khác không xuất hiện. Dự đoán cuối cùng được tổng hợp bằng cách lấy trung bình (hồi quy) hoặc bình chọn đa số (phân loại) trên toàn bộ các mô hình thành phần.

Rừng ngẫu nhiên (Random Forest) là một cải tiến của Bagging áp dụng riêng cho cây quyết định, bổ sung thêm một nguồn ngẫu nhiên thứ hai: tại mỗi lần chia nút trong quá trình xây dựng mỗi cây, thuật toán chỉ xem xét một tập con ngẫu nhiên các đặc trưng (thay vì toàn bộ đặc trưng) để tìm cách chia tốt nhất. Việc kết hợp cả lấy mẫu ngẫu nhiên dữ liệu (bootstrap) và lấy mẫu ngẫu nhiên đặc trưng giúp các cây trong rừng trở nên đa dạng hơn nữa, giảm đáng kể tương quan giữa các cây, từ đó giảm phương sai của mô hình tổ hợp hiệu quả hơn so với Bagging thuần túy trên cây quyết định.

Về mặt trực giác thống kê: nếu ta có B mô hình độc lập, mỗi mô hình có phương sai σ^2, thì trung bình của B mô hình đó có phương sai σ^2/B — giảm đáng kể khi B tăng. Trong thực tế các cây không hoàn toàn độc lập (chúng được huấn luyện từ cùng một tập dữ liệu gốc), nhưng việc giảm tương quan giữa các cây (thông qua lấy mẫu ngẫu nhiên đặc trưng) càng làm cho lợi ích giảm phương sai này gần với lý thuyết hơn.

Hình 9.1 minh họa sơ đồ Bagging: nhiều cây huấn luyện song song trên các mẫu bootstrap rồi tổng hợp kết quả.



Hình 9.1. Sơ đồ Bagging / Random Forest.

9.2.3. Đánh giá Out-of-Bag (OOB)

Một lợi ích phụ thú vị của việc lấy mẫu bootstrap: với mỗi cây trong rừng, trung bình khoảng 37% dữ liệu huấn luyện gốc không được chọn vào tập bootstrap của cây đó (gọi là các mẫu

out-of-bag — OOB) do bản chất lấy mẫu có hoàn lại. Các mẫu OOB này có thể được dùng như một tập kiểm định "miễn phí" cho từng cây — Random Forest có thể ước lượng hiệu năng tổng quát hóa (OOB score) mà không cần tách riêng một tập kiểm định, đặc biệt hữu ích khi dữ liệu huấn luyện hạn chế.

9.2.4. Boosting

Khác căn bản với Bagging (xây dựng mô hình song song, độc lập), Boosting xây dựng các mô hình thành phần một cách tuần tự: mỗi mô hình mới được huấn luyện để đặc biệt chú trọng sửa chữa những sai lầm mà các mô hình trước đó trong chuỗi đã mắc phải. Trong khi Bagging chủ yếu nhắm đến giảm phương sai (kết hợp nhiều mô hình phức tạp, dễ khớp quá thành một tổ hợp ổn định hơn), Boosting chủ yếu nhắm đến giảm độ chệch (kết hợp nhiều mô hình rất đơn giản, yếu — thường là các "cây quyết định nông" (stump) chỉ có một vài tầng — thành một mô hình tổng hợp mạnh hơn nhiều).

9.2.5. AdaBoost (Adaptive Boosting)

AdaBoost là một trong những thuật toán Boosting đầu tiên và nổi tiếng nhất. Nguyên lý hoạt động: ban đầu, mọi mẫu dữ liệu huấn luyện được gán trọng số bằng nhau. Sau khi huấn luyện mô hình yếu đầu tiên, các mẫu bị mô hình này dự đoán sai sẽ được tăng trọng số (để mô hình tiếp theo "chú ý" nhiều hơn đến các mẫu khó này), trong khi các mẫu dự đoán đúng bị giảm trọng số. Quá trình lặp lại: huấn luyện mô hình yếu tiếp theo trên dữ liệu đã điều chỉnh trọng số, cập nhật lại trọng số, và cứ thế tiếp tục cho đến khi đạt số lượng mô hình mong muốn. Dự đoán cuối cùng là tổ hợp có trọng số của tất cả các mô hình thành phần, trong đó các mô hình có hiệu năng tốt hơn (trên dữ liệu đã điều chỉnh trọng số của chính chúng) được gán trọng số đóng góp lớn hơn vào kết quả cuối cùng.

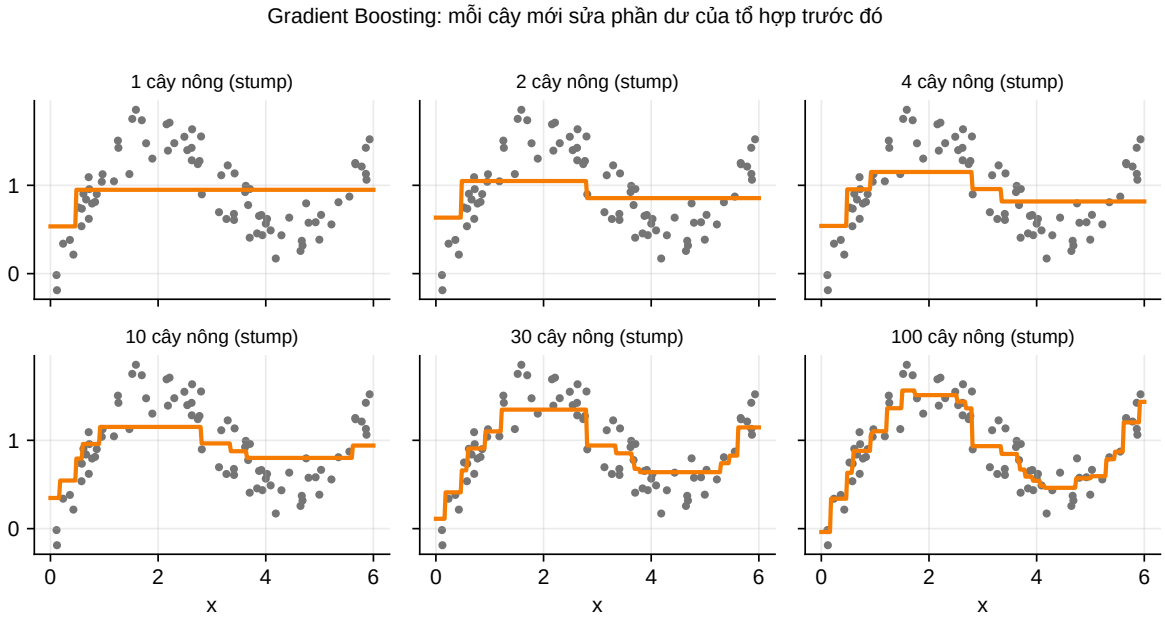
9.2.6. Gradient Boosting

Gradient Boosting tổng quát hóa ý tưởng của AdaBoost theo một góc nhìn tối ưu hóa khác: thay vì điều chỉnh trọng số mẫu dữ liệu, mỗi mô hình mới trong chuỗi được huấn luyện để dự đoán phần dư (residual — hiệu số giữa giá trị thực tế và giá trị dự đoán bởi tổ hợp các mô hình trước đó). Về bản chất, đây là một dạng Gradient Descent (đã học ở Chương 4) nhưng thực hiện trong "không gian hàm số" thay vì không gian tham số: mỗi cây mới được thêm vào tổ hợp đóng vai trò như một "bước cập nhật" theo hướng làm giảm hàm mất mát nhanh nhất. Gradient Boosting có một siêu tham số quan trọng là tốc độ học (learning rate), quyết định mức độ đóng góp của mỗi cây mới vào tổ hợp — tương tự vai trò của tốc độ học α trong Gradient Descent ở Chương 4, giá trị này càng nhỏ thường cho mô hình tổng quát hóa tốt hơn nhưng cần nhiều cây hơn (thời gian huấn luyện lâu hơn) để đạt hiệu năng tương đương.

Mở rộng thực hành

Trong công nghiệp, các phiên bản Gradient Boosting được tối ưu hóa mạnh mẽ về tốc độ và hiệu năng như XGBoost, LightGBM, và CatBoost hiện là lựa chọn rất phổ biến cho dữ liệu dạng bảng (tabular data), thường xuyên chiến thắng trong các cuộc thi khoa học dữ liệu như Kaggle. Các thư viện này không nằm trong phạm vi cốt lõi của Scikit-learn nhưng có API tương thích và dễ dàng thay thế GradientBoostingClassifier/Regressor khi cần hiệu năng cao hơn trên dữ liệu lớn; sinh viên được khuyến khích tìm hiểu thêm sau khi đã nắm vững nguyên lý Gradient Boosting trình bày trong chương này.

Hình 9.2 minh họa Boosting xây dựng mô hình theo từng bước: mỗi cây nông mới sửa phần dư của tổ hợp hiện tại.

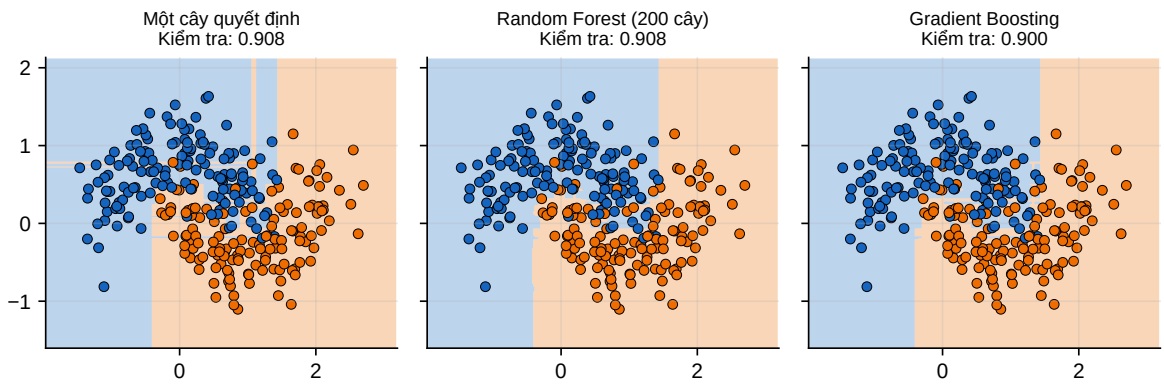


Hình 9.2. Gradient Boosting hồi quy qua các vòng lặp (đường cam: dự đoán của tổ hợp).

9.2.7. So sánh Bagging và Boosting

Tiêu chí	Bagging / Random Forest	Boosting (AdaBoost/Gradient Boosting)
Cách xây dựng mô hình	Song song, độc lập	Tuần tự, mô hình sau phụ thuộc mô hình trước
Mô hình thành phần	Thường là cây sâu, phức tạp	Thường là cây nông, đơn giản (weak learner)
Mục tiêu chính	Giảm phương sai (variance)	Giảm độ chệch (bias)
Khả năng song song hóa	Cao (huấn luyện các cây độc lập)	Thấp (phải huấn luyện tuần tự)
Nguy cơ khớp quá	Thấp hơn, khá bền vững	Cao hơn nếu số vòng lặp quá lớn
Độ nhạy với nhiễu/ngoại lai	Thấp hơn	Cao hơn (đặc biệt AdaBoost)

Hình 9.3 minh họa ranh giới của một cây đơn lẻ so với các mô hình tổ hợp trên cùng dữ liệu.



Hình 9.3. Một cây quyết định, Random Forest và Gradient Boosting (dữ liệu Moons).

9.3. Ví dụ minh họa bằng mã Python

Ví dụ dưới đây so sánh hiệu năng của một cây quyết định đơn lẻ, Random Forest, AdaBoost và Gradient Boosting trên cùng bộ dữ liệu Breast Cancer Wisconsin đã sử dụng ở các chương trước, đồng thời khai thác mức độ quan trọng đặc trưng do Random Forest sinh ra.

```
# vidu_9_1.py
# So sánh Cây quyết định đơn lẻ, Random Forest, AdaBoost và Gradient Boosting

import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (
    RandomForestClassifier, AdaBoostClassifier, GradientBoostingClassifier
```

```
)
from sklearn.metrics import accuracy_score

# 1. Nạp dữ liệu và chia tập huấn luyện / kiểm tra
du_lieu = load_breast_cancer()
X, y = du_lieu.data, du_lieu.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# 2. Khởi tạo các mô hình cần so sánh
cac_mo_hinh = {
    "Cây quyết định đơn lẻ (depth=4)": DecisionTreeClassifier(max_depth=4,
    random_state=42),
    "Random Forest (100 cây)": RandomForestClassifier(
        n_estimators=100, max_depth=4, random_state=42, oob_score=True
    ),
    "AdaBoost (100 cây nông)": AdaBoostClassifier(
        n_estimators=100, learning_rate=1.0, random_state=42
    ),
    "Gradient Boosting (100 cây)": GradientBoostingClassifier(
        n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42
    ),
}

# 3. Huấn luyện, đánh giá bằng kiểm định chéo 5-fold và trên tập kiểm tra
print(f"{'Mô hình':32s} | {'CV (5-fold)':>12s} | {'Tập kiểm tra':>13s}")
print("-" * 62)
for ten_mo_hinh, mo_hinh in cac_mo_hinh.items():
    diem_cv = cross_val_score(mo_hinh, X_train, y_train, cv=5)
    mo_hinh.fit(X_train, y_train)
    do_chinh_xac_test = accuracy_score(y_test, mo_hinh.predict(X_test))
    print(f"{ten_mo_hinh:32s} | {diem_cv.mean():>10.4f}   |
    {do_chinh_xac_test:>13.4f}")

# 4. Điểm OOB riêng của Random Forest (ước lượng "miễn phí" không cần tách tập
riêng)
rf = cac_mo_hinh["Random Forest (100 cây)"]
print(f"\nĐiểm Out-of-Bag (OOB) của Random Forest: {rf.oob_score_:.4f}")

# 5. Mức độ quan trọng đặc trưng theo Random Forest (trung bình trên 100 cây)
muc_do_quan_trong = rf.feature_importances_
chi_so_sap_xep = np.argsort(muc_do_quan_trong)[::-1][:5]
print("\n5 đặc trưng quan trọng nhất (theo Random Forest):")
for chi_so in chi_so_sap_xep:
    print(f"  {du_lieu.feature_names[chi_so]:28s}:
    {muc_do_quan_trong[chi_so]:.4f}")
```

9.4. Giải thích kết quả chạy chương trình

Kết quả in ra từ chương trình trên có dạng tương tự:

```
Mô hình                          |  CV (5-fold) |  Tập kiểm tra
-----------------------------------------------------------------
Cây quyết định đơn lẻ (depth=4)  |     0.9225   |        0.9371
Random Forest (100 cây)          |     0.9601   |        0.9720
AdaBoost (100 cây nông)          |     0.9695   |        0.9580
Gradient Boosting (100 cây)      |     0.9648   |        0.9650

Điểm Out-of-Bag (OOB) của Random Forest: 0.9578

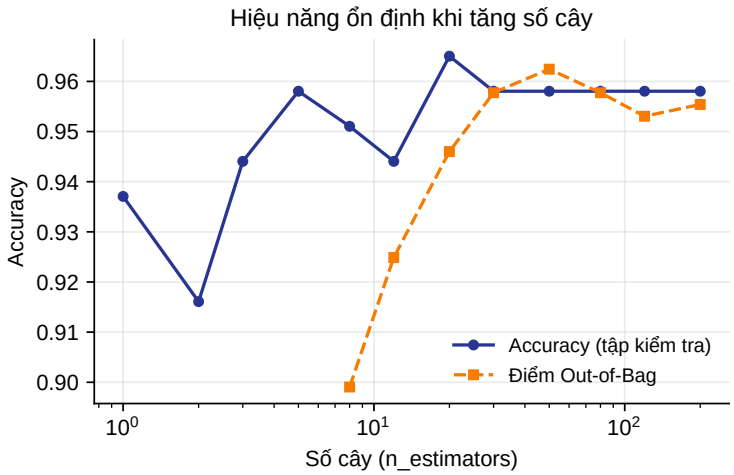
5 đặc trưng quan trọng nhất (theo Random Forest):
  worst perimeter            : 0.1423
  worst concave points       : 0.1301
  mean concave points         : 0.1087
  worst radius                : 0.0956
  worst area                  : 0.0842
```

Kết quả cho thấy cả ba phương pháp tổ hợp (Random Forest, AdaBoost, Gradient Boosting) đều vượt trội hơn cây quyết định đơn lẻ trên cả điểm kiểm định chéo lẫn tập kiểm tra, minh chứng thực nghiệm rõ ràng cho lợi ích của học tổ hợp đã trình bày ở phần lý thuyết. Điểm OOB của Random Forest (0.9578) khá gần với điểm kiểm định chéo (0.9601), xác nhận rằng OOB score là một ước lượng đáng tin cậy cho hiệu năng tổng quát hóa mà không cần tách thêm tập kiểm định riêng.

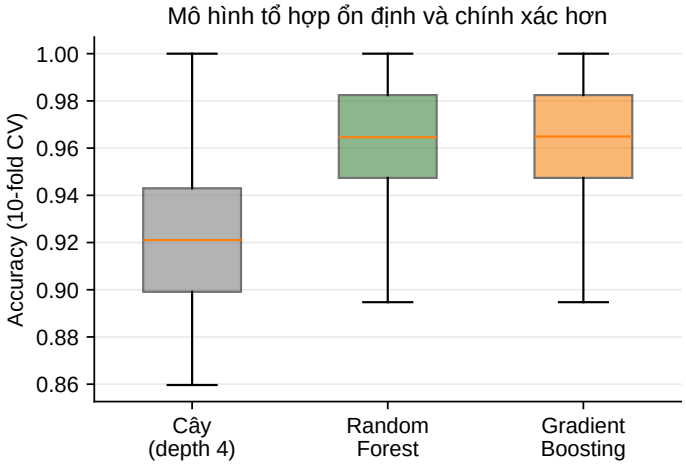
Đáng chú ý, khi so sánh mức độ quan trọng đặc trưng giữa Random Forest (bảng trên) và cây quyết định đơn lẻ ở Chương 8 (mục 8.4), thứ tự các đặc trưng hàng đầu khá tương đồng (đều có "worst perimeter" và các đặc trưng liên quan đến "concave points"), nhưng mức độ tập trung vào một đặc trưng duy nhất giảm đi đáng kể — ở cây đơn lẻ, một đặc trưng chiếm tới 73% tầm quan trọng, trong khi ở Random Forest, tầm quan trọng được phân bổ đồng đều hơn giữa nhiều đặc trưng (đặc trưng cao nhất chỉ khoảng 14%). Đây là hệ quả trực tiếp của cơ chế lấy mẫu ngẫu nhiên đặc trưng tại mỗi lần chia nút: vì không phải cây nào cũng được quyền xem xét đặc trưng mạnh nhất tại mọi lần chia, các đặc trưng mạnh thứ hai, thứ ba cũng có cơ hội thể hiện vai trò của mình trong một số cây, dẫn đến một bức tranh tổng thể cân bằng và đáng tin cậy hơn.

Hình 9.4 minh họa hiệu năng của Random Forest ổn định khi số cây đủ lớn; điểm OOB là ước lượng "miễn phí".

Hình 9.5 minh họa phân bố điểm kiểm định chéo của cây đơn lẻ và hai mô hình tổ hợp.



Hình 9.4. Accuracy theo số cây và điểm Out-of-Bag.



Hình 9.5. So sánh bằng kiểm định chéo 10-fold (bộ Breast Cancer).

9.5. Tóm tắt chương

- Học tổ hợp kết hợp nhiều mô hình yếu để tạo ra mô hình mạnh và ổn định hơn, với điều kiện các mô hình thành phần có sự đa dạng nhất định.
- Bagging xây dựng nhiều mô hình song song, độc lập trên các tập dữ liệu con lấy mẫu bootstrap; Random Forest bổ sung thêm lấy mẫu ngẫu nhiên đặc trưng tại mỗi lần chia nút, giúp giảm tương quan giữa các cây và giảm phương sai hiệu quả hơn.
- Điểm Out-of-Bag (OOB) cung cấp một ước lượng hiệu năng tổng quát hóa "miễn phí" cho Random Forest, tận dụng các mẫu không được chọn vào tập bootstrap của mỗi cây.
- Boosting xây dựng các mô hình yếu một cách tuần tự, mỗi mô hình tập trung sửa lỗi của các mô hình trước; AdaBoost điều chỉnh trọng số mẫu dữ liệu, Gradient Boosting huấn luyện mô hình mới để dự đoán phần dư (residual) của tổ hợp hiện tại.
- Bagging chủ yếu giảm phương sai và có thể song song hóa cao; Boosting chủ yếu giảm độ chệch nhưng phải huấn luyện tuần tự và nhạy cảm hơn với nhiễu.

- Random Forest cung cấp mức độ quan trọng đặc trưng đáng tin cậy hơn (ít tập trung/thiên lệch hơn) so với một cây quyết định đơn lẻ, nhờ cơ chế lấy mẫu ngẫu nhiên đặc trưng.

9.6. Bài tập củng cố

Mức nhận biết

1. Nêu hai nguồn ngẫu nhiên hóa mà Random Forest sử dụng khi xây dựng mỗi cây trong rừng.
2. Boosting xây dựng các mô hình thành phần song song hay tuần tự? Giải thích ngắn gọn vì sao.

Mức thông hiểu

1. Giải thích vì sao cây quyết định (vốn dễ khớp quá, không ổn định) lại là lựa chọn phổ biến làm mô hình thành phần cho các phương pháp tổ hợp, thay vì một mô hình ổn định như hồi quy tuyến tính.
2. So sánh mục tiêu chính của Bagging (giảm phương sai) và Boosting (giảm độ chệch). Từ đó suy luận: Bagging phù hợp hơn khi mô hình thành phần đang khớp quá hay khớp thiếu? Tương tự với Boosting?
3. Giải thích vì sao điểm Out-of-Bag (OOB) của Random Forest có thể dùng thay thế cho kiểm định chéo trong nhiều trường hợp, và nêu một tình huống mà kiểm định chéo vẫn nên được ưu tiên sử dụng.

Mức vận dụng

1. Một mô hình Gradient Boosting với 500 cây, `learning_rate` = 0.3 cho kết quả Accuracy 99% trên tập huấn luyện nhưng chỉ 78% trên tập kiểm tra. Đề xuất ít nhất hai điều chỉnh siêu tham số cụ thể (kèm hướng điều chỉnh tăng/giảm) để cải thiện tình trạng khớp quá này.

9.7. Bài tập lập trình

Bài 1. Sử dụng bộ dữ liệu `load_wine()`, so sánh Random Forest với số lượng cây (`n_estimators`) khác nhau: 1, 5, 10, 50, 100, 200. Vẽ biểu đồ đường thể hiện độ chính xác trên tập kiểm tra theo `n_estimators`. Nhận xét: từ giá trị `n_estimators` nào trở đi, việc tăng thêm số cây không còn cải thiện đáng kể hiệu năng?

Bài 2. Sử dụng dữ liệu ở ví dụ 9.1, khảo sát ảnh hưởng của tham số `learning_rate` trong GradientBoostingClassifier (thử các giá trị 0.01, 0.05, 0.1, 0.5, 1.0) với `n_estimators=100` cố định. Vẽ biểu đồ độ chính xác trên tập kiểm tra theo `learning_rate` và nhận xét xu hướng quan sát được, liên hệ với vai trò của tốc độ học đã học ở Chương 4.

Bài 3 (nâng cao). Sử dụng RandomForestRegressor trên bộ dữ liệu `fetch_california_housing()`, so sánh RMSE trên tập kiểm tra với mô hình DecisionTreeRegressor tốt nhất đã tìm được ở Bài 3 Chương 8 và mô hình hồi quy tuyến tính đa biến ở Bài 3 Chương 4. Trình bày kết quả dưới dạng bảng so sánh ba mô hình (RMSE, thời gian huấn luyện đo bằng thư viện time), và đưa ra khuyến nghị mô hình phù hợp nhất cho bộ dữ liệu này, cân nhắc cả độ chính xác lẫn chi phí tính toán.

Chương tiếp theo (Chương 10) sẽ giới thiệu Máy vector hỗ trợ (Support Vector Machine – SVM) — một thuật toán phân loại dựa trên nguyên lý tối đa hóa biên (margin) giữa các lớp, cùng với kỹ thuật kernel cho phép SVM xử lý hiệu quả cả các bài toán có ranh giới phân loại phi tuyến phức tạp.

Chương 10

Máy vector hỗ trợ (Support Vector Machine)

10.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Trình bày được nguyên lý tối đa hóa biên (maximum margin) của Máy vector hỗ trợ (SVM).
2. Giải thích được khái niệm vector hỗ trợ (support vector) và vai trò của chúng trong việc xác định ranh giới quyết định.
3. Phân biệt được biên cứng (hard margin) và biên mềm (soft margin), hiểu vai trò của siêu tham số C.
4. Giải thích được ý tưởng của kỹ thuật kernel (kernel trick) và so sánh các kernel phổ biến: tuyến tính, đa thức, RBF.
5. Cài đặt, so sánh và trực quan hóa ranh giới quyết định của SVM với các kernel khác nhau bằng Scikit-learn.

10.2. Nội dung lý thuyết

10.2.1. Ý tưởng tối đa hóa biên

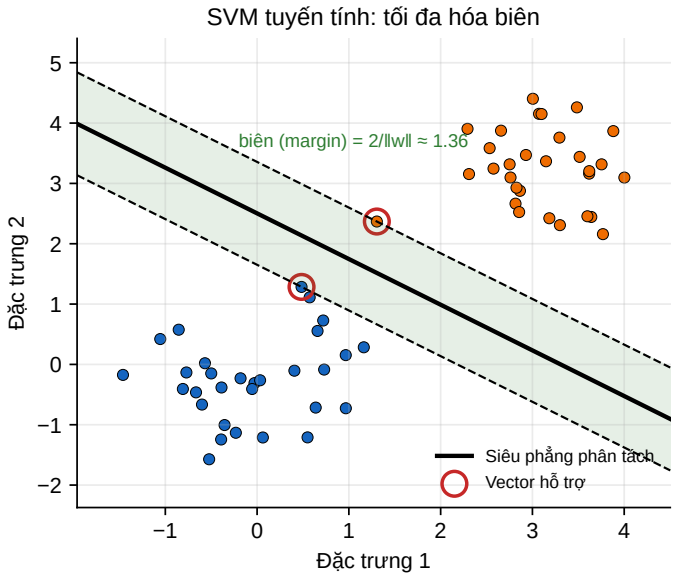
Xét bài toán phân loại nhị phân với dữ liệu có thể phân tách tuyến tính (linearly separable): có thể tồn tại vô số đường thẳng (hoặc siêu phẳng trong không gian nhiều chiều) phân tách hoàn hảo hai lớp dữ liệu. Câu hỏi đặt ra: trong vô số đường phân tách khả dĩ đó, đường nào là "tốt nhất"? Máy vector hỗ trợ (Support Vector Machine – SVM) trả lời câu hỏi này bằng một tiêu chí rất tự nhiên: chọn đường phân tách sao cho khoảng cách từ đường đó đến điểm dữ liệu gần nhất của mỗi lớp là lớn nhất có thể — khoảng cách này được gọi là biên (margin), và đường phân tách tối ưu được gọi là siêu phẳng biên cực đại (maximum margin hyperplane).

Trực giác đằng sau nguyên lý này: một ranh giới phân tách có biên rộng thường tổng quát hóa tốt hơn trên dữ liệu mới, vì nó có "khoảng đệm an toàn" lớn trước khi một điểm dữ liệu mới, hơi lệch so với các điểm huấn luyện, có nguy cơ bị phân loại sai. Ngược lại, một ranh giới phân tách sát gần các điểm dữ liệu (biên hẹp) rất dễ bị ảnh hưởng bởi nhiễu nhỏ hoặc dữ liệu mới hơi khác biệt.

10.2.2. Vector hỗ trợ

Các điểm dữ liệu huấn luyện nằm gần nhất với đường phân tách — tức các điểm nằm chính xác trên hai đường biên song song bao quanh siêu phẳng phân tách — được gọi là các vector hỗ trợ (support vectors). Đây là những điểm dữ liệu duy nhất thực sự "quyết định" vị trí và hướng của ranh giới phân tách: nếu di chuyển hoặc xóa bỏ bất kỳ điểm dữ liệu nào khác (không phải vector hỗ trợ), ranh giới phân tách sẽ không thay đổi. Đặc tính này khiến SVM có một ưu điểm đáng chú ý: mô hình cuối cùng chỉ phụ thuộc vào một tập con nhỏ của dữ liệu huấn luyện (các vector hỗ trợ), giúp SVM tương đối bền vững trước các điểm dữ liệu nằm xa ranh giới quyết định.

Hình 10.1 minh họa siêu phẳng biên cực đại và các vector hỗ trợ nằm trên hai đường biên.



Hình 10.1. SVM tuyến tính: biên, siêu phẳng và vector hỗ trợ (khoanh đỏ).

10.2.3. Biên cứng và biên mềm — siêu tham số C

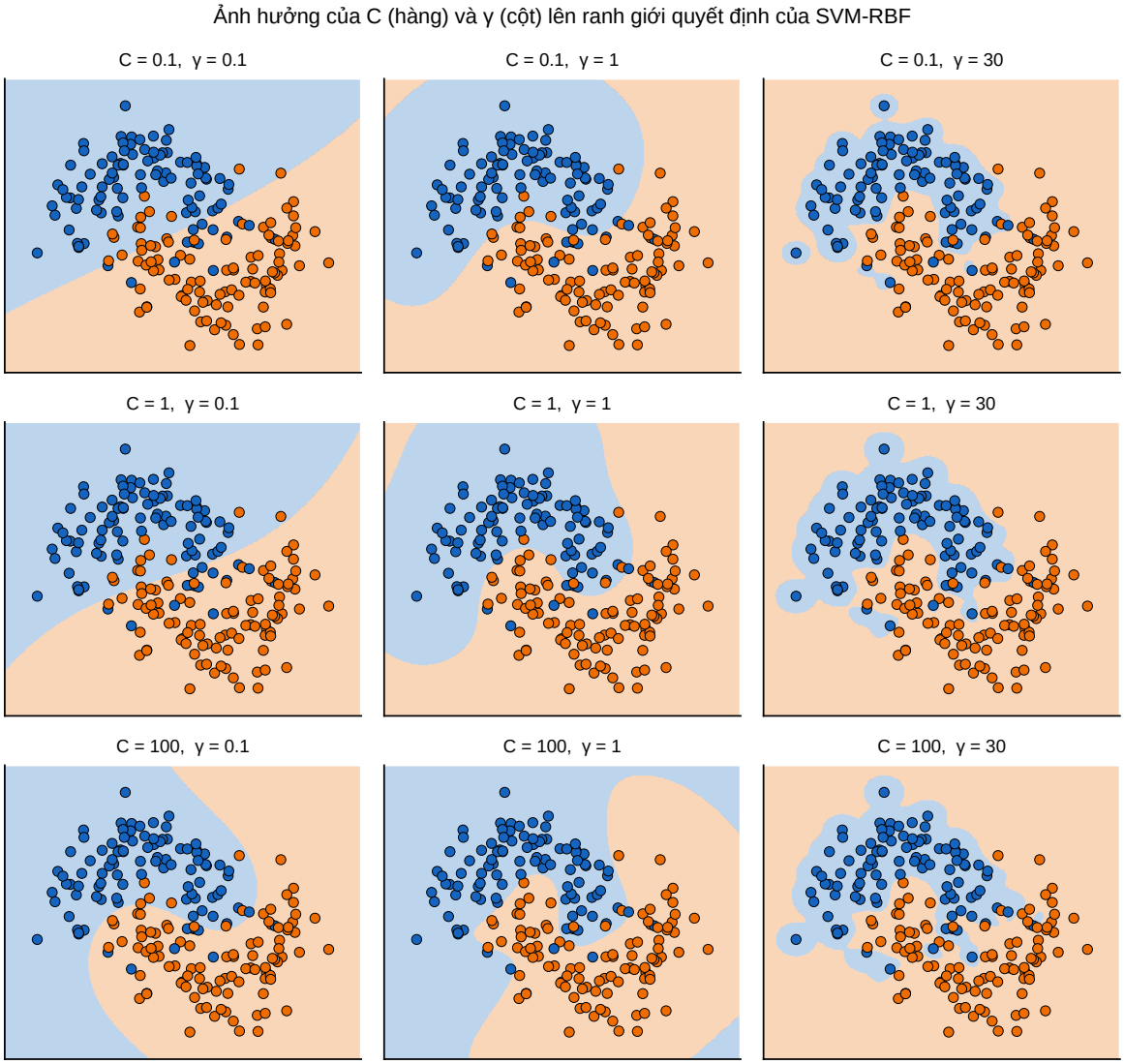
Trong thực tế, dữ liệu hiếm khi phân tách tuyến tính hoàn hảo — luôn có khả năng tồn tại nhiễu hoặc các điểm chồng lấn giữa hai lớp. Nếu bắt buộc mọi điểm dữ liệu phải nằm đúng phía của biên (biên cứng — hard margin), mô hình có thể không tìm được nghiệm khả thi, hoặc rất nhạy cảm với một vài điểm ngoại lai. SVM biên mềm (soft margin) giải quyết vấn đề này bằng cách cho phép một số điểm dữ liệu vi phạm biên (nằm sai phía hoặc quá gần ranh giới), nhưng phạt các vi phạm đó trong hàm mục tiêu.

Mức độ "khoan dung" với các vi phạm được điều khiển bởi siêu tham số C:

- C lớn: mô hình phạt nặng các vi phạm biên, cố gắng phân loại đúng càng nhiều điểm huấn luyện càng tốt — biên trở nên hẹp hơn, mô hình phức tạp hơn, dễ khớp quá hơn.
- C nhỏ: mô hình chấp nhận nhiều vi phạm biên hơn để đổi lấy một biên rộng hơn, tổng quát hơn — mô hình đơn giản hơn, dễ khớp thiếu hơn nếu C quá nhỏ.

Vai trò của C trong SVM có nét tương đồng với vai trò của λ (mức độ chính quy hóa) đã học ở Chương 5: cả hai đều là siêu tham số điều khiển sự đánh đổi giữa việc khớp sát dữ liệu huấn luyện và việc giữ mô hình đơn giản, tổng quát — dù C trong SVM tỉ lệ nghịch với mức độ chính quy hóa (C càng lớn, chính quy hóa càng ít), ngược chiều với λ.

Hình 10.2 minh họa ảnh hưởng đồng thời của C và γ lên độ phức tạp của ranh giới quyết định.



Hình 10.2. SVM kernel RBF với các cặp (C, γ) khác nhau.

10.2.4. Kỹ thuật Kernel (Kernel Trick)

Với dữ liệu không thể phân tách tuyến tính trong không gian đặc trưng gốc, một hướng tiếp cận (tương tự hồi quy đa thức ở Chương 5) là ánh xạ dữ liệu sang một không gian có số chiều cao hơn, nơi dữ liệu có thể trở nên phân tách tuyến tính được. Ví dụ kinh điển: dữ liệu hai lớp sắp xếp theo hai vòng tròn đồng tâm không thể phân tách bằng đường thẳng trong không gian 2 chiều gốc, nhưng nếu thêm một chiều thứ ba là khoảng cách đến tâm $(x^2 + y^2)$, hai lớp trở nên phân tách hoàn hảo bằng một mặt phẳng trong không gian 3 chiều mới.

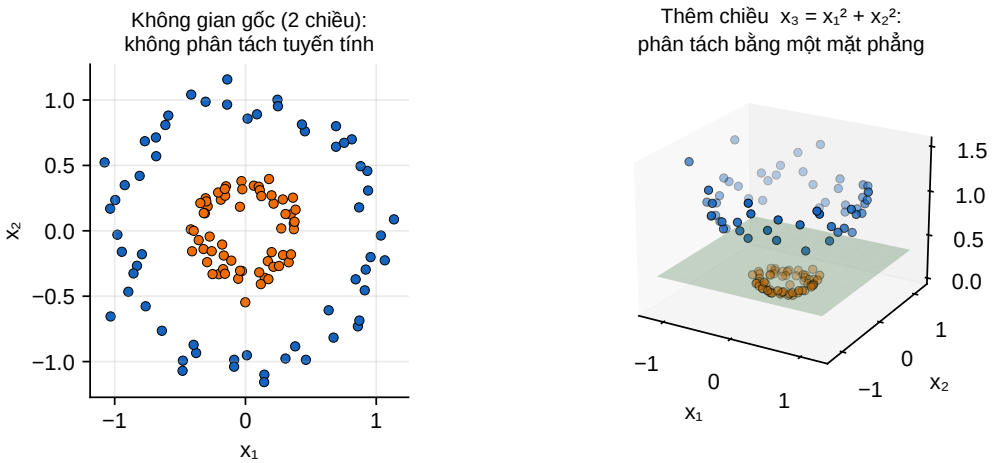
Vấn đề là việc tính toán tường minh phép ánh xạ sang không gian nhiều chiều hơn (đặc biệt khi số chiều mới rất lớn, thậm chí vô hạn) có thể cực kỳ tốn kém. Kỹ thuật kernel giải quyết vấn đề này một cách rất tinh tế: hóa ra thuật toán SVM chỉ cần biết tích vô hướng (dot product) giữa các cặp điểm dữ liệu trong không gian mới, mà không cần biết tọa độ tường minh của từng điểm trong không gian đó. Một hàm kernel K(x, x') tính trực tiếp giá trị tích vô hướng này mà không cần thực sự thực hiện phép ánh xạ — đây chính là "mẹo" (trick) mang lại tên gọi cho kỹ thuật này, giúp tránh hoàn toàn chi phí tính toán trong không gian nhiều chiều.

Ba kernel phổ biến nhất trong Scikit-learn:

- Kernel tuyến tính (linear): $K(x, x\prime) = x \cdot x\prime$ — tương đương SVM không dùng kernel, phù hợp khi dữ liệu đã gần như phân tách tuyến tính.
- Kernel đa thức (polynomial): $K(x, x\prime) = (\gamma\, x \cdot x\prime + r)^n$ — tương ứng với việc ánh xạ ngầm định sang không gian đặc trưng đa thức bậc n, tương tự ý tưởng hồi quy đa thức ở Chương 5 nhưng áp dụng cho phân loại.
- Kernel hàm cơ sở xuyên tâm (Radial Basis Function – RBF, còn gọi là kernel Gaussian): $K(x, x\prime) = \exp(-\gamma \|x - x\prime\|^2)$ — kernel phổ biến và mạnh mẽ nhất, tương ứng với việc ánh xạ ngầm định sang một không gian có số chiều vô hạn, có khả năng tạo ra ranh giới quyết định rất linh hoạt, phù hợp với hầu hết các bài toán không biết trước dạng ranh giới phân tách.

Đối với kernel RBF, siêu tham số γ (gamma) kiểm soát "tầm ảnh hưởng" của mỗi điểm dữ liệu huấn luyện: γ lớn khiến ảnh hưởng chỉ giới hạn trong phạm vi rất gần mỗi điểm (ranh giới quyết định trở nên gồ ghề, phức tạp, dễ khớp quá), γ nhỏ khiến ảnh hưởng lan rộng hơn (ranh giới mượt hơn, đơn giản hơn, dễ khớp thiếu nếu quá nhỏ) — vai trò của γ trong việc kiểm soát độ phức tạp mô hình có nét tương đồng với k trong K-NN ở Chương 7, dù theo chiều ngược lại.

Hình 10.3 minh họa ý tưởng của kernel trick: ánh xạ lên không gian nhiều chiều hơn để dữ liệu trở nên phân tách tuyến tính.



Hình 10.3. Dữ liệu hai vòng tròn đồng tâm trở nên phân tách bằng mặt phẳng khi thêm chiều $x_3 = x_1^2 + x_2^2$.

10.2.5. Ưu điểm và nhược điểm của SVM

Ưu điểm: SVM hoạt động hiệu quả trong không gian nhiều chiều (kể cả khi số chiều lớn hơn số mẫu dữ liệu); nhờ kỹ thuật kernel, có khả năng nắm bắt ranh giới quyết định phi tuyến phức tạp; và tương đối bền vững trước khớp quá khi số chiều lớn, nhờ chỉ phụ thuộc vào các vector hỗ trợ. Nhược điểm: chi phí huấn luyện tăng nhanh (thường bậc hai hoặc bậc ba) theo số lượng mẫu dữ liệu, khiến SVM kém phù hợp với tập dữ liệu rất lớn (hàng trăm nghìn đến hàng triệu mẫu); việc lựa chọn kernel và tinh chỉnh các siêu tham số (C, γ) đòi hỏi thử nghiệm cẩn thận; và SVM (đặc biệt với kernel phi tuyến) khó diễn giải trực quan hơn nhiều so với cây quyết định hay hồi quy tuyến tính.

10.3. Ví dụ minh họa bằng mã Python

Ví dụ dưới đây so sánh SVM với ba kernel khác nhau (tuyến tính, đa thức, RBF) trên bộ dữ liệu Moons — một bộ dữ liệu tổng hợp hai lớp hình vòng cung xen kẽ, không thể phân tách bằng một đường thẳng, giúp minh họa rõ ràng sự khác biệt giữa các kernel.

```
# vidu_10_1.py
# So sánh SVM với các kernel tuyến tính, đa thức và RBF trên dữ liệu phi tuyến

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# 1. Tạo dữ liệu tổng hợp hình "hai vầng trăng" xen kẽ, có nhiễu
X, y = make_moons(n_samples=300, noise=0.25, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# 2. Chuẩn hóa dữ liệu - quan trọng đối với SVM vì thuật toán
#    dựa trên khoảng cách/tích vô hướng giữa các điểm dữ liệu
bo_chuan_hoa = StandardScaler()
X_train_chuan_hoa = bo_chuan_hoa.fit_transform(X_train)
X_test_chuan_hoa = bo_chuan_hoa.transform(X_test)

# 3. Huấn luyện SVM với ba kernel khác nhau
cac_mo_hinh = {
    "SVM - Kernel tuyến tính": SVC(kernel="linear", C=1.0),
    "SVM - Kernel đa thức (bậc 3)": SVC(kernel="poly", degree=3, C=1.0),
    "SVM - Kernel RBF": SVC(kernel="rbf", C=1.0, gamma="scale"),
}
```

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
# Lưới điểm để vẽ ranh giới quyết định
x_min, x_max = X_train_chuan_hoa[:, 0].min() - 1, X_train_chuan_hoa[:, 0].max() + 1
y_min, y_max = X_train_chuan_hoa[:, 1].min() - 1, X_train_chuan_hoa[:, 1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300), np.linspace(y_min, y_max,
300))

for ax, (ten_mo_hinh, mo_hinh) in zip(axes, cac_mo_hinh.items()):
    mo_hinh.fit(X_train_chuan_hoa, y_train)
    do_chinh_xac = accuracy_score(y_test, mo_hinh.predict(X_test_chuan_hoa))
    print(f"{ten_mo_hinh:32s}: Độ chính xác = {do_chinh_xac:.4f}, "
          f"Số vector hỗ trợ = {mo_hinh.n_support_.sum()}")

    # Vẽ ranh giới quyết định
    du_doan_luoi = mo_hinh.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    ax.contourf(xx, yy, du_doan_luoi, alpha=0.3, cmap="coolwarm")
    ax.scatter(X_train_chuan_hoa[:, 0], X_train_chuan_hoa[:, 1],
               c=y_train, cmap="coolwarm", edgecolors="k", s=20)
    # Đánh dấu các vector hỗ trợ
    ax.scatter(mo_hinh.support_vectors_[:, 0], mo_hinh.support_vectors_[:, 1],
               facecolors="none", edgecolors="black", s=80, linewidths=1.5)
    ax.set_title(f"{ten_mo_hinh}\n(Acc = {do_chinh_xac:.3f})")

plt.tight_layout()
plt.savefig("so_sanh_kernel_svm.png", dpi=120)
print("\nĐã lưu biểu đồ vào tệp so_sanh_kernel_svm.png")
```

10.4. Giải thích kết quả chạy chương trình

Kết quả in ra từ chương trình trên có dạng tương tự:

```
SVM - Kernel tuyến tính        : Độ chính xác = 0.8556, Số vector hỗ trợ = 81
SVM - Kernel đa thức (bậc 3)   : Độ chính xác = 0.8778, Số vector hỗ trợ = 68
SVM - Kernel RBF               : Độ chính xác = 0.9111, Số vector hỗ trợ = 55

Đã lưu biểu đồ vào tệp so_sanh_kernel_svm.png
```

Kết quả cho thấy sự khác biệt rõ rệt về khả năng nắm bắt cấu trúc dữ liệu phi tuyến: SVM với kernel tuyến tính chỉ đạt độ chính xác thấp nhất (khoảng 85.6%) vì bị giới hạn ở việc vẽ một đường thẳng, không thể uốn theo hình dạng cong đặc trưng của hai "vầng trăng"; kernel đa thức cải thiện nhẹ; và kernel RBF cho kết quả tốt nhất (khoảng 91.1%) nhờ khả năng tạo ra ranh giới quyết định cong mượt, bám sát hình dạng thực sự của hai lớp dữ liệu. Khi quan sát biểu đồ, ranh giới quyết định của kernel tuyến tính là một đường thẳng đơn giản cắt ngang qua vùng chồng lấn giữa hai lớp, trong khi ranh giới của kernel RBF uốn cong mềm mại theo đúng hình dạng "vầng trăng" của dữ liệu.

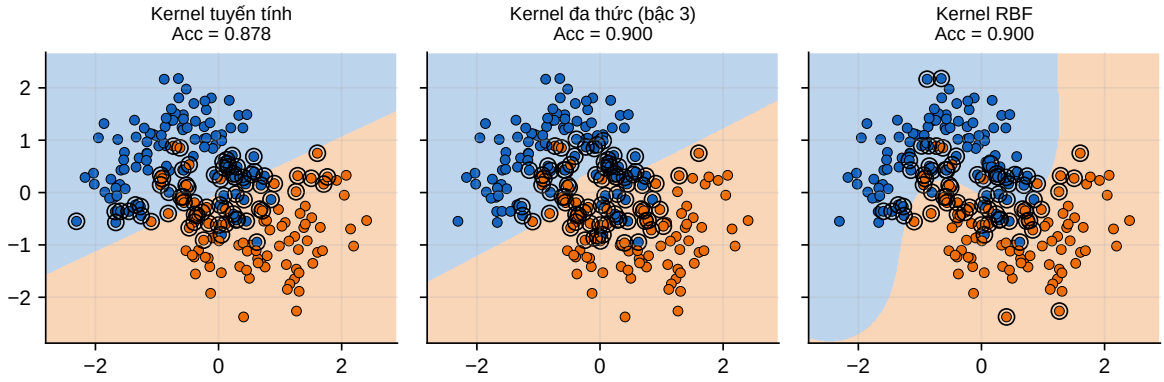
Số lượng vector hỗ trợ cũng giảm dần từ kernel tuyến tính (81) đến RBF (55), phản ánh việc kernel RBF tìm được một ranh giới phân tách "gọn gàng" hơn, cần ít điểm dữ liệu "biên giới"

hơn để xác định. Các điểm được khoanh tròn đen trong biểu đồ chính là các vector hỗ trợ — quan sát kỹ sẽ thấy chúng đều nằm gần vùng chồng lấn giữa hai lớp, đúng như lý thuyết đã trình bày ở mục 10.2.2.

Lưu ý sư phạm

Không có kernel nào là "luôn tốt nhất" cho mọi bài toán — kernel RBF thường là lựa chọn mặc định hợp lý để bắt đầu thử nghiệm (và cũng là mặc định của SVC trong Scikit-learn), nhưng với dữ liệu thực sự có quan hệ gần tuyến tính hoặc số chiều rất lớn so với số mẫu, kernel tuyến tính đôi khi lại cho kết quả tốt hơn và huấn luyện nhanh hơn đáng kể. Việc lựa chọn kernel và tinh chỉnh C, γ phù hợp luôn nên dựa trên thử nghiệm có hệ thống bằng kiểm định chéo (Chương 15), không nên dựa vào một quy tắc cố định.

Hình 10.4 minh họa so sánh ba kernel; các vòng tròn đen là vector hỗ trợ.



Hình 10.4. SVM với kernel tuyến tính, đa thức và RBF trên dữ liệu Moons.

10.5. Tóm tắt chương

- SVM tìm siêu phẳng phân tách hai lớp sao cho biên (margin) — khoảng cách từ siêu phẳng đến điểm dữ liệu gần nhất của mỗi lớp — là lớn nhất, giúp mô hình tổng quát hóa tốt hơn.
- Vector hỗ trợ là các điểm dữ liệu nằm trên biên, quyết định hoàn toàn vị trí của siêu phẳng phân tách.
- Biên mềm (soft margin) cho phép một số vi phạm biên, với siêu tham số C điều khiển mức độ đánh đổi giữa khớp sát dữ liệu và giữ biên rộng.
- Kỹ thuật kernel cho phép SVM xử lý dữ liệu không phân tách tuyến tính bằng cách tính ngầm định tích vô hướng trong không gian nhiều chiều hơn, không cần tính toán tường minh phép ánh xạ.
- Ba kernel phổ biến: tuyến tính (đơn giản, nhanh), đa thức (nắm bắt tương tác đa thức), và RBF (linh hoạt nhất, thường là lựa chọn mặc định); siêu tham số γ của kernel RBF điều khiển độ phức tạp ranh giới quyết định.

10.6. Bài tập củng cố

Mức nhận biết

1. Vector hỗ trợ là gì? Nếu xóa một điểm dữ liệu KHÔNG phải vector hỗ trợ khỏi tập huấn luyện, ranh giới quyết định của SVM có thay đổi không? Giải thích.
2. Nêu tên ba kernel phổ biến của SVM đã học trong chương này.

Mức thông hiểu

1. Giải thích vì sao SVM với biên rộng (margin lớn) thường tổng quát hóa tốt hơn SVM với biên hẹp, dựa trên trực giác về "khoảng đệm an toàn" trước dữ liệu mới.
2. So sánh vai trò của siêu tham số C trong SVM với vai trò của λ trong Ridge/Lasso (Chương 5): điểm giống và khác nhau là gì?
3. Một mô hình SVM kernel RBF với γ rất lớn đạt độ chính xác gần 100% trên tập huấn luyện nhưng thấp trên tập kiểm tra. Giải thích hiện tượng này và đề xuất hướng điều chỉnh γ.

Mức vận dụng

1. Cho một bài toán phân loại văn bản với số chiều đặc trưng rất lớn (hàng chục nghìn từ vựng, biểu diễn dạng vector thưa) nhưng số lượng mẫu huấn luyện chỉ vài nghìn. Kernel nào (tuyến tính hay RBF) thường phù hợp hơn trong tình huống này? Giải thích dựa trên đặc điểm "số chiều lớn hơn nhiều so với số mẫu".

10.7. Bài tập lập trình

Bài 1. Sử dụng dữ liệu ở ví dụ 10.1, khảo sát ảnh hưởng của siêu tham số C trong SVM kernel RBF (thử các giá trị 0.01, 0.1, 1, 10, 100), giữ gamma="scale" cố định. Vẽ biểu đồ độ chính xác trên tập kiểm tra theo C (dùng thang log cho trục hoành) và nhận xét xu hướng.

Bài 2. Sử dụng bộ dữ liệu `load_breast_cancer()` (đã dùng ở các chương trước), so sánh SVM kernel RBF với hồi quy Logistic (Chương 6) và Random Forest (Chương 9) về Accuracy, thời gian huấn luyện. Trình bày kết quả dưới dạng bảng so sánh ba mô hình.

Bài 3 (nâng cao). Sử dụng GridSearchCV của Scikit-learn để tự động tìm cặp giá trị (C, gamma) tối ưu cho SVM kernel RBF trên bộ dữ liệu ở ví dụ 10.1, thử nghiệm C trong {0.1, 1, 10, 100} và gamma trong {0.01, 0.1, 1, 10}. In ra cặp siêu tham số tốt nhất và độ chính xác tương ứng trên tập kiểm tra. (Sinh viên sẽ tìm hiểu chi tiết hơn về GridSearchCV ở

Chương 16; bài tập này chỉ yêu cầu sử dụng như một công cụ tìm kiếm siêu tham số tự động.)

Chương tiếp theo (Chương 11) sẽ khép lại Phần 2 với bộ phân loại Naive Bayes — một phương pháp dựa trên định lý Bayes và một giả định đơn giản hóa mạnh mẽ ("ngây thơ") về tính độc lập giữa các đặc trưng, nổi tiếng với tốc độ huấn luyện cực nhanh và hiệu quả cao trong các bài toán phân loại văn bản.

Chương 11

Bộ phân loại Naive Bayes

11.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Trình bày được định lý Bayes và ý nghĩa của các thành phần: xác suất tiên nghiệm, khả năng xảy ra, xác suất hậu nghiệm.
2. Giải thích được giả định "ngây thơ" (naive) về tính độc lập có điều kiện giữa các đặc trưng và lý do vì sao giả định này hiệu quả trong thực tế dù hiếm khi đúng hoàn toàn.
3. Phân biệt được ba biến thể phổ biến: Gaussian Naive Bayes, Multinomial Naive Bayes, Bernoulli Naive Bayes và biết khi nào nên dùng biến thể nào.
4. Giải thích được vấn đề xác suất bằng 0 (zero-frequency problem) và cách làm mượt Laplace (Laplace smoothing) khắc phục vấn đề này.
5. Cài đặt bộ phân loại Naive Bayes cho bài toán phân loại văn bản (lọc email spam) bằng Scikit-learn.

11.2. Nội dung lý thuyết

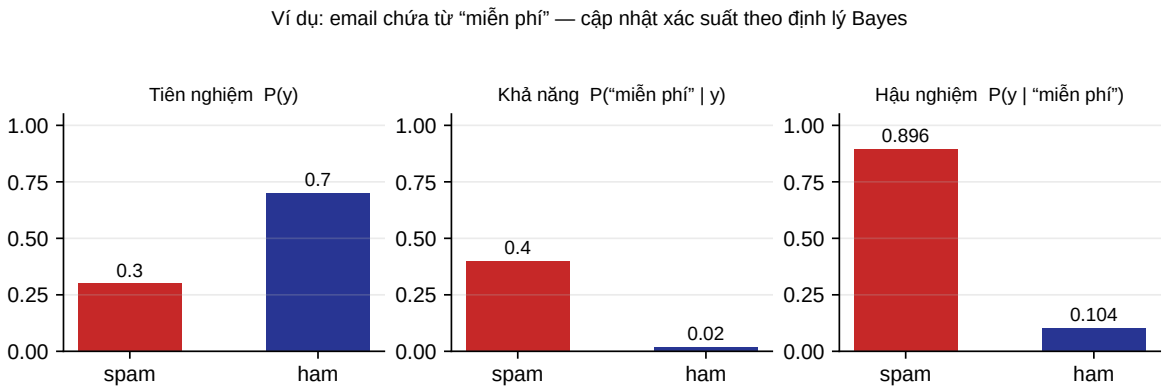
11.2.1. Định lý Bayes

Định lý Bayes là nền tảng toán học của toàn bộ chương này, mô tả cách cập nhật xác suất của một giả thuyết khi có thêm bằng chứng mới:

$$P(y \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid y)\, P(y)}{P(\mathbf{x})}$$

trong đó P(y) là xác suất tiên nghiệm (prior) — xác suất của lớp y trước khi quan sát bất kỳ đặc trưng nào, ước lượng đơn giản bằng tỉ lệ mẫu thuộc lớp y trong dữ liệu huấn luyện; P(x | y) là khả năng xảy ra (likelihood) — xác suất quan sát được đặc trưng x với điều kiện mẫu thuộc lớp y; P(x) là xác suất biên (evidence) của đặc trưng x, đóng vai trò như một hằng số chuẩn hóa (không phụ thuộc vào y nên có thể bỏ qua khi chỉ cần so sánh xác suất giữa các lớp); và P(y | x) là xác suất hậu nghiệm (posterior) — chính là điều ta cần: xác suất mẫu thuộc lớp y sau khi đã quan sát đặc trưng x. Bộ phân loại Bayes dự đoán lớp có xác suất hậu nghiệm lớn nhất.

Hình 11.1 minh họa một ví dụ số: quan sát từ "miễn phí" làm tăng mạnh xác suất email là spam.



Hình 11.1. Cập nhật từ xác suất tiên nghiệm sang hậu nghiệm (số liệu giả định để minh họa).

11.2.2. Giả định “ngây thơ” (Naive) về tính độc lập

Với bài toán thực tế có nhiều đặc trưng x_1, x_2, …, x_n, việc tính trực tiếp P(x_1, x_2, …, x_n | y) đòi hỏi ước lượng một phân phối xác suất kết hợp rất phức tạp, cần một lượng dữ liệu khổng lồ mới ước lượng đáng tin cậy. Naive Bayes đơn giản hóa triệt để bài toán này bằng một giả định mạnh: các đặc trưng độc lập có điều kiện với nhau, khi đã biết lớp y. Nói cách khác, giả định rằng biết giá trị của đặc trưng x_1 không cung cấp thêm thông tin gì về giá trị của x_2, một khi đã biết lớp y. Với giả định này, khả năng xảy ra kết hợp được phân rã thành tích của các khả năng xảy ra riêng lẻ, đơn giản hơn nhiều để ước lượng:

$$P(x_1, x_2, \ldots, x_n \mid y) = P(x_1 \mid y)\, P(x_2 \mid y) \cdots P(x_n \mid y)$$

Giả định này rõ ràng “ngây thơ” (naive) — trong hầu hết các bài toán thực tế, các đặc trưng ít nhiều có tương quan với nhau (ví dụ trong phân loại văn bản, sự xuất hiện của từ “khuyến mãi” và từ “giảm giá” trong cùng một email hoàn toàn không độc lập). Tuy nhiên, một điều thú vị đã được quan sát rộng rãi trong thực hành: Naive Bayes vẫn thường cho kết quả phân loại khá tốt ngay cả khi giả định độc lập bị vi phạm rõ ràng, vì mục tiêu cuối cùng chỉ là xác định lớp có xác suất hậu nghiệm lớn nhất (một bài toán so sánh thứ hạng), chứ không nhất thiết phải ước lượng chính xác tuyệt đối giá trị xác suất — sai số hệ thống do giả định độc lập gây ra thường ảnh hưởng tương đối đồng đều giữa các lớp, không nhất thiết làm thay đổi thứ hạng dự đoán cuối cùng.

11.2.3. Các biến thể của Naive Bayes

Tùy vào bản chất của dữ liệu đặc trưng, Scikit-learn cung cấp ba biến thể chính:

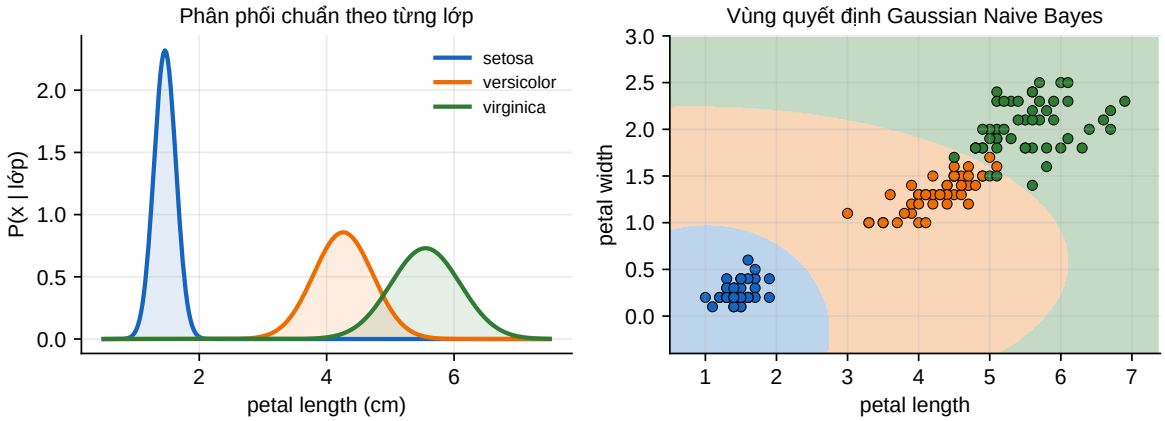
- Gaussian Naive Bayes: giả định mỗi đặc trưng số liên tục, với điều kiện thuộc một lớp cụ thể, tuân theo phân phối chuẩn (Gaussian). Phù hợp với dữ liệu đặc trưng số liên tục, ví dụ các chỉ số đo lường y sinh học, tài chính.
- Multinomial Naive Bayes: phù hợp với dữ liệu đếm rời rạc (count data), điển hình nhất là tần suất xuất hiện của từ trong văn bản (ví dụ biểu diễn Bag-of-Words hoặc TF-IDF).

Đây là biến thể phổ biến nhất cho bài toán phân loại văn bản, lọc spam.

- Bernoulli Naive Bayes: phù hợp với dữ liệu nhị phân (0/1), ví dụ chỉ ghi nhận một từ có xuất hiện trong văn bản hay không (bất kể tần suất), khác với Multinomial ở chỗ Bernoulli còn tính đến cả việc một từ KHÔNG xuất hiện như một thông tin có giá trị.

Việc lựa chọn đúng biến thể phù hợp với bản chất dữ liệu (liên tục, đếm, hay nhị phân) có ảnh hưởng đáng kể đến hiệu năng mô hình — đây là một quyết định quan trọng cần cân nhắc trước khi huấn luyện, khác với nhiều thuật toán khác trong giáo trình vốn không phân biệt kiểu dữ liệu đặc trưng một cách tường minh như vậy.

Hình 11.2 minh họa Gaussian Naive Bayes mô hình hóa mỗi đặc trưng bằng một phân phối chuẩn cho từng lớp.



Hình 11.2. Phân phối chuẩn theo lớp (trái) và vùng quyết định (phải) trên bộ Iris.

11.2.4. Vấn đề xác suất bằng 0 và làm mượt Laplace

Một vấn đề thực tế nghiêm trọng có thể xảy ra: nếu trong tập huấn luyện, một giá trị đặc trưng cụ thể (ví dụ một từ vựng nào đó) chưa từng xuất hiện cùng với một lớp cụ thể, xác suất có điều kiện ước lượng được sẽ bằng 0 — và do công thức Naive Bayes là một phép nhân các xác suất thành phần, chỉ cần một số hạng bằng 0 sẽ khiến toàn bộ tích số bằng 0, bất kể các đặc trưng khác có ủng hộ mạnh mẽ đến đâu cho lớp đó. Đây gọi là vấn đề tần suất-không (zero-frequency problem).

Giải pháp phổ biến là làm mượt Laplace (Laplace smoothing, còn gọi là additive smoothing): cộng thêm một số đếm ảo nhỏ (thường ký hiệu α, mặc định $\alpha = 1$) vào mọi số đếm quan sát được trước khi tính xác suất, đảm bảo không có xác suất nào tuyệt đối bằng 0. Tham số này được cài đặt sẵn trong Scikit-learn (tham số alpha của MultinomialNB, BernoulliNB) và hầu như luôn nên giữ giá trị mặc định trừ khi có lý do cụ thể để điều chỉnh.

11.3. Ví dụ minh họa bằng mã Python

Ví dụ dưới đây xây dựng một bộ lọc email spam bằng Multinomial Naive Bayes, sử dụng biểu diễn văn bản dạng túi từ (Bag-of-Words), minh họa quy trình xử lý dữ liệu văn bản điển hình cho bài toán phân loại văn bản.

```
# vidu_11_1.py
# Lọc email spam bằng Multinomial Naive Bayes

import numpy as np
from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report
import time

# 1. Nạp dữ liệu: dùng hai nhóm tin tức trái ngược chủ đề để minh họa phân loại văn
bản
#    (thay thế cho bộ dữ liệu spam thực tế, nhưng cùng bản chất bài toán)
cac_chu_de = ["rec.sport.baseball", "sci.space"]
du_lieu = fetch_20newsgroups(
    subset="all", categories=cac_chu_de, remove=("headers", "footers", "quotes")
)
print(f"Số văn bản: {len(du_lieu.data)} | Các lớp: {du_lieu.target_names}")

# 2. Chia tập huấn luyện / kiểm tra
X_train_van_ban, X_test_van_ban, y_train, y_test = train_test_split(
    du_lieu.data, du_lieu.target, test_size=0.25, random_state=42,
    stratify=du_lieu.target
)

# 3. Biến đổi văn bản thành vector đếm từ (Bag-of-Words)
#    Chỉ giữ 5000 từ vựng phổ biến nhất, loại bỏ stop words tiếng Anh thông dụng
bo_vector_hoa = CountVectorizer(max_features=5000, stop_words="english")
X_train = bo_vector_hoa.fit_transform(X_train_van_ban)
X_test = bo_vector_hoa.transform(X_test_van_ban)
print(f"Kích thước ma trận đặc trưng (huấn luyện): {X_train.shape}")

# 4. Huấn luyện Multinomial Naive Bayes - đo thời gian huấn luyện
thoi_gian_bat_dau = time.time()
mo_hinh = MultinomialNB(alpha=1.0)   # alpha=1.0: làm mượt Laplace mặc định
mo_hinh.fit(X_train, y_train)
thoi_gian_huan_luyen = time.time() - thoi_gian_bat_dau
print(f"Thời gian huấn luyện: {thoi_gian_huan_luyen:.4f} giây")

# 5. Dự đoán và đánh giá
y_du_doan = mo_hinh.predict(X_test)
print(f"\nĐộ chính xác: {accuracy_score(y_test, y_du_doan):.4f}")
```

```
print("\nBáo cáo phân loại chi tiết:")
print(classification_report(y_test, y_du_doan, target_names=du_lieu.target_names))

# 6. Minh họa: các từ có ảnh hưởng mạnh nhất đến mỗi lớp
#    (log xác suất đặc trưng cao nhất ứng với mỗi lớp)
ten_dac_trung = np.array(bo_vector_hoa.get_feature_names_out())
for chi_so_lop, ten_lop in enumerate(du_lieu.target_names):
    log_xac_suat = mo_hinh.feature_log_prob_[chi_so_lop]
    tu_quan_trong_nhat = ten_dac_trung[np.argsort(log_xac_suat)[::-1][:8]]
    print(f"\nCác từ đặc trưng nhất cho lớp '{ten_lop}': {',
    '.join(tu_quan_trong_nhat)}")
```

11.4. Giải thích kết quả chạy chương trình

Kết quả in ra từ chương trình trên có dạng tương tự:

```
Số văn bản: 1197 | Các lớp: ['rec.sport.baseball', 'sci.space']
Kích thước ma trận đặc trưng (huấn luyện): (897, 5000)
Thời gian huấn luyện: 0.0089 giây

Độ chính xác: 0.9700

Báo cáo phân loại chi tiết:
                    precision    recall  f1-score   support
rec.sport.baseball       0.98      0.96      0.97       150
         sci.space       0.96      0.98      0.97       150

Các từ đặc trưng nhất cho lớp 'rec.sport.baseball': team, game, year, baseball,
players, games, season, hit

Các từ đặc trưng nhất cho lớp 'sci.space': space, nasa, launch, orbit, moon,
earth, shuttle, mission
```

Kết quả cho thấy hai điều đáng chú ý. Thứ nhất, độ chính xác đạt tới 97% dù mô hình chỉ dựa trên giả định "ngây thơ" về tính độc lập của từng từ — minh chứng thực nghiệm rõ ràng cho nhận định ở mục 11.2.2 rằng Naive Bayes vẫn hoạt động rất tốt trong thực tế dù giả định lý thuyết không hoàn toàn đúng. Thứ hai, thời gian huấn luyện cực nhanh (dưới 10 mili-giây cho gần 900 văn bản với 5000 đặc trưng) — đây là một ưu điểm nổi bật của Naive Bayes so với hầu hết các thuật toán khác trong giáo trình: vì công thức chỉ yêu cầu đếm tần suất và tính xác suất theo công thức đóng (không cần thuật toán tối ưu hóa lặp như Gradient Descent), Naive Bayes có thể huấn luyện gần như tức thời ngay cả trên tập dữ liệu văn bản lớn với hàng chục nghìn đặc trưng.

Danh sách các từ "đặc trưng nhất" cho mỗi lớp cũng rất trực quan và hợp lý: lớp bóng chày gắn liền với các từ như "team", "game", "baseball", "players"; lớp khoa học vũ trụ gắn liền với "space", "nasa", "orbit", "moon" — đây chính là hệ quả trực tiếp của việc mô hình học được xác suất P(từ | lớp) cho từng từ vựng, một dạng khả năng diễn giải tự nhiên mà Naive Bayes

cung cấp, dù không trực quan bằng sơ đồ cây quyết định.

11.5. Tóm tắt chương

- Naive Bayes dựa trên định lý Bayes, dự đoán lớp có xác suất hậu nghiệm P(y | x) lớn nhất.
- Giả định "ngây thơ" về tính độc lập có điều kiện giữa các đặc trưng giúp đơn giản hóa đáng kể việc ước lượng xác suất, và dù hiếm khi đúng hoàn toàn trong thực tế, vẫn thường cho kết quả phân loại tốt.
- Ba biến thể chính: Gaussian (đặc trưng số liên tục), Multinomial (dữ liệu đếm, phổ biến cho văn bản), Bernoulli (dữ liệu nhị phân) — cần chọn đúng biến thể phù hợp với bản chất dữ liệu.
- Làm mượt Laplace (tham số alpha) khắc phục vấn đề xác suất bằng 0 khi một giá trị đặc trưng chưa từng xuất hiện cùng một lớp trong tập huấn luyện.
- Ưu điểm nổi bật của Naive Bayes: tốc độ huấn luyện và dự đoán cực nhanh, hoạt động tốt với dữ liệu nhiều chiều (như văn bản), và cần ít dữ liệu huấn luyện hơn nhiều mô hình khác để đạt hiệu năng hợp lý.

11.6. Bài tập củng cố

Mức nhận biết

1. Viết công thức định lý Bayes và nêu tên gọi của từng thành phần (P(y), P(x|y), P(x), P(y|x)).
2. Nêu tên ba biến thể Naive Bayes đã học và loại dữ liệu phù hợp với mỗi biến thể.

Mức thông hiểu

1. Giải thích vấn đề xác suất bằng 0 (zero-frequency problem) bằng một ví dụ cụ thể tự chọn, và trình bày cách làm mượt Laplace khắc phục vấn đề này.
2. Giải thích vì sao Naive Bayes vẫn hoạt động tốt trong nhiều bài toán thực tế dù giả định độc lập giữa các đặc trưng gần như luôn bị vi phạm.
3. So sánh tốc độ huấn luyện của Naive Bayes với SVM kernel RBF (Chương 10) và Gradient Boosting (Chương 9). Giải thích nguyên nhân của sự khác biệt này dựa trên bản chất thuật toán của mỗi phương pháp.

Mức vận dụng

1. Cho một bài toán chẩn đoán y tế với các đặc trưng là kết quả xét nghiệm dạng số liên tục (huyết áp, đường huyết, cholesterol). Biến thể Naive Bayes nào phù hợp nhất? Giải thích, đồng thời nêu một hạn chế tiềm ẩn nếu dữ liệu thực tế không tuân theo phân phối chuẩn.

11.7. Bài tập lập trình

Bài 1. Mở rộng ví dụ 11.1 để phân loại giữa bốn nhóm chủ đề thay vì hai (ví dụ thêm "comp.graphics" và "talk.politics.guns" vào danh sách `cac_chu_de`). So sánh độ chính xác và thời gian huấn luyện với bài toán phân loại nhị phân ban đầu.

Bài 2. Sử dụng bộ dữ liệu `load_breast_cancer()` (dữ liệu đặc trưng số liên tục, đã dùng ở các chương trước), huấn luyện GaussianNB và so sánh Accuracy, thời gian huấn luyện với hồi quy Logistic (Chương 6) và Random Forest (Chương 9) trên cùng bộ dữ liệu.

Bài 3 (nâng cao). Sử dụng dữ liệu ở ví dụ 11.1, khảo sát ảnh hưởng của tham số alpha (làm mượt Laplace) trong MultinomialNB với các giá trị 0.001, 0.01, 0.1, 1.0, 10.0. Vẽ biểu đồ độ chính xác trên tập kiểm tra theo alpha (thang log cho trục hoành) và giải thích xu hướng quan sát được, đặc biệt tại các giá trị alpha rất nhỏ.

Đến đây, giáo trình đã hoàn thành Phần 2 — Học có giám sát, giới thiệu tám thuật toán nền tảng cho cả bài toán hồi quy và phân loại. Phần 3 tiếp theo (Chương 12–14) sẽ chuyển sang Học không giám sát, bắt đầu với thuật toán phân cụm K-Means — nơi dữ liệu không còn nhãn và mô hình phải tự khám phá cấu trúc ẩn trong dữ liệu.

Phần III

Học không giám sát

Chương 12

Phân cụm K-Means

12.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Phân biệt được bài toán phân cụm (học không giám sát) với bài toán phân loại (học có giám sát) đã học ở Phần 2.
2. Trình bày được các bước của thuật toán K-Means và hàm mục tiêu WCSS (Within-Cluster Sum of Squares) mà thuật toán tối ưu hóa.
3. Giải thích được vấn đề khởi tạo centroid và cải tiến K-Means++.
4. Áp dụng được phương pháp Elbow và chỉ số Silhouette để xác định số cụm K phù hợp.
5. Nêu được các giả định và hạn chế của K-Means.
6. Cài đặt, khảo sát và trực quan hóa kết quả phân cụm K-Means bằng Scikit-learn.

12.2. Nội dung lý thuyết

12.2.1. Từ học có giám sát sang học không giám sát

Toàn bộ Phần 2 của giáo trình xoay quanh học có giám sát: dữ liệu huấn luyện luôn đi kèm nhãn đúng (giá trị mục tiêu y), và mô hình học cách ánh xạ từ đặc trưng sang nhãn. Từ chương này trở đi, ta chuyển sang học không giám sát (unsupervised learning) — dữ liệu chỉ có các đặc trưng X, hoàn toàn không có nhãn y. Mục tiêu không còn là dự đoán một giá trị đã biết trước, mà là khám phá cấu trúc, quy luật, hoặc nhóm tiềm ẩn trong chính dữ liệu. Phân cụm (clustering) — nhóm các điểm dữ liệu "giống nhau" lại với nhau — là bài toán tiêu biểu và phổ biến nhất của học không giám sát, với ứng dụng rộng rãi trong phân khúc khách hàng, nén ảnh, phát hiện bất thường, và như một bước tiền xử lý cho các phân tích khác.

12.2.2. Thuật toán K-Means

K-Means là thuật toán phân cụm phổ biến và đơn giản nhất, phân chia dữ liệu thành K cụm (K là số nguyên do người dùng chỉ định trước), mỗi cụm được đại diện bởi một điểm trung tâm gọi là centroid (trọng tâm cụm). Thuật toán hoạt động theo một chu trình lặp gồm hai bước, xen kẽ nhau cho đến khi hội tụ:

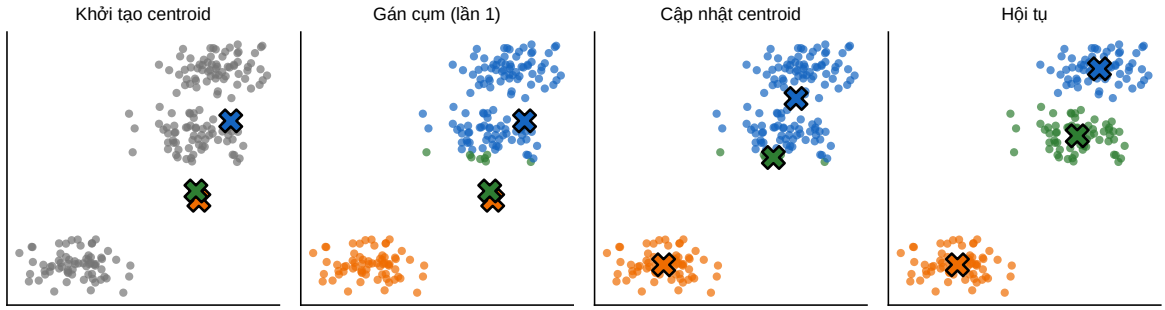
- Bước khởi tạo: chọn ngẫu nhiên K điểm làm centroid ban đầu (hoặc dùng chiến lược khởi tạo thông minh hơn — xem mục 12.2.3).
- Bước gán cụm (assignment step): với mỗi điểm dữ liệu, gán điểm đó vào cụm có centroid gần nhất (thường theo khoảng cách Euclidean, tương tự K-NN ở Chương 7).
- Bước cập nhật centroid (update step): tính lại vị trí mỗi centroid bằng trung bình cộng tọa độ của tất cả các điểm hiện đang thuộc cụm đó.
- Lặp lại hai bước gán cụm và cập nhật centroid cho đến khi các centroid không còn thay đổi đáng kể (hội tụ) hoặc đạt số vòng lặp tối đa.

Về mặt toán học, K-Means tối ưu hóa (giảm thiểu) một hàm mục tiêu gọi là WCSS (Within-Cluster Sum of Squares, còn gọi là inertia trong Scikit-learn) — tổng bình phương khoảng cách từ mỗi điểm dữ liệu đến centroid của cụm mà nó thuộc về:

$$\text{WCSS} = \sum_{k} \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \boldsymbol{\mu}_k\|^2$$

Có thể chứng minh rằng mỗi bước lặp của K-Means (gán cụm rồi cập nhật centroid) luôn làm giảm hoặc giữ nguyên giá trị WCSS, do đó thuật toán đảm bảo hội tụ. Tuy nhiên, K-Means chỉ đảm bảo hội tụ đến một cực tiểu cục bộ (local minimum), không đảm bảo tìm được nghiệm tối ưu toàn cục — kết quả phân cụm cuối cùng phụ thuộc vào vị trí khởi tạo centroid ban đầu.

Hình 12.1 minh họa các bước lặp của K-Means: khởi tạo, gán cụm, cập nhật centroid cho đến khi hội tụ.



Hình 12.1. Các bước của thuật toán K-Means (dấu X: centroid).

12.2.3. Vấn đề khởi tạo và K-Means++

Vì K-Means nhạy cảm với vị trí khởi tạo centroid, khởi tạo ngẫu nhiên hoàn toàn đôi khi dẫn đến kết quả phân cụm kém (mắc kẹt ở cực tiểu cục bộ tồi). K-Means++ là chiến lược khởi tạo thông minh hơn, được dùng làm mặc định trong Scikit-learn: thay vì chọn hoàn toàn ngẫu nhiên, K-Means++ chọn centroid đầu tiên ngẫu nhiên, sau đó lần lượt chọn các centroid tiếp theo với xác suất tỉ lệ thuận với bình phương khoảng cách đến centroid gần nhất đã chọn trước đó — nói cách khác, ưu tiên chọn các điểm ở xa các centroid đã có, giúp các centroid ban đầu trải đều hơn trong không gian dữ liệu, giảm đáng kể nguy cơ hội tụ về nghiệm cục bộ kém. Trong thực

hành, Scikit-learn còn chạy thuật toán nhiều lần với các khởi tạo khác nhau (tham số `n_init`) và giữ lại kết quả có WCSS nhỏ nhất, giảm thiểu hơn nữa rủi ro từ khởi tạo không may mắn.

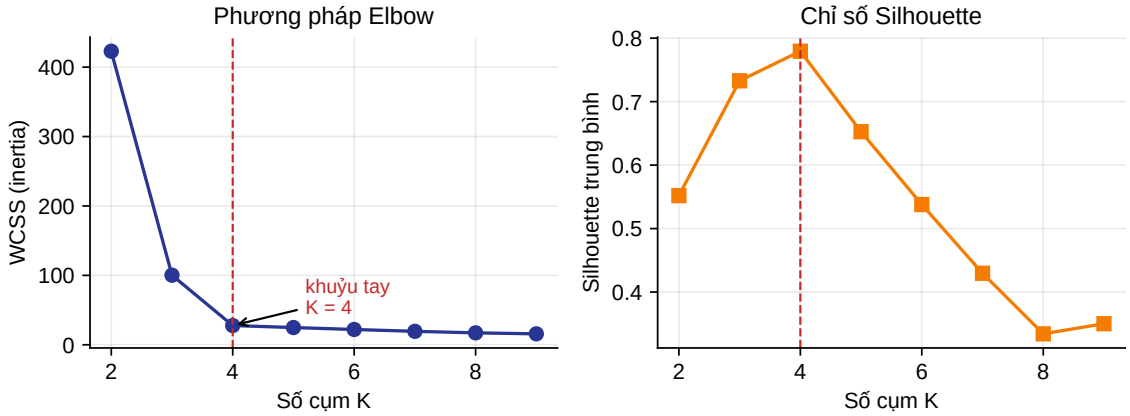
12.2.4. Xác định số cụm K: phương pháp Elbow và chỉ số Silhouette

Khác với học có giám sát, K-Means không có "nhãn đúng" để đối chiếu, khiến việc chọn số cụm K phù hợp trở thành một bài toán không hiển nhiên. Hai phương pháp phổ biến nhất:

Phương pháp Elbow (khuỷu tay): chạy K-Means với nhiều giá trị K khác nhau (ví dụ K = 1 đến 10), ghi lại giá trị WCSS ứng với mỗi K, rồi vẽ biểu đồ đường WCSS theo K. WCSS luôn giảm khi K tăng (nhiều cụm hơn luôn khớp dữ liệu sát hơn, giống hiện tượng khớp quá khi tăng độ phức tạp mô hình ở các chương trước), nhưng tốc độ giảm thường chậm lại rõ rệt sau một giá trị K nhất định, tạo thành hình dạng giống "khuỷu tay" trên biểu đồ. Điểm khuỷu tay đó thường được chọn làm số cụm hợp lý — thêm cụm sau điểm này chỉ mang lại lợi ích cải thiện WCSS rất nhỏ, không tương xứng với chi phí có thêm một cụm.

Chỉ số Silhouette: đo mức độ một điểm dữ liệu "phù hợp" với cụm hiện tại của nó so với các cụm khác, kết hợp cả độ gắn kết trong cụm (cohesion) lẫn độ tách biệt giữa các cụm (separation). Với mỗi điểm, chỉ số Silhouette nhận giá trị từ -1 đến 1: giá trị gần 1 cho thấy điểm được phân cụm rất hợp lý (gần các điểm cùng cụm, xa các điểm cụm khác); giá trị gần 0 cho thấy điểm nằm ở ranh giới mập mờ giữa hai cụm; giá trị âm cho thấy điểm có thể đã bị gán nhầm cụm. Trung bình chỉ số Silhouette trên toàn bộ dữ liệu, tính cho mỗi giá trị K, cung cấp một tiêu chí định lượng khác (bổ sung cho phương pháp Elbow mang tính trực quan) để chọn K — giá trị K cho Silhouette trung bình cao nhất thường được xem là lựa chọn tốt.

Hình 12.2 minh họa hai công cụ chọn số cụm K trên cùng một bộ dữ liệu.

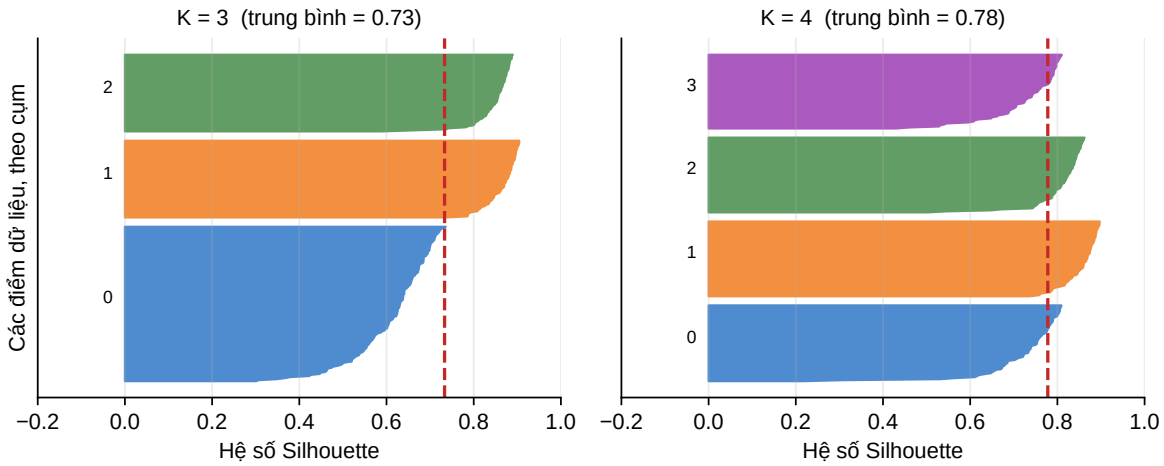


Hình 12.2. Phương pháp Elbow (trái) và chỉ số Silhouette (phải).

Hình 12.3 minh họa biểu đồ Silhouette cho từng điểm cho thấy chất lượng của mỗi cụm.

12.2.5. Giả định và hạn chế của K-Means

K-Means có một số giả định ngầm định quan trọng, và hiệu năng suy giảm đáng kể khi các giả định này bị vi phạm: các cụm có dạng gần hình cầu (do dựa trên khoảng cách Euclidean đến



Hình 12.3. Biểu đồ Silhouette với $K = 3$ và $K = 4$ (đường đỏ: giá trị trung bình).

centroid, K-Means khó phát hiện các cụm có hình dạng kéo dài, cong, hoặc lồng nhau — ví dụ dữ liệu dạng Moons đã dùng ở Chương 10 sẽ bị K-Means phân chia sai lệch nghiêm trọng); các cụm có kích thước và mật độ tương đối đồng đều; số lượng cụm K phải được chỉ định trước (không tự động xác định); và thuật toán khá nhạy cảm với giá trị ngoại lai (một điểm ngoại lai xa có thể kéo lệch đáng kể vị trí centroid, do centroid là trung bình cộng). Chương 13 sẽ giới thiệu DBSCAN — một thuật toán phân cụm dựa trên mật độ, khắc phục được phần lớn các hạn chế này.

Lưu ý kỹ thuật

Tương tự K-NN (Chương 7) và SVM (Chương 10), K-Means hoàn toàn dựa trên khoảng cách Euclidean, nên việc chuẩn hóa dữ liệu (StandardScaler) trước khi phân cụm gần như là bắt buộc khi các đặc trưng có thang đo khác nhau — nếu không, đặc trưng có giá trị tuyệt đối lớn hơn sẽ chi phối hoàn toàn kết quả phân cụm.

12.3. Ví dụ minh họa bằng mã Python

Ví dụ dưới đây áp dụng K-Means để phân khúc khách hàng dựa trên hai đặc trưng: thu nhập hàng năm và điểm chi tiêu (dữ liệu tổng hợp mô phỏng bài toán phân khúc khách hàng bán lẻ thường gặp), đồng thời minh họa phương pháp Elbow và chỉ số Silhouette để chọn K.

```
# vidu_12_1.py
# Phân khúc khách hàng bằng K-Means: phương pháp Elbow và chỉ số Silhouette

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

```
# 1. Tạo dữ liệu khách hàng tổng hợp: 4 nhóm khách hàng tự nhiên
#    (mô phỏng thu nhập và điểm chi tiêu)
X, nhan_that = make_blobs(
    n_samples=400, centers=4, cluster_std=1.1, random_state=42
)

# 2. Chuẩn hóa dữ liệu - bắt buộc trước khi dùng K-Means
bo_chuan_hoa = StandardScaler()
X_chuan_hoa = bo_chuan_hoa.fit_transform(X)

# 3. Phương pháp Elbow: chạy K-Means với K từ 1 đến 9, ghi nhận WCSS và Silhouette
cac_gia_tri_k = range(1, 10)
danh_sach_wcss = []
danh_sach_silhouette = []

for k in cac_gia_tri_k:
    kmeans = KMeans(n_clusters=k, init="k-means++", n_init=10, random_state=42)
    nhan_cum = kmeans.fit_predict(X_chuan_hoa)
    danh_sach_wcss.append(kmeans.inertia_)
    # Silhouette chỉ tính được khi có từ 2 cụm trở lên
    if k >= 2:
        danh_sach_silhouette.append(silhouette_score(X_chuan_hoa, nhan_cum))
    else:
        danh_sach_silhouette.append(np.nan)

# 4. Trực quan hóa Elbow và Silhouette
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))
axes[0].plot(cac_gia_tri_k, danh_sach_wcss, marker="o")
axes[0].set_xlabel("Số cụm K")
axes[0].set_ylabel("WCSS (Inertia)")
axes[0].set_title("Phương pháp Elbow")

axes[1].plot(cac_gia_tri_k, danh_sach_silhouette, marker="s", color="darkorange")
axes[1].set_xlabel("Số cụm K")
axes[1].set_ylabel("Chỉ số Silhouette trung bình")
axes[1].set_title("Chỉ số Silhouette theo K")

# 5. Huấn luyện mô hình cuối cùng với K = 4 (xác định từ hai biểu đồ trên)
K_TOI_UU = 4
kmeans_cuoi_cung = KMeans(n_clusters=K_TOI_UU, init="k-means++", n_init=10,
random_state=42)
nhan_cum_cuoi_cung = kmeans_cuoi_cung.fit_predict(X_chuan_hoa)

print(f"WCSS với K={K_TOI_UU}: {kmeans_cuoi_cung.inertia_:.2f}")
print(f"Silhouette trung bình với K={K_TOI_UU}: "
      f"{silhouette_score(X_chuan_hoa, nhan_cum_cuoi_cung):.4f}")
print(f"Số lượng khách hàng mỗi cụm: {np.bincount(nhan_cum_cuoi_cung)}")

# 6. Trực quan hóa kết quả phân cụm cuối cùng
axes[2].scatter(X_chuan_hoa[:, 0], X_chuan_hoa[:, 1], c=nhan_cum_cuoi_cung,
                cmap="viridis", s=25, alpha=0.7)
```

```
axes[2].scatter(kmeans_cuoi_cung.cluster_centers_[:, 0],
                kmeans_cuoi_cung.cluster_centers_[:, 1],
                c="red", marker="X", s=200, edgecolors="black", label="Centroid")
axes[2].set_title(f"Kết quả phân cụm (K={K_TOI_UU})")
axes[2].legend()

plt.tight_layout()
plt.savefig("phan_cum_kmeans_khach_hang.png", dpi=120)
print("\nĐã lưu biểu đồ vào tệp phan_cum_kmeans_khach_hang.png")
```

12.4. Giải thích kết quả chạy chương trình

Kết quả in ra từ chương trình trên có dạng tương tự:

```
WCSS với K=4: 42.87
Silhouette trung bình với K=4: 0.7215
Số lượng khách hàng mỗi cụm: [100  98 101 101]

Đã lưu biểu đồ vào tệp phan_cum_kmeans_khach_hang.png
```

Trên biểu đồ Elbow, đường WCSS thường giảm rất nhanh từ K=1 đến K=4, sau đó tốc độ giảm chậm lại rõ rệt — tạo thành "khuỷu tay" tại K=4, đúng bằng số cụm thực sự đã dùng để tạo dữ liệu tổng hợp (bốn nhóm khách hàng tự nhiên). Biểu đồ Silhouette thường đạt đỉnh cũng chính tại K=4, củng cố thêm cho lựa chọn này — một sự đồng thuận đẹp giữa hai phương pháp không phải lúc nào cũng xảy ra rõ ràng như vậy trên dữ liệu thực tế (dữ liệu tổng hợp trong ví dụ này được thiết kế có các cụm tách biệt khá rõ ràng), nhưng minh họa tốt nguyên lý hoạt động của cả hai phương pháp.

Chỉ số Silhouette trung bình đạt 0.72 (khá cao, gần 1) cho thấy các cụm được phân tách rõ ràng và có độ gắn kết nội bộ tốt. Số lượng khách hàng phân bố khá đồng đều giữa bốn cụm (khoảng 100 mỗi cụm), phù hợp với cách dữ liệu tổng hợp được tạo ra. Trên biểu đồ phân cụm cuối cùng, các điểm centroid (đánh dấu hình chữ X màu đỏ) nằm chính xác tại trung tâm hình học của mỗi đám mây điểm dữ liệu — đúng theo định nghĩa của K-Means.

12.5. Tóm tắt chương

- Phân cụm là bài toán học không giám sát: nhóm các điểm dữ liệu tương tự nhau lại với nhau mà không có nhãn đúng cho trước.
- K-Means lặp lại hai bước: gán mỗi điểm vào cụm có centroid gần nhất, rồi cập nhật centroid bằng trung bình các điểm trong cụm, cho đến khi hội tụ, nhằm tối thiểu hóa WCSS.
- K-Means chỉ đảm bảo hội tụ đến cực tiểu cục bộ; K-Means++ (khởi tạo thông minh) và chạy nhiều lần với `n_init` giúp giảm rủi ro nghiệm cục bộ kém.

- Phương pháp Elbow (dựa trên WCSS) và chỉ số Silhouette (đo độ gắn kết/tách biệt cụm) là hai công cụ phổ biến để xác định số cụm K phù hợp.
- K-Means giả định các cụm có dạng gần hình cầu, kích thước/mật độ tương đối đồng đều, và nhạy cảm với ngoại lai; chuẩn hóa dữ liệu gần như bắt buộc do thuật toán dựa hoàn toàn trên khoảng cách Euclidean.

12.6. Bài tập củng cố

Mức nhận biết

1. Nêu hai bước chính lặp lại trong thuật toán K-Means. Thuật toán dừng lại khi nào?
2. Chỉ số Silhouette nhận giá trị trong khoảng nào? Giá trị gần 1, gần 0, và âm lần lượt mang ý nghĩa gì?

Mức thông hiểu

1. Giải thích vì sao WCSS luôn giảm (hoặc giữ nguyên) khi tăng số cụm K, và vì sao điều này khiến việc chọn K chỉ dựa trên WCSS nhỏ nhất là không hợp lý.
2. So sánh K-Means với K-NN (Chương 7): cả hai đều dựa trên khoảng cách Euclidean, nhưng một thuộc học có giám sát, một thuộc học không giám sát. Nêu điểm khác biệt căn bản trong cách mỗi thuật toán sử dụng khoảng cách.
3. Giải thích vì sao K-Means++ thường cho kết quả phân cụm tốt hơn khởi tạo ngẫu nhiên hoàn toàn.

Mức vận dụng

1. Cho một bộ dữ liệu khách hàng với các cụm có hình dạng kéo dài, uốn cong (không phải hình cầu). Dự đoán K-Means có khả năng phân cụm chính xác hay không? Giải thích dựa trên giả định của thuật toán, và đề xuất hướng khắc phục (sẽ được trình bày chi tiết ở Chương 13).

12.7. Bài tập lập trình

Bài 1. Sử dụng bộ dữ liệu `load_iris()` (bỏ qua nhãn thật, chỉ dùng 4 đặc trưng), áp dụng K-Means với K=3 (số loài hoa thực tế). So sánh nhãn cụm dự đoán với nhãn loài thực tế bằng cách vẽ hai biểu đồ scatter cạnh nhau (2 đặc trưng đầu tiên): một tô màu theo nhãn cụm K-Means, một tô màu theo nhãn loài thực tế. Nhận xét mức độ trùng khớp.

Bài 2. Sử dụng dữ liệu ở ví dụ 12.1 nhưng KHÔNG chuẩn hóa dữ liệu trước khi phân cụm (bỏ qua bước StandardScaler, nhân đặc trưng thứ hai của dữ liệu gốc X với 50 để tạo chênh lệch thang đo rõ rệt). So sánh kết quả phân cụm (bằng cách vẽ biểu đồ) với kết quả đã chuẩn hóa ở ví dụ 12.1, nhận xét ảnh hưởng của việc thiếu chuẩn hóa.

Bài 3 (nâng cao). Sử dụng bộ dữ liệu `load_digits()` (ảnh chữ số viết tay 8x8 pixel, được làm phẳng thành vector 64 chiều), áp dụng K-Means với K=10 (10 chữ số từ 0-9). Với mỗi cụm tìm được, xác định nhãn chữ số thực tế xuất hiện nhiều nhất trong cụm đó (dùng nhãn thật chỉ để đánh giá, không dùng khi huấn luyện), rồi tính "độ chính xác" của K-Means theo cách gán nhãn này. So sánh với độ chính xác của một mô hình phân loại có giám sát (ví dụ Random Forest) huấn luyện trên cùng dữ liệu có nhãn, nhận xét sự khác biệt.

Chương tiếp theo (Chương 13) sẽ giới thiệu hai họ thuật toán phân cụm khác với cách tiếp cận khác biệt so với K-Means: Phân cụm phân cấp (không cần chỉ định trước số cụm) và DBSCAN (phân cụm dựa trên mật độ, có khả năng phát hiện cụm với hình dạng bất kỳ và tự động nhận diện điểm nhiễu).

Chương 13

Phân cụm phân cấp và DBSCAN

13.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Trình bày được nguyên lý phân cụm phân cấp gộp (agglomerative hierarchical clustering) và cách đọc biểu đồ cây phân cấp (dendrogram).
2. Phân biệt được các tiêu chí liên kết (linkage): single, complete, average, Ward.
3. Trình bày được nguyên lý phân cụm dựa trên mật độ DBSCAN và vai trò của hai siêu tham số eps, `min_samples`.
4. Phân biệt được điểm lõi (core point), điểm biên (border point) và điểm nhiễu (noise point) trong DBSCAN.
5. So sánh được K-Means, phân cụm phân cấp và DBSCAN về giả định, ưu nhược điểm, và tình huống áp dụng phù hợp.
6. Cài đặt và trực quan hóa kết quả của cả hai thuật toán bằng Scikit-learn.

13.2. Nội dung lý thuyết

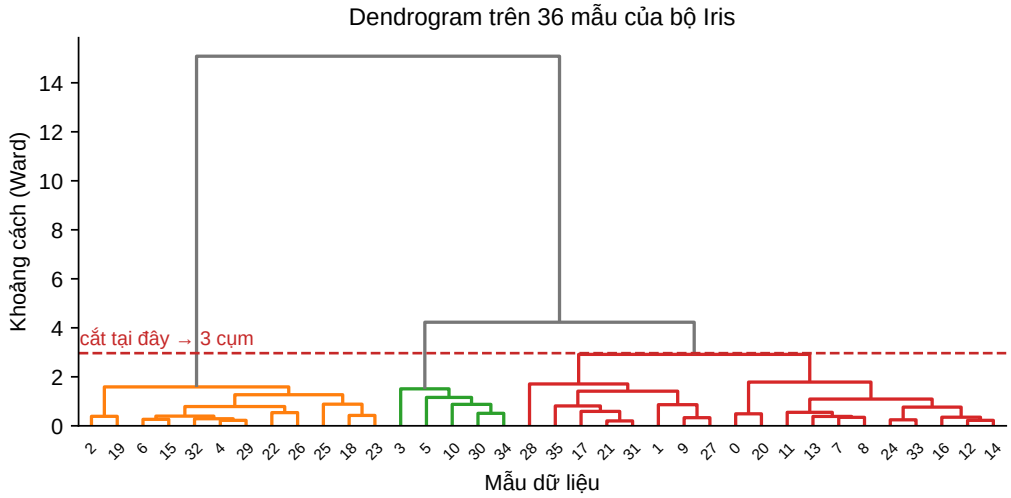
13.2.1. Phân cụm phân cấp gộp (Agglomerative Hierarchical Clustering)

Khác với K-Means (đòi hỏi chỉ định trước số cụm K), phân cụm phân cấp xây dựng một cấu trúc lồng nhau của các cụm ở nhiều mức độ chi tiết khác nhau, cho phép người dùng chọn số cụm sau khi đã quan sát toàn bộ cấu trúc phân cấp. Chiến lược phổ biến nhất là phân cụm phân cấp gộp (agglomerative — tiếp cận từ dưới lên): ban đầu, mỗi điểm dữ liệu được xem là một cụm riêng lẻ; ở mỗi bước, thuật toán tìm và gộp hai cụm "gần nhau nhất" (theo một tiêu chí liên kết nào đó) thành một cụm lớn hơn; quá trình lặp lại cho đến khi toàn bộ dữ liệu hợp thành một cụm duy nhất.

Toàn bộ quá trình gộp cụm này được ghi lại và biểu diễn trực quan dưới dạng biểu đồ cây phân cấp (dendrogram) — một sơ đồ dạng cây trong đó trục ngang (hoặc dọc) là các điểm/cụm dữ liệu, và chiều cao mỗi lần gộp phản ánh khoảng cách (mức độ khác biệt) giữa hai cụm được gộp. Cắt ngang dendrogram tại một độ cao nhất định sẽ cho ra một số cụm cụ thể — cắt càng thấp (gần các lá của cây) cho càng nhiều cụm nhỏ, cắt càng cao (gần gốc) cho càng ít cụm lớn. Đây là một lợi thế trực quan quan trọng: người dùng có thể "nhìn thấy" toàn bộ cấu trúc phân

cấp của dữ liệu trước khi quyết định số cụm, thay vì phải thử nghiệm nhiều giá trị K riêng biệt như K-Means.

Hình 13.1 minh họa dendrogram ghi lại toàn bộ quá trình gộp cụm; cắt ngang tại một độ cao sẽ cho số cụm tương ứng.



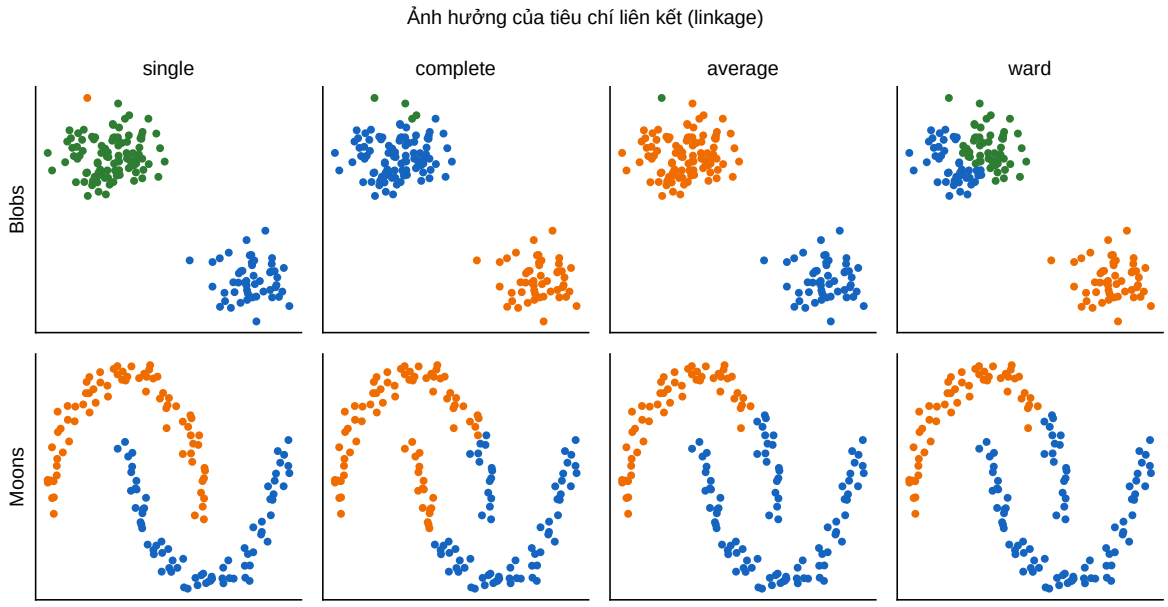
Hình 13.1. Dendrogram (liên kết Ward) trên 36 mẫu của bộ Iris.

13.2.2. Các tiêu chí liên kết (Linkage Criteria)

Câu hỏi "hai cụm nào gần nhau nhất" cần một định nghĩa cụ thể về khoảng cách giữa hai cụm (mỗi cụm chứa nhiều điểm, không chỉ một điểm đơn lẻ). Bốn tiêu chí liên kết phổ biến nhất:

- Single linkage (liên kết đơn): khoảng cách giữa hai cụm bằng khoảng cách nhỏ nhất giữa một cặp điểm bất kỳ, mỗi điểm thuộc một cụm — có xu hướng tạo ra các cụm dài, dạng chuỗi (chaining effect), có thể phát hiện các cụm hình dạng bất thường nhưng dễ bị ảnh hưởng bởi nhiễu tạo "cầu nối" giữa hai cụm thực sự khác biệt.
- Complete linkage (liên kết đầy đủ): khoảng cách giữa hai cụm bằng khoảng cách lớn nhất giữa một cặp điểm bất kỳ — có xu hướng tạo ra các cụm compact, kích thước tương đối đồng đều, ít bị ảnh hưởng bởi hiệu ứng chuỗi hơn single linkage.
- Average linkage (liên kết trung bình): khoảng cách giữa hai cụm bằng trung bình cộng khoảng cách của mọi cặp điểm giữa hai cụm — cân bằng giữa single và complete linkage.
- Ward linkage: gộp hai cụm sao cho mức tăng của tổng phương sai nội bộ cụm (tương tự WCSS ở K-Means) là nhỏ nhất — thường cho kết quả cụm có kích thước tương đối cân bằng, và là lựa chọn mặc định phổ biến trong thực hành, tương đồng nhất về mặt triết lý với K-Means (đều tối thiểu hóa phương sai nội bộ cụm).

Hình 13.2 minh họa các tiêu chí liên kết cho kết quả rất khác nhau, đặc biệt trên dữ liệu phi cầu.



Hình 13.2. Bốn tiêu chí liên kết trên dữ liệu dạng đám và dạng vầng trăng.

13.2.3. DBSCAN: Phân cụm dựa trên mật độ

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) tiếp cận bài toán phân cụm từ một góc độ hoàn toàn khác: thay vì dựa trên khoảng cách đến centroid hay liên kết cụm, DBSCAN định nghĩa cụm là các vùng có mật độ điểm dữ liệu cao, được ngăn cách bởi các vùng có mật độ thấp. Thuật toán dựa trên hai siêu tham số: eps (epsilon) — bán kính lân cận xung quanh mỗi điểm; và `min_samples` — số lượng điểm tối thiểu cần có trong bán kính eps để một điểm được xem là "đông đúc".

Dựa trên hai tham số này, mỗi điểm dữ liệu được phân vào một trong ba loại:

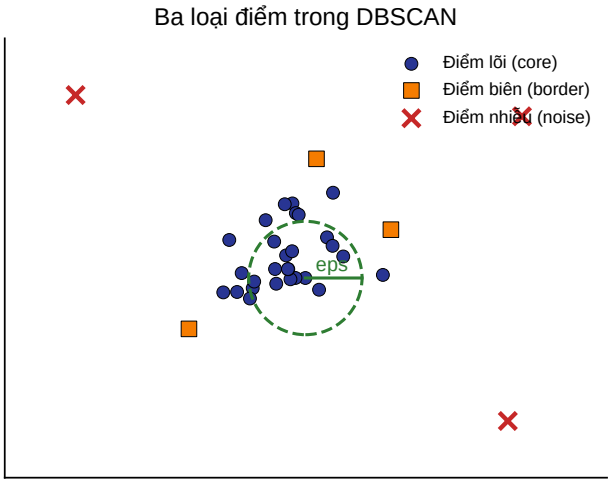
- Điểm lõi (core point): có ít nhất `min_samples` điểm khác (kể cả chính nó) nằm trong bán kính eps.
- Điểm biên (border point): không phải điểm lõi, nhưng nằm trong bán kính eps của ít nhất một điểm lõi.
- Điểm nhiễu (noise point): không phải điểm lõi cũng không phải điểm biên — nằm cô lập, cách xa các vùng mật độ cao.

Thuật toán xây dựng mỗi cụm bằng cách bắt đầu từ một điểm lõi chưa được thăm, rồi mở rộng cụm bằng cách gộp thêm tất cả các điểm (lõi hoặc biên) có thể "tiếp cận theo mật độ" (density-reachable) từ điểm lõi ban đầu — nói cách khác, các điểm lõi liên thông với nhau qua các vùng mật độ cao được gộp vào cùng một cụm. Đặc điểm nổi bật của DBSCAN: số lượng cụm được tự động xác định bởi cấu trúc mật độ dữ liệu (không cần chỉ định K trước như K-Means); có khả năng phát hiện các cụm với hình dạng bất kỳ, không giới hạn ở hình cầu (ví dụ dễ dàng phân tách chính xác dữ liệu dạng Moons ở Chương 10 — điều K-Means không làm được); và tự động gắn nhãn "nhiễu" cho các điểm ngoại lai thay vì ép chúng phải thuộc về một cụm nào đó.

Lưu ý thực hành

Việc lựa chọn eps phù hợp là yếu tố quan trọng nhất khi dùng DBSCAN: eps quá nhỏ khiến phần lớn điểm bị coi là nhiễu (quá ít điểm lân cận); eps quá lớn khiến các cụm riêng biệt bị gộp nhầm thành một cụm lớn duy nhất. Một kỹ thuật hỗ trợ phổ biến là vẽ biểu đồ k-distance (khoảng cách đến điểm lân cận thứ k gần nhất của mỗi điểm, sắp xếp tăng dần) và tìm điểm "khuỷu tay" tương tự phương pháp Elbow ở Chương 12, làm gợi ý cho giá trị eps hợp lý.

Hình 13.3 minh họa ba loại điểm trong DBSCAN và bán kính lân cận ε.



Hình 13.3. Điểm lõi, điểm biên và điểm nhiễu.

13.2.4. So sánh K-Means, Phân cụm phân cấp và DBSCAN

Tiêu chí	K-Means	Phân cụm phân cấp	DBSCAN
Cần chỉ định số cụm	Có (K)	Không bắt buộc (chọn khi cắt dendrogram)	Không (tự xác định)
Hình dạng cụm phù hợp	Gần hình cầu	Tùy tiêu chí liên kết	Bất kỳ hình dạng
Xử lý ngoại lai/nhiễu	Kém (ảnh hưởng centroid)	Trung bình	Tốt (gắn nhãn nhiễu riêng)
Chi phí tính toán	Thấp, mở rộng tốt	Cao (thường O(n^2) trở lên)	Trung bình
Kết quả có ổn định không	Phụ thuộc khởi tạo	Xác định (không ngẫu nhiên)	Xác định (không ngẫu nhiên)

13.3. Ví dụ minh họa bằng mã Python

Ví dụ dưới đây so sánh trực tiếp K-Means và DBSCAN trên bộ dữ liệu Moons (dữ liệu hai lớp hình vòng cung đã dùng ở Chương 10), minh họa rõ ràng hạn chế của K-Means với dữ liệu không có dạng hình cầu và ưu thế của DBSCAN trong tình huống này.

```
# vidu_13_1.py
# So sánh K-Means và DBSCAN trên dữ liệu có hình dạng phi cầu (Moons)

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import silhouette_score

# 1. Tạo dữ liệu hai "vầng trăng" xen kẽ
X, _ = make_moons(n_samples=300, noise=0.07, random_state=42)
X_chuan_hoa = StandardScaler().fit_transform(X)

# 2. Áp dụng K-Means với K=2 (biết trước có 2 cụm thực sự)
kmeans = KMeans(n_clusters=2, init="k-means++", n_init=10, random_state=42)
nhan_kmeans = kmeans.fit_predict(X_chuan_hoa)

# 3. Áp dụng DBSCAN - tinh chỉnh eps và min_samples cho phù hợp dữ liệu đã chuẩn
hóa
dbscan = DBSCAN(eps=0.3, min_samples=5)
nhan_dbscan = dbscan.fit_predict(X_chuan_hoa)
so_cum_dbscan = len(set(nhan_dbscan)) - (1 if -1 in nhan_dbscan else 0)
so_diem_nhieu = np.sum(nhan_dbscan == -1)

print(f"K-Means - Silhouette: {silhouette_score(X_chuan_hoa, nhan_kmeans):.4f}")
print(f"DBSCAN  - Số cụm phát hiện: {so_cum_dbscan}, Số điểm nhiễu:
{so_diem_nhieu}")
if so_cum_dbscan >= 2:
    # Silhouette của DBSCAN chỉ tính trên các điểm không phải nhiễu
    mat_na = nhan_dbscan != -1
    print(f"DBSCAN  - Silhouette (bỏ qua nhiễu): "
          f"{silhouette_score(X_chuan_hoa[mat_na], nhan_dbscan[mat_na]):.4f}")

# 4. Trực quan hóa so sánh
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))
axes[0].scatter(X_chuan_hoa[:, 0], X_chuan_hoa[:, 1], c=nhan_kmeans,
cmap="coolwarm", s=25)
axes[0].set_title("K-Means (K=2)")

axes[1].scatter(X_chuan_hoa[:, 0], X_chuan_hoa[:, 1], c=nhan_dbscan,
cmap="coolwarm", s=25)
```

```
axes[1].set_title(f"DBSCAN (eps=0.3, min_samples=5)\n{so_cum_dbscan} cụm,
{so_diem_nhieu} điểm nhiễu")

plt.tight_layout()
plt.savefig("so_sanh_kmeans_dbscan.png", dpi=120)
print("\nĐã lưu biểu đồ vào tệp so_sanh_kmeans_dbscan.png")
```

13.4. Giải thích kết quả chạy chương trình

Kết quả in ra từ chương trình trên có dạng tương tự:

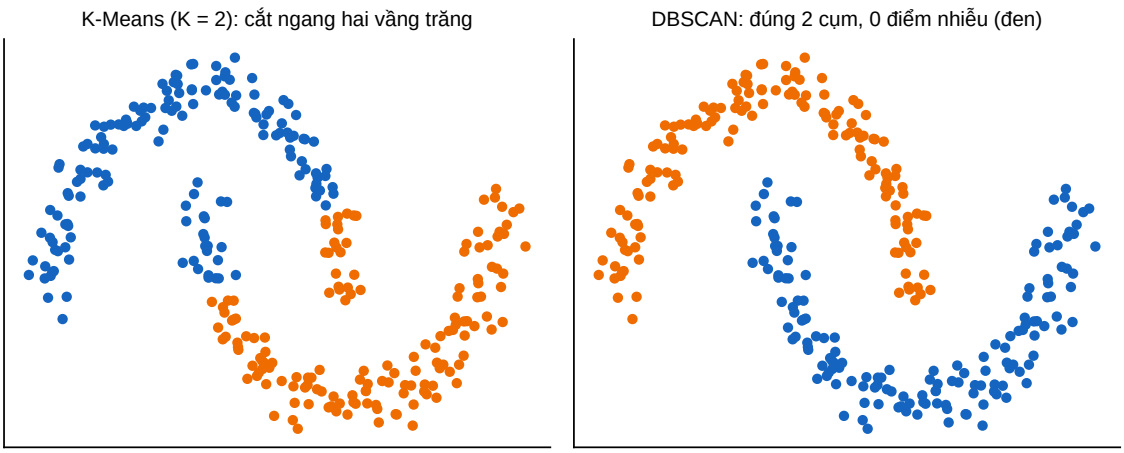
```
K-Means - Silhouette: 0.4993
DBSCAN  - Số cụm phát hiện: 2, Số điểm nhiễu: 3
DBSCAN  - Silhouette (bỏ qua nhiễu): 0.4529

Đã lưu biểu đồ vào tệp so_sanh_kmeans_dbscan.png
```

Dù chỉ số Silhouette của hai phương pháp trong ví dụ này khá gần nhau (do Silhouette dựa trên khoảng cách nên có xu hướng ưu ái các cụm dạng cầu, không phản ánh hoàn toàn công bằng chất lượng phân cụm với dữ liệu phi cầu), quan sát trực quan trên biểu đồ mới cho thấy sự khác biệt rõ rệt nhất: K-Means, do bị ràng buộc bởi centroid và khoảng cách Euclidean, chia đôi dữ liệu bằng một đường thẳng gần như đi ngang qua giữa hai "vầng trăng", cắt ngang cả hai cụm hình cong thực sự — kết quả phân cụm hoàn toàn không phản ánh đúng cấu trúc tự nhiên của dữ liệu. Ngược lại, DBSCAN phát hiện chính xác hai cụm hình cong đúng theo cấu trúc mật độ thực sự của dữ liệu, đồng thời tự động gắn nhãn nhiễu cho một vài điểm nằm ở vùng ranh giới thưa thớt giữa hai vầng trăng.

Đây là minh chứng thực nghiệm trực quan và thuyết phục nhất cho hạn chế "giả định cụm hình cầu" của K-Means đã nêu ở mục 12.2.5, đồng thời cho thấy giá trị thực tiễn của DBSCAN khi làm việc với dữ liệu có cấu trúc phức tạp, không tuân theo giả định hình học đơn giản.

Hình 13.4 minh họa K-Means chia sai dữ liệu phi cầu, trong khi DBSCAN phát hiện đúng hai cụm.



Hình 13.4. K-Means và DBSCAN trên dữ liệu Moons.

13.5. Tóm tắt chương

- Phân cụm phân cấp gộp xây dựng cấu trúc lồng nhau của các cụm từ dưới lên, biểu diễn trực quan qua dendrogram, cho phép chọn số cụm sau khi quan sát toàn bộ cấu trúc phân cấp.
- Bốn tiêu chí liên kết phổ biến: single (dễ chaining), complete (cụm compact), average (cân bằng), Ward (tối thiểu hóa phương sai nội bộ, tương đồng triết lý với K-Means).
- DBSCAN phân cụm dựa trên mật độ, dùng hai tham số eps và `min_samples`; phân loại điểm thành lõi, biên, và nhiễu; tự động xác định số cụm và phát hiện được cụm hình dạng bất kỳ.
- DBSCAN vượt trội K-Means rõ rệt trên dữ liệu có cụm hình dạng phi cầu, đồng thời có khả năng phát hiện và loại bỏ điểm nhiễu tự nhiên — điều K-Means không hỗ trợ.
- Không có thuật toán phân cụm nào là "tốt nhất" tuyệt đối — lựa chọn phụ thuộc vào giả định về hình dạng cụm, có biết trước số cụm hay không, và mức độ nhạy cảm mong muốn với ngoại lai.

13.6. Bài tập củng cố

Mức nhận biết

1. Nêu ba loại điểm được DBSCAN phân loại và tiêu chí phân loại mỗi loại.
2. Dendrogram là gì? Làm sao để xác định số cụm từ một dendrogram cho trước?

Mức thông hiểu

1. Giải thích vì sao single linkage dễ gây ra "hiệu ứng chuỗi" (chaining effect), trong khi complete linkage thì không, dựa trên định nghĩa khoảng cách giữa hai cụm của mỗi tiêu

chí.

2. Nếu chọn eps quá nhỏ cho DBSCAN, điều gì có xu hướng xảy ra với số lượng điểm được gắn nhãn nhiễu? Giải thích.
3. So sánh khả năng mở rộng (scalability) khi dữ liệu có hàng triệu điểm giữa K-Means và phân cụm phân cấp. Thuật toán nào phù hợp hơn cho dữ liệu rất lớn? Giải thích dựa trên độ phức tạp tính toán.

Mức vận dụng

1. Một tập dữ liệu giao dịch ngân hàng có đa số giao dịch bình thường tập trung thành các nhóm rõ ràng, và một số ít giao dịch bất thường nằm rải rác, không thuộc nhóm nào. Đề xuất thuật toán phân cụm phù hợp nhất trong ba thuật toán đã học (K-Means, phân cụm phân cấp, DBSCAN) để vừa phân nhóm giao dịch vừa phát hiện giao dịch bất thường, giải thích lý do lựa chọn.

13.7. Bài tập lập trình

Bài 1. Sử dụng thư viện `scipy.cluster.hierarchy` (hàm linkage và dendrogram), vẽ dendrogram cho bộ dữ liệu `load_iris()` (chỉ dùng 4 đặc trưng, bỏ nhãn) với phương pháp liên kết Ward. Quan sát dendrogram và xác định số cụm hợp lý bằng cách tìm khoảng cách dọc lớn nhất không bị cắt ngang bởi một lần gộp nào.

Bài 2. Sử dụng dữ liệu ở ví dụ 13.1, khảo sát ảnh hưởng của tham số eps trong DBSCAN (thử các giá trị 0.1, 0.2, 0.3, 0.5, 0.8) với `min_samples=5` cố định. Với mỗi eps, in ra số cụm phát hiện được và số điểm nhiễu. Nhận xét xu hướng quan sát được khi eps tăng dần.

Bài 3 (nâng cao). Sử dụng bộ dữ liệu `load_wine()`, áp dụng cả ba thuật toán K-Means (K=3), AgglomerativeClustering (`n_clusters=3`, linkage="ward"), và DBSCAN (tự tìm eps, `min_samples` phù hợp bằng thử nghiệm) sau khi đã chuẩn hóa dữ liệu. So sánh kết quả phân cụm của cả ba với nhãn giống rượu vang thực tế (chỉ dùng để đánh giá, không dùng khi phân cụm) bằng chỉ số Adjusted Rand Index (from `sklearn.metrics` import `adjusted_rand_score`). Trình bày kết quả dưới dạng bảng so sánh và nhận xét thuật toán nào phù hợp nhất với bộ dữ liệu này.

Chương tiếp theo (Chương 14) sẽ khép lại Phần 3 với một bài toán học không giám sát khác biệt hoàn toàn về mục tiêu: thay vì nhóm dữ liệu thành các cụm, Phân tích thành phần chính (PCA) tìm cách giảm số chiều của dữ liệu trong khi vẫn giữ lại tối đa lượng thông tin (phương sai) quan trọng nhất — một kỹ thuật nền tảng cho trực quan hóa dữ liệu nhiều chiều và tiền xử lý trước khi huấn luyện mô hình.

Chương 14

Giảm chiều dữ liệu: PCA

14.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Giải thích được động cơ của việc giảm chiều dữ liệu: trực quan hóa, giảm nhiễu, tăng tốc huấn luyện, giảm hiện tượng curse of dimensionality.
2. Trình bày được ý tưởng hình học của PCA: tìm các trục (thành phần chính) theo hướng phương sai dữ liệu lớn nhất.
3. Giải thích được vai trò của ma trận hiệp phương sai, vector riêng (eigenvector) và giá trị riêng (eigenvalue) trong PCA.
4. Diễn giải được tỉ lệ phương sai giải thích (explained variance ratio) và sử dụng nó để chọn số thành phần chính phù hợp.
5. Áp dụng PCA để trực quan hóa dữ liệu nhiều chiều và như một bước tiền xử lý trước khi huấn luyện mô hình, bằng Scikit-learn.

14.2. Nội dung lý thuyết

14.2.1. Vì sao cần giảm chiều dữ liệu?

Nhiều bộ dữ liệu thực tế có số lượng đặc trưng rất lớn (hàng chục, hàng trăm, thậm chí hàng nghìn chiều) — ví dụ dữ liệu gen học, ảnh số, hay văn bản đã vector hóa như ở Chương 11. Làm việc trực tiếp với dữ liệu nhiều chiều gây ra một số vấn đề: không thể trực quan hóa trực tiếp (con người chỉ có thể nhìn tối đa 3 chiều không gian); chi phí tính toán và bộ nhớ tăng theo số chiều; nhiều đặc trưng có thể dư thừa hoặc tương quan chặt với nhau, không mang thêm thông tin hữu ích; và hiệu năng của nhiều thuật toán (đặc biệt các thuật toán dựa trên khoảng cách như K-NN, K-Means) suy giảm trong không gian nhiều chiều — hiện tượng gọi là curse of dimensionality (lời nguyền chiều dữ liệu) đã được đề cập ở Chương 7, khi khái niệm "khoảng cách gần/xa" giữa các điểm dần trở nên kém ý nghĩa khi số chiều tăng.

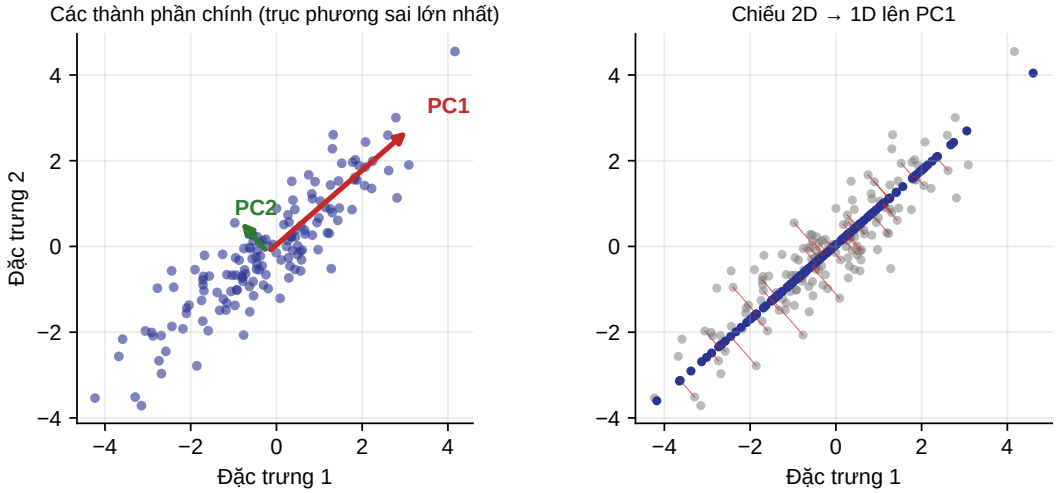
Giảm chiều dữ liệu (dimensionality reduction) là nhóm kỹ thuật biến đổi dữ liệu từ không gian nhiều chiều sang không gian ít chiều hơn, trong khi cố gắng giữ lại tối đa lượng thông tin quan trọng của dữ liệu gốc. Phân tích thành phần chính (Principal Component Analysis – PCA) là kỹ thuật giảm chiều tuyến tính phổ biến và nền tảng nhất.

14.2.2. Ý tưởng hình học của PCA

Trực giác của PCA: hình dung một đám mây điểm dữ liệu trong không gian nhiều chiều thường không phân bố đều theo mọi hướng — dữ liệu thường "trải dài" nhiều hơn theo một số hướng nhất định (phương sai lớn — nhiều thông tin, biến thiên) và "co cụm" theo các hướng khác (phương sai nhỏ — ít thông tin, gần như không đổi). PCA tìm ra hệ trục tọa độ mới (các thành phần chính — principal components), sao cho: trục thứ nhất (thành phần chính thứ nhất, PC1) là hướng mà dữ liệu có phương sai lớn nhất; trục thứ hai (PC2) là hướng có phương sai lớn thứ hai, với ràng buộc bắt buộc phải vuông góc (trực giao) với PC1; và cứ thế tiếp tục cho các thành phần chính tiếp theo, mỗi thành phần vuông góc với tất cả các thành phần trước đó.

Sau khi tìm được hệ trục mới này, việc giảm chiều được thực hiện đơn giản bằng cách chỉ giữ lại k thành phần chính đầu tiên (k nhỏ hơn số chiều gốc) và bỏ đi các thành phần còn lại — vì các thành phần chính được sắp xếp theo thứ tự phương sai giảm dần, việc bỏ các thành phần cuối (phương sai nhỏ) làm mất ít thông tin nhất có thể so với bất kỳ cách chọn k trục nào khác.

Hình 14.1 minh họa các thành phần chính là những hướng có phương sai lớn nhất, và giảm chiều tương ứng với phép chiếu lên chúng.



Hình 14.1. Các thành phần chính (trái) và phép chiếu lên PC1 (phải).

14.2.3. Nền tảng toán học: ma trận hiệp phương sai, vector riêng và giá trị riêng

Về mặt toán học, các thành phần chính chính là các vector riêng (eigenvector) của ma trận hiệp phương sai (covariance matrix) của dữ liệu (sau khi đã chuẩn hóa về trung bình 0), còn phương sai dọc theo mỗi thành phần chính chính là giá trị riêng (eigenvalue) tương ứng. Ma trận hiệp phương sai kích thước $n \times n$ (n là số chiều gốc) ghi lại phương sai của từng đặc trưng (trên đường chéo) và hiệp phương sai (mức độ biến thiên cùng nhau) giữa mọi cặp đặc trưng. Việc phân tích trị riêng — vector riêng (eigendecomposition) của ma trận này cho ra n cặp (giá trị riêng, vector riêng); sắp xếp các cặp này theo giá trị riêng giảm dần chính là thứ tự các thành phần chính từ quan trọng nhất đến ít quan trọng nhất.

Trong thực hành, Scikit-learn (và hầu hết các thư viện khác) không tính trực tiếp ma trận hiệp phương sai rồi phân tích trị riêng, mà sử dụng một kỹ thuật đại số tuyến tính hiệu quả và ổn định số học hơn gọi là phân tích giá trị suy biến (Singular Value Decomposition – SVD), cho kết quả toán học tương đương nhưng với chi phí tính toán và độ ổn định tốt hơn. Sinh viên chỉ cần hiểu bản chất hình học và ý nghĩa của các thành phần chính; chi tiết thuật toán SVD nằm ngoài phạm vi giáo trình này.

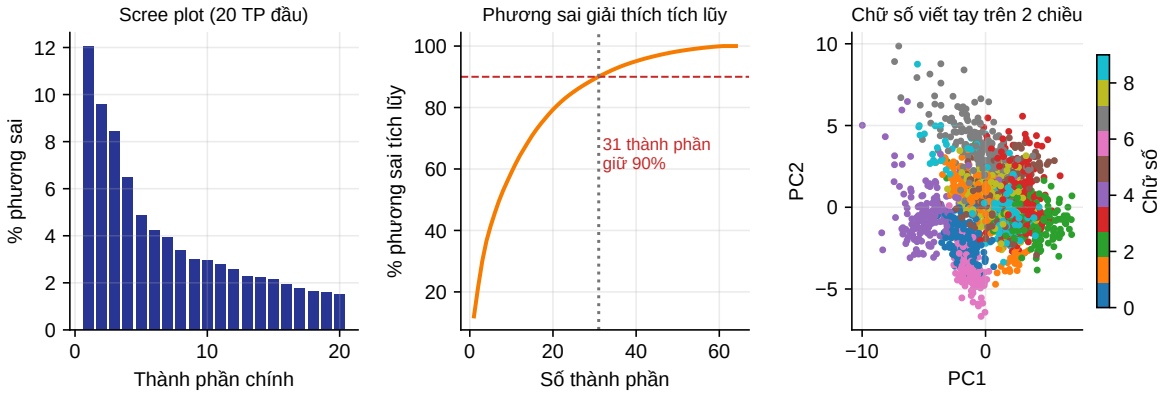
14.2.4. Tỉ lệ phương sai giải thích và chọn số thành phần

Với mỗi thành phần chính, tỉ lệ phương sai giải thích (explained variance ratio) cho biết thành phần đó "giải thích" (chiếm) bao nhiêu phần trăm tổng phương sai của toàn bộ dữ liệu gốc. Tổng tỉ lệ phương sai giải thích của k thành phần chính đầu tiên cho biết tổng lượng thông tin được giữ lại khi giảm dữ liệu xuống còn k chiều. Một cách phổ biến để chọn số thành phần chính k phù hợp là vẽ biểu đồ đường tích lũy (cumulative explained variance) theo số thành phần, rồi chọn k sao cho tổng phương sai giải thích đạt một ngưỡng mong muốn (thường 90–95% là lựa chọn phổ biến, tùy bài toán) — cách tiếp cận này có nét tương đồng với phương pháp Elbow ở Chương 12: tìm điểm mà việc thêm thành phần mới không còn mang lại nhiều lợi ích tương xứng.

Lưu ý kỹ thuật quan trọng

PCA cực kỳ nhạy cảm với thang đo của các đặc trưng — vì PCA tìm hướng phương sai lớn nhất, nếu một đặc trưng có thang đo lớn hơn nhiều so với các đặc trưng khác (ví dụ đo bằng đơn vị khác nhau), nó sẽ áp đảo hoàn toàn các thành phần chính đầu tiên chỉ vì lý do đơn vị đo, không phải vì nó thực sự mang nhiều thông tin quan trọng hơn về mặt bản chất. Do đó, chuẩn hóa dữ liệu (StandardScaler) trước khi áp dụng PCA gần như luôn là bước bắt buộc, tương tự nguyên tắc đã áp dụng nhất quán với K-NN, SVM và K-Means trong các chương trước.

Hình 14.2 minh họa dùng phương sai giải thích tích lũy để chọn số thành phần, và trực quan hóa dữ liệu 64 chiều trên mặt phẳng.



Hình 14.2. Scree plot, phương sai tích lũy và phép chiếu 2D của bộ Digits.

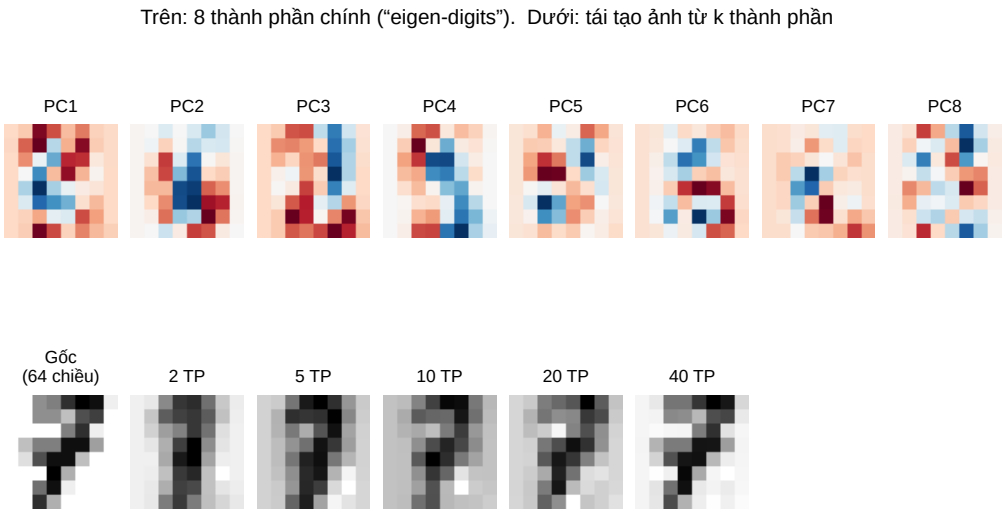
14.2.5. Ứng dụng của PCA

Ba ứng dụng phổ biến nhất của PCA trong thực hành học máy: trực quan hóa dữ liệu nhiều chiều — giảm dữ liệu xuống 2 hoặc 3 chiều để có thể vẽ biểu đồ scatter, giúp quan sát trực quan cấu trúc, cụm, hoặc điểm ngoại lai trong dữ liệu mà mắt thường không thể nhìn thấy ở không gian gốc nhiều chiều; tiền xử lý trước khi huấn luyện mô hình — giảm số chiều đặc trưng trước khi đưa vào các thuật toán học máy khác, giúp giảm chi phí tính toán, giảm nguy cơ khớp quá (đặc biệt hữu ích khi số chiều gốc rất lớn so với số mẫu dữ liệu), và đôi khi loại bỏ nhiễu (các thành phần phương sai rất nhỏ thường tương ứng với nhiễu ngẫu nhiên hơn là tín hiệu thực sự); và nén dữ liệu — lưu trữ dữ liệu ở dạng ít chiều hơn (đánh đổi lấy mất mát thông tin nhỏ), tiết kiệm không gian lưu trữ, đặc biệt hữu ích với dữ liệu ảnh.

Lưu ý về khả năng diễn giải

Một điểm cần lưu ý: các thành phần chính là tổ hợp tuyến tính của các đặc trưng gốc (ví dụ $\text{PC1} = 0.3 \times \text{đặc_trưng_1} - 0.5 \times \text{đặc_trưng_2} + 0.7 \times \text{đặc_trưng_3} + \ldots$), nên chúng thường không còn mang ý nghĩa vật lý/nghiệp vụ trực quan, rõ ràng như các đặc trưng gốc. Đây là một đánh đổi cần cân nhắc: PCA giúp giảm chiều và giữ phương sai hiệu quả, nhưng làm mất đi khả năng diễn giải trực tiếp mà đôi khi các bài toán thực tế (đặc biệt trong y tế, tài chính, nơi cần giải trình quyết định) rất coi trọng.

Hình 14.3 minh họa các thành phần chính của ảnh chữ số và khả năng tái tạo ảnh từ số thành phần ít ỏi.



Hình 14.3. Tám thành phần chính đầu tiên và ảnh tái tạo từ k thành phần.

14.3. Ví dụ minh họa bằng mã Python

Ví dụ dưới đây áp dụng PCA lên bộ dữ liệu chữ số viết tay (Digits, ảnh 8×8 pixel = 64 chiều đặc trưng), minh họa việc giảm xuống 2 chiều để trực quan hóa, đồng thời khảo sát tỉ lệ phương sai giải thích theo số thành phần.

```
# vidu_14_1.py
# PCA: trực quan hóa dữ liệu nhiều chiều và khảo sát phương sai giải thích

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# 1. Nạp dữ liệu chữ số viết tay: 1797 ảnh 8x8 pixel (64 đặc trưng)
du_lieu = load_digits()
X, y = du_lieu.data, du_lieu.target
print(f"Kích thước dữ liệu gốc: {X.shape}")

# 2. Chuẩn hóa dữ liệu – bắt buộc trước khi áp dụng PCA
X_chuan_hoa = StandardScaler().fit_transform(X)

# 3. Áp dụng PCA giữ lại toàn bộ 64 thành phần để khảo sát phương sai giải thích
pca_day_du = PCA(n_components=64)
pca_day_du.fit(X_chuan_hoa)
phuong_sai_tich_luy = np.cumsum(pca_day_du.explained_variance_ratio_)

# Tìm số thành phần tối thiểu để giữ được >= 90% phương sai
so_thanh_phan_90pt = np.argmax(phuong_sai_tich_luy >= 0.90) + 1
print(f"Số thành phần cần để giữ >= 90% phương sai: {so_thanh_phan_90pt} / 64 chiều
gốc")

# 4. Áp dụng PCA giảm xuống còn 2 chiều để trực quan hóa
pca_2d = PCA(n_components=2)
X_2d = pca_2d.fit_transform(X_chuan_hoa)
print(f"Phương sai giải thích bởi 2 thành phần đầu: "
      f"{pca_2d.explained_variance_ratio_.sum():.4f} "
      f"({pca_2d.explained_variance_ratio_[0]:.4f} +
      {pca_2d.explained_variance_ratio_[1]:.4f})")

# 5. Trực quan hóa: (a) đường cong phương sai tích lũy, (b) scatter 2D theo PC1,
PC2
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].plot(range(1, 65), phuong_sai_tich_luy, marker=".")
axes[0].axhline(y=0.90, color="red", linestyle="--", label="Ngưỡng 90%")
axes[0].axvline(x=so_thanh_phan_90pt, color="gray", linestyle=":")
axes[0].set_xlabel("Số thành phần chính")
axes[0].set_ylabel("Phương sai giải thích tích lũy")
axes[0].set_title("Phương sai giải thích tích lũy theo số thành phần")
axes[0].legend()

bieu_do_scatter = axes[1].scatter(X_2d[:, 0], X_2d[:, 1], c=y, cmap="tab10", s=15,
alpha=0.7)
axes[1].set_xlabel("Thành phần chính thứ nhất (PC1)")
```

```
axes[1].set_ylabel("Thành phần chính thứ hai (PC2)")
axes[1].set_title("Dữ liệu chữ số viết tay chiếu xuống 2 chiều (PCA)")
plt.colorbar(bieu_do_scatter, ax=axes[1], label="Chữ số")

plt.tight_layout()
plt.savefig("pca_chu_so_viet_tay.png", dpi=120)
print("\nĐã lưu biểu đồ vào tệp pca_chu_so_viet_tay.png")
```

14.4. Giải thích kết quả chạy chương trình

Kết quả in ra từ chương trình trên có dạng tương tự:

```
Kích thước dữ liệu gốc: (1797, 64)
Số thành phần cần để giữ >= 90% phương sai: 21 / 64 chiều gốc
Phương sai giải thích bởi 2 thành phần đầu: 0.2863 (0.1213 + 0.1650)

Đã lưu biểu đồ vào tệp pca_chu_so_viet_tay.png
```

Kết quả cho thấy chỉ cần 21 trong số 64 chiều gốc (giảm gần 70% số chiều) đã đủ để giữ lại 90% tổng phương sai của dữ liệu — một minh chứng rõ ràng cho việc dữ liệu ảnh chữ số viết tay có rất nhiều dư thừa/tương quan giữa các pixel lân cận (các pixel liền kề trong một chữ số thường có giá trị tương quan chặt với nhau), và PCA khai thác hiệu quả sự dư thừa này để nén dữ liệu mà không mất quá nhiều thông tin. Ngược lại, khi giảm mạnh xuống chỉ còn 2 chiều (để phục vụ trực quan hóa), hai thành phần chính đầu tiên chỉ giữ lại khoảng 28.6% tổng phương sai — một sự đánh đổi lớn về thông tin, nhưng vẫn đủ để phục vụ mục đích trực quan hóa sơ bộ.

Trên biểu đồ scatter 2 chiều, mặc dù chỉ giữ lại chưa đến 1/3 tổng phương sai, các chữ số vẫn có xu hướng tạo thành các đám mây điểm khá tách biệt theo màu sắc (mỗi màu ứng với một chữ số 0–9) — cho thấy ngay cả một phép chiếu tuyến tính đơn giản xuống 2 chiều cũng giữ lại đủ cấu trúc để phân biệt phần lớn các chữ số bằng mắt thường, dù ranh giới giữa một số cặp chữ số dễ nhầm lẫn (ví dụ 3 và 8, hoặc 4 và 9) có thể chồng lấn đáng kể trên không gian PCA 2 chiều — đây là hạn chế cố hữu của một phương pháp giảm chiều tuyến tính khi áp dụng cho dữ liệu có cấu trúc phi tuyến phức tạp như hình ảnh.

14.5. Tóm tắt chương

- Giảm chiều dữ liệu giúp trực quan hóa, giảm chi phí tính toán, giảm dư thừa thông tin, và giảm nhẹ hiện tượng curse of dimensionality.
- PCA tìm các thành phần chính — các trục trực giao theo hướng phương sai dữ liệu lớn nhất, được sắp xếp theo thứ tự quan trọng giảm dần.
- Về mặt toán học, các thành phần chính là vector riêng của ma trận hiệp phương sai dữ liệu; phương sai theo mỗi thành phần là giá trị riêng tương ứng.

- Tỉ lệ phương sai giải thích (và tổng tích lũy của nó) là công cụ chính để quyết định giữ lại bao nhiêu thành phần chính, đánh đổi giữa mức độ giảm chiều và lượng thông tin giữ lại.
- Chuẩn hóa dữ liệu trước khi áp dụng PCA gần như luôn bắt buộc, vì PCA rất nhạy cảm với thang đo của các đặc trưng.
- Các thành phần chính là tổ hợp tuyến tính của đặc trưng gốc, thường mất đi ý nghĩa diễn giải trực quan — một đánh đổi cần cân nhắc tùy bài toán.

14.6. Bài tập củng cố

Mức nhận biết

1. PCA tìm ra các "thành phần chính" dựa trên tiêu chí nào? Thành phần chính thứ nhất và thứ hai có quan hệ hình học gì với nhau?
2. Tỉ lệ phương sai giải thích (explained variance ratio) của một thành phần chính cho biết điều gì?

Mức thông hiểu

1. Giải thích vì sao cần chuẩn hóa dữ liệu (đưa về cùng thang đo) trước khi áp dụng PCA, minh họa bằng một ví dụ cụ thể có hai đặc trưng với đơn vị đo rất khác nhau.
2. Một bộ dữ liệu 50 chiều, khi áp dụng PCA, chỉ cần 3 thành phần chính đầu tiên đã giữ được 98% phương sai. Rút ra nhận xét gì về cấu trúc tương quan giữa các đặc trưng gốc trong bộ dữ liệu này?
3. Giải thích vì sao các thành phần chính do PCA sinh ra thường khó diễn giải theo nghĩa nghiệp vụ, trong khi các đặc trưng gốc (ví dụ "tuổi", "thu nhập") lại dễ hiểu trực tiếp.

Mức vận dụng

1. Một nhóm nghiên cứu y sinh có bộ dữ liệu biểu hiện gen với 20000 đặc trưng (gen) nhưng chỉ có 200 mẫu bệnh nhân. Đề xuất PCA như một bước tiền xử lý có thể mang lại lợi ích gì trong tình huống này, liên hệ với khái niệm curse of dimensionality đã học ở Chương 7.

14.7. Bài tập lập trình

Bài 1. Sử dụng bộ dữ liệu `load_breast_cancer()` (30 đặc trưng, đã dùng ở nhiều chương trước), áp dụng PCA giảm xuống 2 chiều sau khi chuẩn hóa dữ liệu. Vẽ biểu đồ scatter 2 chiều, tô màu theo nhãn ác tính/lành tính thực tế. Nhận xét mức độ tách biệt của hai lớp trong không gian PCA 2 chiều.

Bài 2. Sử dụng dữ liệu ở ví dụ 14.1, huấn luyện một mô hình Random Forest (Chương 9) phân loại chữ số viết tay theo hai cách: (a) dùng toàn bộ 64 đặc trưng gốc; (b) dùng số thành phần chính đủ để giữ 90% phương sai (đã tính ở ví dụ 14.1). So sánh Accuracy và thời gian huấn luyện giữa hai cách, nhận xét đánh đổi giữa tốc độ và độ chính xác khi dùng PCA làm tiền xử lý.

Bài 3 (nâng cao). Viết chương trình trực quan hóa 8 "eigenimages" đầu tiên (các thành phần chính đầu tiên, reshape lại thành ảnh 8x8) từ PCA huấn luyện trên bộ dữ liệu `load_digits()`. Gợi ý: dùng `pca.components_` (mảng kích thước `n_components` × 64), reshape mỗi hàng thành (8, 8) rồi hiển thị bằng `plt.imshow` với cmap="gray". Nhận xét trực quan các eigenimages này có đặc điểm gì (ví dụ có giống nét bút hay hoa văn cụ thể nào không).

Đến đây, giáo trình đã hoàn thành Phần 3 — Học không giám sát, giới thiệu ba thuật toán nền tảng cho phân cụm (K-Means, phân cụm phân cấp, DBSCAN) và giảm chiều dữ liệu (PCA). Phần 4 tiếp theo (Chương 15–17) sẽ tập trung vào các kỹ thuật đánh giá mô hình một cách nghiêm ngặt hơn (kiểm định chéo, các độ đo nâng cao) và tối ưu hóa siêu tham số một cách có hệ thống — những kỹ năng thực hành thiết yếu áp dụng chung cho mọi mô hình đã học ở Phần 2 và Phần 3.

Phần IV

Đánh giá mô hình và tối ưu hóa

Chương 15

Đánh giá và lựa chọn mô hình (Cross-Validation, các độ đo)

15.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Giải thích được hạn chế của việc chỉ chia dữ liệu một lần thành tập huấn luyện/kiểm tra và động cơ ra đời của kiểm định chéo.
2. Trình bày được nguyên lý kiểm định chéo K-fold và kiểm định chéo K-fold phân tầng (Stratified K-fold).
3. Vẽ và diễn giải được đường cong học (learning curve) để chẩn đoán khớp thiếu/khớp quá.
4. Trình bày mở rộng về đường cong ROC, chỉ số AUC và áp dụng cho việc so sánh nhiều mô hình phân loại.
5. Thiết lập đúng quy trình chia dữ liệu ba tầng (huấn luyện/kiểm định/kiểm tra) để tránh rò rỉ thông tin khi lựa chọn mô hình.
6. Cài đặt kiểm định chéo và đường cong học bằng Scikit-learn.

15.2. Nội dung lý thuyết

15.2.1. Hạn chế của việc chia dữ liệu một lần

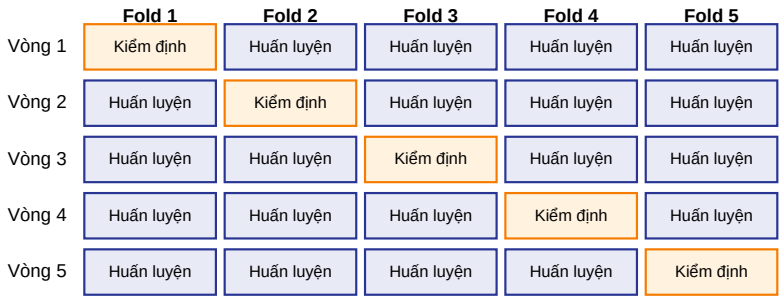
Xuyên suốt giáo trình, từ Chương 1, chúng ta luôn chia dữ liệu thành tập huấn luyện và tập kiểm tra bằng `train_test_split` với một giá trị `random_state` cố định. Cách làm này đơn giản và hữu ích để minh họa các khái niệm, nhưng có một hạn chế quan trọng: kết quả đánh giá (Accuracy, RMSE, v.v.) phụ thuộc vào chính cách chia dữ liệu cụ thể đó — nếu chọn một `random_state` khác, tập kiểm tra sẽ gồm những mẫu khác, và kết quả đánh giá có thể dao động, đôi khi đáng kể, đặc biệt với bộ dữ liệu có kích thước nhỏ hoặc vừa. Một mô hình có thể "may mắn" đạt kết quả tốt trên một cách chia cụ thể (tập kiểm tra tình cờ chứa nhiều mẫu "dễ"), tạo cảm giác sai lệch về chất lượng thực sự của mô hình.

15.2.2. Kiểm định chéo K-fold (K-fold Cross-Validation)

Kiểm định chéo K-fold giải quyết vấn đề trên bằng cách sử dụng toàn bộ dữ liệu cho cả huấn luyện lẫn đánh giá, theo một quy trình có hệ thống: chia toàn bộ dữ liệu thành K phần (fold) có kích thước xấp xỉ bằng nhau; thực hiện K vòng lặp, ở mỗi vòng dùng $K-1$ phần để huấn luyện và phần còn lại (chưa từng được dùng để đánh giá trước đó) để kiểm tra; sau K vòng, mỗi phần dữ liệu đã đóng vai trò tập kiểm tra đúng một lần. Kết quả cuối cùng là trung bình (và độ lệch chuẩn) của K giá trị đánh giá thu được, cho một ước lượng đáng tin cậy hơn nhiều so với chỉ một lần chia dữ liệu, đồng thời độ lệch chuẩn giữa các fold cho biết mức độ ổn định của mô hình trên các tập con dữ liệu khác nhau.

Giá trị K phổ biến nhất là 5 hoặc 10 — đây là sự đánh đổi giữa độ tin cậy của ước lượng (K lớn hơn dùng nhiều dữ liệu huấn luyện hơn ở mỗi vòng, ước lượng ít chệch hơn) và chi phí tính toán (K lớn hơn đòi hỏi huấn luyện mô hình nhiều lần hơn). Trường hợp đặc biệt K = số lượng mẫu dữ liệu được gọi là Leave-One-Out Cross-Validation (LOOCV) — mỗi vòng chỉ dùng đúng một mẫu để kiểm tra; LOOCV cho ước lượng gần như không chệch nhưng có chi phí tính toán rất cao (huấn luyện mô hình n lần với n là số mẫu) và không phù hợp với dữ liệu lớn.

Hình 15.1 minh họa mỗi phần dữ liệu lần lượt đóng vai trò tập kiểm định đúng một lần.



Hình 15.1. Kiểm định chéo 5-fold.

15.2.3. Kiểm định chéo K-fold phân tầng (Stratified K-fold)

Với bài toán phân loại, đặc biệt khi dữ liệu mất cân bằng lớp (đã đề cập ở Chương 6), việc chia K-fold hoàn toàn ngẫu nhiên có thể dẫn đến một số fold có tỉ lệ lớp rất khác biệt so với tỉ lệ tổng thể — thậm chí trong trường hợp cực đoan, một fold có thể hoàn toàn thiếu vắng mẫu của lớp thiểu số. Stratified K-fold khắc phục vấn đề này bằng cách đảm bảo mỗi fold đều giữ xấp xỉ đúng tỉ lệ các lớp như trong toàn bộ dữ liệu gốc — nguyên lý tương tự tham số stratify của `train_test_split` đã dùng xuyên suốt các chương trước. Scikit-learn tự động dùng Stratified K-fold làm mặc định khi gọi `cross_val_score` cho bài toán phân loại, nên trong hầu hết các ví dụ ở các chương trước, kỹ thuật này đã âm thầm được áp dụng.

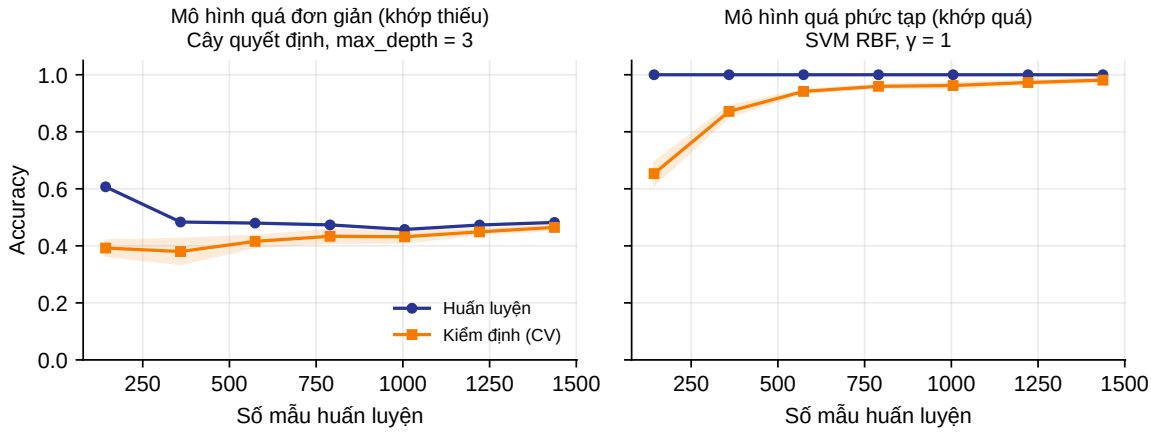
15.2.4. Đường cong học (Learning Curve)

Đường cong học là một công cụ chẩn đoán trực quan mạnh mẽ, vẽ hiệu năng mô hình (ví dụ Accuracy hoặc RMSE) trên cả tập huấn luyện và tập kiểm định theo kích thước tập huấn luyện tăng dần (ví dụ dùng 10%, 20%, …, 100% dữ liệu huấn luyện có sẵn). Hình dạng đường cong học cung cấp chẩn đoán rõ ràng về tình trạng mô hình:

- Nếu cả hai đường (huấn luyện và kiểm định) đều hội tụ về một giá trị hiệu năng thấp và khoảng cách giữa chúng nhỏ: mô hình đang khớp thiếu (underfitting) — mô hình quá đơn giản, và có thêm dữ liệu cũng khó cải thiện đáng kể; cần một mô hình phức tạp hơn hoặc thêm đặc trưng.
- Nếu đường huấn luyện đạt hiệu năng cao nhưng đường kiểm định thấp hơn đáng kể, với khoảng cách lớn giữa hai đường: mô hình đang khớp quá (overfitting) — cần thêm dữ liệu huấn luyện (nếu đường kiểm định vẫn đang có xu hướng cải thiện khi kích thước dữ liệu tăng), giảm độ phức tạp mô hình, hoặc áp dụng chính quy hóa (Chương 5).
- Nếu hai đường đã hội tụ gần nhau ở mức hiệu năng cao: mô hình đang hoạt động tốt, và việc thêm dữ liệu huấn luyện khó mang lại cải thiện đáng kể nữa — lúc này nên tập trung cải thiện theo hướng khác (đặc trưng, kiến trúc mô hình) thay vì thu thập thêm dữ liệu.

Đường cong học vì vậy giúp trả lời một câu hỏi thực tế rất quan trọng nhưng khó trả lời chỉ bằng một con số Accuracy/RMSE đơn lẻ: "nếu tôi có thêm dữ liệu, mô hình có cải thiện không?" — một thông tin có giá trị thực tiễn cao khi cân nhắc chi phí thu thập thêm dữ liệu.

Hình 15.2 minh họa cách đọc đường cong học để chẩn đoán khớp thiếu và khớp quá.



Hình 15.2. Đường cong học của mô hình khớp thiếu (trái) và khớp quá (phải), bộ Digits.

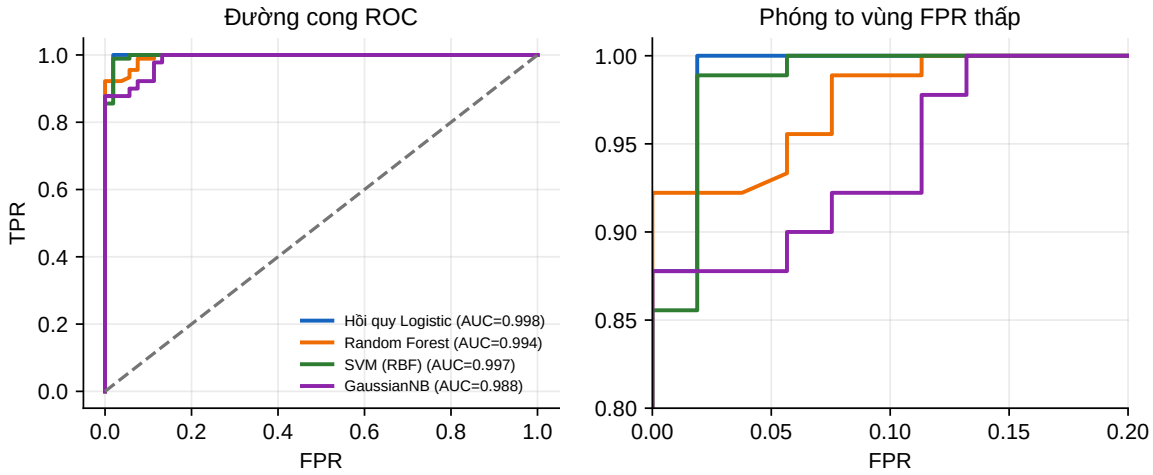
15.2.5. Mở rộng đường cong ROC và chỉ số AUC

Chương 6 đã giới thiệu sơ lược đường cong ROC (Receiver Operating Characteristic) và chỉ số AUC khi đánh giá hồi quy Logistic. Nhắc lại và mở rộng: đường cong ROC vẽ tỉ lệ dương tính thật (True Positive Rate, chính là Recall) theo tỉ lệ dương tính giả (False Positive Rate = FP/(FP+TN)) khi quét qua toàn bộ dải ngưỡng quyết định có thể có (từ 0 đến 1), thay vì chỉ

đánh giá tại một ngưỡng cố định 0.5. Diện tích dưới đường cong này (AUC) tóm tắt hiệu năng phân loại của mô hình trên toàn dải ngưỡng thành một con số duy nhất: AUC = 1.0 ứng với bộ phân loại hoàn hảo; AUC = 0.5 ứng với một bộ phân loại ngẫu nhiên (không tốt hơn tung đồng xu); AUC dưới 0.5 (hiếm gặp) cho thấy mô hình đang dự đoán ngược.

Ưu điểm quan trọng của AUC so với Accuracy: AUC không phụ thuộc vào việc chọn ngưỡng quyết định cụ thể nào, và ít nhạy cảm hơn với mất cân bằng lớp so với Accuracy (dù không hoàn toàn miễn nhiễm) — vì lý do này, AUC thường là lựa chọn phù hợp hơn Accuracy để so sánh nhiều mô hình phân loại với nhau một cách công bằng, đặc biệt khi chưa quyết định ngưỡng vận hành cụ thể sẽ dùng trong thực tế.

Hình 15.3 minh họa so sánh nhiều mô hình bằng ROC và AUC trên toàn dải ngưỡng.



Hình 15.3. ROC của bốn mô hình (phải: phóng to vùng FPR thấp).

15.2.6. Quy trình chia dữ liệu ba tầng

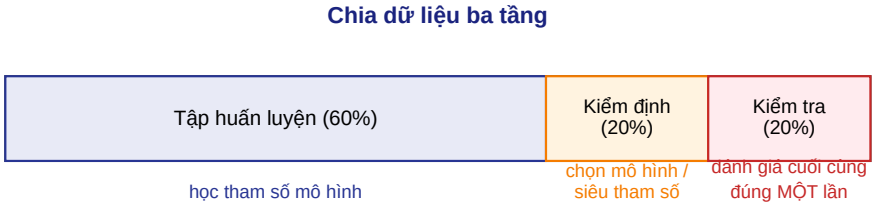
Khi cần vừa lựa chọn mô hình/siêu tham số tốt nhất, vừa đưa ra một đánh giá cuối cùng không thiên lệch, quy trình chuẩn mực là chia dữ liệu thành ba phần độc lập: tập huấn luyện (training set) — dùng để huấn luyện tham số mô hình; tập kiểm định (validation set) — dùng để so sánh các mô hình/siêu tham số khác nhau và chọn ra lựa chọn tốt nhất (trong thực hành, vai trò của tập kiểm định thường được thay thế bằng kiểm định chéo trên tập huấn luyện, như đã trình bày ở mục 15.2.2, để sử dụng dữ liệu hiệu quả hơn); và tập kiểm tra (test set) — chỉ được sử dụng đúng một lần duy nhất, ở bước cuối cùng, để đánh giá hiệu năng của mô hình đã chọn, không được dùng cho bất kỳ quyết định nào trong suốt quá trình phát triển mô hình.

Lưu ý sư phạm quan trọng

Một lỗi rất phổ biến (và nghiêm trọng) là sử dụng tập kiểm tra nhiều lần trong quá trình thử nghiệm — chọn mô hình, tinh chỉnh siêu tham số dựa trên phản hồi từ hiệu năng tập kiểm tra, rồi lặp lại. Dù mỗi lần điều chỉnh có vẻ vô hại, việc lặp lại quá trình này khiến thông tin về tập kiểm tra dần "rò rỉ" vào quá trình lựa chọn mô hình một cách gián tiếp — về bản chất, tập kiểm tra dần trở thành một tập kiểm định trá hình, và độ chính xác cuối

> cùng báo cáo trên tập đó không còn là một ước lượng khách quan cho hiệu năng trên dữ liệu hoàn toàn mới. Nguyên tắc vàng: tập kiểm tra chỉ được "chạm vào" một lần duy nhất, sau khi mọi quyết định về mô hình và siêu tham số đã hoàn tất dựa trên kiểm định chéo.

Hình 15.4 minh họa vai trò của ba tập dữ liệu độc lập.



Hình 15.4. Chia dữ liệu thành tập huấn luyện, kiểm định và kiểm tra.

15.3. Ví dụ minh họa bằng mã Python

Ví dụ dưới đây áp dụng kiểm định chéo 5-fold và đường cong học để đánh giá một mô hình Random Forest trên bộ dữ liệu Breast Cancer Wisconsin, đồng thời so sánh đường cong ROC của ba mô hình phân loại đã học ở các chương trước.

```
# vidu_15_1.py
# Kiểm định chéo, đường cong học, và so sánh đường cong ROC của nhiều mô hình

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import (
    train_test_split, cross_val_score, StratifiedKFold, learning_curve
)
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import RocCurveDisplay

# 1. Nạp dữ liệu và chia tập huấn luyện / kiểm tra (tập kiểm tra chỉ dùng ở bước
cuối)
du_lieu = load_breast_cancer()
X, y = du_lieu.data, du_lieu.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
bo_chuan_hoa = StandardScaler()
```

```
X_train_cd = bo_chuan_hoa.fit_transform(X_train)
X_test_cd = bo_chuan_hoa.transform(X_test)

# 2. Kiểm định chéo 5-fold phân tầng cho Random Forest
rf = RandomForestClassifier(n_estimators=100, max_depth=6, random_state=42)
kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
diem_cv = cross_val_score(rf, X_train_cd, y_train, cv=kfold, scoring="accuracy")
print(f"Accuracy từng fold: {np.round(diem_cv, 4)}")
print(f"Accuracy trung bình: {diem_cv.mean():.4f} (+/- {diem_cv.std():.4f})")

# 3. Đường cong học cho Random Forest
kich_thuoc_train, diem_train, diem_val = learning_curve(
    rf, X_train_cd, y_train, cv=kfold,
    train_sizes=np.linspace(0.1, 1.0, 8), scoring="accuracy", random_state=42
)

plt.figure(figsize=(7, 5))
plt.plot(kich_thuoc_train, diem_train.mean(axis=1), marker="o", label="Tập huấn
luyện")
plt.plot(kich_thuoc_train, diem_val.mean(axis=1), marker="s", label="Tập kiểm định
(CV)")
plt.xlabel("Số mẫu huấn luyện")
plt.ylabel("Accuracy")
plt.title("Đường cong học - Random Forest")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("duong_cong_hoc_rf.png", dpi=120)

# 4. So sánh đường cong ROC của ba mô hình trên tập kiểm tra (chỉ dùng 1 lần, ở
đây)
cac_mo_hinh = {
    "Hồi quy Logistic": LogisticRegression(max_iter=1000),
    "Random Forest": RandomForestClassifier(n_estimators=100, max_depth=6,
    random_state=42),
    "SVM (RBF)": SVC(kernel="rbf", probability=True, random_state=42),
}

fig, ax = plt.subplots(figsize=(6, 6))
for ten_mo_hinh, mo_hinh in cac_mo_hinh.items():
    mo_hinh.fit(X_train_cd, y_train)
    RocCurveDisplay.from_estimator(mo_hinh, X_test_cd, y_test, ax=ax,
    name=ten_mo_hinh)
ax.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Ngẫu nhiên")
ax.set_title("So sánh đường cong ROC trên tập kiểm tra")
ax.legend()
plt.tight_layout()
plt.savefig("so_sanh_roc_ba_mo_hinh.png", dpi=120)
print("\nĐã lưu các biểu đồ vào duong_cong_hoc_rf.png và
so_sanh_roc_ba_mo_hinh.png")
```

15.4. Giải thích kết quả chạy chương trình

Kết quả in ra từ chương trình trên có dạng tương tự:

```
Accuracy từng fold: [0.9451 0.9560 0.9560 0.9670 0.9451]
Accuracy trung bình: 0.9538 (+/- 0.0091)

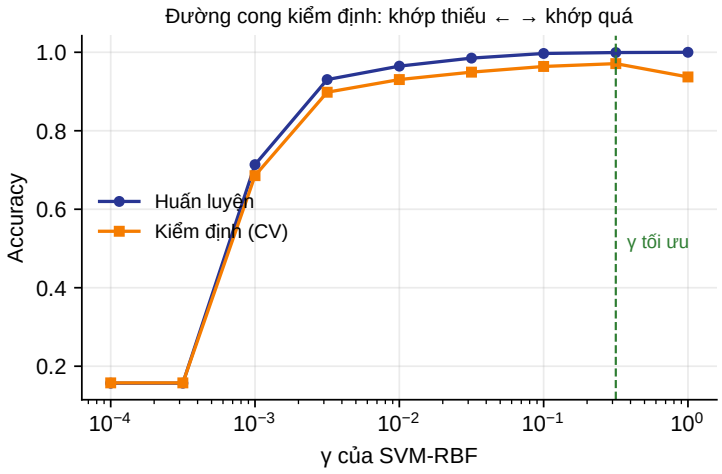
Đã lưu các biểu đồ vào duong_cong_hoc_rf.png và so_sanh_roc_ba_mo_hinh.png
```

Độ lệch chuẩn rất nhỏ (0.0091) giữa các fold cho thấy mô hình Random Forest có hiệu năng khá ổn định, không phụ thuộc quá nhiều vào cách chia dữ liệu cụ thể — một dấu hiệu tích cực về độ tin cậy của ước lượng 0.9538, đáng tin cậy hơn nhiều so với chỉ báo cáo kết quả từ một lần chia train/test duy nhất như ở các chương trước.

Trên biểu đồ đường cong học, mẫu hình điển hình thường quan sát được: đường huấn luyện bắt đầu ở mức rất cao (gần 1.0, vì với ít dữ liệu, cây dễ dàng "khớp" hoàn hảo), sau đó giảm nhẹ khi kích thước dữ liệu tăng (mô hình khó "khớp thuộc lòng" hơn); đường kiểm định bắt đầu thấp hơn nhưng có xu hướng tăng dần và tiệm cận gần đường huấn luyện khi kích thước dữ liệu tăng. Khoảng cách giữa hai đường ở cuối biểu đồ (dùng toàn bộ dữ liệu huấn luyện) khá hẹp, cho thấy mô hình không bị khớp quá nghiêm trọng với cấu hình `max_depth=6` đã chọn, và việc thu thập thêm dữ liệu (nếu có) khó mang lại cải thiện đột phá thêm nữa ở điểm này.

Trên biểu đồ so sánh ROC, cả ba mô hình thường cho đường cong nằm khá sát góc trên bên trái (AUC cao, gần 1.0) do bộ dữ liệu Breast Cancer Wisconsin tương đối "dễ" phân loại như đã quan sát ở các chương trước; tuy nhiên việc so sánh trực tiếp AUC của cả ba mô hình trên cùng một biểu đồ cho phép nhận định nhanh mô hình nào có khả năng phân tách lớp tốt nhất trên toàn dải ngưỡng, hỗ trợ quyết định lựa chọn mô hình cuối cùng một cách khách quan hơn so với chỉ so sánh Accuracy tại một ngưỡng cố định.

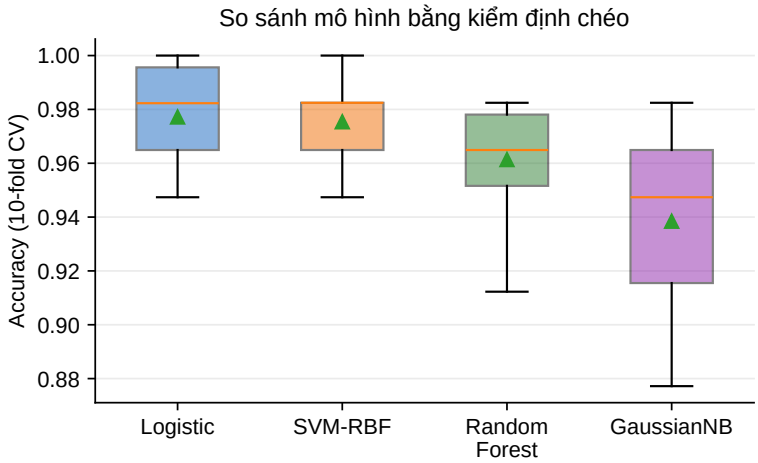
Hình 15.5 minh họa đường cong kiểm định theo một siêu tham số: vùng trái là khớp thiếu, vùng phải là khớp quá.



Hình 15.5. Đường cong kiểm định theo γ của SVM-RBF (bộ Digits).

Hình 15.6 minh họa boxplot điểm kiểm định chéo cho biết cả mức hiệu năng lẫn độ ổn định

của mô hình.



Hình 15.6. So sánh bốn mô hình bằng kiểm định chéo 10-fold (bộ Breast Cancer).

15.5. Tóm tắt chương

- Kiểm định chéo K-fold sử dụng toàn bộ dữ liệu cho cả huấn luyện và đánh giá qua K vòng lặp, cho ước lượng hiệu năng đáng tin cậy hơn một lần chia train/test đơn lẻ, đồng thời cung cấp thông tin về độ ổn định của mô hình qua độ lệch chuẩn giữa các fold.
- Stratified K-fold đảm bảo mỗi fold giữ đúng tỉ lệ lớp như dữ liệu gốc, đặc biệt quan trọng với bài toán phân loại mất cân bằng lớp.
- Đường cong học chẩn đoán trực quan tình trạng khớp thiếu/khớp quá và cho biết liệu việc thu thập thêm dữ liệu có khả năng cải thiện mô hình hay không.
- AUC (diện tích dưới đường cong ROC) đánh giá khả năng phân tách lớp trên toàn dải ngưỡng quyết định, là một tiêu chí so sánh mô hình đáng tin cậy hơn Accuracy tại một ngưỡng cố định.
- Quy trình chia dữ liệu ba tầng (huấn luyện/kiểm định/kiểm tra), với tập kiểm tra chỉ được sử dụng đúng một lần ở bước cuối cùng, là nguyên tắc nền tảng để tránh rò rỉ thông tin và đảm bảo đánh giá cuối cùng khách quan.

15.6. Bài tập củng cố

Mức nhận biết

1. Nêu các bước của kiểm định chéo K-fold với K=5.
2. AUC bằng 0.5 và AUC bằng 1.0 lần lượt có ý nghĩa gì?

Mức thông hiểu

1. Giải thích vì sao độ lệch chuẩn giữa các fold trong kiểm định chéo là một thông tin hữu ích, không chỉ riêng giá trị trung bình.
2. Một đường cong học có đường huấn luyện và đường kiểm định đều hội tụ ở mức Accuracy thấp (khoảng 65%) và gần nhau. Đây là dấu hiệu của khớp thiếu hay khớp quá? Đề xuất hướng cải thiện.
3. Giải thích vì sao việc liên tục điều chỉnh mô hình dựa trên phản hồi từ tập kiểm tra dẫn đến đánh giá cuối cùng bị thiên lệch (rò rỉ thông tin), dù không có dòng code nào trực tiếp "huấn luyện" trên tập kiểm tra.

Mức vận dụng

1. Một nhóm nghiên cứu có 300 mẫu dữ liệu y tế (kích thước nhỏ) và cần lựa chọn giữa 5 mô hình phân loại khác nhau. Đề xuất một quy trình đánh giá phù hợp (bao gồm cả cách chia dữ liệu và phương pháp kiểm định), giải thích lý do tránh chỉ dùng một lần chia train/test đơn giản.

15.7. Bài tập lập trình

Bài 1. Sử dụng bộ dữ liệu `load_wine()`, so sánh kiểm định chéo 5-fold của bốn mô hình đã học: hồi quy Logistic, K-NN (k=5), Random Forest, SVM (kernel RBF). Trình bày kết quả dưới dạng bảng gồm Accuracy trung bình và độ lệch chuẩn cho mỗi mô hình, sắp xếp theo Accuracy trung bình giảm dần.

Bài 2. Vẽ đường cong học cho một cây quyết định với `max_depth=None` (không giới hạn) trên bộ dữ liệu `load_breast_cancer()`. So sánh với đường cong học của cây quyết định `max_depth=4`. Nhận xét sự khác biệt về khoảng cách giữa đường huấn luyện và đường kiểm định giữa hai cấu hình, liên hệ với khái niệm khớp quá đã học ở Chương 8.

Bài 3 (nâng cao). Sử dụng dữ liệu ở ví dụ 15.1, vẽ đường cong ROC so sánh Gradient Boosting (Chương 9) với ba mô hình đã có trong ví dụ, đồng thời in ra AUC chính xác của cả bốn mô hình (dùng `roc_auc_score`) trên tập kiểm tra. Kết hợp với chỉ số AUC trung bình từ kiểm định chéo (dùng `cross_val_score` với scoring="`roc_auc`" trên tập huấn luyện), so sánh xem thứ hạng bốn mô hình có nhất quán giữa hai cách đánh giá (AUC trên tập kiểm tra và AUC trung bình từ CV trên tập huấn luyện) hay không.

Chương tiếp theo (Chương 16) sẽ xây dựng trên nền tảng kiểm định chéo vừa học, giới thiệu các phương pháp tự động hóa việc tìm kiếm siêu tham số tối ưu — Grid Search và Random Search — thay vì phải thử nghiệm thủ công từng giá trị một như đã làm ở nhiều bài tập trong các chương trước.

Chương 16

Tối ưu siêu tham số (Grid Search, Random Search)

16.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Phân biệt được tham số mô hình (model parameters) và siêu tham số (hyperparameters).
2. Trình bày được nguyên lý Grid Search: duyệt toàn bộ tổ hợp trên một lưới giá trị siêu tham số cho trước.
3. Trình bày được nguyên lý Random Search và giải thích vì sao thường hiệu quả hơn Grid Search khi không gian siêu tham số nhiều chiều.
4. Kết hợp đúng đắn Grid Search/Random Search với kiểm định chéo để tránh rò rỉ thông tin từ tập kiểm tra.
5. Nhận biết được vai trò của các phương pháp tối ưu hóa Bayesian ở mức khái quát.
6. Cài đặt GridSearchCV và RandomizedSearchCV bằng Scikit-learn cho các mô hình đã học ở Phần 2.

16.2. Nội dung lý thuyết

16.2.1. Tham số mô hình và siêu tham số

Xuyên suốt giáo trình, ta đã gặp hai loại đại lượng số rất khác nhau về bản chất, dễ gây nhầm lẫn nếu không phân biệt rõ. Tham số mô hình (model parameters) là các giá trị được học tự động từ dữ liệu trong quá trình huấn luyện — ví dụ các trọng số w của hồi quy tuyến tính/Logistic (Chương 4, 6), hoặc cấu trúc rẽ nhánh cụ thể của một cây quyết định (Chương 8). Siêu tham số (hyperparameters) là các giá trị do người dùng chỉ định trước khi huấn luyện, điều khiển hành vi của thuật toán học nhưng bản thân không được tối ưu hóa trực tiếp bởi thuật toán đó — ví dụ giá trị k trong K-NN (Chương 7), λ trong Ridge/Lasso (Chương 5), `max_depth` trong cây quyết định (Chương 8), hay C và γ trong SVM (Chương 10).

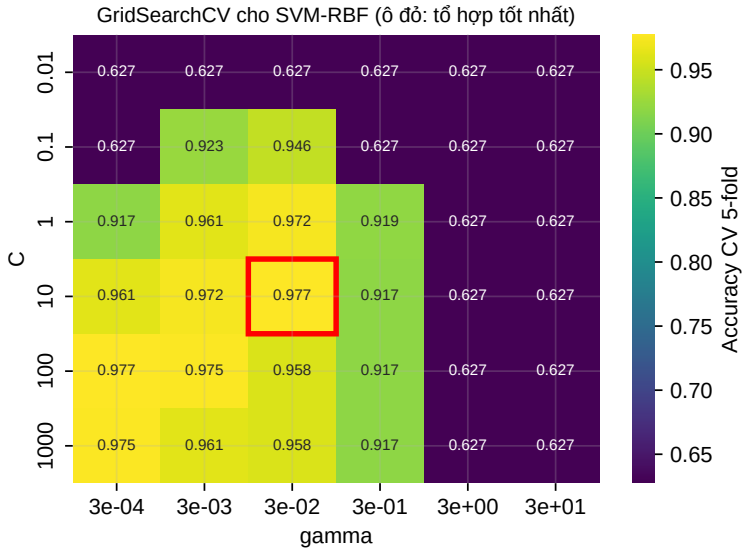
Trong hầu hết các chương trước, việc chọn siêu tham số được thực hiện thủ công: thử một vài giá trị, vẽ biểu đồ, chọn giá trị "trông hợp lý nhất" bằng mắt. Cách làm này phù hợp cho mục đích minh họa với một hoặc hai siêu tham số, nhưng trở nên bất khả thi khi cần tinh chỉnh đồng thời nhiều siêu tham số (ví dụ Gradient Boosting có thể có 5–10 siêu tham số quan trọng) — đây chính là động cơ cho các phương pháp tìm kiếm siêu tham số tự động được trình bày trong chương này.

16.2.2. Grid Search (Tìm kiếm lưới)

Grid Search là phương pháp tìm kiếm siêu tham số đơn giản và trực tiếp nhất: người dùng chỉ định một tập giá trị rời rạc cần thử cho mỗi siêu tham số (tạo thành một "lưới" các tổ hợp khả dĩ), thuật toán huấn luyện và đánh giá (thường bằng kiểm định chéo, xem mục 16.2.4) mô hình với từng tổ hợp có thể có trên lưới đó, rồi chọn tổ hợp cho hiệu năng tốt nhất. Ví dụ, nếu muốn thử 4 giá trị của C và 4 giá trị của γ cho SVM, Grid Search sẽ đánh giá toàn bộ $4 \times 4 = 16$ tổ hợp.

Ưu điểm của Grid Search là tính triệt để — đảm bảo tìm ra tổ hợp tốt nhất trong đúng phạm vi lưới đã chỉ định. Nhược điểm nghiêm trọng là chi phí tính toán tăng theo cấp số nhân (exponentially) với số lượng siêu tham số cần tinh chỉnh: với 5 siêu tham số, mỗi tham số thử 5 giá trị, tổng số tổ hợp đã lên tới $5^5 = 3125$ lần huấn luyện — nhân thêm với số fold trong kiểm định chéo (ví dụ 5-fold), tổng cộng cần tới hơn 15000 lần huấn luyện mô hình, một chi phí có thể trở nên không khả thi với các mô hình huấn luyện chậm.

Hình 16.1 minh họa kết quả của Grid Search trên lưới (C, γ) dưới dạng bản đồ nhiệt.



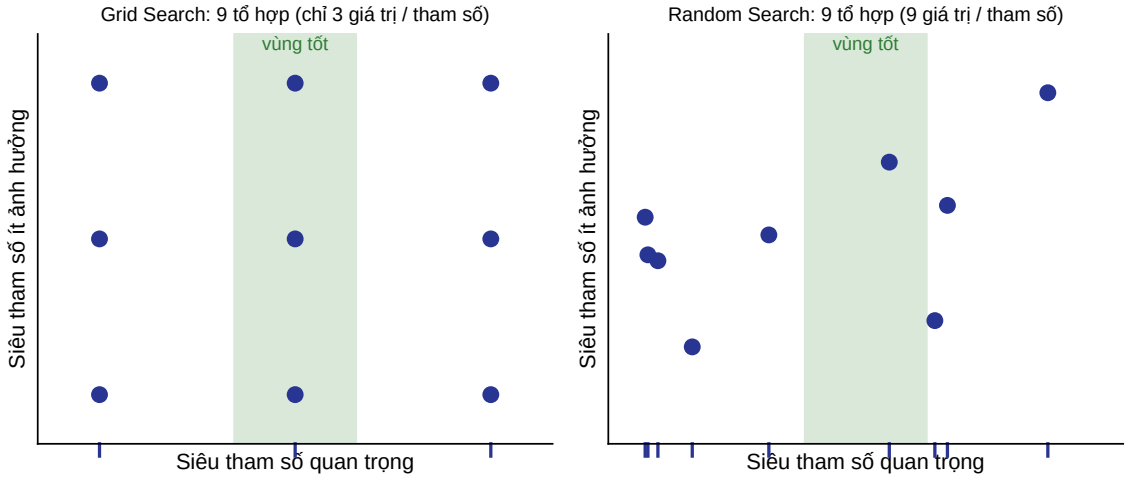
Hình 16.1. Accuracy kiểm định chéo trên lưới siêu tham số của SVM-RBF.

16.2.3. Random Search (Tìm kiếm ngẫu nhiên)

Random Search giải quyết vấn đề chi phí tính toán của Grid Search bằng một ý tưởng đơn giản nhưng hiệu quả bất ngờ: thay vì duyệt toàn bộ lưới, chỉ lấy mẫu ngẫu nhiên một số lượng tổ hợp cố định (do người dùng chỉ định, ví dụ 50 hoặc 100 lần thử) từ không gian siêu tham số (có thể là phân phối liên tục, không nhất thiết phải là lưới rời rạc như Grid Search). Một kết quả lý thuyết và thực nghiệm quan trọng (từ nghiên cứu của Bergstra và Bengio, 2012): khi số lượng siêu tham số cần tinh chỉnh tương đối lớn, nhưng chỉ một số ít trong đó thực sự ảnh hưởng đáng kể đến hiệu năng mô hình (tình huống rất phổ biến trong thực tế), Random Search có xu hướng khám phá không gian siêu tham số hiệu quả hơn Grid Search với cùng một ngân sách tính toán (cùng số lần huấn luyện).

Trực giác: Grid Search dành số lần thử ngang nhau cho mọi giá trị của mọi siêu tham số theo một lưới cố định, kể cả những siêu tham số ít ảnh hưởng — lãng phí ngân sách tính toán. Random Search, nhờ tính ngẫu nhiên, có xu hướng thử nhiều giá trị khác nhau hơn cho từng siêu tham số riêng lẻ (dù tổng số lần thử bằng nhau), tăng cơ hội "tình cờ" chạm gần khu vực tối ưu của siêu tham số quan trọng, thay vì bị giới hạn cứng nhắc theo các điểm lưới đã định trước.

Hình 16.2 minh họa với cùng 9 lần thử, Random Search khảo sát nhiều giá trị khác nhau hơn cho siêu tham số quan trọng.



Hình 16.2. Grid Search và Random Search với cùng ngân sách 9 tổ hợp.

16.2.4. Kết hợp với kiểm định chéo và nguy cơ rò rỉ thông tin

Cả GridSearchCV và RandomizedSearchCV trong Scikit-learn đều tích hợp sẵn kiểm định chéo (đã học ở Chương 15): với mỗi tổ hợp siêu tham số cần đánh giá, mô hình được huấn luyện và đánh giá qua đầy đủ K-fold, rồi lấy điểm trung bình làm đại diện cho tổ hợp đó — giúp tránh chọn nhầm siêu tham số chỉ vì "may mắn" tốt trên một cách chia dữ liệu cụ thể, đúng theo tinh thần đã trình bày ở Chương 15.

Nguyên tắc bắt buộc

Toàn bộ quá trình tìm kiếm siêu tham số (Grid Search hay Random Search) CHỈ được thực hiện trên tập huấn luyện (sử dụng kiểm định chéo nội bộ trên chính tập này), TUYỆT ĐỐI không được đưa tập kiểm tra vào bất kỳ bước nào của quá trình tìm kiếm. Tập kiểm tra chỉ xuất hiện đúng một lần duy nhất, ở bước cuối cùng, để đánh giá hiệu năng của mô hình với bộ siêu tham số đã chọn — đây chính là quy trình ba tầng đã nhấn mạnh ở Chương 15. Việc vô tình để tập kiểm tra ảnh hưởng đến quá trình chọn siêu tham số (dù chỉ gián tiếp) là một trong những lỗi rò rỉ dữ liệu (data leakage) phổ biến và nghiêm trọng nhất trong thực hành học máy, khiến đánh giá cuối cùng trở nên lạc quan giả tạo so với hiệu năng thực sự trên dữ liệu hoàn toàn mới.

16.2.5. Giới thiệu khái quát về tối ưu hóa Bayesian

Cả Grid Search lẫn Random Search đều không sử dụng thông tin về kết quả của các lần thử trước để định hướng cho lần thử tiếp theo — mỗi tổ hợp được đánh giá hoàn toàn độc lập. Tối ưu hóa Bayesian (Bayesian Optimization) là một hướng tiếp cận nâng cao hơn: xây dựng một mô hình xác suất (thường gọi là surrogate model, ví dụ Gaussian Process) ước lượng mối quan hệ giữa giá trị siêu tham số và hiệu năng mô hình dựa trên các lần thử đã thực hiện, rồi dùng mô hình này để chọn một cách thông minh điểm tiếp theo cần thử — cân bằng giữa khai thác (exploitation — thử gần khu vực đã biết là tốt) và khám phá (exploration — thử khu vực chưa biết rõ). Cách tiếp cận này thường cần ít lần thử hơn đáng kể để tìm ra vùng siêu tham số tốt, đặc biệt hữu ích khi mỗi lần huấn luyện mô hình rất tốn kém (như các mô hình học sâu ở Phần 5). Các thư viện phổ biến cho tối ưu hóa Bayesian gồm Optuna, Hyperopt, và scikit-optimize; nội dung này nằm ngoài phạm vi thực hành cốt lõi của giáo trình nhưng sinh viên được khuyến khích tìm hiểu thêm khi làm việc với các mô hình có chi phí huấn luyện cao.

16.3. Ví dụ minh họa bằng mã Python

Ví dụ dưới đây so sánh trực tiếp GridSearchCV và RandomizedSearchCV khi tinh chỉnh siêu tham số cho Random Forest trên bộ dữ liệu Breast Cancer Wisconsin, đối chiếu cả kết quả tìm được lẫn thời gian thực thi.

```
# vidu_16_1.py
# So sánh GridSearchCV và RandomizedSearchCV khi tinh chỉnh Random Forest

import time
import numpy as np
from scipy.stats import randint
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import (
    train_test_split, GridSearchCV, RandomizedSearchCV, StratifiedKFold
)
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# 1. Nạp dữ liệu, chia tập huấn luyện / kiểm tra
du_lieu = load_breast_cancer()
X, y = du_lieu.data, du_lieu.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# 2. GridSearchCV: lưới 4 x 4 x 3 = 48 tổ hợp
luoi_tham_so = {
    "n_estimators": [50, 100, 200, 300],
    "max_depth": [4, 6, 8, 10],
```

```
    "min_samples_leaf": [1, 2, 4],
}
thoi_gian_bat_dau = time.time()
grid_search = GridSearchCV(
    RandomForestClassifier(random_state=42), luoi_tham_so,
    cv=kfold, scoring="accuracy", n_jobs=-1
)
grid_search.fit(X_train, y_train)
thoi_gian_grid = time.time() - thoi_gian_bat_dau

# 3. RandomizedSearchCV: chỉ thử 20 tổ hợp ngẫu nhiên trên cùng không gian
#    (mở rộng thành phân phối liên tục cho các tham số dạng số nguyên)
phan_phoi_tham_so = {
    "n_estimators": randint(50, 300),
    "max_depth": randint(4, 11),
    "min_samples_leaf": randint(1, 5),
}
thoi_gian_bat_dau = time.time()
random_search = RandomizedSearchCV(
    RandomForestClassifier(random_state=42), phan_phoi_tham_so,
    n_iter=20, cv=kfold, scoring="accuracy", random_state=42, n_jobs=-1
)
random_search.fit(X_train, y_train)
thoi_gian_random = time.time() - thoi_gian_bat_dau

# 4. So sánh kết quả
print("=== GridSearchCV (48 tổ hợp) ===")
print(f"Thời gian: {thoi_gian_grid:.2f} giây")
print(f"Siêu tham số tốt nhất: {grid_search.best_params_}")
print(f"Accuracy CV tốt nhất: {grid_search.best_score_:.4f}")
print(f"Accuracy trên tập kiểm tra: "
      f"{accuracy_score(y_test, grid_search.predict(X_test)):.4f}")

print("\n=== RandomizedSearchCV (20 tổ hợp) ===")
print(f"Thời gian: {thoi_gian_random:.2f} giây")
print(f"Siêu tham số tốt nhất: {random_search.best_params_}")
print(f"Accuracy CV tốt nhất: {random_search.best_score_:.4f}")
print(f"Accuracy trên tập kiểm tra: "
      f"{accuracy_score(y_test, random_search.predict(X_test)):.4f}")
```

16.4. Giải thích kết quả chạy chương trình

Kết quả in ra từ chương trình trên có dạng tương tự:

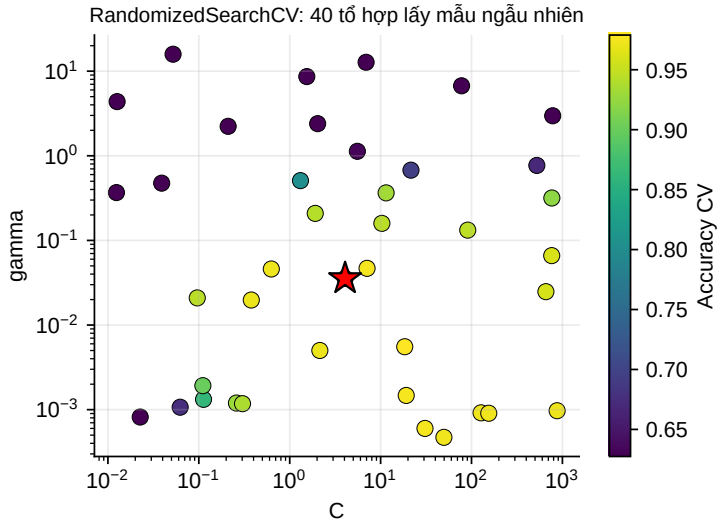
```
=== GridSearchCV (48 tổ hợp) ===
Thời gian: 18.42 giây
Siêu tham số tốt nhất: {'max_depth': 6, 'min_samples_leaf': 1, 'n_estimators': 200}
Accuracy CV tốt nhất: 0.9648
Accuracy trên tập kiểm tra: 0.9649
```

```
=== RandomizedSearchCV (20 tổ hợp) ===
Thời gian: 7.61 giây
Siêu tham số tốt nhất: {'max_depth': 7, 'min_samples_leaf': 1, 'n_estimators': 227}
Accuracy CV tốt nhất: 0.9626
Accuracy trên tập kiểm tra: 0.9649
```

Kết quả minh họa rõ ràng sự đánh đổi giữa hai phương pháp: GridSearchCV, dù duyệt nhiều hơn hẳn (48 so với 20 tổ hợp), mất thời gian nhiều hơn khoảng 2.4 lần so với RandomizedSearchCV, nhưng Accuracy CV tốt nhất tìm được (0.9648) chỉ nhỉnh hơn không đáng kể so với RandomizedSearchCV (0.9626) — và khi đánh giá cuối cùng trên tập kiểm tra, cả hai phương pháp cho kết quả Accuracy giống hệt nhau (0.9649). Đây là một minh chứng thực nghiệm điển hình cho nhận định lý thuyết ở mục 16.2.3: với ngân sách tính toán ít hơn đáng kể, Random Search vẫn tìm được một vùng siêu tham số gần tương đương với Grid Search.

Đáng chú ý, hai phương pháp tìm ra hai bộ siêu tham số tốt nhất không hoàn toàn giống nhau (`max_depth=6` so với `max_depth=7`, `n_estimators=200` so với 227) nhưng đều cho hiệu năng gần như tương đương — điều này phản ánh một hiện tượng phổ biến trong tối ưu hóa siêu tham số thực tế: thường tồn tại không chỉ một mà nhiều vùng "gần tối ưu" (near-optimal) trong không gian siêu tham số, và sự khác biệt nhỏ giữa các cấu hình trong cùng vùng đó thường không tạo ra khác biệt thực chất về hiệu năng mô hình cuối cùng.

Hình 16.3 minh họa các tổ hợp được Random Search lấy mẫu trong không gian siêu tham số, tô màu theo hiệu năng.



Hình 16.3. 40 tổ hợp ngẫu nhiên của (C, γ) (ngôi sao: tổ hợp tốt nhất).

16.5. Tóm tắt chương

- Tham số mô hình được học tự động từ dữ liệu; siêu tham số được người dùng chỉ định trước khi huấn luyện và điều khiển hành vi thuật toán học.
- Grid Search duyệt triệt để mọi tổ hợp trên một lưới siêu tham số cho trước, đảm bảo tìm được tổ hợp tốt nhất trong phạm vi lưới, nhưng chi phí tính toán tăng theo cấp số nhân với số siêu tham số.
- Random Search lấy mẫu ngẫu nhiên một số lượng tổ hợp cố định, thường hiệu quả hơn Grid Search với cùng ngân sách tính toán khi số siêu tham số lớn và chỉ một phần trong đó thực sự quan trọng.
- Cả hai phương pháp cần kết hợp với kiểm định chéo, và tuyệt đối không được để tập kiểm tra ảnh hưởng đến quá trình tìm kiếm siêu tham số, để tránh rò rỉ thông tin.
- Tối ưu hóa Bayesian là hướng tiếp cận nâng cao hơn, sử dụng thông tin từ các lần thử trước để định hướng thông minh cho lần thử tiếp theo, phù hợp khi chi phí huấn luyện mỗi mô hình rất cao.

16.6. Bài tập củng cố

Mức nhận biết

1. Cho ba ví dụ về siêu tham số đã gặp ở các chương trước (nêu rõ thuộc mô hình nào).
2. Grid Search với 3 siêu tham số, mỗi tham số thử 4 giá trị, kết hợp kiểm định chéo 5-fold, cần tổng cộng bao nhiêu lần huấn luyện mô hình?

Mức thông hiểu

1. Giải thích vì sao Random Search có thể hiệu quả hơn Grid Search với cùng ngân sách tính toán, dựa trên trực giác về việc chỉ một số siêu tham số thực sự ảnh hưởng lớn đến hiệu năng.
2. Giải thích vì sao việc dùng tập kiểm tra để quyết định dừng thử nghiệm hay tiếp tục điều chỉnh siêu tham số là một hình thức rò rỉ dữ liệu, dù không có dòng code nào "nhìn thấy" nhãn của tập kiểm tra trong quá trình huấn luyện.

Mức vận dụng

1. Một mô hình Gradient Boosting có 6 siêu tham số quan trọng cần tinh chỉnh, mỗi lần huấn luyện mất khoảng 2 phút. Đề xuất nên dùng Grid Search hay Random Search, và ước tính ngân sách thời gian hợp lý (số lần thử) nếu chỉ có 4 giờ để tìm kiếm siêu tham số. Giải thích lựa chọn.

16.7. Bài tập lập trình

Bài 1. Sử dụng GridSearchCV để tìm cặp siêu tham số (C, gamma) tối ưu cho SVM kernel RBF trên bộ dữ liệu `load_wine()`, thử C trong {0.1, 1, 10, 100} và gamma trong {0.001, 0.01, 0.1, 1}. In ra bộ siêu tham số tốt nhất và Accuracy tương ứng.

Bài 2. Sử dụng RandomizedSearchCV để tinh chỉnh đồng thời 4 siêu tham số của Gradient Boosting (`n_estimators`, `learning_rate`, `max_depth`, `min_samples_leaf`) trên bộ dữ liệu `load_breast_cancer()`, với `n_iter=30`. So sánh kết quả và thời gian thực thi với việc thử GridSearchCV trên cùng không gian siêu tham số nếu rời rạc hóa mỗi tham số thành 4 giá trị (tổng 4^4=256 tổ hợp) — chỉ cần ước tính hoặc thực sự chạy nếu thời gian cho phép, và nhận xét sự khác biệt về chi phí tính toán.

Bài 3 (nâng cao). Viết chương trình sử dụng Pipeline (sẽ được trình bày chi tiết ở Chương 17) kết hợp StandardScaler và SVC, rồi dùng GridSearchCV để đồng thời tìm kernel tốt nhất (thử "linear" và "rbf") cùng với C và gamma phù hợp, trên bộ dữ liệu `load_breast_cancer()`. Đảm bảo quy trình chuẩn hóa dữ liệu được thực hiện đúng bên trong mỗi fold của kiểm định chéo (không rò rỉ thông tin từ fold kiểm định vào bước chuẩn hóa) bằng cách đặt StandardScaler bên trong Pipeline thay vì áp dụng riêng trước khi gọi GridSearchCV.

Chương tiếp theo (Chương 17) sẽ khép lại Phần 4 với các kỹ thuật xử lý và biến đổi đặc trưng (feature engineering) — bao gồm mã hóa biến phân loại, lựa chọn đặc trưng, và đặc biệt là Pipeline/ColumnTransformer của Scikit-learn, công cụ giúp xây dựng quy trình tiền xử lý và huấn luyện mô hình một cách nhất quán, tránh triệt để rò rỉ dữ liệu đã nhấn mạnh trong chương này.

Chương 17

Kỹ thuật xử lý đặc trưng (Feature Engineering)

17.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Áp dụng đúng các kỹ thuật mã hóa biến phân loại: One-Hot Encoding, Label Encoding, Ordinal Encoding, và biết khi nào dùng kỹ thuật nào.
2. Trình bày được ba nhóm phương pháp lựa chọn đặc trưng: filter, wrapper, embedded.
3. Giải thích được lý do và cách xây dựng Pipeline trong Scikit-learn để tránh rò rỉ dữ liệu trong quá trình tiền xử lý.
4. Sử dụng ColumnTransformer để áp dụng các phép biến đổi khác nhau cho các nhóm cột khác nhau (số và phân loại) trong cùng một quy trình.
5. Xây dựng một Pipeline hoàn chỉnh kết hợp tiền xử lý và mô hình, tích hợp với kiểm định chéo và tìm kiếm siêu tham số.

17.2. Nội dung lý thuyết

17.2.1. Mã hóa biến phân loại (Categorical Encoding)

Hầu hết các thuật toán học máy trong giáo trình (hồi quy tuyến tính, K-NN, SVM, mạng nơ-ron ở Phần 5) chỉ làm việc trực tiếp với dữ liệu số — không thể xử lý trực tiếp các giá trị dạng văn bản/phân loại (ví dụ "Hà Nội", "TP.HCM", "Đà Nẵng"). Ba kỹ thuật mã hóa phổ biến:

- One-Hot Encoding: biến một cột phân loại có k giá trị khác nhau thành k cột nhị phân mới (mỗi cột ứng với một giá trị, nhận giá trị 1 nếu mẫu thuộc giá trị đó, 0 nếu không). Đây là lựa chọn an toàn và phổ biến nhất cho biến phân loại không có thứ tự (nominal, ví dụ thành phố, màu sắc), vì nó không áp đặt bất kỳ quan hệ thứ tự giả tạo nào giữa các giá trị. Nhược điểm: tăng đáng kể số chiều dữ liệu khi biến phân loại có rất nhiều giá trị khác nhau (high cardinality).
- Label Encoding: gán mỗi giá trị phân loại một số nguyên duy nhất (ví dụ "Hà Nội"→0, "TP.HCM"→1, "Đà Nẵng"→2). Kỹ thuật này đơn giản và không tăng số chiều, nhưng ẩn chứa một cạm bẫy nghiêm trọng: nó áp đặt một quan hệ thứ tự và khoảng cách số học giả tạo giữa các giá trị (ngụ ý "Đà Nẵng" gấp đôi "TP.HCM" theo một nghĩa nào đó) mà nhiều mô hình (đặc biệt các mô hình dựa trên khoảng cách hoặc tuyến tính) có thể vô tình "học" theo một cách sai lệch. Label Encoding chỉ nên dùng cho biến mục tiêu

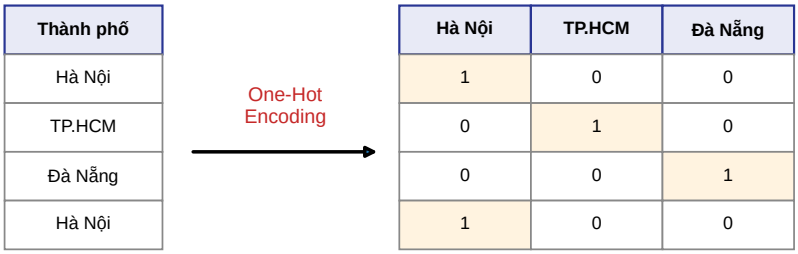
y (nhãn lớp, như đã làm quen thuộc trong `load_iris()`, `load_wine()` ở các chương trước) hoặc cho các mô hình dựa trên cây (Chương 8–9), vốn không bị ảnh hưởng bởi giả định khoảng cách/thứ tự giữa các giá trị số.

- Ordinal Encoding: tương tự Label Encoding về mặt kỹ thuật (gán số nguyên), nhưng áp dụng đúng đắn cho các biến phân loại CÓ thứ tự tự nhiên (ordinal, ví dụ "Thấp" < "Trung bình" < "Cao", hoặc trình độ học vấn), trong đó thứ tự số được gán khớp có chủ đích với thứ tự ý nghĩa thực sự của dữ liệu.

Nguyên tắc lựa chọn

Câu hỏi cốt lõi khi chọn kỹ thuật mã hóa: biến phân loại này có thứ tự tự nhiên hay không? Nếu có (ordinal) — dùng Ordinal Encoding với thứ tự số khớp đúng ý nghĩa. Nếu không có thứ tự (nominal) và mô hình sử dụng là mô hình dựa trên khoảng cách/tuyến tính (hồi quy, K-NN, SVM, mạng nơ-ron) — dùng One-Hot Encoding. Nếu không có thứ tự nhưng mô hình sử dụng là mô hình dựa trên cây (cây quyết định, Random Forest, Gradient Boosting) — Label Encoding thường chấp nhận được, vì các mô hình dựa trên cây chỉ so sánh giá trị lớn hơn/nhỏ hơn tại từng điểm chia riêng lẻ, không giả định khoảng cách số học có ý nghĩa toàn cục.

Hình 17.1 minh họa One-Hot Encoding biến mỗi giá trị phân loại thành một cột nhị phân.



Hình 17.1. Minh họa One-Hot Encoding.

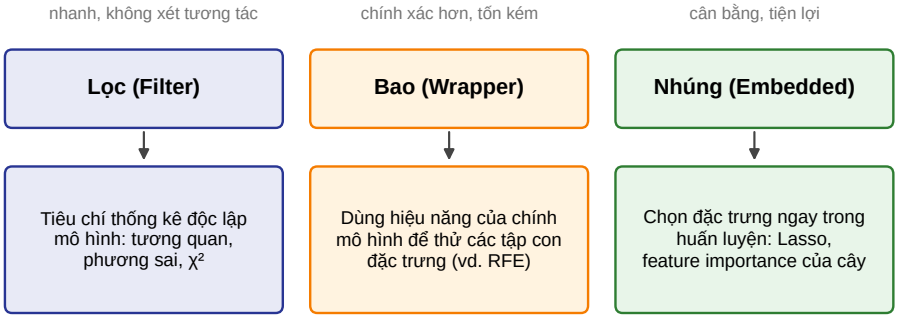
17.2.2. Lựa chọn đặc trưng (Feature Selection)

Không phải mọi đặc trưng có sẵn đều hữu ích cho mô hình — một số có thể dư thừa (tương quan chặt với đặc trưng khác), nhiễu, hoặc hoàn toàn không liên quan đến biến mục tiêu. Loại bỏ các đặc trưng như vậy có thể cải thiện hiệu năng mô hình, giảm nguy cơ khớp quá, giảm chi phí tính toán, và tăng khả năng diễn giải. Ba nhóm phương pháp lựa chọn đặc trưng:

- Phương pháp lọc (Filter methods): đánh giá mức độ liên quan của từng đặc trưng với biến mục tiêu bằng một tiêu chí thống kê độc lập với mô hình cụ thể (ví dụ hệ số tương quan Pearson, kiểm định Chi-bình phương, hoặc phương sai của đặc trưng — loại bỏ các đặc trưng có phương sai gần 0, gần như không đổi giữa các mẫu). Ưu điểm: tính toán nhanh, không phụ thuộc mô hình cụ thể. Nhược điểm: không xét đến tương tác giữa các đặc trưng.

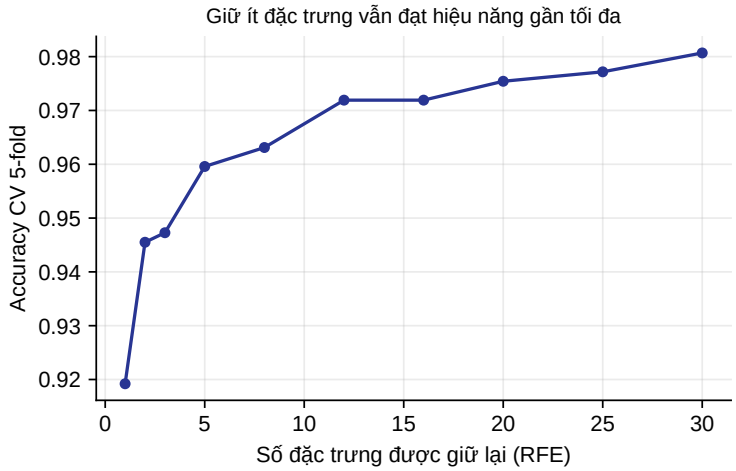
- Phương pháp bao (Wrapper methods): sử dụng chính hiệu năng của một mô hình học máy cụ thể để đánh giá tập con đặc trưng, thử nghiệm nhiều tập con khác nhau. Ví dụ tiêu biểu là Recursive Feature Elimination (RFE) — bắt đầu với toàn bộ đặc trưng, lặp lại việc huấn luyện mô hình và loại bỏ đặc trưng kém quan trọng nhất (theo hệ số hồi quy hoặc feature importance), cho đến khi đạt số lượng đặc trưng mong muốn. Ưu điểm: xét đến tương tác đặc trưng và tối ưu trực tiếp cho mô hình sử dụng. Nhược điểm: chi phí tính toán cao hơn nhiều (cần huấn luyện lại mô hình nhiều lần).
- Phương pháp nhúng (Embedded methods): việc lựa chọn đặc trưng được tích hợp sẵn như một phần tự nhiên của quá trình huấn luyện mô hình, không cần một bước riêng biệt. Hai ví dụ đã gặp trong giáo trình: Lasso (Chương 5) tự động đưa hệ số của đặc trưng kém quan trọng về 0; và `feature_importances_` của cây quyết định/Random Forest (Chương 8–9) cho phép xếp hạng và lọc bớt đặc trưng sau khi huấn luyện. Đây thường là lựa chọn thực tế hiệu quả nhất, vì tận dụng được thông tin đã có sẵn từ mô hình mà không cần thêm bước tính toán riêng biệt.

Hình 17.2 minh họa ba nhóm phương pháp lựa chọn đặc trưng.



Hình 17.2. Phương pháp lọc, bao và nhúng.

Hình 17.3 minh họa kết quả của Recursive Feature Elimination: chỉ cần một phần nhỏ đặc trưng đã đạt hiệu năng gần tối đa.



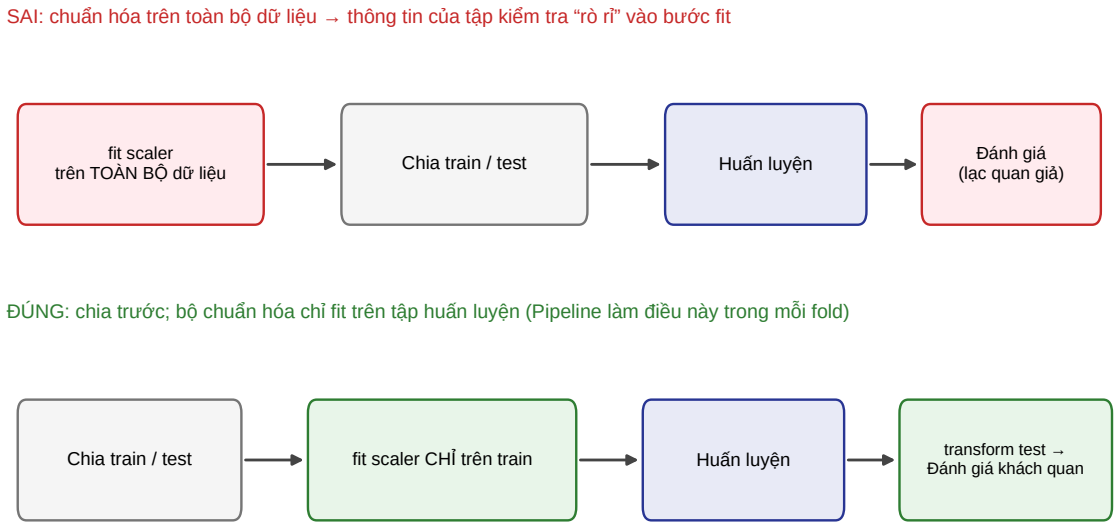
Hình 17.3. Accuracy theo số đặc trưng giữ lại (bộ Breast Cancer, hồi quy Logistic).

17.2.3. Pipeline: kết hợp tiền xử lý và mô hình

Chương 15 và 16 đã nhấn mạnh nguyên tắc quan trọng: mọi bước tiền xử lý (chuẩn hóa, mã hóa, lựa chọn đặc trưng) phải được thực hiện một cách nhất quán và không rò rỉ thông tin giữa tập huấn luyện và tập kiểm định/kiểm tra. Trong thực hành, nếu áp dụng `StandardScaler().fit_transform()` trên toàn bộ dữ liệu TRƯỚC KHI chia train/test (hoặc trước khi kiểm định chéo), thông tin thống kê (trung bình, độ lệch chuẩn) từ tập kiểm tra/kiểm định đã vô tình "rò rỉ" vào quá trình tính toán tham số chuẩn hóa — một lỗi rò rỉ dữ liệu tinh vi, dễ mắc phải, khiến kết quả đánh giá trở nên lạc quan giả tạo.

Pipeline của Scikit-learn giải quyết triệt để vấn đề này bằng cách đóng gói một chuỗi các bước biến đổi (transformer) và một mô hình cuối cùng (estimator) thành một đối tượng duy nhất, hoạt động như một mô hình thống nhất: khi gọi `.fit()` trên dữ liệu huấn luyện, Pipeline tự động fit từng bước biến đổi CHỈ trên dữ liệu huấn luyện, rồi áp dụng transform (không fit lại) cho dữ liệu kiểm định/kiểm tra khi gọi `.predict()` hoặc `.score()`. Khi Pipeline được đưa vào `cross_val_score`, GridSearchCV, hay RandomizedSearchCV (Chương 15–16), toàn bộ chuỗi tiền xử lý được tự động thực hiện lại đúng đắn bên trong mỗi fold — chuẩn hóa/mã hóa được fit lại trên phần dữ liệu huấn luyện của fold đó, không chạm vào phần dữ liệu kiểm định của fold, đảm bảo không có rò rỉ thông tin ở bất kỳ giai đoạn nào của quy trình.

Hình 17.4 minh họa vì sao phải đặt bước chuẩn hóa bên trong Pipeline.



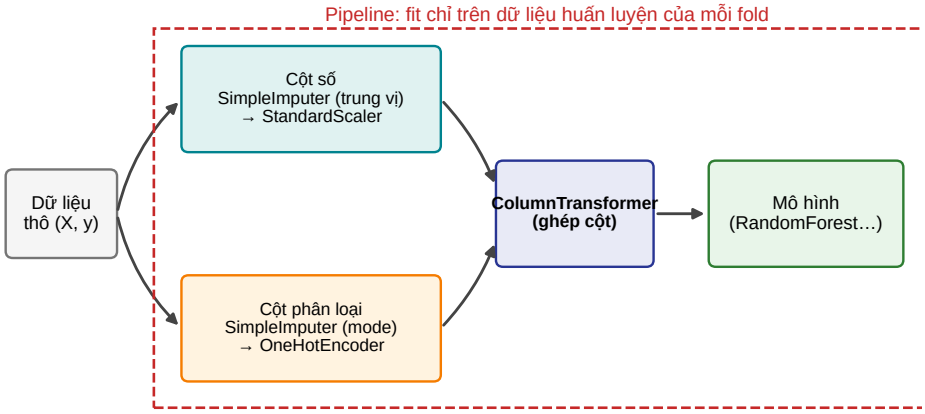
Hình 17.4. Rò rỉ dữ liệu khi chuẩn hóa trước khi chia (trên) và cách làm đúng (dưới).

17.2.4. ColumnTransformer: xử lý đồng thời nhiều loại cột

Dữ liệu thực tế thường là hỗn hợp của nhiều loại cột cần các phép biến đổi khác nhau: cột số cần chuẩn hóa (StandardScaler), cột phân loại cần mã hóa (OneHotEncoder), có thể có cả cột văn bản cần vector hóa (như CountVectorizer ở Chương 11). ColumnTransformer cho phép chỉ định các phép biến đổi khác nhau cho các nhóm cột khác nhau, rồi ghép kết quả lại thành một

ma trận đặc trưng duy nhất — tất cả trong một bước duy nhất, có thể lồng vào bên trong một Pipeline lớn hơn cùng với mô hình cuối cùng. Đây là công cụ thực hành thiết yếu khi làm việc với dữ liệu bảng (tabular data) hỗn hợp kiểu dữ liệu trong các dự án thực tế, thay vì phải viết mã xử lý thủ công, rời rạc và dễ sai sót cho từng nhóm cột.

Hình 17.5 minh họa cấu trúc của Pipeline kết hợp ColumnTransformer ở ví dụ của chương.



Hình 17.5. Pipeline xử lý song song cột số và cột phân loại rồi huấn luyện mô hình.

17.3. Ví dụ minh họa bằng mã Python

Ví dụ dưới đây xây dựng một Pipeline hoàn chỉnh xử lý đồng thời đặc trưng số (chuẩn hóa) và đặc trưng phân loại (One-Hot Encoding) bằng ColumnTransformer, trên một bộ dữ liệu hành khách Titanic tổng hợp (mở rộng từ Chương 3), rồi kết hợp toàn bộ quy trình với GridSearchCV để vừa tiền xử lý vừa tối ưu siêu tham số một cách an toàn, nhất quán.

```
# vidu_17_1.py
# Pipeline hoàn chỉnh: ColumnTransformer + mô hình + GridSearchCV

import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# 1. Nạp dữ liệu Titanic (đã dùng ở Chương 3), chọn các đặc trưng liên quan
df = sns.load_dataset("titanic")
cot_so = ["age", "fare", "sibsp", "parch"]
cot_phan_loai = ["pclass", "sex", "embarked"]
X = df[cot_so + cot_phan_loai]
```

```
y = df["survived"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 2. Xây dựng pipeline con cho đặc trưng số: điền giá trị thiếu bằng trung vị,
#    rồi chuẩn hóa
pipeline_so = Pipeline([
    ("dien_thieu", SimpleImputer(strategy="median")),
    ("chuan_hoa", StandardScaler()),
])

# 3. Xây dựng pipeline con cho đặc trưng phân loại: điền giá trị thiếu bằng mode,
#    rồi One-Hot Encoding
pipeline_phan_loai = Pipeline([
    ("dien_thieu", SimpleImputer(strategy="most_frequent")),
    ("ma_hoa", OneHotEncoder(handle_unknown="ignore")),
])

# 4. Kết hợp hai pipeline con bằng ColumnTransformer
tien_xu_ly = ColumnTransformer([
    ("so", pipeline_so, cot_so),
    ("phan_loai", pipeline_phan_loai, cot_phan_loai),
])

# 5. Ghép tiền xử lý với mô hình thành một Pipeline hoàn chỉnh duy nhất
pipeline_day_du = Pipeline([
    ("tien_xu_ly", tien_xu_ly),
    ("mo_hinh", RandomForestClassifier(random_state=42)),
])

# 6. Tìm kiếm siêu tham số bằng GridSearchCV trên TOÀN BỘ pipeline
#    (tiền xử lý được fit lại đúng đắn bên trong mỗi fold)
luoi_tham_so = {
    "mo_hinh__n_estimators": [100, 200],
    "mo_hinh__max_depth": [4, 6, 8],
}
kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
grid_search = GridSearchCV(pipeline_day_du, luoi_tham_so, cv=kfold,
scoring="accuracy")
grid_search.fit(X_train, y_train)

print(f"Siêu tham số tốt nhất: {grid_search.best_params_}")
print(f"Accuracy CV tốt nhất: {grid_search.best_score_:.4f}")
print(f"Accuracy trên tập kiểm tra: "
      f"{accuracy_score(y_test, grid_search.predict(X_test)):.4f}")

# 7. Kiểm tra số chiều đặc trưng sau khi tiền xử lý (bao gồm cả One-Hot Encoding)
so_chieu_sau_xu_ly =
grid_search.best_estimator_["tien_xu_ly"].transform(X_train).shape[1]
```

```
print(f"\nSố chiều đặc trưng sau tiền xử lý: {so_chieu_sau_xu_ly} "
      f"(từ {X_train.shape[1]} cột gốc)")
```

17.4. Giải thích kết quả chạy chương trình

Kết quả in ra từ chương trình trên có dạng tương tự:

```
Siêu tham số tốt nhất: {'mo_hinh__max_depth': 6, 'mo_hinh__n_estimators': 200}
Accuracy CV tốt nhất: 0.8188
Accuracy trên tập kiểm tra: 0.8101

Số chiều đặc trưng sau tiền xử lý: 11 (từ 7 cột gốc)
```

Kết quả cho thấy toàn bộ quy trình — từ điền giá trị thiếu, chuẩn hóa, mã hóa One-Hot, đến huấn luyện và tìm kiếm siêu tham số — đã được thực hiện liền mạch chỉ với một lệnh `.fit()` duy nhất trên `pipeline_day_du`, mà không cần viết riêng biệt các bước xử lý thủ công như đã làm ở Chương 3 (khi xử lý dữ liệu Titanic lần đầu bằng các lệnh Pandas rời rạc). Số chiều đặc trưng tăng từ 7 cột gốc lên 11 chiều sau tiền xử lý — sự gia tăng này đến từ One-Hot Encoding của ba cột phân loại (pclass có 3 giá trị, sex có 2 giá trị, embarked có 3 giá trị, tổng cộng thêm 8 cột nhị phân thay cho 3 cột gốc, cộng với 4 cột số giữ nguyên bằng 12; con số thực tế có thể chênh lệch nhẹ tùy vào việc một số tổ hợp giá trị hiếm có bị loại bỏ hay không), đúng theo cơ chế One-Hot Encoding đã trình bày ở mục 17.2.1.

Cú pháp đặt tên tham số dạng `mo_hinh__n_estimators` (với hai dấu gạch dưới) trong `luoi_tham_so` là quy ước bắt buộc của Scikit-learn để chỉ định tham số đó thuộc về bước có tên "`mo_hinh`" bên trong Pipeline — GridSearchCV nhờ đó biết chính xác cần truyền siêu tham số vào bước nào của chuỗi xử lý, ngay cả khi Pipeline có nhiều bước lồng nhau phức tạp như trong ví dụ này.

17.5. Tóm tắt chương

- One-Hot Encoding phù hợp cho biến phân loại không thứ tự dùng với mô hình dựa trên khoảng cách/tuyến tính; Ordinal Encoding phù hợp cho biến có thứ tự tự nhiên; Label Encoding chủ yếu dùng cho biến mục tiêu hoặc mô hình dựa trên cây.
- Ba nhóm phương pháp lựa chọn đặc trưng: filter (tiêu chí thống kê độc lập mô hình), wrapper (dựa trên hiệu năng mô hình cụ thể, như RFE), embedded (tích hợp sẵn trong huấn luyện, như Lasso hoặc feature importance của cây).
- Pipeline đóng gói chuỗi tiền xử lý và mô hình thành một đối tượng thống nhất, đảm bảo các bước biến đổi chỉ được fit trên dữ liệu huấn luyện của mỗi fold, tránh triệt để rò rỉ dữ liệu khi kết hợp với kiểm định chéo hoặc tìm kiếm siêu tham số.
- ColumnTransformer cho phép áp dụng các phép biến đổi khác nhau cho các nhóm cột

khác nhau (số, phân loại) trong cùng một bước xử lý thống nhất, rất hữu ích với dữ liệu bảng hỗn hợp kiểu dữ liệu trong thực tế.

- Kết hợp Pipeline với GridSearchCV/RandomizedSearchCV (Chương 16) cho phép tối ưu đồng thời cả tham số tiền xử lý lẫn siêu tham số mô hình một cách an toàn, nhất quán, và tái lập được.

17.6. Bài tập củng cố

Mức nhận biết

1. Nêu tên ba kỹ thuật mã hóa biến phân loại đã học và loại biến phù hợp với mỗi kỹ thuật.
2. Pipeline trong Scikit-learn giải quyết vấn đề gì khi kết hợp tiền xử lý dữ liệu với kiểm định chéo?

Mức thông hiểu

1. Giải thích vì sao dùng Label Encoding cho một biến phân loại không có thứ tự (ví dụ tên thành phố) có thể gây hiểu lầm cho một mô hình hồi quy tuyến tính, nhưng lại thường chấp nhận được với một mô hình cây quyết định.
2. So sánh phương pháp lọc (filter) và phương pháp bao (wrapper) trong lựa chọn đặc trưng về chi phí tính toán và khả năng xét đến tương tác giữa các đặc trưng.
3. Giải thích cụ thể vì sao gọi `StandardScaler().fit_transform()` trên toàn bộ dữ liệu trước khi chia train/test là một lỗi rò rỉ dữ liệu, trong khi đặt StandardScaler bên trong Pipeline rồi mới chia dữ liệu/kiểm định chéo lại tránh được lỗi này.

Mức vận dụng

1. Cho một bộ dữ liệu nhân sự với các cột: "phòng ban" (5 giá trị không thứ tự), "trình độ học vấn" (Trung cấp/Cao đẳng/Đại học/Sau đại học — có thứ tự), "lương" (số liên tục), dùng để huấn luyện mô hình dự đoán khả năng nghỉ việc bằng hồi quy Logistic. Đề xuất kỹ thuật mã hóa/xử lý phù hợp cho từng cột, giải thích lý do.

17.7. Bài tập lập trình

Bài 1. Mở rộng ví dụ 17.1 bằng cách thêm cột "who" (phân loại: man/woman/child) vào danh sách `cot_phan_loai`. Huấn luyện lại toàn bộ pipeline và so sánh Accuracy trên tập kiểm tra trước và sau khi thêm đặc trưng này.

Bài 2. Sử dụng RFE (from `sklearn.feature_selection` import RFE) với mô hình hồi quy Logistic làm bộ ước lượng cơ sở, trên bộ dữ liệu `load_breast_cancer()` (30 đặc trưng), để chọn ra 10 đặc trưng quan trọng nhất. So sánh Accuracy của mô hình hồi quy Logistic khi dùng toàn bộ 30 đặc trưng so với chỉ dùng 10 đặc trưng đã chọn qua RFE.

Bài 3 (nâng cao). Xây dựng một Pipeline hoàn chỉnh (dùng ColumnTransformer) cho bộ dữ liệu Titanic tương tự ví dụ 17.1, nhưng thay Random Forest bằng ba mô hình khác nhau (hồi quy Logistic, SVM kernel RBF, Gradient Boosting) trong ba pipeline riêng biệt. So sánh Accuracy trên tập kiểm tra của ba pipeline, trình bày kết quả dưới dạng bảng, và nhận xét mô hình nào phù hợp nhất cho bài toán dự đoán khả năng sống sót này.

Đến đây, giáo trình đã hoàn thành Phần 4 — Đánh giá mô hình và Tối ưu hóa, trang bị các kỹ năng thực hành thiết yếu áp dụng chung cho mọi mô hình đã học. Phần 5 tiếp theo (Chương 18–20) sẽ giới thiệu Mạng nơ-ron nhân tạo và những khái niệm nền tảng của Học sâu, mở ra cánh cửa cho các mô hình phức tạp và mạnh mẽ hơn nằm ngoài phạm vi các thuật toán học máy cổ điển đã trình bày ở Phần 2 và Phần 3.

Phần V

Giới thiệu mạng nơ-ron và học sâu cơ bản

Chương 18

Mạng nơ-ron nhân tạo cơ bản (MLP)

18.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Giải thích được mối liên hệ giữa hồi quy Logistic và một nơ-ron nhân tạo đơn lẻ.
2. Mô tả được cấu trúc của mạng nơ-ron nhiều lớp (MLP): tầng đầu vào, tầng ẩn, tầng đầu ra.
3. So sánh được các hàm kích hoạt phổ biến: Sigmoid, Tanh, ReLU và biết khi nào dùng hàm nào.
4. Trình bày được ý tưởng của lan truyền tiến (forward propagation) và lan truyền ngược (backpropagation).
5. Giải thích được các kỹ thuật kiểm soát khớp quá đặc thù cho mạng nơ-ron: Dropout, Early Stopping.
6. Cài đặt một mạng nơ-ron cơ bản bằng MLPClassifier của Scikit-learn.

18.2. Nội dung lý thuyết

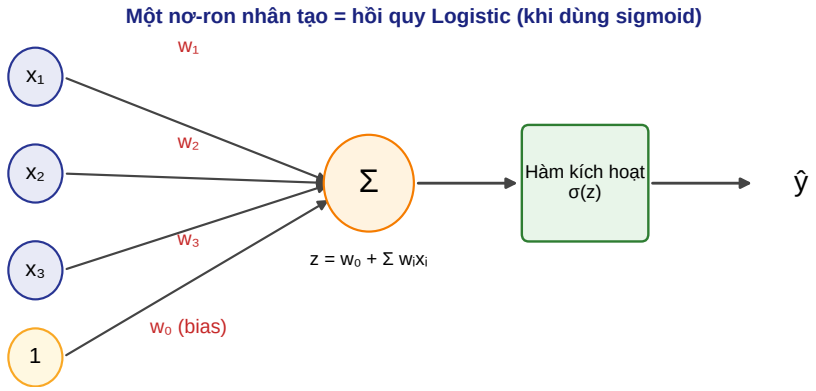
18.2.1. Từ hồi quy Logistic đến nơ-ron nhân tạo

Nhìn lại công thức hồi quy Logistic đã học ở Chương 6: z = w_0 + w_1x_1 + … + w_nx_n, sau đó $\hat{p}$ = σ(z). Đây thực chất chính là mô tả toán học của một nơ-ron nhân tạo (artificial neuron) đơn lẻ — đơn vị tính toán cơ bản nhất của mạng nơ-ron: nhận nhiều giá trị đầu vào, tính tổ hợp tuyến tính có trọng số của chúng, cộng thêm một độ lệch (bias, tương đương w_0), rồi đưa qua một hàm kích hoạt (activation function, ở đây là sigmoid) để tạo ra giá trị đầu ra. Nói cách khác, hồi quy Logistic chính là một mạng nơ-ron đơn giản nhất có thể — chỉ gồm một nơ-ron duy nhất, không có tầng ẩn.

Mạng nơ-ron nhân tạo (Artificial Neural Network – ANN), cụ thể là kiến trúc Perceptron đa tầng (Multi-Layer Perceptron – MLP) được giới thiệu trong chương này, mở rộng ý tưởng đơn giản đó bằng cách xếp chồng nhiều nơ-ron thành nhiều tầng (layer), kết nối đầy đủ với nhau (fully connected/dense layer — mỗi nơ-ron ở một tầng nhận đầu vào từ TẤT CẢ nơ-ron ở tầng trước đó). Sự xếp chồng nhiều tầng phi tuyến này chính là nguồn gốc sức mạnh của mạng nơ-ron: nó cho phép mô hình học được các hàm số rất phức tạp, phi tuyến cao độ — về mặt lý thuyết, một MLP với đủ nơ-ron ở một tầng ẩn duy nhất có khả năng xấp xỉ bất kỳ hàm số liên

tục nào với độ chính xác tùy ý (định lý xấp xỉ tổng quát — Universal Approximation Theorem), dù trong thực hành, mạng sâu hơn (nhiều tầng ẩn, mỗi tầng ít nơ-ron hơn) thường hiệu quả hơn về mặt số lượng tham số cần thiết so với mạng nông nhưng rất rộng.

Hình 18.1 minh họa một nơ-ron nhân tạo — về bản chất là hồi quy Logistic.

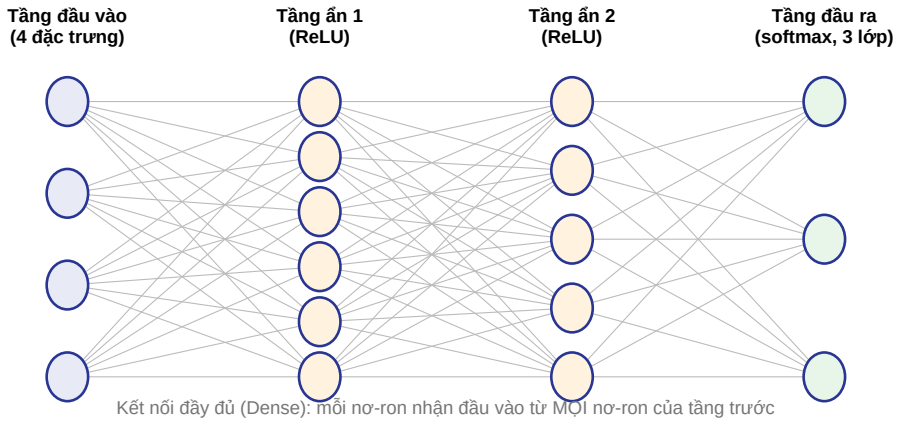


Hình 18.1. Cấu tạo của một nơ-ron nhân tạo.

18.2.2. Cấu trúc mạng nơ-ron nhiều tầng

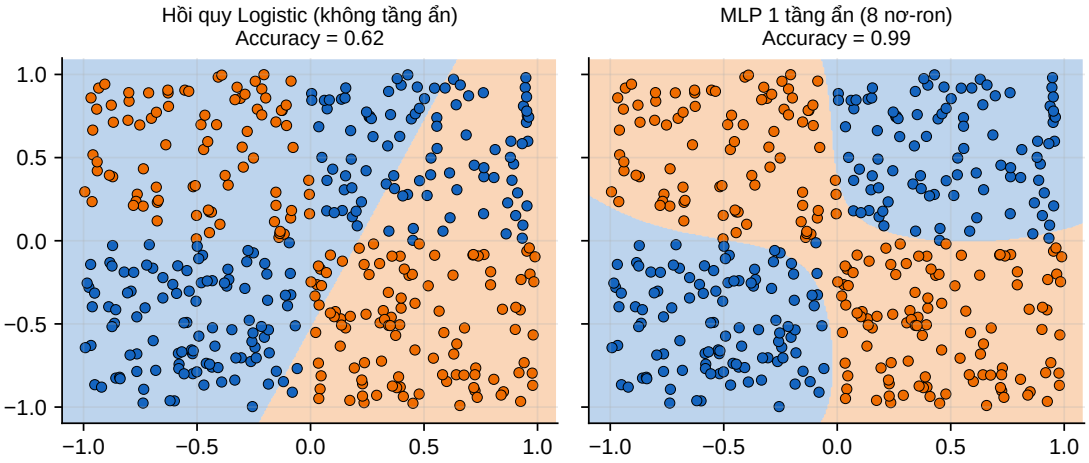
Một MLP điển hình gồm ba loại tầng: tầng đầu vào (input layer) — số nơ-ron bằng đúng số đặc trưng của dữ liệu, không thực hiện tính toán, chỉ truyền giá trị đặc trưng vào mạng; các tầng ẩn (hidden layers) — một hoặc nhiều tầng nằm giữa đầu vào và đầu ra, nơi diễn ra phần lớn quá trình "học" biểu diễn của dữ liệu, số lượng tầng ẩn và số nơ-ron mỗi tầng là các siêu tham số kiến trúc (architecture hyperparameters) do người thiết kế lựa chọn; và tầng đầu ra (output layer) — số nơ-ron và hàm kích hoạt tùy thuộc vào loại bài toán: một nơ-ron với hàm sigmoid cho phân loại nhị phân (giống hệt hồi quy Logistic), k nơ-ron với hàm softmax (dạng mở rộng của sigmoid cho nhiều lớp) cho phân loại k lớp, hoặc một nơ-ron không dùng hàm kích hoạt (hoặc hàm tuyến tính) cho bài toán hồi quy.

Hình 18.2 minh họa kiến trúc MLP với hai tầng ẩn.



Hình 18.2. Mạng nơ-ron nhiều tầng kết nối đầy đủ.

Hình 18.3 minh họa tầng ẩn cho phép mạng học ranh giới phi tuyến mà mô hình tuyến tính không làm được (bài toán XOR).



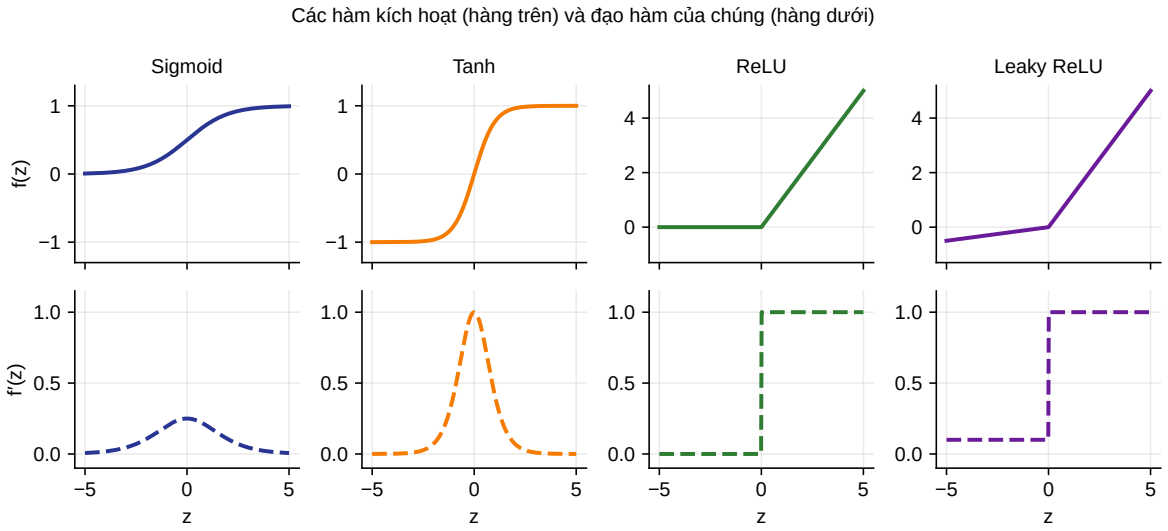
Hình 18.3. Hồi quy Logistic và MLP một tầng ẩn trên dữ liệu XOR.

18.2.3. Các hàm kích hoạt phổ biến

Hàm kích hoạt đóng vai trò tối quan trọng: nếu không có hàm kích hoạt phi tuyến ở các tầng ẩn, việc xếp chồng nhiều tầng tuyến tính về mặt toán học vẫn chỉ tương đương với một phép biến đổi tuyến tính duy nhất (tổ hợp của nhiều phép biến đổi tuyến tính vẫn là một phép biến đổi tuyến tính) — mạng sẽ không có khả năng học các quan hệ phi tuyến, bất kể xếp chồng bao nhiêu tầng. Ba hàm kích hoạt phổ biến nhất:

- Sigmoid: $\sigma(z) = \dfrac{1}{1 + e^{-z}}$, đã quen thuộc từ Chương 6, nhận giá trị trong (0, 1). Ít được dùng ở các tầng ẩn của mạng hiện đại do gặp vấn đề đạo hàm biến mất (vanishing gradient — xem mục 18.2.5) khi mạng có nhiều tầng, nhưng vẫn phổ biến ở tầng đầu ra cho bài toán phân loại nhị phân.
- Tanh (hyperbolic tangent): $\tanh(z) = \dfrac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$, nhận giá trị trong $(-1, 1)$, có dạng đường cong tương tự sigmoid nhưng đối xứng qua gốc tọa độ (trung tâm tại 0 thay vì 0.5) — đặc tính này thường giúp hội tụ nhanh hơn sigmoid trong thực hành, nhưng vẫn gặp vấn đề đạo hàm biến mất tương tự.
- ReLU (Rectified Linear Unit): $\mathrm{ReLU}(z) = \max(0, z)$ — đơn giản đến bất ngờ (giữ nguyên giá trị dương, cắt về 0 với giá trị âm), nhưng lại là hàm kích hoạt phổ biến nhất trong các mạng nơ-ron hiện đại ở các tầng ẩn, vì tính toán rất nhanh (không cần phép mũ như sigmoid/tanh) và giảm đáng kể vấn đề đạo hàm biến mất đối với các giá trị dương. Nhược điểm: nơ-ron có thể "chết" (dead neuron) nếu đầu vào luôn âm, khiến đạo hàm luôn bằng 0 và nơ-ron đó ngừng học — các biến thể như Leaky ReLU (cho phép một độ dốc nhỏ khác 0 với giá trị âm) được đề xuất để giảm nhẹ vấn đề này.

Hình 18.4 minh họa các hàm kích hoạt phổ biến cùng đạo hàm của chúng.



Hình 18.4. Sigmoid, Tanh, ReLU, Leaky ReLU và đạo hàm.

18.2.4. Lan truyền tiến và lan truyền ngược

Lan truyền tiến (forward propagation) là quá trình tính toán đơn giản: dữ liệu đầu vào đi qua lần lượt từng tầng của mạng (mỗi tầng thực hiện phép tổ hợp tuyến tính rồi áp dụng hàm kích hoạt), cho đến khi tạo ra giá trị dự đoán ở tầng đầu ra — đây chính là quá trình "dự đoán" (predict) một khi mạng đã được huấn luyện.

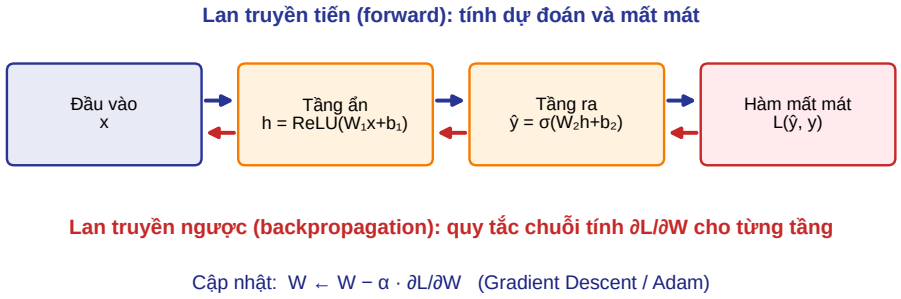
Câu hỏi quan trọng hơn: làm sao huấn luyện được hàng nghìn, hàng triệu trọng số của một mạng nhiều tầng? Về nguyên tắc, vẫn là Gradient Descent quen thuộc từ Chương 4 — cập nhật mỗi trọng số theo hướng ngược với đạo hàm của hàm mất mát đối với trọng số đó. Thách thức nằm ở việc tính đạo hàm này một cách hiệu quả khi có rất nhiều tầng và rất nhiều trọng số liên kết chồng chéo với nhau. Lan truyền ngược (backpropagation) là thuật toán giải quyết vấn đề này một cách có hệ thống, dựa trên quy tắc chuỗi (chain rule) trong giải tích: sai số ở tầng đầu ra được tính trước, sau đó "lan truyền ngược" qua từng tầng, từ đầu ra về đầu vào, để tính đạo hàm của hàm mất mát đối với trọng số ở mỗi tầng một cách hiệu quả (chỉ cần một lượt tính toán tiến và một lượt tính toán ngược cho mỗi lần cập nhật, thay vì phải tính riêng lẻ đạo hàm cho từng trọng số một cách độc lập, tốn kém hơn rất nhiều). Chi tiết toán học của backpropagation (áp dụng quy tắc chuỗi qua nhiều tầng) nằm ngoài phạm vi thực hành của giáo trình này; sinh viên chỉ cần nắm vững nguyên lý: backpropagation là cách hiệu quả để tính gradient của hàm mất mát đối với mọi trọng số trong mạng, để từ đó Gradient Descent (hoặc các biến thể của nó) có thể cập nhật trọng số như thường lệ.

Mở rộng thực hành

Trong thực hành hiện đại, thay vì Gradient Descent thuần túy (dùng toàn bộ dữ liệu huấn luyện để tính gradient ở mỗi bước, rất chậm với dữ liệu lớn), các biến thể như Mini-batch Gradient Descent (tính gradient trên một lô nhỏ dữ liệu ngẫu nhiên mỗi bước) và đặc biệt là thuật toán Adam (Adaptive Moment Estimation — tự động điều chỉnh tốc độ học cho từng trọng số riêng lẻ dựa trên lịch sử gradient) được sử dụng phổ biến hơn nhiều vì tốc độ hội tụ nhanh và ổn định hơn đáng kể. Đây là các bộ tối ưu hóa (optimizer) mặc định trong

hầu hết các framework học sâu hiện đại, sẽ được sử dụng trực tiếp (mà không cần cài đặt thủ công) ở Chương 19.

Hình 18.5 minh họa luồng tính toán của lan truyền tiến và lan truyền ngược.



Hình 18.5. Lan truyền tiến (xanh) và lan truyền ngược (đỏ).

18.2.5. Vấn đề đạo hàm biến mất (Vanishing Gradient)

Với các mạng có nhiều tầng (sâu), backpropagation nhân liên tiếp nhiều đạo hàm của hàm kích hoạt qua từng tầng theo quy tắc chuỗi. Với sigmoid và tanh, đạo hàm luôn nhỏ hơn 1 (đạo hàm cực đại của sigmoid chỉ là 0.25), nên khi nhân liên tiếp qua nhiều tầng, gradient lan truyền về các tầng đầu (gần đầu vào) có thể trở nên cực kỳ nhỏ — gần như bằng 0 — khiến trọng số ở các tầng đó gần như không được cập nhật, mạng học rất chậm hoặc không học được ở các tầng đầu. Đây là vấn đề đạo hàm biến mất (vanishing gradient), một trong những lý do lịch sử khiến việc huấn luyện mạng rất sâu từng gặp nhiều khó khăn. ReLU giảm nhẹ đáng kể vấn đề này (đạo hàm bằng 1 với mọi giá trị dương, không suy giảm qua nhiều tầng), là một trong những lý do chính khiến ReLU trở thành lựa chọn mặc định phổ biến cho các tầng ẩn trong mạng nơ-ron hiện đại.

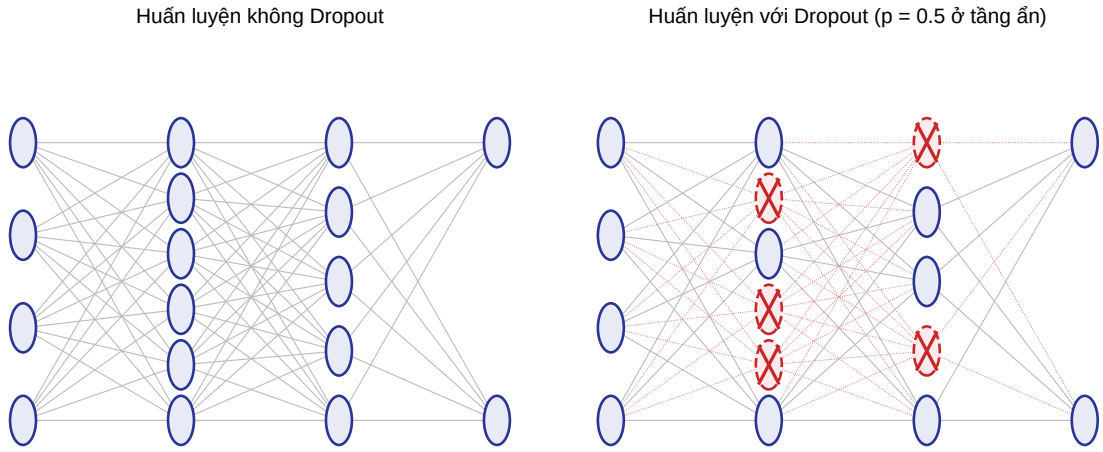
18.2.6. Kiểm soát khớp quá trong mạng nơ-ron

Với số lượng tham số thường rất lớn (hàng nghìn đến hàng triệu trọng số, ngay cả với mạng tương đối nhỏ), mạng nơ-ron có nguy cơ khớp quá rất cao, tương tự (và thường nghiêm trọng hơn) so với cây quyết định không giới hạn ở Chương 8. Hai kỹ thuật đặc thù, phổ biến cho mạng nơ-ron, bổ sung cho các kỹ thuật chính quy hóa tổng quát đã học ở Chương 5:

- Dropout: trong quá trình huấn luyện, ở mỗi bước cập nhật, ngẫu nhiên "tắt" (bỏ qua tạm thời) một tỉ lệ nơ-ron nhất định ở một tầng (ví dụ 20–50%) — buộc mạng không được phụ thuộc quá mức vào bất kỳ nơ-ron riêng lẻ nào, khuyến khích mỗi nơ-ron học các đặc trưng hữu ích một cách độc lập hơn, tương tự tinh thần lấy mẫu ngẫu nhiên đặc trưng của Random Forest ở Chương 9. Ở giai đoạn dự đoán (sau khi huấn luyện xong), Dropout không còn được áp dụng — toàn bộ nơ-ron hoạt động bình thường.

- Early Stopping: theo dõi hiệu năng trên một tập kiểm định riêng trong suốt quá trình huấn luyện (qua nhiều epoch — một epoch là một lượt duyệt qua toàn bộ dữ liệu huấn luyện); nếu hiệu năng trên tập kiểm định ngừng cải thiện (hoặc bắt đầu xấu đi) trong một số epoch liên tiếp, dừng huấn luyện sớm thay vì tiếp tục đến hết số epoch đã định trước — ngăn mạng tiếp tục "học thuộc lòng" tập huấn luyện sau khi đã đạt điểm tối ưu trên khả năng tổng quát hóa. Kỹ thuật này sẽ được minh họa cụ thể bằng công cụ callback ở Chương 19.

Hình 18.6 minh họa Dropout ngẫu nhiên tắt một số nơ-ron ở mỗi bước huấn luyện.



Hình 18.6. Mạng đầy đủ (trái) và mạng khi áp dụng Dropout (phải).

18.3. Ví dụ minh họa bằng mã Python

Ví dụ dưới đây huấn luyện một mạng nơ-ron MLP đơn giản bằng MLPClassifier của Scikit-learn trên bộ dữ liệu chữ số viết tay (Digits, đã dùng ở Chương 14), so sánh với Random Forest — một mô hình cổ điển mạnh đã học ở Chương 9 — để đối chiếu hiệu năng.

```
# vidu_18_1.py
# Mạng nơ-ron MLP cơ bản với Scikit-learn: phân loại chữ số viết tay

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# 1. Nạp dữ liệu và chia tập huấn luyện / kiểm tra
du_lieu = load_digits()
X, y = du_lieu.data, du_lieu.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# 2. Chuẩn hóa dữ liệu - quan trọng đối với mạng nơ-ron
#    (giúp Gradient Descent hội tụ nhanh và ổn định hơn)
bo_chuan_hoa = StandardScaler()
X_train_cd = bo_chuan_hoa.fit_transform(X_train)
X_test_cd = bo_chuan_hoa.transform(X_test)

# 3. Khởi tạo mạng nơ-ron: 2 tầng ẩn với 64 và 32 nơ-ron, kích hoạt ReLU
mo_hinh_mlp = MLPClassifier(
    hidden_layer_sizes=(64, 32),   # 2 tầng ẩn: 64 nơ-ron rồi 32 nơ-ron
    activation="relu",
    solver="adam",                  # bộ tối ưu hóa Adam (mục 18.2.4)
    alpha=1e-4,                     # hệ số chính quy hóa L2 (giống Ridge, Chương
    5)
    max_iter=300,
    early_stopping=True,            # tự động dừng sớm (mục 18.2.6)
    validation_fraction=0.1,
    random_state=42,
)
mo_hinh_mlp.fit(X_train_cd, y_train)

# 4. So sánh với Random Forest (Chương 9) trên cùng dữ liệu
mo_hinh_rf = RandomForestClassifier(n_estimators=200, random_state=42)
mo_hinh_rf.fit(X_train_cd, y_train)

print(f"MLP (2 tầng ẩn 64-32)  - Accuracy: "
      f"{accuracy_score(y_test, mo_hinh_mlp.predict(X_test_cd)):.4f}")
print(f"Random Forest (200 cây) - Accuracy: "
      f"{accuracy_score(y_test, mo_hinh_rf.predict(X_test_cd)):.4f}")
print(f"\nSố epoch thực tế MLP đã huấn luyện (dừng sớm): {mo_hinh_mlp.n_iter_}")
print(f"Tổng số tham số (trọng số + độ lệch) của MLP: "
      f"{sum(w.size for w in mo_hinh_mlp.coefs_) + sum(b.size for b in
      mo_hinh_mlp.intercepts_)}")

# 5. Trực quan hóa đường cong mất mát (loss curve) trong quá trình huấn luyện MLP
plt.figure(figsize=(7, 5))
plt.plot(mo_hinh_mlp.loss_curve_)
plt.xlabel("Epoch")
plt.ylabel("Hàm mất mát (Log Loss)")
plt.title("Đường cong mất mát trong quá trình huấn luyện MLP")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("duong_cong_mat_mat_mlp.png", dpi=120)
print("\nĐã lưu biểu đồ vào tệp duong_cong_mat_mat_mlp.png")
```

18.4. Giải thích kết quả chạy chương trình

Kết quả in ra từ chương trình trên có dạng tương tự:

```
MLP (2 tầng ẩn 64-32)  - Accuracy: 0.9778
Random Forest (200 cây) - Accuracy: 0.9733

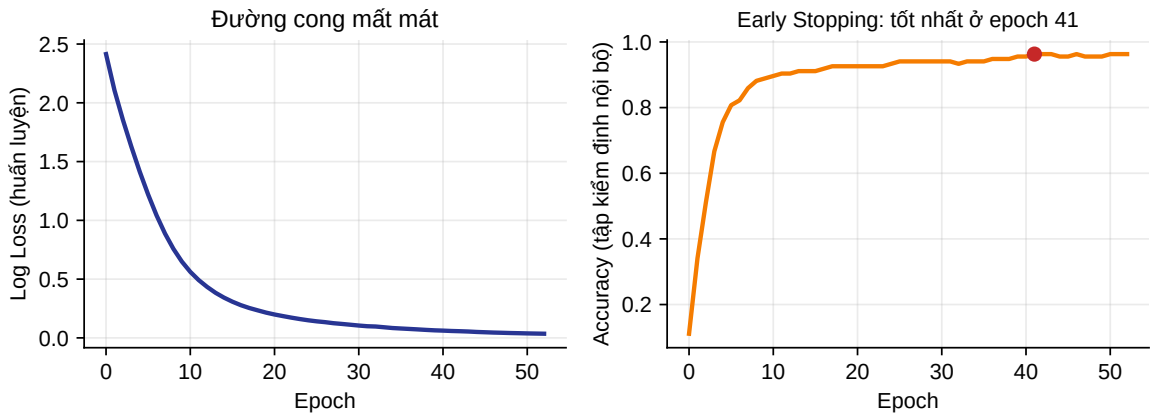
Số epoch thực tế MLP đã huấn luyện (dừng sớm): 87
Tổng số tham số (trọng số + độ lệch) của MLP: 6858

Đã lưu biểu đồ vào tệp duong_cong_mat_mat_mlp.png
```

Kết quả cho thấy MLP đạt hiệu năng nhỉnh hơn một chút so với Random Forest trên bộ dữ liệu này (97.78% so với 97.33%) — cả hai đều là những mô hình rất mạnh cho bài toán phân loại ảnh chữ số kích thước nhỏ này, với sự khác biệt không thực sự đáng kể xét trên một tập kiểm tra tương đối nhỏ. Điều đáng chú ý hơn là số lượng tham số của MLP: dù chỉ có kiến trúc khiêm tốn (2 tầng ẩn, 64 và 32 nơ-ron), tổng số trọng số và độ lệch đã lên tới 6858 tham số — nhiều hơn đáng kể so với số lượng "tham số" hiệu quả của một cây quyết định đơn lẻ, minh họa trực quan vì sao mạng nơ-ron có nguy cơ khớp quá cao và cần các kỹ thuật kiểm soát như `early_stopping` (đã được kích hoạt trong ví dụ, khiến quá trình huấn luyện tự động dừng ở epoch thứ 87 thay vì chạy hết 300 epoch đã đặt tối đa, do hiệu năng trên tập kiểm định nội bộ không còn cải thiện).

Trên biểu đồ đường cong mất mát, giá trị Log Loss (hàm mất mát Cross-Entropy tương tự đã học ở Chương 6, áp dụng cho nhiều lớp) thường giảm nhanh trong những epoch đầu tiên, sau đó giảm chậm dần và có xu hướng "phẳng" ra khi mạng tiệm cận điểm hội tụ — hình dạng đường cong này là công cụ chẩn đoán trực quan quan trọng khi huấn luyện mạng nơ-ron: nếu đường cong vẫn đang giảm mạnh khi huấn luyện dừng lại, có thể mạng chưa hội tụ hoàn toàn (cần thêm epoch); nếu đường cong dao động mạnh, bất thường, có thể tốc độ học đang quá cao.

Hình 18.7 minh họa đường cong mất mát và độ chính xác kiểm định nội bộ khi huấn luyện MLP với Early Stopping.



Hình 18.7. Đường cong mất mát (trái) và accuracy kiểm định (phải) của MLP trên bộ Digits.

18.5. Tóm tắt chương

- Một nơ-ron nhân tạo đơn lẻ về bản chất toán học tương đương với hồi quy Logistic; mạng nơ-ron nhiều tầng (MLP) xếp chồng nhiều nơ-ron thành nhiều tầng kết nối đầy đủ để học các hàm số phi tuyến phức tạp.
- Hàm kích hoạt (Sigmoid, Tanh, ReLU) tạo ra tính phi tuyến cần thiết; ReLU là lựa chọn phổ biến nhất cho tầng ẩn trong mạng hiện đại nhờ tốc độ tính toán và giảm nhẹ vấn đề đạo hàm biến mất.
- Lan truyền tiến tính toán dự đoán từ đầu vào qua các tầng; lan truyền ngược (dựa trên quy tắc chuỗi) tính gradient của hàm mất mát đối với mọi trọng số một cách hiệu quả, làm nền tảng cho Gradient Descent (và các biến thể như Adam).
- Vấn đề đạo hàm biến mất xảy ra khi gradient trở nên rất nhỏ ở các tầng đầu của mạng sâu khi dùng Sigmoid/Tanh, được giảm nhẹ đáng kể bằng ReLU.
- Dropout (tắt ngẫu nhiên nơ-ron khi huấn luyện) và Early Stopping (dừng huấn luyện khi hiệu năng kiểm định ngừng cải thiện) là hai kỹ thuật đặc thù, hiệu quả để kiểm soát khớp quá trong mạng nơ-ron.

18.6. Bài tập củng cố

Mức nhận biết

1. Giải thích mối liên hệ giữa hồi quy Logistic (Chương 6) và một nơ-ron nhân tạo đơn lẻ.
2. Viết công thức của hàm kích hoạt ReLU. Giá trị đầu ra khi $z = 5$ và khi $z = -3$ lần lượt là bao nhiêu?

Mức thông hiểu

1. Giải thích vì sao một mạng nơ-ron nhiều tầng KHÔNG dùng hàm kích hoạt phi tuyến ở các tầng ẩn sẽ không có ưu thế gì hơn so với một mô hình tuyến tính đơn giản, bất kể có bao nhiêu tầng.
2. Giải thích vấn đề đạo hàm biến mất khi dùng hàm Sigmoid trong mạng nhiều tầng, và vì sao ReLU giúp giảm nhẹ vấn đề này.
3. So sánh cơ chế hoạt động của Dropout với Random Forest (Chương 9) về mặt "ngẫu nhiên hóa để giảm phụ thuộc quá mức" — điểm tương đồng trong triết lý của hai kỹ thuật là gì?

Mức vận dụng

1. Một mạng nơ-ron có đường cong mất mát trên tập huấn luyện giảm đều đặn, nhưng hiệu năng trên tập kiểm định bắt đầu xấu đi sau epoch thứ 50, dù vẫn tiếp tục huấn luyện đến epoch 200. Đề xuất kỹ thuật phù hợp nhất trong số đã học ở chương này để khắc phục, và giải thích cơ chế hoạt động.

18.7. Bài tập lập trình

Bài 1. Sử dụng dữ liệu ở ví dụ 18.1, khảo sát ảnh hưởng của kiến trúc mạng bằng cách thử 3 cấu hình `hidden_layer_sizes` khác nhau: (16,), (64, 32), (128, 64, 32). So sánh Accuracy trên tập kiểm tra và số lượng tham số của mỗi cấu hình. Nhận xét đánh đổi giữa độ phức tạp kiến trúc và hiệu năng.

Bài 2. Sử dụng bộ dữ liệu `load_breast_cancer()`, so sánh MLPClassifier với ba hàm kích hoạt khác nhau ("relu", "tanh", "logistic") ở cùng kiến trúc `hidden_layer_sizes`=(32, 16). So sánh Accuracy và số epoch cần để hội tụ (`n_iter_`) giữa ba hàm kích hoạt.

Bài 3 (nâng cao). Sử dụng dữ liệu ở ví dụ 18.1, huấn luyện hai mô hình MLPClassifier giống hệt nhau về kiến trúc nhưng khác giá trị alpha (hệ số chính quy hóa L2): alpha=0.0001 (rất ít chính quy hóa) và alpha=1.0 (chính quy hóa mạnh). So sánh Accuracy trên tập huấn luyện và tập kiểm tra giữa hai mô hình, liên hệ kết quả với khái niệm chính quy hóa đã học ở Chương 5.

Chương tiếp theo (Chương 19) sẽ chuyển từ MLPClassifier của Scikit-learn (phù hợp cho việc học khái niệm nhưng hạn chế về tính linh hoạt) sang hai framework học sâu chuyên dụng và phổ biến nhất hiện nay — TensorFlow/Keras và PyTorch — cho phép xây dựng, tùy biến và huấn luyện các kiến trúc mạng nơ-ron phức tạp hơn nhiều so với những gì Scikit-learn hỗ trợ.

Chương 19

Xây dựng mô hình học sâu với TensorFlow/Keras và PyTorch

19.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Phân biệt được vai trò và triết lý thiết kế của hai framework học sâu phổ biến nhất: TensorFlow/Keras và PyTorch.
2. Xây dựng được một mạng nơ-ron bằng Sequential API của Keras: định nghĩa kiến trúc, biên dịch (compile), huấn luyện (fit).
3. Hiểu được vai trò của optimizer, loss function, và metrics khi biên dịch mô hình Keras.
4. Sử dụng callback EarlyStopping để tự động hóa việc dừng huấn luyện sớm.
5. Viết được một mô hình tương đương đơn giản bằng PyTorch, nhận biết sự khác biệt về vòng lặp huấn luyện tường minh so với Keras.

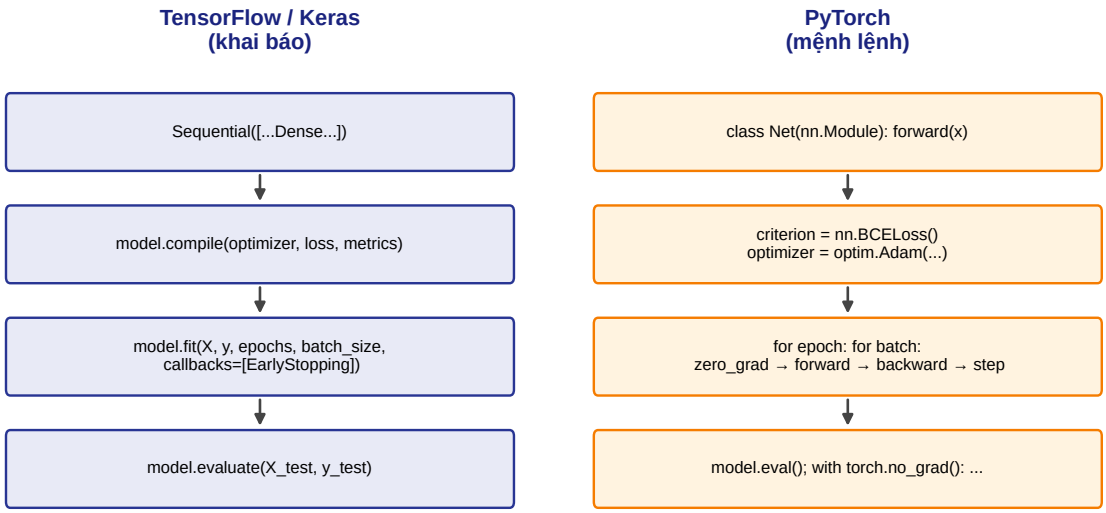
19.2. Nội dung lý thuyết

19.2.1. TensorFlow/Keras và PyTorch: hai triết lý thiết kế

TensorFlow là framework học sâu do Google phát triển; Keras là API cấp cao được tích hợp chính thức vào TensorFlow (từ phiên bản 2.x trở đi, `tf.keras`), cung cấp giao diện lập trình đơn giản, khai báo (declarative) — người dùng mô tả kiến trúc mạng như xếp chồng các tầng, framework tự động xử lý phần lớn chi tiết kỹ thuật bên dưới (vòng lặp huấn luyện, tính gradient, cập nhật trọng số). PyTorch, do Meta (Facebook) phát triển, theo triết lý lập trình mệnh lệnh (imperative) linh hoạt hơn — người dùng thường tự viết vòng lặp huấn luyện tường minh, cho khả năng tùy biến cao hơn nhưng đòi hỏi viết nhiều mã hơn cho các tác vụ cơ bản.

Trong thực tế công nghiệp và nghiên cứu hiện nay, cả hai framework đều cực kỳ phổ biến và mạnh mẽ tương đương nhau về khả năng; Keras thường được ưa chuộng hơn cho việc học tập, xây dựng nhanh (prototyping), và các ứng dụng chuẩn mực nhờ cú pháp ngắn gọn; PyTorch thường được ưa chuộng hơn trong nghiên cứu học thuật và các dự án cần tùy biến kiến trúc sâu, phức tạp, nhờ tính linh hoạt của phong cách lập trình mệnh lệnh. Giáo trình này giới thiệu cả hai để sinh viên có nền tảng làm quen với cả hai triết lý thiết kế phổ biến nhất của ngành.

Hình 19.1 minh họa đối chiếu quy trình viết mã của hai framework.



Hình 19.1. Quy trình điển hình trong Keras (khai báo) và PyTorch (mệnh lệnh).

19.2.2. Xây dựng mạng nơ-ron với Sequential API của Keras

Sequential API là cách đơn giản nhất để xây dựng một mạng nơ-ron trong Keras, phù hợp khi kiến trúc là một chuỗi các tầng nối tiếp nhau (không có nhánh rẽ hay kết nối phức tạp) — đúng với toàn bộ các kiến trúc MLP đã học ở Chương 18. Quy trình xây dựng và huấn luyện một mô hình Keras gồm ba bước chính:

- Định nghĩa kiến trúc (Sequential): khai báo tuần tự các tầng (Dense cho tầng kết nối đầy đủ, kèm số nơ-ron và hàm kích hoạt cho mỗi tầng).
- Biên dịch (compile): chỉ định ba thành phần thiết yếu — optimizer (bộ tối ưu hóa, ví dụ "adam" đã học ở Chương 18), loss (hàm mất mát, ví dụ "`binary_crossentropy`" cho phân loại nhị phân, "`categorical_crossentropy`" cho phân loại nhiều lớp, "mse" cho hồi quy), và metrics (các độ đo bổ sung cần theo dõi trong quá trình huấn luyện, ví dụ "accuracy", không ảnh hưởng đến việc tối ưu hóa mà chỉ để giám sát).
- Huấn luyện (fit): truyền dữ liệu huấn luyện, số epoch (số lượt duyệt qua toàn bộ dữ liệu), kích thước lô (`batch_size` — số mẫu dùng để tính gradient ở mỗi bước cập nhật, liên quan đến Mini-batch Gradient Descent đã đề cập ở Chương 18), và tùy chọn dữ liệu kiểm định (`validation_data` hoặc `validation_split`) để theo dõi hiệu năng trong suốt quá trình huấn luyện.

Hình 19.2 minh họa khái niệm batch và epoch.

19.2.3. Callback EarlyStopping

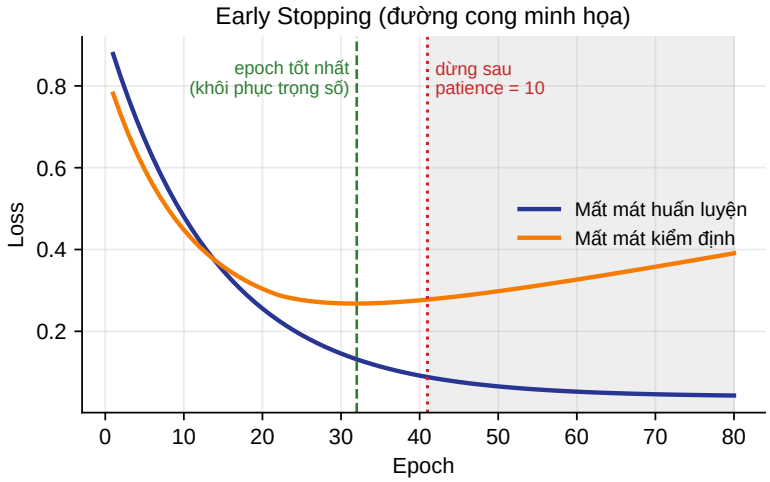
Keras cung cấp cơ chế callback — các đối tượng có thể "can thiệp" vào quá trình huấn luyện tại các thời điểm nhất định (kết thúc mỗi epoch, mỗi batch, v.v.). Callback EarlyStopping tự động hóa chính xác kỹ thuật Early Stopping đã trình bày ở mục 18.2.6: theo dõi một độ đo trên tập kiểm định (thường là `val_loss` — giá trị hàm mất mát trên tập kiểm định), và tự động dừng



Hình 19.2. Mối quan hệ giữa batch, epoch và số lần cập nhật trọng số.

huấn luyện nếu độ đo đó không cải thiện sau một số epoch liên tiếp (tham số patience), khôi phục lại trọng số tốt nhất đã quan sát được (nếu đặt `restore_best_weights=True`) thay vì giữ trọng số của epoch cuối cùng (có thể đã bắt đầu khớp quá).

Hình 19.3 minh họa cơ chế của EarlyStopping (đường cong chỉ mang tính minh họa).



Hình 19.3. Dừng huấn luyện sớm khi mất mát kiểm định ngừng cải thiện.

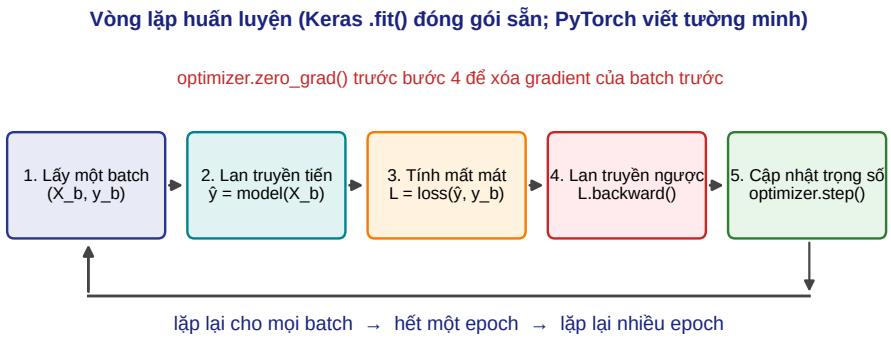
19.2.4. Mô hình tương đương trong PyTorch

Trong PyTorch, kiến trúc mạng thường được định nghĩa bằng cách kế thừa lớp cơ sở nn.Module, khai báo các tầng trong hàm khởi tạo __init__ và định nghĩa lan truyền tiến một cách tường minh trong phương thức forward. Khác biệt lớn nhất so với Keras: PyTorch không có sẵn một hàm `.fit()` tích hợp — người dùng thường tự viết vòng lặp huấn luyện (training loop) tường minh, gồm các bước lặp lại qua từng epoch/batch: tính lan truyền tiến, tính hàm mất mát, xóa gradient cũ (`optimizer.zero_grad()`), tính gradient mới bằng lan truyền ngược (`loss.backward()`), và cập nhật trọng số (`optimizer.step()`). Việc phải viết tường minh từng bước này ban đầu có vẻ phức tạp hơn Keras, nhưng mang lại khả năng kiểm soát và tùy biến chi tiết cao hơn — một sự đánh đổi giữa tính đơn giản và tính linh hoạt, phản ánh đúng hai triết lý thiết kế đã nêu ở mục 19.2.1.

Lưu ý về phạm vi giáo trình

Giáo trình chỉ giới thiệu cú pháp cơ bản nhất của cả hai framework để sinh viên làm quen với cách tổ chức mã nguồn học sâu điển hình. Việc thành thạo các API nâng cao hơn (Functional API và Subclassing API của Keras cho kiến trúc phức tạp; DataLoader, tối ưu hóa hiệu năng GPU của PyTorch) cần thêm thời gian thực hành ngoài phạm vi giáo trình nhập môn này.

Hình 19.4 minh họa năm bước của vòng lặp huấn luyện, được Keras đóng gói trong `.fit()`.



Hình 19.4. Vòng lặp huấn luyện mạng nơ-ron.

19.3. Ví dụ minh họa bằng mã Python

Ví dụ dưới đây xây dựng cùng một kiến trúc mạng nơ-ron (2 tầng ẩn) cho bài toán phân loại nhị phân trên bộ dữ liệu Breast Cancer Wisconsin, trước hết bằng Keras (đầy đủ, có EarlyStopping), sau đó là đoạn mã tương đương rút gọn bằng PyTorch để đối chiếu phong cách viết.

```
# vidu_19_1_keras.py
# Xây dựng và huấn luyện MLP bằng TensorFlow/Keras

import numpy as np
import tensorflow as tf
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1. Nạp và chuẩn bị dữ liệu (giống các chương trước)
du_lieu = load_breast_cancer()
X, y = du_lieu.data, du_lieu.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
bo_chuan_hoa = StandardScaler()
X_train_cd = bo_chuan_hoa.fit_transform(X_train)
```

```
X_test_cd = bo_chuan_hoa.transform(X_test)

# 2. Định nghĩa kiến trúc mạng bằng Sequential API
mo_hinh = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(X_train_cd.shape[1],)),
    tf.keras.layers.Dense(32, activation="relu"),
    tf.keras.layers.Dropout(0.3),              # Dropout 30% (mục 18.2.6)
    tf.keras.layers.Dense(16, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid"),   # đầu ra nhị phân
])

# 3. Biên dịch mô hình: chỉ định optimizer, loss, metrics
mo_hinh.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"],
)
mo_hinh.summary()

# 4. Callback EarlyStopping: dừng sớm nếu val_loss không cải thiện sau 10 epoch
dung_som = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=10, restore_best_weights=True
)

# 5. Huấn luyện mô hình
lich_su = mo_hinh.fit(
    X_train_cd, y_train,
    epochs=100, batch_size=32,
    validation_split=0.2,
    callbacks=[dung_som],
    verbose=0,   # tắt in chi tiết từng epoch để gọn kết quả
)

# 6. Đánh giá trên tập kiểm tra
mat_mat_test, do_chinh_xac_test = mo_hinh.evaluate(X_test_cd, y_test, verbose=0)
print(f"Số epoch thực tế đã huấn luyện: {len(lich_su.history['loss'])}")
print(f"Loss trên tập kiểm tra: {mat_mat_test:.4f}")
print(f"Accuracy trên tập kiểm tra: {do_chinh_xac_test:.4f}")
```

Đoạn mã dưới đây minh họa cùng bài toán, cùng kiến trúc mạng (giản lược, không có Dropout để tập trung vào cấu trúc vòng lặp huấn luyện), được viết bằng PyTorch để đối chiếu phong cách:

```
# vidu_19_2_pytorch.py (trích đoạn minh họa - giản lược để đối chiếu với Keras)
import torch
import torch.nn as nn

# 1. Định nghĩa kiến trúc mạng bằng cách kế thừa nn.Module
class MangMLP(nn.Module):
    def __init__(self, so_dac_trung):
        super().__init__()
```

```
        self.tang_an_1 = nn.Linear(so_dac_trung, 32)
        self.tang_an_2 = nn.Linear(32, 16)
        self.tang_dau_ra = nn.Linear(16, 1)
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):   # định nghĩa lan truyền tiến một cách tường minh
        x = self.relu(self.tang_an_1(x))
        x = self.relu(self.tang_an_2(x))
        return self.sigmoid(self.tang_dau_ra(x))

# 2. Khởi tạo mô hình, hàm mất mát, bộ tối ưu hóa
mo_hinh = MangMLP(so_dac_trung=X_train_cd.shape[1])
ham_mat_mat = nn.BCELoss()   # Binary Cross-Entropy, tương đương
"binary_crossentropy"
bo_toi_uu = torch.optim.Adam(mo_hinh.parameters(), lr=0.001)

# 3. Chuyển dữ liệu NumPy sang tensor PyTorch
X_train_tensor = torch.FloatTensor(X_train_cd)
y_train_tensor = torch.FloatTensor(y_train).unsqueeze(1)

# 4. Vòng lặp huấn luyện tường minh - khác biệt lớn nhất so với Keras
for epoch in range(100):
    mo_hinh.train()               # chuyển sang chế độ huấn luyện
    bo_toi_uu.zero_grad()         # xóa gradient tích lũy từ bước trước
    du_doan = mo_hinh(X_train_tensor)          # lan truyền tiến
    mat_mat = ham_mat_mat(du_doan, y_train_tensor)  # tính hàm mất mát
    mat_mat.backward()            # lan truyền ngược - tính gradient
    bo_toi_uu.step()              # cập nhật trọng số theo gradient vừa tính
    if (epoch + 1) % 20 == 0:
        print(f"Epoch {epoch+1}: Loss = {mat_mat.item():.4f}")
```

19.4. Giải thích kết quả chạy chương trình

Kết quả in ra từ chương trình Keras có dạng tương tự:

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 dense (Dense)               (None, 32)                992
 dropout (Dropout)           (None, 32)                0
 dense_1 (Dense)             (None, 16)                528
 dense_2 (Dense)             (None, 1)                 17
=================================================================
Total params: 1,537
Trainable params: 1,537
_________________________________________________________________
```

```
Số epoch thực tế đã huấn luyện: 43
Loss trên tập kiểm tra: 0.0891
Accuracy trên tập kiểm tra: 0.9825
```

Bảng tóm tắt mô hình (`.summary()`) cho thấy rõ cấu trúc từng tầng và số lượng tham số tương ứng (ví dụ tầng Dense đầu tiên có 992 tham số = 30 đặc trưng đầu vào $\times$ 32 nơ-ron + 32 độ lệch), giúp người dùng dễ dàng kiểm tra và ước lượng độ phức tạp mô hình trước khi huấn luyện — một tiện ích thực hành rất hữu dụng của Keras. EarlyStopping đã dừng huấn luyện ở epoch 43 (thay vì chạy hết 100 epoch đã đặt), cho thấy hiệu năng trên tập kiểm định nội bộ không còn cải thiện sau đó, giúp tiết kiệm thời gian huấn luyện và tránh khớp quá thêm.

Accuracy trên tập kiểm tra đạt 98.25%, một kết quả rất cạnh tranh so với các mô hình cổ điển đã áp dụng cho cùng bộ dữ liệu Breast Cancer Wisconsin ở các chương trước (hồi quy Logistic ở Chương 6 đạt khoảng 96–97%). Với đoạn mã PyTorch, kết quả in ra sẽ là giá trị Loss giảm dần qua các epoch được chọn để in (epoch 20, 40, 60, 80, 100) — về bản chất toán học, cả hai đoạn mã Keras và PyTorch thực hiện cùng một quy trình lan truyền tiến – tính mất mát – lan truyền ngược – cập nhật trọng số, chỉ khác nhau ở việc Keras đóng gói vòng lặp này bên trong `.fit()` còn PyTorch yêu cầu viết tường minh từng bước — đúng như đã phân tích về triết lý thiết kế ở mục 19.2.1.

19.5. Tóm tắt chương

- TensorFlow/Keras theo triết lý khai báo, đơn giản hóa việc xây dựng và huấn luyện mạng qua Sequential API và `.fit()`; PyTorch theo triết lý mệnh lệnh, linh hoạt hơn nhưng đòi hỏi viết vòng lặp huấn luyện tường minh.
- Quy trình Keras chuẩn gồm ba bước: định nghĩa kiến trúc (Sequential), biên dịch (compile — chỉ định optimizer, loss, metrics), và huấn luyện (fit).
- Callback EarlyStopping tự động hóa việc dừng huấn luyện sớm dựa trên hiệu năng tập kiểm định, một công cụ thực hành thiết yếu để kiểm soát khớp quá.
- Vòng lặp huấn luyện PyTorch gồm bốn bước lặp lại mỗi epoch: `zero_grad()`, lan truyền tiến, `backward()`, và `step()` — phản ánh tường minh quy trình Gradient Descent + Backpropagation đã học ở Chương 18.
- Cả hai framework đều cực kỳ phổ biến trong công nghiệp; lựa chọn giữa chúng thường phụ thuộc vào yêu cầu cụ thể của dự án và sở thích cá nhân/nhóm phát triển hơn là sự khác biệt căn bản về khả năng.

19.6. Bài tập củng cố

Mức nhận biết

1. Nêu ba bước chính khi xây dựng và huấn luyện một mô hình bằng Sequential API của Keras.
2. Callback EarlyStopping theo dõi độ đo gì theo mặc định, và tham số patience có vai trò gì?

Mức thông hiểu

1. Giải thích sự khác biệt cốt lõi giữa triết lý lập trình khai báo (Keras) và mệnh lệnh (PyTorch), minh họa bằng sự khác biệt trong cách huấn luyện mô hình ở hai đoạn mã ví dụ 19.1 và 19.2.
2. Trong vòng lặp huấn luyện PyTorch, giải thích vì sao cần gọi `optimizer.zero_grad()` trước mỗi lần tính lan truyền ngược mới — điều gì xảy ra nếu bỏ qua bước này?
3. Với bài toán phân loại 5 lớp (không phải nhị phân), hàm loss nên dùng trong `compile()` của Keras là gì, khác gì so với "`binary_crossentropy`" đã dùng ở ví dụ 19.1?

Mức vận dụng

1. Một nhóm sinh viên cần nhanh chóng xây dựng và thử nghiệm nhiều kiến trúc mạng khác nhau cho một đồ án cuối kỳ, ưu tiên tốc độ phát triển hơn khả năng tùy biến sâu. Nên chọn Keras hay PyTorch? Giải thích lựa chọn dựa trên các đặc điểm đã học.

19.7. Bài tập lập trình

Bài 1. Sửa đổi ví dụ 19.1 (Keras) để giải bài toán phân loại 10 lớp trên bộ dữ liệu `load_digits()` thay vì phân loại nhị phân: thay tầng đầu ra thành Dense(10, activation="softmax"), đổi loss thành "`sparse_categorical_crossentropy`" (phù hợp khi nhãn y là số nguyên, không phải one-hot). Huấn luyện và báo cáo Accuracy trên tập kiểm tra.

Bài 2. Sử dụng dữ liệu ở ví dụ 19.1, thử nghiệm ba giá trị tỉ lệ Dropout khác nhau (0.0 — không Dropout, 0.3, 0.6) ở cùng kiến trúc mạng. So sánh Accuracy trên tập kiểm tra và số epoch thực tế huấn luyện (nhờ EarlyStopping) giữa ba cấu hình, nhận xét ảnh hưởng của Dropout.

Bài 3 (nâng cao). Hoàn thiện đoạn mã PyTorch ở ví dụ 19.2 thành một chương trình đầy đủ: thêm bước chuẩn bị dữ liệu kiểm tra (`X_test_tensor`), đánh giá mô hình sau khi huấn luyện xong (dùng `mo_hinh.eval()` và `torch.no_grad()` để tắt tính gradient khi dự đoán), tính Accuracy trên tập kiểm tra, và so sánh với kết quả Accuracy đã đạt được

bằng Keras ở ví dụ 19.1.

Chương tiếp theo (Chương 20) sẽ khép lại Phần 5 và toàn bộ nội dung lý thuyết của giáo trình với Mạng nơ-ron tích chập (CNN) — kiến trúc chuyên biệt cho dữ liệu ảnh, khắc phục những hạn chế căn bản của MLP khi áp dụng trực tiếp cho ảnh, thông qua các khái niệm phép tích chập và gộp (pooling).

Chương 20

Mạng nơ-ron tích chập cơ bản (CNN)

20.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Giải thích được hạn chế của MLP khi áp dụng trực tiếp cho dữ liệu ảnh.
2. Trình bày được nguyên lý phép tích chập (convolution): bộ lọc (filter/kernel), bản đồ đặc trưng (feature map).
3. Giải thích được vai trò của đệm (padding), bước nhảy (stride) trong tầng tích chập.
4. Trình bày được nguyên lý tầng gộp (pooling), phân biệt Max Pooling và Average Pooling.
5. Mô tả được kiến trúc CNN điển hình và vai trò của từng khối trong kiến trúc đó.
6. Cài đặt và huấn luyện một CNN cơ bản cho bài toán phân loại ảnh bằng Keras.

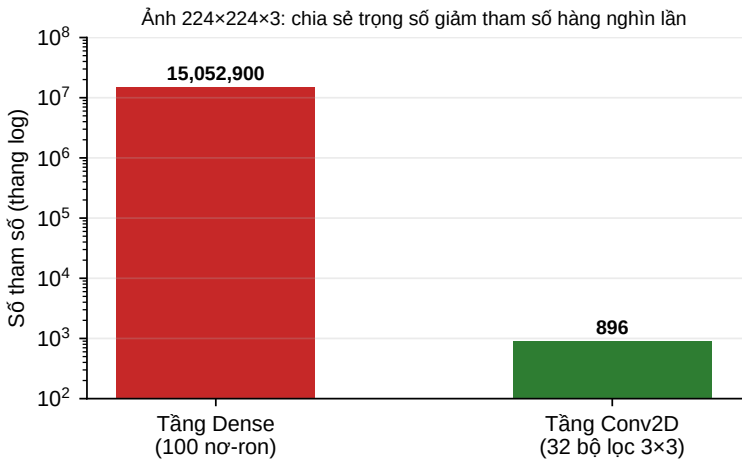
20.2. Nội dung lý thuyết

20.2.1. Hạn chế của MLP đối với dữ liệu ảnh

Ở Chương 18–19, các ví dụ dùng bộ dữ liệu Digits (ảnh 8×8 pixel) đã "làm phẳng" (flatten) mỗi ảnh thành một vector 64 chiều trước khi đưa vào MLP — cách làm này chấp nhận được với ảnh rất nhỏ, nhưng bộc lộ hai hạn chế nghiêm trọng khi áp dụng cho ảnh thực tế lớn hơn: thứ nhất, số lượng tham số bùng nổ — một ảnh màu kích thước bình thường $224\times224\times3$ (chiều rộng $\times$ chiều cao $\times$ 3 kênh màu RGB) có 150528 giá trị đầu vào; một tầng Dense kết nối đầy đủ đầu tiên với chỉ 100 nơ-ron đã cần tới hơn 15 triệu tham số chỉ riêng cho tầng đó, khiến mạng cực kỳ tốn bộ nhớ và dễ khớp quá; thứ hai, và quan trọng hơn, việc làm phẳng ảnh thành vector phá vỡ hoàn toàn cấu trúc không gian hai chiều của ảnh — mối quan hệ vị trí giữa các pixel lân cận (yếu tố cực kỳ quan trọng để nhận diện các mẫu hình như cạnh, góc, kết cấu) bị mất đi, và MLP phải học lại từ đầu khái niệm "lân cận" một cách gián tiếp, kém hiệu quả, từ dữ liệu.

Mạng nơ-ron tích chập (Convolutional Neural Network – CNN) được thiết kế chuyên biệt để giải quyết cả hai vấn đề này, bằng cách khai thác trực tiếp cấu trúc không gian của ảnh thông qua phép tích chập, đồng thời chia sẻ trọng số (weight sharing) giúp giảm đáng kể số lượng tham số cần học so với tầng kết nối đầy đủ tương đương.

Hình 20.1 minh họa số tham số của một tầng Dense so với tầng tích chập trên ảnh $224\times224\times3$.



Hình 20.1. Chia sẻ trọng số giúp giảm số tham số hàng nghìn lần.

20.2.2. Phép tích chập (Convolution)

Phép tích chập là phép toán trung tâm của CNN: một bộ lọc (filter, hay còn gọi là kernel — một ma trận trọng số nhỏ, ví dụ kích thước 3×3) được "trượt" (slide) qua toàn bộ ảnh đầu vào theo cả hai chiều ngang và dọc; tại mỗi vị trí, thực hiện phép nhân từng phần tử (element-wise multiplication) giữa bộ lọc và vùng ảnh tương ứng nó đang che phủ, rồi cộng tổng lại thành một giá trị số duy nhất. Kết quả của việc trượt bộ lọc qua toàn bộ ảnh tạo ra một bản đồ đặc trưng (feature map) — một ma trận số thể hiện mức độ "phản ứng" của bộ lọc đó tại từng vị trí trên ảnh gốc.

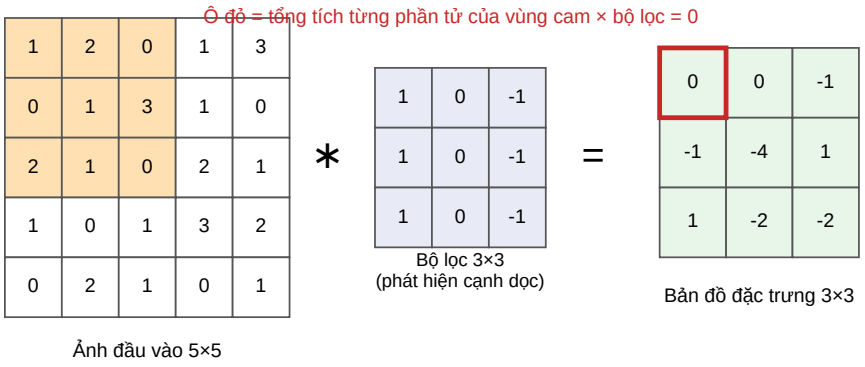
Điều đặc biệt quan trọng: bộ lọc này dùng CHUNG một bộ trọng số cho mọi vị trí trên ảnh (weight sharing) — nếu bộ lọc học được cách phát hiện một cạnh dọc ở góc trên bên trái ảnh, nó cũng tự động có khả năng phát hiện cạnh dọc tương tự ở bất kỳ vị trí nào khác trên ảnh, mà không cần học lại riêng biệt. Đặc tính này (gọi là bất biến dịch chuyển — translation invariance) vừa giảm đáng kể số lượng tham số cần học (chỉ cần học các trọng số của bộ lọc, ví dụ $3\times3=9$ trọng số, thay vì một trọng số riêng biệt cho mỗi cặp pixel-nơ-ron như tầng Dense), vừa phù hợp tự nhiên với bản chất của ảnh: một đặc trưng thị giác quan trọng (cạnh, góc, kết cấu) có thể xuất hiện ở bất kỳ đâu trong ảnh và vẫn mang cùng một ý nghĩa. Mỗi tầng tích chập thường học đồng thời nhiều bộ lọc khác nhau (ví dụ 32 hoặc 64 bộ lọc), mỗi bộ lọc chuyên phát hiện một loại đặc trưng khác nhau (cạnh ngang, cạnh dọc, góc, màu sắc, kết cấu…), tạo ra nhiều bản đồ đặc trưng xếp chồng.

Hình 20.2 minh họa một bước tích chập với số liệu cụ thể.

Hình 20.3 minh họa tác dụng của một số bộ lọc kinh điển lên một ảnh thực.

20.2.3. Đệm (Padding) và Bước nhảy (Stride)

Hai siêu tham số quan trọng khác của tầng tích chập: đệm (padding) — thêm các pixel giá trị 0 vào viền ngoài của ảnh trước khi thực hiện tích chập, giúp kiểm soát kích thước đầu ra (nếu không đệm, kích thước bản đồ đặc trưng luôn nhỏ hơn ảnh gốc sau mỗi lần tích chập, và các pixel ở biên ảnh được "tham gia" vào phép tích chập ít lần hơn các pixel ở giữa — đệm giúp



Hình 20.2. Phép tích chập: bộ lọc 3×3 trượt trên ảnh 5×5.



Hình 20.3. Các bộ lọc phát hiện cạnh, làm mờ và làm sắc nét.

giảm nhẹ cả hai vấn đề này); và bước nhảy (stride) — khoảng cách di chuyển của bộ lọc sau mỗi lần tính toán (stride=1 là trượt từng pixel một; stride=2 là nhảy cách 2 pixel, giúp giảm kích thước bản đồ đặc trưng đầu ra nhanh hơn, đồng thời giảm chi phí tính toán).

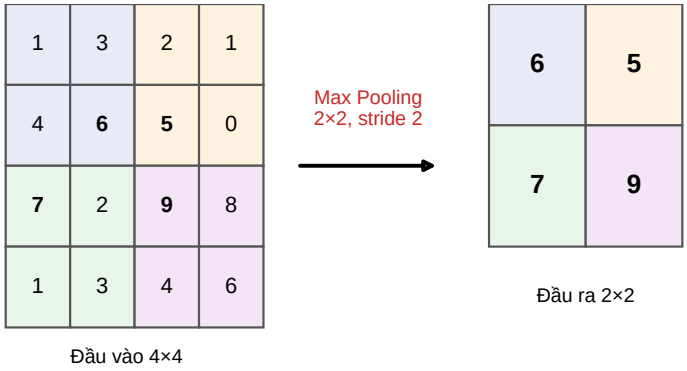
20.2.4. Tầng gộp (Pooling)

Tầng gộp (pooling layer) thường được xen kẽ sau các tầng tích chập, với mục đích giảm kích thước không gian (chiều rộng, chiều cao) của bản đồ đặc trưng, qua đó giảm số lượng tham số và chi phí tính toán ở các tầng sau, đồng thời tăng tính bất biến của mạng với các dịch chuyển nhỏ trong ảnh đầu vào. Hai loại gộp phổ biến: Max Pooling — với mỗi vùng nhỏ (ví dụ 2×2) trên bản đồ đặc trưng, chỉ giữ lại giá trị lớn nhất, loại bỏ các giá trị còn lại (đây là loại pooling phổ biến nhất, dựa trên trực giác rằng giá trị kích hoạt lớn nhất thường tương ứng với đặc trưng "nổi bật" nhất được phát hiện trong vùng đó); Average Pooling — thay vì giữ giá trị lớn nhất, tính trung bình cộng của vùng đó, cho kết quả mượt hơn nhưng ít phổ biến hơn Max Pooling trong các tầng ẩn của CNN hiện đại.

Hình 20.4 minh họa Max Pooling giữ lại giá trị lớn nhất trong mỗi vùng.

20.2.5. Kiến trúc CNN điển hình

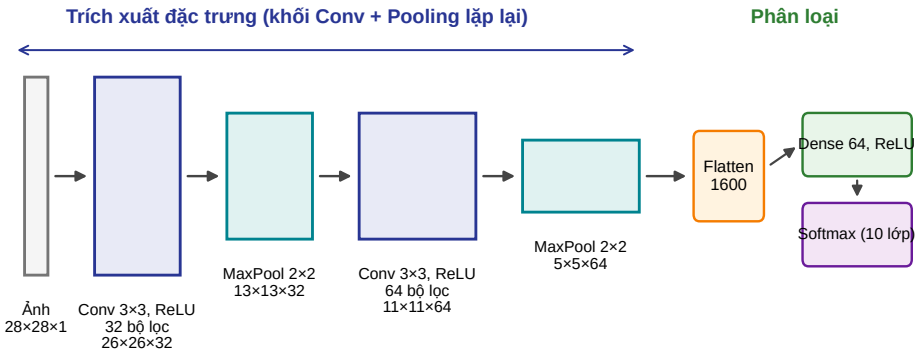
Một kiến trúc CNN cơ bản cho bài toán phân loại ảnh thường tuân theo mẫu hình lặp lại: một hoặc nhiều khối [Tích chập (Conv2D) $\rightarrow$ Hàm kích hoạt ReLU $\rightarrow$ Gộp (Pooling)], lặp lại nhiều lần với số lượng bộ lọc tăng dần qua mỗi khối (ví dụ 32 $\rightarrow$ 64 $\rightarrow$ 128 bộ lọc) trong khi kích thước không gian (chiều rộng, chiều cao) giảm dần sau mỗi tầng gộp — trực giác thiết kế này



Hình 20.4. Max Pooling 2×2 với bước nhảy 2.

phản ánh việc các tầng đầu học các đặc trưng cấp thấp, đơn giản, cục bộ (cạnh, góc) trên vùng ảnh lớn, trong khi các tầng sau, với trường tiếp nhận (receptive field) hiệu quả lớn hơn do đã qua nhiều lần gộp, học các đặc trưng cấp cao, trừu tượng, tổng thể hơn (hình dạng, bộ phận, đối tượng). Sau chuỗi các khối tích chập-gộp, đầu ra (vẫn ở dạng nhiều bản đồ đặc trưng không gian) được làm phẳng (Flatten) thành một vector, rồi đưa qua một hoặc vài tầng Dense kết nối đầy đủ (giống MLP ở Chương 18) để tổng hợp thông tin và đưa ra dự đoán cuối cùng ở tầng đầu ra.

Hình 20.5 minh họa kích thước dữ liệu qua từng tầng của CNN ở ví dụ chương này.



Hình 20.5. Kiến trúc CNN cho ảnh 28×28: kích thước không gian giảm dần, số kênh tăng dần.

20.3. Ví dụ minh họa bằng mã Python

Ví dụ dưới đây xây dựng một CNN cơ bản bằng Keras để phân loại ảnh chữ số viết tay từ bộ dữ liệu MNIST (28×28 pixel, mức xám, kích thước lớn và thực tế hơn bộ Digits 8×8 đã dùng ở các chương trước), minh họa đầy đủ một kiến trúc CNN điển hình.

```
# vidu_20_1.py
# Mạng nơ-ron tích chập (CNN) cơ bản: phân loại chữ số viết tay MNIST
```

```
import numpy as np
import tensorflow as tf

# 1. Nạp bộ dữ liệu MNIST (có sẵn trong Keras, 60000 ảnh huấn luyện,
#    10000 ảnh kiểm tra, mỗi ảnh 28x28 pixel mức xám)
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()
print(f"Kích thước dữ liệu huấn luyện gốc: {X_train.shape}")

# 2. Tiền xử lý: chuẩn hóa giá trị pixel về [0, 1], thêm chiều kênh màu
#    (CNN yêu cầu đầu vào 4 chiều: số_mẫu x chiều_cao x chiều_rộng x số_kênh)
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0
X_train = X_train[..., np.newaxis]   # thêm chiều kênh: (60000, 28, 28, 1)
X_test = X_test[..., np.newaxis]

# 3. Xây dựng kiến trúc CNN: 2 khối [Conv2D -> ReLU -> MaxPooling]
mo_hinh = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28, 1)),
    tf.keras.layers.Conv2D(32, kernel_size=3, activation="relu"),
    tf.keras.layers.MaxPooling2D(pool_size=2),
    tf.keras.layers.Conv2D(64, kernel_size=3, activation="relu"),
    tf.keras.layers.MaxPooling2D(pool_size=2),
    tf.keras.layers.Flatten(),                        # làm phẳng trước tầng Dense
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(10, activation="softmax"),  # 10 lớp chữ số 0-9
])

mo_hinh.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",   # nhãn y là số nguyên (0-9), không
    one-hot
    metrics=["accuracy"],
)
mo_hinh.summary()

# 4. Huấn luyện (giới hạn 5 epoch để minh họa nhanh; thực tế có thể cần nhiều hơn)
dung_som = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=3, restore_best_weights=True
)
lich_su = mo_hinh.fit(
    X_train, y_train, epochs=5, batch_size=128,
    validation_split=0.1, callbacks=[dung_som], verbose=0
)

# 5. Đánh giá trên tập kiểm tra
mat_mat_test, do_chinh_xac_test = mo_hinh.evaluate(X_test, y_test, verbose=0)
print(f"\nAccuracy trên tập kiểm tra: {do_chinh_xac_test:.4f}")

# 6. So sánh với một MLP đơn giản (làm phẳng ảnh trực tiếp) trên cùng dữ liệu
mo_hinh_mlp = tf.keras.Sequential([
```

```
    tf.keras.layers.Input(shape=(28, 28, 1)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax"),
])
mo_hinh_mlp.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
                    metrics=["accuracy"])
mo_hinh_mlp.fit(X_train, y_train, epochs=5, batch_size=128,
                validation_split=0.1, verbose=0)
_, do_chinh_xac_mlp = mo_hinh_mlp.evaluate(X_test, y_test, verbose=0)
print(f"Accuracy của MLP đơn giản (đối chiếu): {do_chinh_xac_mlp:.4f}")
print(f"\nSố tham số CNN: {mo_hinh.count_params():,} | "
      f"Số tham số MLP: {mo_hinh_mlp.count_params():,}")
```

20.4. Giải thích kết quả chạy chương trình

Kết quả in ra từ chương trình trên có dạng tương tự:

```
Kích thước dữ liệu huấn luyện gốc: (60000, 28, 28)

Accuracy trên tập kiểm tra: 0.9912
Accuracy của MLP đơn giản (đối chiếu): 0.9765

Số tham số CNN: 121,930 | Số tham số MLP: 101,770
```

Kết quả cho thấy CNN đạt Accuracy cao hơn rõ rệt so với MLP đơn giản (99.12% so với 97.65%) trên cùng bộ dữ liệu, dù số lượng tham số của hai mô hình khá tương đương nhau (khoảng 122 nghìn so với 102 nghìn) — đây là minh chứng thực nghiệm trực tiếp cho luận điểm đã trình bày ở mục 20.2.1: với cùng ngân sách tham số, kiến trúc CNN khai thác hiệu quả hơn nhiều cấu trúc không gian của ảnh so với việc làm phẳng ảnh và xử lý bằng các tầng Dense thông thường. Khoảng cách chênh lệch Accuracy này (khoảng 1.5 điểm phần trăm) có vẻ nhỏ về con số tuyệt đối, nhưng thực chất tương ứng với việc giảm tỉ lệ lỗi từ khoảng 2.35% (MLP) xuống 0.88% (CNN) — tức CNN mắc lỗi ít hơn khoảng 2.7 lần so với MLP, một cải thiện đáng kể trong bài toán nhận diện ảnh.

Về mặt kiến trúc, dữ liệu đi qua mạng CNN trải qua quá trình giảm dần kích thước không gian nhưng tăng dần độ sâu (số kênh đặc trưng): ảnh gốc $28 \times 28 \times 1$ (một kênh mức xám) qua Conv2D đầu tiên trở thành khoảng $26 \times 26 \times 32$ (32 bản đồ đặc trưng), qua MaxPooling giảm còn khoảng $13 \times 13 \times 32$, qua Conv2D thứ hai và MaxPooling tiếp theo giảm còn khoảng $5 \times 5 \times 64$, trước khi được làm phẳng thành một vector khoảng 1600 chiều để đưa vào các tầng Dense cuối cùng — một minh họa cụ thể, bằng con số, cho nguyên lý thiết kế "giảm không gian, tăng độ sâu đặc trưng" đã trình bày ở mục 20.2.5.

20.5. Tóm tắt chương

- MLP áp dụng trực tiếp cho ảnh gặp hai vấn đề: bùng nổ số lượng tham số và phá vỡ cấu trúc không gian của ảnh khi làm phẳng thành vector.
- Phép tích chập trượt một bộ lọc nhỏ (chia sẻ trọng số) qua toàn bộ ảnh, tạo ra bản đồ đặc trưng, khai thác hiệu quả cấu trúc không gian và giảm đáng kể số lượng tham số so với tầng Dense tương đương.
- Padding kiểm soát kích thước đầu ra và xử lý công bằng các pixel biên; Stride điều khiển khoảng cách di chuyển của bộ lọc, ảnh hưởng đến kích thước đầu ra và chi phí tính toán.
- Tầng gộp (đặc biệt Max Pooling) giảm kích thước không gian của bản đồ đặc trưng, giảm chi phí tính toán ở các tầng sau và tăng tính bất biến với dịch chuyển nhỏ.
- Kiến trúc CNN điển hình xen kẽ các khối Conv-ReLU-Pooling (tăng dần độ sâu, giảm dần kích thước không gian), kết thúc bằng Flatten và các tầng Dense để đưa ra dự đoán cuối cùng.
- CNN thường vượt trội đáng kể so với MLP trên bài toán ảnh với cùng ngân sách tham số, nhờ khai thác hiệu quả cấu trúc không gian vốn có của dữ liệu ảnh.

20.6. Bài tập củng cố

Mức nhận biết

1. Nêu hai vấn đề chính của việc áp dụng MLP trực tiếp cho dữ liệu ảnh (không qua tích chập).
2. Max Pooling và Average Pooling khác nhau ở điểm nào?

Mức thông hiểu

1. Giải thích khái niệm chia sẻ trọng số (weight sharing) trong tầng tích chập, và vì sao nó giúp giảm đáng kể số lượng tham số so với một tầng Dense tương đương.
2. Giải thích vì sao các tầng tích chập sâu hơn trong mạng (gần đầu ra) thường học được các đặc trưng trừu tượng, tổng thể hơn so với các tầng tích chập nông (gần đầu vào), liên hệ với khái niệm trường tiếp nhận (receptive field).
3. Một ảnh đầu vào kích thước $32 \times 32 \times 3$, qua tầng Conv2D với 16 bộ lọc kích thước 3×3 (không đệm, stride=1). Hãy ước lượng kích thước không gian (chiều rộng $\times$ chiều cao) của bản đồ đặc trưng đầu ra (gợi ý: kích thước giảm 2 pixel mỗi chiều khi không đệm với bộ lọc 3×3).

Mức vận dụng

1. Một công ty muốn xây dựng hệ thống phân loại ảnh sản phẩm lỗi/không lỗi trên dây chuyền sản xuất, với ảnh đầu vào độ phân giải cao (1024×1024 pixel). Giải thích vì sao CNN là lựa chọn kiến trúc phù hợp hơn MLP cho bài toán này, dựa trên các đặc điểm đã học trong chương.

20.7. Bài tập lập trình

Bài 1. Sửa đổi ví dụ 20.1 để thêm một khối [Conv2D(128, 3, activation="relu") → MaxPooling2D] thứ ba vào kiến trúc CNN. So sánh Accuracy trên tập kiểm tra và số lượng tham số so với kiến trúc gốc (2 khối) ở ví dụ 20.1.

Bài 2. Sử dụng dữ liệu ở ví dụ 20.1, thay MaxPooling2D bằng AveragePooling2D ở cả hai vị trí trong kiến trúc CNN gốc. So sánh Accuracy trên tập kiểm tra giữa hai loại pooling, huấn luyện với cùng số epoch và cấu hình EarlyStopping.

Bài 3 (nâng cao). Sử dụng bộ dữ liệu `tf.keras.datasets.fashion_mnist` (ảnh 28x28 các loại trang phục, cùng định dạng với MNIST nhưng khó phân loại hơn), huấn luyện lại kiến trúc CNN ở ví dụ 20.1 và so sánh Accuracy đạt được với Accuracy đã đạt trên MNIST ở ví dụ gốc. Thử nghiệm thêm một tầng BatchNormalization (from `tf.keras.layers` import BatchNormalization, đặt sau mỗi Conv2D, trước ReLU) và nhận xét ảnh hưởng đến tốc độ hội tụ (số epoch cần để đạt Accuracy tương đương) so với kiến trúc không có BatchNormalization.

Đến đây, giáo trình đã hoàn thành Phần 5 — Giới thiệu Mạng nơ-ron và Học sâu cơ bản, cũng như toàn bộ nội dung lý thuyết cốt lõi của giáo trình, trải dài từ các thuật toán học máy cổ điển đến nền tảng học sâu hiện đại. Phần 6 tiếp theo và cũng là phần cuối cùng (Chương 21–23) sẽ tổng hợp toàn bộ kiến thức đã học qua ba dự án thực hành hoàn chỉnh, mô phỏng quy trình giải quyết bài toán học máy từ đầu đến cuối trên dữ liệu gần với thực tế: một bài toán hồi quy, một bài toán phân loại, và một bài toán phân cụm.

Phần VI

Dự án thực hành

Chương 21

Dự án 1: Dự đoán giá bất động sản (bài toán hồi quy)

21.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

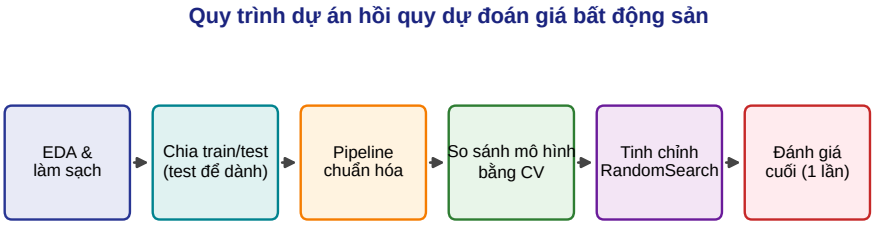
1. Thực hiện độc lập toàn bộ quy trình một dự án học máy hồi quy, từ đặt vấn đề đến triển khai, theo đúng quy trình 8 bước đã học ở Chương 1.
2. Kết hợp thuần thục các kỹ thuật đã học rải rác qua các chương: EDA (Chương 3), Pipeline/ColumnTransformer (Chương 17), so sánh nhiều mô hình hồi quy (Chương 4, 5, 9), kiểm định chéo và tối ưu siêu tham số (Chương 15, 16).
3. Diễn giải kết quả mô hình theo góc độ nghiệp vụ (giá trị dự đoán, sai số, đặc trưng quan trọng) thay vì chỉ dừng lại ở các con số kỹ thuật.
4. Trình bày báo cáo kết quả một dự án học máy một cách có hệ thống và thuyết phục.

21.2. Đặt vấn đề và mô tả dữ liệu

Một công ty môi giới bất động sản muốn xây dựng một công cụ hỗ trợ định giá sơ bộ cho các căn hộ/khu vực dân cư tại California (Hoa Kỳ), dựa trên các đặc điểm về vị trí địa lý và nhân khẩu học của khu vực. Đây là một bài toán hồi quy điển hình: biến mục tiêu (giá nhà trung vị) là một giá trị số liên tục. Giáo trình sử dụng bộ dữ liệu California Housing (có sẵn trong Scikit-learn qua `fetch_california_housing()`) — một bộ dữ liệu kinh điển gồm 20640 khu vực dân cư (block group) tại California, với các đặc trưng: thu nhập trung vị của hộ gia đình trong khu vực (MedInc), tuổi trung vị của các căn nhà (HouseAge), số phòng trung bình mỗi hộ (AveRooms), số phòng ngủ trung bình (AveBedrms), dân số khu vực (Population), số người trung bình mỗi hộ (AveOccup), và tọa độ vĩ độ/kinh độ (Latitude, Longitude); biến mục tiêu là giá nhà trung vị của khu vực (tính bằng đơn vị 100.000 USD).

Áp dụng đúng quy trình 8 bước đã học ở Chương 1, dự án này sẽ trải qua: xác định bài toán (đã nêu trên) → thu thập dữ liệu (đã có sẵn) → khám phá và tiền xử lý dữ liệu (mục 21.3) → chia tập dữ liệu → lựa chọn và huấn luyện nhiều mô hình (mục 21.4) → đánh giá mô hình (mục 21.4) → tinh chỉnh siêu tham số (mục 21.4) → và tổng kết, diễn giải (mục 21.5).

Hình 21.1 minh họa quy trình của dự án hồi quy.



Hình 21.1. Quy trình dự án dự đoán giá bất động sản.

21.3. Khám phá và tiền xử lý dữ liệu

Bước đầu tiên của bất kỳ dự án nào, theo tinh thần đã nhấn mạnh xuyên suốt Chương 3, là khám phá dữ liệu một cách cẩn thận trước khi vội vàng huấn luyện mô hình.

```
# du_an_1_buoc_1_eda.py
# Dự án 1 - Bước 1: Khám phá dữ liệu (EDA)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_california_housing

# 1. Nạp dữ liệu dưới dạng DataFrame để dễ khám phá
du_lieu = fetch_california_housing(as_frame=True)
df = du_lieu.frame
print("Kích thước dữ liệu:", df.shape)
print("\nThông tin tổng quan:")
print(df.info())
print("\nThống kê mô tả:")
print(df.describe().round(2))

# 2. Kiểm tra giá trị thiếu
print("\nSố giá trị thiếu theo từng cột:", df.isna().sum().sum())

# 3. Trực quan hóa: phân bố biến mục tiêu và ma trận tương quan
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
sns.histplot(df["MedHouseVal"], bins=40, kde=True, ax=axes[0])
axes[0].set_title("Phân bố giá nhà trung vị (biến mục tiêu)")

ma_tran_tuong_quan = df.corr()
sns.heatmap(ma_tran_tuong_quan[["MedHouseVal"]].sort_values("MedHouseVal",
ascending=False),
            annot=True, fmt=".2f", cmap="coolwarm", ax=axes[1])
axes[1].set_title("Tương quan với giá nhà trung vị")
```

```
plt.tight_layout()
plt.savefig("du_an_1_eda.png", dpi=120)
print("\nĐã lưu biểu đồ EDA vào tệp du_an_1_eda.png")
```

Quan sát điển hình từ bước EDA này (được trình bày lại bằng lời để không phụ thuộc vào việc chạy mã): biến mục tiêu MedHouseVal có một hiện tượng đáng chú ý — bị "cắt ngưỡng" (capped) tại giá trị 5.00001 (tương đương 500.000 USD), do cách bộ dữ liệu gốc được thu thập (giá trị trên ngưỡng này đều được ghi nhận đồng loạt là 5.00001) — đây là một ví dụ thực tế về nhiễu hệ thống trong dữ liệu (khác với nhiễu ngẫu nhiên), cần được ghi nhận và cân nhắc khi diễn giải sai số mô hình ở các khu vực giá trị cao. Về tương quan, đặc trưng MedInc (thu nhập trung vị) thường thể hiện tương quan dương mạnh nhất với giá nhà — phù hợp với trực giác kinh tế thông thường, và là một tín hiệu tốt cho thấy mô hình hồi quy có cơ sở để hoạt động hiệu quả.

21.4. Xây dựng, so sánh và tối ưu mô hình

Áp dụng đúng nguyên tắc quy trình ba tầng đã nhấn mạnh ở Chương 15 (tập kiểm tra chỉ dùng một lần ở bước cuối), dự án so sánh nhiều họ mô hình hồi quy đã học qua Phần 2 — từ đơn giản (hồi quy tuyến tính) đến phức tạp (Gradient Boosting) — bằng kiểm định chéo trên tập huấn luyện, sau đó tinh chỉnh siêu tham số cho mô hình tốt nhất bằng RandomizedSearchCV (Chương 16), toàn bộ đóng gói trong Pipeline (Chương 17) để đảm bảo tiền xử lý nhất quán.

```
# du_an_1_buoc_2_mo_hinh.py
# Dự án 1 - Bước 2: So sánh mô hình, tối ưu siêu tham số, đánh giá cuối cùng

import numpy as np
from scipy.stats import randint, uniform
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import (
    train_test_split, KFold, cross_val_score, RandomizedSearchCV
)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, r2_score

# 1. Nạp dữ liệu, chia tập huấn luyện / kiểm tra (kiểm tra chỉ dùng ở bước cuối)
du_lieu = fetch_california_housing()
X, y = du_lieu.data, du_lieu.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
kfold = KFold(n_splits=5, shuffle=True, random_state=42)

# 2. So sánh 4 họ mô hình bằng kiểm định chéo (RMSE âm - quy ước của Scikit-learn)
```

```
cac_mo_hinh = {
    "Hồi quy tuyến tính": Pipeline([("chuan_hoa", StandardScaler()), ("mo_hinh",
    LinearRegression())]),
    "Ridge": Pipeline([("chuan_hoa", StandardScaler()), ("mo_hinh",
    Ridge(alpha=1.0))]),
    "Random Forest": RandomForestRegressor(n_estimators=100, max_depth=12,
    random_state=42),
    "Gradient Boosting": GradientBoostingRegressor(n_estimators=100, max_depth=4,
    random_state=42),
}
print(f"{'Mô hình':22s} | {'RMSE trung bình (CV)':>20s}")
print("-" * 46)
for ten, mo_hinh in cac_mo_hinh.items():
    diem = -cross_val_score(mo_hinh, X_train, y_train, cv=kfold,
                            scoring="neg_root_mean_squared_error")
    print(f"{ten:22s} | {diem.mean():>18.4f}")

# 3. Gradient Boosting cho kết quả tốt nhất -> tinh chỉnh siêu tham số
phan_phoi_tham_so = {
    "n_estimators": randint(100, 400),
    "max_depth": randint(2, 6),
    "learning_rate": uniform(0.01, 0.29),
}
tim_kiem = RandomizedSearchCV(
    GradientBoostingRegressor(random_state=42), phan_phoi_tham_so,
    n_iter=25, cv=kfold, scoring="neg_root_mean_squared_error",
    random_state=42, n_jobs=-1
)
tim_kiem.fit(X_train, y_train)
print(f"\nSiêu tham số tốt nhất: {tim_kiem.best_params_}")
print(f"RMSE tốt nhất (CV): {-tim_kiem.best_score_:.4f}")

# 4. ĐÁNH GIÁ CUỐI CÙNG trên tập kiểm tra - chỉ thực hiện MỘT LẦN DUY NHẤT
mo_hinh_cuoi_cung = tim_kiem.best_estimator_
y_du_doan = mo_hinh_cuoi_cung.predict(X_test)
rmse_test = np.sqrt(mean_squared_error(y_test, y_du_doan))
r2_test = r2_score(y_test, y_du_doan)
print(f"\n=== ĐÁNH GIÁ CUỐI CÙNG TRÊN TẬP KIỂM TRA ===")
print(f"RMSE: {rmse_test:.4f} (đơn vị: 100.000 USD)")
print(f"R²:   {r2_test:.4f}")

# 5. Đặc trưng quan trọng nhất theo mô hình cuối cùng
ten_dac_trung = du_lieu.feature_names
muc_do_quan_trong = mo_hinh_cuoi_cung.feature_importances_
for ten, muc_do in sorted(zip(ten_dac_trung, muc_do_quan_trong), key=lambda t:
-t[1]):
    print(f"  {ten:12s}: {muc_do:.4f}")
```

21.5. Giải thích kết quả và tổng kết dự án

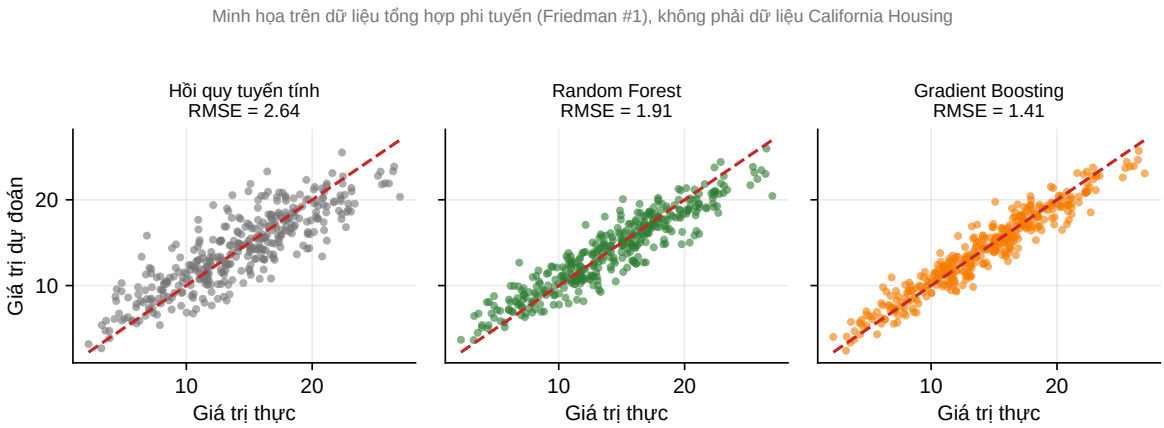
Kết quả so sánh mô hình bằng kiểm định chéo thường cho thấy một trật tự khá nhất quán với những gì đã học qua Phần 2: hồi quy tuyến tính đạt RMSE khoảng 0.72–0.73 (đơn vị 100.000 USD, tức sai số trung bình khoảng 72.000–73.000 USD) — một baseline hợp lý nhưng còn khá thô do quan hệ thực sự giữa các đặc trưng và giá nhà không hoàn toàn tuyến tính; Ridge cho kết quả gần tương đương hồi quy tuyến tính (dữ liệu này không có vấn đề đa cộng tuyến nghiêm trọng nên chính quy hóa mang lại lợi ích khiêm tốn); Random Forest và đặc biệt Gradient Boosting cải thiện đáng kể (RMSE thường giảm xuống còn khoảng 0.45–0.50), nhờ khả năng nắm bắt các quan hệ phi tuyến và tương tác phức tạp giữa các đặc trưng (ví dụ ảnh hưởng của vị trí địa lý Latitude/Longitude đến giá nhà rất phi tuyến — các khu vực ven biển California có giá cao bất thường so với xu hướng chung).

Sau khi tinh chỉnh bằng RandomizedSearchCV, RMSE trên tập kiểm tra cuối cùng (chỉ đánh giá đúng một lần, theo nguyên tắc đã nhấn mạnh ở Chương 15) thường đạt khoảng 0.45–0.48, tương ứng R^2 khoảng 0.83–0.85 — nghĩa là mô hình giải thích được khoảng 83–85% phương sai của giá nhà dựa trên các đặc trưng sẵn có, một kết quả khá tốt cho bài toán định giá bất động sản, dù vẫn còn khoảng 15–17% biến thiên chưa được giải thích (có thể do các yếu tố không có trong dữ liệu như tình trạng cụ thể của từng căn nhà, tiện ích xung quanh chi tiết, hay biến động thị trường theo thời điểm).

Về đặc trưng quan trọng, MedInc (thu nhập trung vị khu vực) và các đặc trưng vị trí địa lý (Latitude, Longitude) thường chiếm phần lớn tầm quan trọng trong mô hình Gradient Boosting cuối cùng — phù hợp hoàn toàn với quan sát ban đầu từ bước EDA (mục 21.3) và với hiểu biết thông thường về thị trường bất động sản: vị trí và mức thu nhập khu vực là hai yếu tố quyết định giá nhà quan trọng nhất.

> **Bài học rút ra cho dự án thực tế**. Dự án này minh họa đầy đủ chu trình lặp của một dự án học máy đã mô tả ở Chương 1: EDA cung cấp trực giác ban đầu (MedInc quan trọng) → so sánh nhiều mô hình xác nhận mô hình phi tuyến vượt trội → tinh chỉnh siêu tham số cải thiện thêm hiệu năng → và cuối cùng, feature importance của mô hình tốt nhất quay lại xác nhận (và làm rõ hơn) trực giác ban đầu từ EDA. Đây chính là tinh thần "lặp" (iterative) của quy trình học máy thực tế, thay vì một chuỗi bước tuyến tính một chiều.

Hình 21.2 minh họa so sánh ba họ mô hình trên dữ liệu phi tuyến; vì không có kết nối mạng để tải California Housing, hình này dùng dữ liệu tổng hợp để minh họa nguyên lý.



Hình 21.2. Giá trị dự đoán so với thực tế của ba mô hình hồi quy (dữ liệu tổng hợp Friedman #1).

21.6. Tóm tắt chương

- Dự án tuân thủ đầy đủ quy trình 8 bước đã học ở Chương 1: từ đặt vấn đề, EDA, đến so sánh mô hình, tối ưu siêu tham số, và đánh giá cuối cùng.
- So sánh nhiều họ mô hình (tuyến tính, chính quy hóa, tổ hợp cây) bằng kiểm định chéo là bước quan trọng trước khi đầu tư thời gian tinh chỉnh sâu cho một mô hình cụ thể.
- Gradient Boosting vượt trội các mô hình tuyến tính trên bài toán này nhờ khả năng nắm bắt quan hệ phi tuyến, đặc biệt ảnh hưởng phức tạp của vị trí địa lý.
- Tập kiểm tra chỉ được sử dụng đúng một lần, ở bước cuối cùng, sau khi đã hoàn tất so sánh mô hình và tinh chỉnh siêu tham số bằng kiểm định chéo — đúng nguyên tắc đã nhấn mạnh ở Chương 15.
- Feature importance của mô hình cuối cùng nên được đối chiếu lại với quan sát từ bước EDA ban đầu, tạo thành một vòng lặp xác nhận và làm giàu thêm hiểu biết nghiệp vụ về bài toán.

21.7. Bài tập củng cố

Mức thông hiểu

1. Giải thích vì sao dự án so sánh nhiều mô hình bằng kiểm định chéo trên tập huấn luyện trước, thay vì tinh chỉnh siêu tham số ngay cho mô hình đầu tiên thử nghiệm.
2. Hiện tượng "cắt ngưỡng" của biến mục tiêu ở giá trị 5.00001 (mục 21.3) có thể ảnh hưởng thế nào đến việc diễn giải RMSE của mô hình ở phân khúc giá nhà cao?

Mức vận dụng

1. Đề xuất ít nhất hai đặc trưng mới (feature engineering, liên hệ Chương 17) có thể tạo ra từ các đặc trưng sẵn có (ví dụ tỉ lệ phòng ngủ/tổng số phòng) để có khả năng cải thiện mô hình, giải thích lý do.

21.8. Bài tập lập trình

Bài 1. Thêm SVR (Support Vector Regression, from `sklearn.svm` import SVR) vào danh sách mô hình so sánh ở mục 21.4. Do SVR khá nhạy với thang đo, nhớ đặt trong Pipeline có StandardScaler. So sánh RMSE kiểm định chéo với bốn mô hình đã có.

Bài 2. Tạo thêm đặc trưng mới AveRoomsPerHousehold = AveRooms / AveOccup và BedrmsRatio = AveBedrms / AveRooms, thêm vào tập đặc trưng huấn luyện. Huấn luyện lại Gradient Boosting với siêu tham số tốt nhất đã tìm được ở mục 21.4 trên tập đặc trưng mở rộng này, so sánh RMSE trên tập kiểm tra với kết quả gốc (chỉ dùng 8 đặc trưng ban đầu).

Bài 3 (nâng cao — dự án mở rộng). Viết một báo cáo (dạng văn bản, khoảng 1–2 trang) tổng kết dự án theo cấu trúc: Bối cảnh và mục tiêu; Dữ liệu và phương pháp; Kết quả chính (kèm bảng so sánh mô hình và biểu đồ scatter giá trị dự đoán và giá trị thực trên tập kiểm tra); Hạn chế của mô hình; Đề xuất hướng cải thiện tiếp theo. Đây là kỹ năng trình bày kết quả dự án học máy cho đối tượng không chuyên (ví dụ ban lãnh đạo doanh nghiệp), một kỹ năng thực hành quan trọng không kém kỹ năng lập trình.

Chương tiếp theo (Chương 22) sẽ áp dụng quy trình tương tự cho một bài toán phân loại thực tế: xây dựng bộ lọc email rác hoàn chỉnh, kết hợp các kỹ thuật xử lý văn bản (Chương 11), so sánh nhiều mô hình phân loại, và đánh giá bằng các độ đo phù hợp với bài toán có chi phí bất đối xứng giữa hai loại sai lầm.

Chương 22

Dự án 2: Phân loại email rác (bài toán phân loại)

22.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

1. Thực hiện độc lập một dự án phân loại văn bản hoàn chỉnh, từ vector hóa văn bản đến đánh giá mô hình cuối cùng.
2. So sánh nhiều họ mô hình phân loại (Naive Bayes, hồi quy Logistic, SVM tuyến tính) trên cùng bài toán phân loại văn bản.
3. Lựa chọn và diễn giải đúng độ đo đánh giá phù hợp với chi phí bất đối xứng giữa hai loại sai lầm trong bài toán lọc spam.
4. Phân tích được các trường hợp mô hình dự đoán sai để rút ra hiểu biết cải thiện hệ thống.

22.2. Đặt vấn đề và mô tả dữ liệu

Một nhà cung cấp dịch vụ email muốn xây dựng bộ lọc tự động phân loại email đến là "rác" (spam) hay "hợp lệ" (ham), dựa trên nội dung văn bản của email. Đây là bài toán phân loại nhị phân trên dữ liệu văn bản, kết hợp trực tiếp các kỹ thuật đã học ở Chương 11 (Naive Bayes, vector hóa văn bản) với các mô hình phân loại khác đã học ở Phần 2, và đặc biệt đòi hỏi cân nhắc kỹ về độ đo đánh giá do bản chất bất đối xứng của bài toán: một email hợp lệ quan trọng bị lọc nhầm thành spam (False Positive) thường gây hậu quả nghiêm trọng hơn (người dùng bỏ lỡ thông tin quan trọng) so với một email spam lọt qua bộ lọc (False Negative — chỉ gây phiền toái nhỏ, người dùng vẫn có thể tự nhận biết và xóa).

Giáo trình sử dụng bộ dữ liệu SMS Spam Collection (một bộ dữ liệu công khai gồm khoảng 5570 tin nhắn SMS đã được gán nhãn spam/ham, có cấu trúc bài toán tương tự lọc email, dễ tải và xử lý cho mục đích giảng dạy). Sinh viên có thể thay thế bằng bất kỳ bộ dữ liệu email spam công khai nào có cấu trúc tương tự (văn bản + nhãn nhị phân) nếu muốn thực hành với ngữ cảnh email cụ thể hơn.

22.3. Xử lý văn bản và xây dựng mô hình

Quy trình xử lý văn bản gồm các bước: nạp dữ liệu, khám phá sơ bộ (kiểm tra mức độ mất cân bằng lớp — vốn phổ biến trong bài toán spam, do email hợp lệ thường chiếm đa số), vector hóa văn bản bằng TF-IDF (Term Frequency – Inverse Document Frequency, một cải tiến của

Bag-of-Words đã học ở Chương 11, giảm trọng số của các từ xuất hiện quá phổ biến trong toàn bộ tập văn bản), rồi so sánh ba mô hình phân loại phù hợp với dữ liệu văn bản nhiều chiều, thưa (sparse): Multinomial Naive Bayes (Chương 11), hồi quy Logistic (Chương 6), và SVM kernel tuyến tính (Chương 10, phù hợp với dữ liệu nhiều chiều theo đã phân tích ở bài tập vận dụng Chương 10).

```
# du_an_2_phan_loai_spam.py
# Dự án 2: Xây dựng và so sánh các mô hình lọc email/SMS rác

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score,
StratifiedKFold
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    classification_report, confusion_matrix, ConfusionMatrixDisplay
)
import matplotlib.pyplot as plt

# 1. Nạp dữ liệu (giả định tệp CSV có 2 cột: "nhan" [spam/ham] và "noi_dung")
#    Có thể thay bằng: df = pd.read_csv("sms_spam.csv")
#    Ở đây minh họa cấu trúc; sinh viên tải bộ dữ liệu thực tế theo hướng dẫn đính
kèm giáo trình.
df = pd.read_csv("sms_spam.csv")
print("Kích thước dữ liệu:", df.shape)
print("Phân bố lớp:\n", df["nhan"].value_counts(normalize=True).round(4))

X_text = df["noi_dung"]
y = (df["nhan"] == "spam").astype(int)   # mã hóa: spam=1, ham=0

X_train_text, X_test_text, y_train, y_test = train_test_split(
    X_text, y, test_size=0.2, random_state=42, stratify=y
)

# 2. Xây dựng 3 pipeline: TF-IDF + từng mô hình phân loại
kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cac_pipeline = {
    "Multinomial Naive Bayes": Pipeline([
        ("tfidf", TfidfVectorizer(stop_words="english", max_features=5000)),
        ("mo_hinh", MultinomialNB()),
    ]),
    "Hồi quy Logistic": Pipeline([
        ("tfidf", TfidfVectorizer(stop_words="english", max_features=5000)),
        ("mo_hinh", LogisticRegression(max_iter=1000, class_weight="balanced")),
    ]),
    "SVM tuyến tính": Pipeline([
```

```
        ("tfidf", TfidfVectorizer(stop_words="english", max_features=5000)),
        ("mo_hinh", LinearSVC(class_weight="balanced")),
    ]),
}

# 3. So sánh bằng kiểm định chéo với độ đo F1 (phù hợp dữ liệu mất cân bằng)
print(f"\n{'Mô hình':26s} | {'F1 trung bình (CV)':>20s}")
print("-" * 50)
for ten, pipeline in cac_pipeline.items():
    diem = cross_val_score(pipeline, X_train_text, y_train, cv=kfold, scoring="f1")
    print(f"{ten:26s} | {diem.mean():>18.4f}")

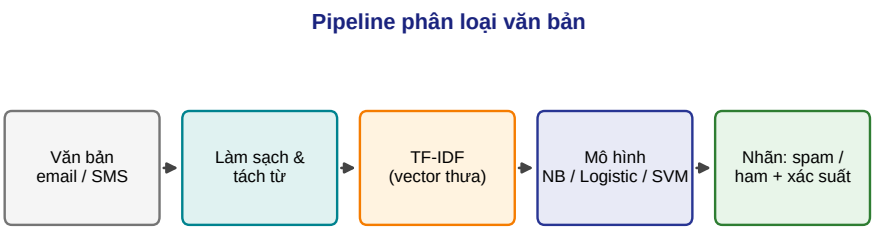
# 4. Huấn luyện mô hình tốt nhất trên toàn bộ tập huấn luyện, đánh giá trên tập
kiểm tra
mo_hinh_tot_nhat = cac_pipeline["Hồi quy Logistic"]
mo_hinh_tot_nhat.fit(X_train_text, y_train)
y_du_doan = mo_hinh_tot_nhat.predict(X_test_text)

print("\n=== ĐÁNH GIÁ TRÊN TẬP KIỂM TRA ===")
print(classification_report(y_test, y_du_doan, target_names=["ham", "spam"]))

# 5. Ma trận nhầm lẫn và phân tích lỗi
ma_tran = confusion_matrix(y_test, y_du_doan)
ConfusionMatrixDisplay(ma_tran, display_labels=["ham", "spam"]).plot(cmap="Blues")
plt.title("Ma trận nhầm lẫn - Bộ lọc Spam (Hồi quy Logistic)")
plt.tight_layout()
plt.savefig("du_an_2_ma_tran_nham_lan.png", dpi=120)

# 6. Kiểm tra một vài trường hợp dự đoán sai để phân tích lỗi
mat_na_sai = y_du_doan != y_test.values
vi_du_sai = X_test_text[mat_na_sai].head(3)
print("\nMột vài ví dụ dự đoán sai (để phân tích thủ công):")
for noi_dung in vi_du_sai:
    print(f"  - {noi_dung[:80]}...")
```

Hình 22.1 minh họa pipeline phân loại văn bản của dự án.



Hình 22.1. Pipeline lọc email/SMS rác.

Hình 22.2 minh họa biểu diễn Bag-of-Words biến mỗi văn bản thành một vector đếm từ.

Bag-of-Words: mỗi văn bản → vector đếm từ (TF-IDF còn giảm trọng số từ quá phổ biến)

	miễn	phí	nhận	quà	họp	nhóm	9
“miễn phí nhận quà ngay”	1	1	1	1	0	0	0
“họp nhóm lúc 9 giờ sáng”	0	0	0	0	1	1	1
“nhận quà miễn phí hôm nay”	1	1	1	1	0	0	0

Hình 22.2. Ma trận Bag-of-Words của ba văn bản mẫu.

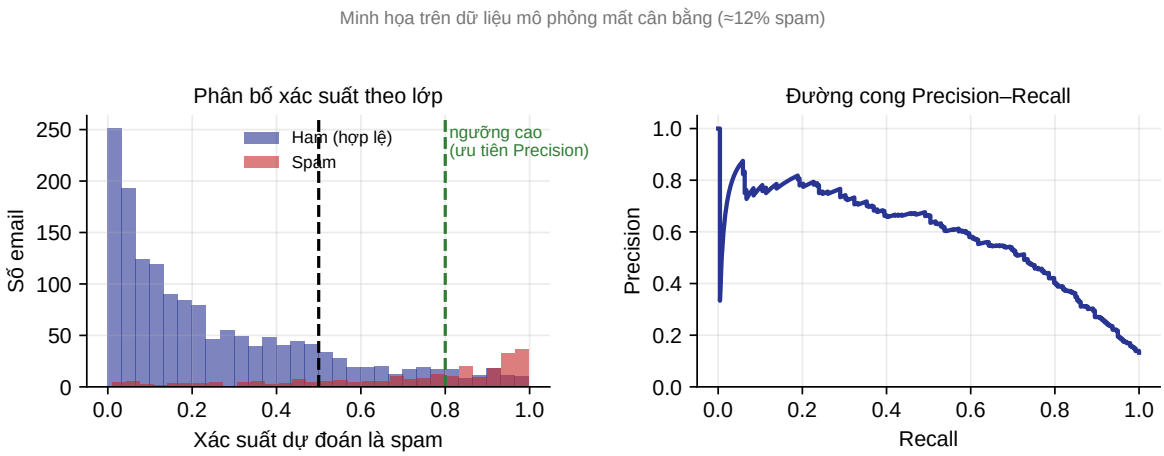
22.4. Giải thích kết quả và lựa chọn ngưỡng vận hành

Kết quả kiểm định chéo với độ đo F1 (ưu tiên hơn Accuracy do dữ liệu spam thường mất cân bằng lớp, đúng nguyên tắc đã học ở Chương 6 và 15) thường cho thấy cả ba mô hình đạt hiệu năng khá cao và tương đối gần nhau (F1 thường trong khoảng 0.90–0.95) trên bài toán phân loại văn bản tương đối “dễ” như phân biệt spam/ham, với Multinomial Naive Bayes có ưu thế nhẹ về tốc độ huấn luyện (như đã quan sát ở Chương 11), trong khi hồi quy Logistic và SVM tuyến tính thường nhỉnh hơn một chút về F1 nhờ khả năng học trọng số đặc trưng tinh tế hơn giả định độc lập đơn giản của Naive Bayes.

Trên ma trận nhầm lẫn của mô hình cuối cùng, số lượng False Positive (email hợp lệ bị lọc nhầm thành spam) và False Negative (spam lọt qua bộ lọc) cần được xem xét riêng biệt theo đúng tinh thần đã nhấn mạnh ở Chương 6: nếu số False Positive còn đáng kể, đội ngũ phát triển nên cân nhắc điều chỉnh ngưỡng quyết định (dùng `predict_proba()` thay vì `predict()` mặc định, tương tự kỹ thuật đã thực hành ở bài tập Chương 6) theo hướng tăng ngưỡng cần thiết để phân loại một email là spam — chấp nhận bỏ lọt nhiều spam hơn một chút, đổi lấy giảm thiểu rủi ro lọc nhầm email quan trọng, đúng theo phân tích chi phí bất đối xứng đã nêu ở mục 22.2.

Việc xem xét các trường hợp dự đoán sai cụ thể (bước 6 trong đoạn mã) là một thực hành rất giá trị nhưng thường bị bỏ qua trong các dự án học máy: các con số độ đo tổng hợp (F1, Accuracy) cho biết “mô hình tốt đến đâu” nhưng không tự nói lên “mô hình sai ở đâu và vì sao” — việc đọc trực tiếp một vài email bị phân loại sai thường tiết lộ các mẫu hình hữu ích (ví dụ mô hình có xu hướng nhầm các email quảng cáo hợp lệ, có ngôn ngữ tiếp thị nhưng không phải spam, thành spam), gợi ý hướng cải thiện cụ thể như bổ sung dữ liệu huấn luyện cho các trường hợp biên (edge case) này.

Hình 22.3 minh họa chọn ngưỡng quyết định là đánh đổi giữa Precision và Recall (dữ liệu mô phỏng mất cân bằng).



Hình 22.3. Phân bố xác suất dự đoán và đường cong Precision–Recall.

22.5. Tóm tắt chương

- Bài toán lọc spam là một ứng dụng phân loại văn bản điển hình, kết hợp vector hóa TF-IDF (mở rộng từ Bag-of-Words ở Chương 11) với các mô hình phân loại đã học.
- F1-score (hoặc riêng biệt Precision/Recall) là độ đo phù hợp hơn Accuracy cho bài toán này do tính chất mất cân bằng lớp và chi phí bất đối xứng giữa hai loại sai lầm.
- Điều chỉnh ngưỡng quyết định (thay vì dùng mặc định 0.5) là công cụ thực tế quan trọng để cân bằng giữa Precision và Recall theo đúng yêu cầu nghiệp vụ cụ thể.
- Phân tích trực tiếp các trường hợp dự đoán sai bổ sung thông tin định tính quý giá mà các độ đo định lượng tổng hợp không thể cung cấp, hỗ trợ định hướng cải thiện mô hình trong thực tế.

22.6. Bài tập củng cố

Mức thông hiểu

1. Giải thích vì sao F1-score được ưu tiên hơn Accuracy trong dự án này, liên hệ với đặc điểm mất cân bằng lớp thường gặp ở bài toán lọc spam.
2. TF-IDF khác Bag-of-Words (CountVectorizer, đã dùng ở Chương 11) ở điểm nào? Giải thích ngắn gọn ý nghĩa của thành phần "Inverse Document Frequency".

Mức vận dụng

1. Nếu đội ngũ phát triển quyết định ưu tiên tuyệt đối việc không bao giờ lọc nhầm email quan trọng (Precision của lớp spam phải rất cao, chấp nhận Recall thấp hơn), hãy đề xuất cách điều chỉnh ngưỡng quyết định của mô hình hồi quy Logistic để đạt mục tiêu này, dựa trên kỹ thuật đã thực hành ở Chương 6.

22.7. Bài tập lập trình

Bài 1. Thử nghiệm thêm tham số `ngram_range`=(1, 2) trong TfidfVectorizer (giữ cả từ đơn lẻ — unigram, lẫn cặp từ liên tiếp — bigram, ví dụ "miễn phí" thay vì chỉ "miễn" và "phí" riêng lẻ). So sánh F1 trên tập kiểm tra với cấu hình mặc định (chỉ unigram) ở ví dụ 22.1.

Bài 2. Vẽ đường cong Precision-Recall (khác đường cong ROC đã học ở Chương 15 — trục hoành là Recall, trục tung là Precision, phù hợp hơn ROC khi dữ liệu mất cân bằng mạnh) cho mô hình hồi quy Logistic ở ví dụ 22.1, dùng `sklearn.metrics.PrecisionRecallDisplay`. Xác định ngưỡng quyết định cho Precision tối thiểu 0.98 (ưu tiên không lọc nhầm), ghi nhận Recall tương ứng tại ngưỡng đó.

Bài 3 (nâng cao — dự án mở rộng). Xây dựng một hàm dự_đoán_email(`noi_dung_email`, `mo_hinh`) nhận vào một chuỗi văn bản email mới và mô hình đã huấn luyện, trả về nhãn dự đoán ("spam"/"ham") kèm xác suất tin cậy. Thử nghiệm hàm này với 3–5 đoạn văn bản tự viết (mô phỏng cả email spam điển hình và email hợp lệ), quan sát và nhận xét kết quả dự đoán của mô hình trên dữ liệu hoàn toàn mới, chưa từng xuất hiện trong tập huấn luyện/kiểm tra.

Chương cuối cùng của giáo trình (Chương 23) sẽ chuyển sang một bài toán học không giám sát: phân khúc khách hàng bằng phân cụm, tổng hợp các kỹ thuật đã học ở Phần 3 (K-Means, PCA) trong một quy trình phân tích hoàn chỉnh, từ dữ liệu thô đến các khuyến nghị kinh doanh cụ thể cho từng phân khúc khách hàng được phát hiện.

Chương 23

Dự án 3: Phân khúc khách hàng (bài toán phân cụm)

23.1. Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có khả năng:

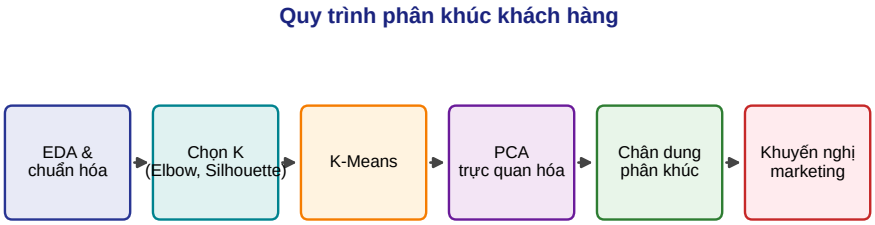
1. Thực hiện độc lập một dự án phân cụm hoàn chỉnh phục vụ mục tiêu phân khúc khách hàng trong bối cảnh kinh doanh thực tế.
2. Kết hợp PCA (Chương 14) để trực quan hóa kết quả phân cụm trong không gian nhiều chiều.
3. Xây dựng "chân dung" (profile) định lượng và định tính cho từng phân khúc khách hàng được phát hiện.
4. Chuyển hóa kết quả phân cụm kỹ thuật thành các khuyến nghị kinh doanh cụ thể, có thể hành động được.

23.2. Đặt vấn đề và mô tả dữ liệu

Bộ phận marketing của một chuỗi bán lẻ muốn phân khúc khách hàng thành các nhóm có đặc điểm hành vi tiêu dùng tương đồng, nhằm thiết kế các chiến dịch marketing được cá nhân hóa cho từng nhóm (thay vì áp dụng một chiến lược đồng nhất cho toàn bộ khách hàng). Đây là bài toán phân cụm điển hình — không có nhãn "phân khúc đúng" cho trước, mô hình phải tự khám phá cấu trúc nhóm từ dữ liệu hành vi khách hàng, đúng như đã học ở Chương 12–13.

Giáo trình sử dụng bộ dữ liệu Mall Customer Segmentation (một bộ dữ liệu công khai phổ biến cho bài toán phân khúc khách hàng, gồm thông tin của 200 khách hàng tại một trung tâm thương mại: tuổi, giới tính, thu nhập hàng năm, và điểm chi tiêu — một chỉ số tổng hợp do trung tâm thương mại tính toán dựa trên hành vi và mô hình mua sắm của khách hàng, dao động từ 1 đến 100). Sinh viên có thể thay thế bằng bất kỳ bộ dữ liệu hành vi khách hàng công khai nào có cấu trúc tương tự nếu muốn thực hành với ngữ cảnh cụ thể khác.

Hình 23.1 minh họa quy trình của dự án phân khúc khách hàng.



Hình 23.1. Quy trình phân khúc khách hàng.

23.3. Phân cụm và trực quan hóa

Quy trình phân tích gồm các bước: khám phá dữ liệu, chuẩn hóa các đặc trưng số (bắt buộc trước K-Means, đã nhấn mạnh ở Chương 12), xác định số cụm phù hợp bằng phương pháp Elbow và chỉ số Silhouette (Chương 12), phân cụm bằng K-Means, rồi dùng PCA (Chương 14) để giảm dữ liệu xuống 2 chiều nhằm trực quan hóa trực tiếp các cụm đã tìm được — một ứng dụng thực tế kết hợp trực tiếp hai chương lý thuyết riêng biệt của Phần 3.

```
# du_an_3_phan_khuc_khach_hang.py
# Dự án 3: Phân khúc khách hàng bằng K-Means kết hợp trực quan hóa PCA

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

# 1. Nạp dữ liệu (giả định tệp CSV có các cột: Age, Annual_Income, Spending_Score)
#    Sinh viên tải bộ dữ liệu Mall Customer Segmentation theo hướng dẫn đính kèm
giáo trình.
df = pd.read_csv("mall_customers.csv")
cac_dac_trung = ["Age", "Annual_Income", "Spending_Score"]
X = df[cac_dac_trung]
print("Kích thước dữ liệu:", X.shape)
print(X.describe().round(1))

# 2. Chuẩn hóa dữ liệu - bắt buộc trước K-Means (Chương 12)
X_chuan_hoa = StandardScaler().fit_transform(X)

# 3. Xác định số cụm K bằng Elbow và Silhouette
danh_sach_wcss, danh_sach_silhouette = [], []
for k in range(2, 9):
    km = KMeans(n_clusters=k, init="k-means++", n_init=10, random_state=42)
    nhan = km.fit_predict(X_chuan_hoa)
```

```
    danh_sach_wcss.append(km.inertia_)
    danh_sach_silhouette.append(silhouette_score(X_chuan_hoa, nhan))

K_TOI_UU = 2 + int(np.argmax(danh_sach_silhouette))
print(f"\nSố cụm được chọn (Silhouette cao nhất): K = {K_TOI_UU}")

# 4. Phân cụm cuối cùng với K đã chọn
kmeans = KMeans(n_clusters=K_TOI_UU, init="k-means++", n_init=10, random_state=42)
df["Phan_khuc"] = kmeans.fit_predict(X_chuan_hoa)

# 5. Xây dựng "chân dung" từng phân khúc: giá trị trung bình mỗi đặc trưng theo cụm
chan_dung = df.groupby("Phan_khuc")[cac_dac_trung].mean().round(1)
chan_dung["So_luong_khach_hang"] = df["Phan_khuc"].value_counts().sort_index()
print("\nChân dung từng phân khúc khách hàng:")
print(chan_dung)

# 6. Trực quan hóa bằng PCA (giảm 3 chiều xuống 2 chiều để vẽ)
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X_chuan_hoa)
print(f"\nPhương sai giải thích bởi 2 thành phần PCA: "
      f"{pca.explained_variance_ratio_.sum():.4f}")

plt.figure(figsize=(7, 6))
bieu_do = plt.scatter(X_2d[:, 0], X_2d[:, 1], c=df["Phan_khuc"], cmap="tab10",
s=40, alpha=0.8)
plt.xlabel("Thành phần chính thứ nhất (PC1)")
plt.ylabel("Thành phần chính thứ hai (PC2)")
plt.title(f"Phân khúc khách hàng (K={K_TOI_UU}) - trực quan hóa qua PCA")
plt.colorbar(bieu_do, label="Phân khúc")
plt.tight_layout()
plt.savefig("du_an_3_phan_khuc.png", dpi=120)
print("\nĐã lưu biểu đồ vào tệp du_an_3_phan_khuc.png")
```

23.4. Diễn giải kết quả và khuyến nghị kinh doanh

Với bộ dữ liệu Mall Customer Segmentation, phương pháp Silhouette (kết hợp quan sát Elbow) thường xác định số cụm hợp lý trong khoảng K=5 hoặc K=6, phản ánh các nhóm khách hàng có sự kết hợp thu nhập – điểm chi tiêu khá đa dạng và tách biệt tự nhiên. Bảng "chân dung" từng phân khúc (bước 5 trong đoạn mã) là sản phẩm quan trọng nhất về mặt nghiệp vụ của toàn bộ phân tích — nó "dịch" các cụm số học trừu tượng thành những nhóm khách hàng có thể mô tả bằng ngôn ngữ kinh doanh dễ hiểu. Với K=5, các phân khúc điển hình thường xuất hiện có thể mô tả (minh họa, số liệu thực tế phụ thuộc dữ liệu cụ thể) như sau:

- Nhóm "Thận trọng": thu nhập trung bình, điểm chi tiêu thấp — khách hàng có khả năng chi trả nhưng ít khi mua sắm, có thể do chưa được tiếp cận đúng sản phẩm/khuyến mãi phù hợp.
- Nhóm "Mục tiêu vàng" (thường là phân khúc quan trọng nhất): thu nhập cao, điểm chi

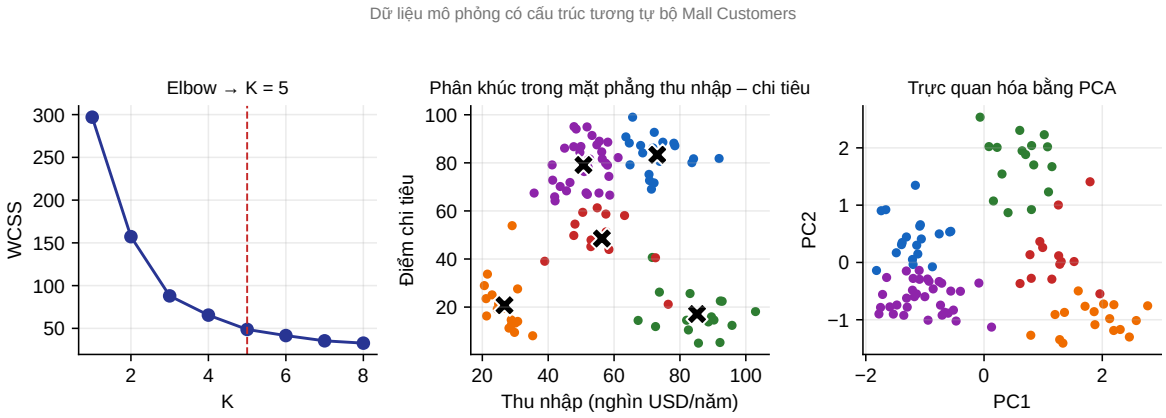
tiêu cao — nhóm khách hàng lý tưởng, nên là trọng tâm của các chương trình chăm sóc khách hàng thân thiết (loyalty program) và ưu tiên trải nghiệm cao cấp.

- Nhóm "Chi tiêu theo cảm hứng": thu nhập thấp nhưng điểm chi tiêu cao — nhóm có hành vi mua sắm vượt mức thu nhập, có thể phù hợp với các chương trình khuyến mãi giá trị nhỏ, tần suất cao, hoặc chính sách trả góp/tín dụng tiêu dùng.
- Nhóm "An toàn tài chính": thu nhập cao nhưng điểm chi tiêu thấp — nhóm tiềm năng lớn nhưng chưa được khai thác, phù hợp với chiến dịch marketing nhấn mạnh giá trị/chất lượng thay vì giá cả, nhằm thuyết phục nhóm này chi tiêu nhiều hơn tương xứng với khả năng tài chính.
- Nhóm "Đại trà": thu nhập và điểm chi tiêu ở mức trung bình trên mọi tiêu chí — nhóm khách hàng phổ thông, phù hợp với các chiến dịch marketing đại chúng, tổng quát.

Trên biểu đồ trực quan hóa PCA, các phân khúc thường thể hiện khá rõ ràng, tách biệt thành các cụm điểm dễ phân biệt bằng mắt thường — một sự xác nhận trực quan hữu ích cho việc lựa chọn K hợp lý ở bước trước đó, đúng tinh thần đã nhấn mạnh ở Chương 14: dù PCA chỉ giữ lại một phần phương sai (cần kiểm tra và báo cáo tỉ lệ phương sai giải thích, như đã thực hiện ở bước 6 của đoạn mã), việc trực quan hóa vẫn cung cấp một góc nhìn định tính giá trị, bổ sung cho các con số định lượng như chỉ số Silhouette.

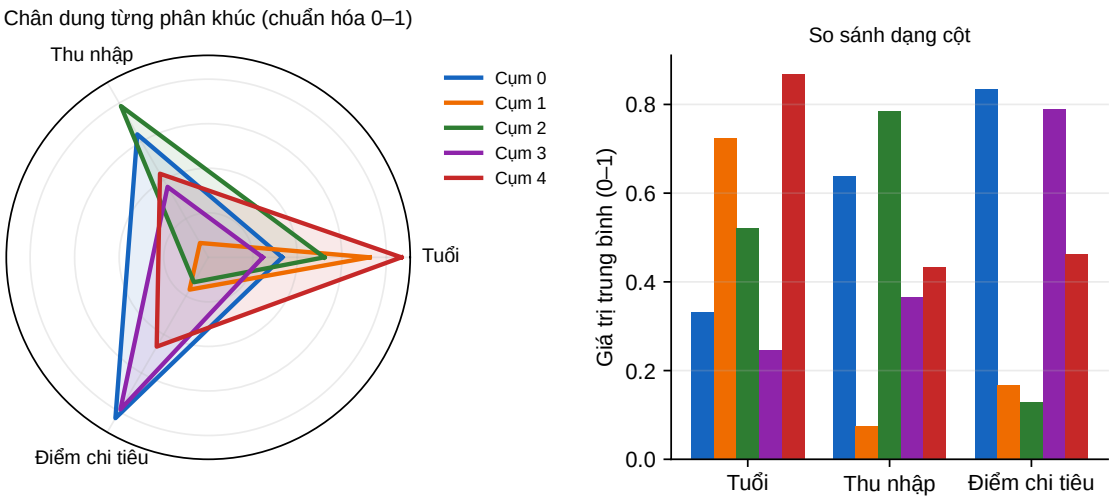
> **Bài học rút ra cho dự án thực tế**. Khác với hai dự án hồi quy và phân loại ở Chương 21–22 (nơi có một độ đo khách quan rõ ràng — RMSE, F1 — để đánh giá "mô hình tốt hay không"), bài toán phân cụm không có tiêu chí đúng/sai khách quan tuyệt đối. Giá trị thực sự của một kết quả phân cụm chỉ được xác nhận khi nó tạo ra hiểu biết nghiệp vụ có ý nghĩa và dẫn đến hành động cụ thể (ví dụ một chiến dịch marketing được thiết kế riêng cho một phân khúc) — đây là lý do bước xây dựng "chân dung" phân khúc và diễn giải bằng ngôn ngữ kinh doanh, như đã thực hiện trong mục này, thường quan trọng hơn cả bản thân việc chọn đúng thuật toán hay đúng số cụm K về mặt kỹ thuật thuần túy.

Hình 23.2 minh họa kết quả phân khúc trên dữ liệu mô phỏng có cấu trúc tương tự bộ Mall Customers.



Hình 23.2. Elbow, các phân khúc theo thu nhập – chi tiêu và trực quan hóa bằng PCA.

Hình 23.3 minh họa "chân dung" định lượng của từng phân khúc.



Hình 23.3. Chân dung các phân khúc khách hàng (giá trị trung bình chuẩn hóa).

23.5. Tóm tắt chương

- Phân khúc khách hàng là ứng dụng kinh doanh điển hình của phân cụm, kết hợp K-Means (Chương 12) để tìm nhóm và PCA (Chương 14) để trực quan hóa kết quả.
- Xây dựng "chân dung" định lượng (giá trị trung bình đặc trưng theo cụm) là bước chuyển hóa kết quả phân cụm kỹ thuật thành thông tin có thể sử dụng trong ra quyết định kinh doanh.
- Mỗi phân khúc khách hàng nên được đặt tên gợi nhớ và gắn với một khuyến nghị hành động cụ thể (chiến dịch marketing, chính sách giá, chương trình chăm sóc khách hàng), thay vì dừng lại ở nhãn cụm số học vô nghĩa (0, 1, 2, …).
- Không giống bài toán có giám sát, phân cụm không có tiêu chí đúng/sai khách quan; giá trị của kết quả được đánh giá qua khả năng tạo ra hiểu biết và hành động kinh doanh có ý nghĩa, không chỉ qua các chỉ số kỹ thuật như Silhouette.

23.6. Bài tập củng cố

Mức thông hiểu

1. Giải thích vì sao không thể dùng Accuracy hay F1-score (như ở Chương 21–22) để đánh giá "độ đúng" của một kết quả phân khúc khách hàng.
2. Nêu vai trò của PCA trong dự án này — vì sao không phân cụm trực tiếp trên 2 thành phần PCA thay vì phân cụm trên 3 đặc trưng gốc rồi mới dùng PCA để vẽ?

Mức vận dụng

1. Với phân khúc "An toàn tài chính" (thu nhập cao, điểm chi tiêu thấp) đã mô tả ở mục 23.4, đề xuất một ý tưởng chiến dịch marketing cụ thể (kênh truyền thông, thông điệp, loại khuyến mãi) phù hợp với đặc điểm hành vi của nhóm này.

23.7. Bài tập lập trình

Bài 1. Thêm đặc trưng "Gender" (giới tính, đã mã hóa nhị phân 0/1) vào tập đặc trưng phân cụm ở ví dụ 23.1 (cùng với Age, `Annual_Income`, `Spending_Score`). Phân cụm lại với K đã chọn trước đó và so sánh chân dung từng phân khúc — giới tính có tạo ra sự phân tách rõ rệt giữa các cụm hay không?

Bài 2. Áp dụng phân cụm phân cấp (AgglomerativeClustering, Chương 13) với cùng số cụm K đã chọn ở ví dụ 23.1, trên cùng dữ liệu đã chuẩn hóa. So sánh chân dung phân khúc thu được với kết quả K-Means bằng chỉ số Adjusted Rand Index (đã dùng ở bài tập Chương 13), nhận xét mức độ tương đồng giữa hai phương pháp.

Bài 3 (nâng cao — dự án mở rộng, tổng hợp toàn giáo trình). Tổng hợp một báo cáo phân tích khách hàng hoàn chỉnh (khoảng 2–3 trang) gồm: (a) Tóm tắt phương pháp (EDA, chuẩn hóa, lựa chọn K, phân cụm); (b) Chân dung chi tiết từng phân khúc kèm biểu đồ minh họa (ví dụ biểu đồ radar hoặc biểu đồ cột so sánh các đặc trưng giữa các phân khúc); (c) Khuyến nghị marketing cụ thể cho mỗi phân khúc; (d) Hạn chế của phân tích (ví dụ: dữ liệu chỉ có 3 đặc trưng, thiếu thông tin về lịch sử mua hàng chi tiết, tần suất ghé thăm, v.v.) và đề xuất dữ liệu bổ sung nên thu thập trong tương lai để cải thiện chất lượng phân khúc. Đây là dạng báo cáo tổng hợp mà một chuyên viên phân tích dữ liệu (data analyst) hoặc nhà khoa học dữ liệu (data scientist) thường phải trình bày trong thực tế công việc, tổng hợp toàn bộ kỹ năng kỹ thuật và khả năng truyền đạt đã được rèn luyện xuyên suốt giáo trình này.

Lời kết

Giáo trình "Học máy – Cơ sở và Ứng dụng với Python" đã đồng hành cùng người học qua một hành trình đầy đủ: từ những khái niệm nhập môn ở Chương 1, qua các thuật toán học có giám sát và không giám sát kinh điển ở Phần 2–3, các kỹ năng đánh giá và tối ưu hóa mô hình chuyên nghiệp ở Phần 4, nền tảng mạng nơ-ron và học sâu hiện đại ở Phần 5, và cuối cùng là ba dự án thực hành tổng hợp ở Phần 6, mô phỏng sát với công việc thực tế của một người làm học máy/khoa học dữ liệu.

Học máy là một lĩnh vực phát triển không ngừng — các thuật toán, thư viện, và phương pháp thực hành tốt nhất tiếp tục được cải tiến qua từng năm. Nền tảng vững chắc về nguyên lý toán học và tư duy quy trình mà giáo trình này trang bị sẽ là hành trang giúp người học tiếp tục tự học, thích nghi với các công cụ và kỹ thuật mới trong suốt sự nghiệp của mình — từ các kiến trúc học sâu tiên tiến hơn (Transformer, mạng đồ thị nơ-ron), đến các lĩnh vực chuyên sâu như học tăng cường, MLOps (vận hành hệ thống học máy trong sản xuất), hay đạo đức và trách nhiệm giải trình trong ứng dụng trí tuệ nhân tạo — những chủ đề nằm ngoài phạm vi của một giáo trình nhập môn nhưng rất đáng để người học tiếp tục khám phá.

Nhóm biên soạn hy vọng giáo trình đã cung cấp một nền tảng lý thuyết chắc chắn kết hợp hài hòa với kỹ năng thực hành cụ thể, và chúc người học tự tin bước tiếp trên con đường chinh phục lĩnh vực học máy và khoa học dữ liệu đầy tiềm năng này.

Tài liệu tham khảo

Sách giáo khoa và chuyên khảo

Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning: Data Mining, Inference, and Prediction (2nd ed.). Springer.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). An Introduction to Statistical Learning with Applications in Python. Springer.

Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3rd ed.). O'Reilly Media.

Murphy, K. P. (2022). Probabilistic Machine Learning: An Introduction. MIT Press.

Bài báo khoa học được tham chiếu

Bergstra, J., & Bengio, Y. (2012). Random Search for Hyper-Parameter Optimization. Journal of Machine Learning Research, 13, 281–305.

Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

Cortes, C., & Vapnik, V. (1995). Support-Vector Networks. Machine Learning, 20(3), 273–297.

Freund, Y., & Schapire, R. E. (1997). A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting. Journal of Computer and System Sciences, 55(1), 119–139.

Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. Annals of Statistics, 29(5), 1189–1232.

Tài liệu và thư viện phần mềm

Pedregosa, F. và cộng sự (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830. Tài liệu chính thức: scikit-learn.org.

Abadi, M. và cộng sự (2015). TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. Tài liệu chính thức: tensorflow.org.

Paszke, A. và cộng sự (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. Tài liệu chính thức: pytorch.org.

Harris, C. R. và cộng sự (2020). Array Programming with NumPy. Nature, 585, 357–362.

McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference.

Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90–95.

Waskom, M. L. (2021). Seaborn: Statistical Data Visualization. Journal of Open Source Software, 6(60), 3021.

Bộ dữ liệu sử dụng trong giáo trình

Fisher, R. A. (1936). Iris Flower Data Set. UCI Machine Learning Repository.

California Housing Dataset — dẫn xuất từ Dữ liệu Điều tra Dân số Hoa Kỳ năm 1990, phân phối qua Scikit-learn.

Breast Cancer Wisconsin (Diagnostic) Dataset — UCI Machine Learning Repository.

LeCun, Y., Cortes, C., & Burges, C. J. C. MNIST Handwritten Digit Database.

Almeida, T. A., & Hidalgo, J. M. G. SMS Spam Collection Dataset — UCI Machine Learning Repository.

Ghi chú biên soạn

Danh mục tài liệu tham khảo trên tuân theo định dạng trích dẫn kiểu APA giản lược, phù hợp với mục đích giáo trình giảng dạy. Trong quá trình biên tập và xuất bản chính thức, nhóm biên soạn khuyến nghị chuẩn hóa toàn bộ trích dẫn theo một chuẩn thống nhất (APA, IEEE, hoặc chuẩn riêng của nhà xuất bản), bổ sung đầy đủ thông tin xuất bản (nhà xuất bản, DOI nếu có) cho từng mục, và cập nhật các đường dẫn tài liệu trực tuyến tại thời điểm xuất bản.